

Nextflow Documentation

Source: <https://docs.seqera.io/nextflow/>

Generated from Seqera Docs on 2026-07-04 16:34 UTC.

Included pages: 98

Contents

1. [Nextflow](https://docs.seqera.io/nextflow/)
<https://docs.seqera.io/nextflow/>
2. [Overview](https://docs.seqera.io/nextflow/overview)
<https://docs.seqera.io/nextflow/overview>
3. [Installation](https://docs.seqera.io/nextflow/install)
<https://docs.seqera.io/nextflow/install>
4. [Environment setup](https://docs.seqera.io/nextflow/developer-env)
<https://docs.seqera.io/nextflow/developer-env>
5. [Your first script](https://docs.seqera.io/nextflow/your-first-script)
<https://docs.seqera.io/nextflow/your-first-script>
6. [Command line interface](https://docs.seqera.io/nextflow/cli)
<https://docs.seqera.io/nextflow/cli>
7. [Scripts](https://docs.seqera.io/nextflow/script)
<https://docs.seqera.io/nextflow/script>
8. [Overview](https://docs.seqera.io/nextflow/modules/)
<https://docs.seqera.io/nextflow/modules/>
9. [Git integration](https://docs.seqera.io/nextflow/git)
<https://docs.seqera.io/nextflow/git>
10. [Amazon Web Services](https://docs.seqera.io/nextflow/aws)
<https://docs.seqera.io/nextflow/aws>
11. [Plugins](https://docs.seqera.io/nextflow/plugins/)
<https://docs.seqera.io/nextflow/plugins/>
12. [Feature flags](https://docs.seqera.io/nextflow/reference/feature-flags)
<https://docs.seqera.io/nextflow/reference/feature-flags>
13. [CLI reference](https://docs.seqera.io/nextflow/reference/cli)
<https://docs.seqera.io/nextflow/reference/cli>
14. [Updating Nextflow](https://docs.seqera.io/nextflow/updating-nextflow)
<https://docs.seqera.io/nextflow/updating-nextflow>
15. [Overview](https://docs.seqera.io/nextflow/developer/)
<https://docs.seqera.io/nextflow/developer/>
16. [Getting started with rnaseq-nf](https://docs.seqera.io/nextflow/tutorials/rnaseq-nf)
<https://docs.seqera.io/nextflow/tutorials/rnaseq-nf>
17. [AWS Java SDK v2](https://docs.seqera.io/nextflow/guides/aws-java-sdk-v2)
<https://docs.seqera.io/nextflow/guides/aws-java-sdk-v2>
18. [Amazon S3](https://docs.seqera.io/nextflow/amazons3)
<https://docs.seqera.io/nextflow/amazons3>
19. [Azure](https://docs.seqera.io/nextflow/azure)
<https://docs.seqera.io/nextflow/azure>
20. [Google Cloud](https://docs.seqera.io/nextflow/google)
<https://docs.seqera.io/nextflow/google>
21. [Kubernetes](https://docs.seqera.io/nextflow/kubernetes)
<https://docs.seqera.io/nextflow/kubernetes>

- 22. [Fusion file system](https://docs.seqera.io/nextflow/fusion)
<https://docs.seqera.io/nextflow/fusion>
- 23. [Configuration](https://docs.seqera.io/nextflow/config)
<https://docs.seqera.io/nextflow/config>
- 24. [Executors](https://docs.seqera.io/nextflow/executor)
<https://docs.seqera.io/nextflow/executor>
- 25. [Caching and resuming](https://docs.seqera.io/nextflow/cache-and-resume)
<https://docs.seqera.io/nextflow/cache-and-resume>
- 26. [Reports](https://docs.seqera.io/nextflow/reports)
<https://docs.seqera.io/nextflow/reports>
- 27. [Containers](https://docs.seqera.io/nextflow/container)
<https://docs.seqera.io/nextflow/container>
- 28. [Conda environments](https://docs.seqera.io/nextflow/conda)
<https://docs.seqera.io/nextflow/conda>
- 29. [Spack environments](https://docs.seqera.io/nextflow/spack)
<https://docs.seqera.io/nextflow/spack>
- 30. [Wave containers](https://docs.seqera.io/nextflow/wave)
<https://docs.seqera.io/nextflow/wave>
- 31. [Workflow diagram](https://docs.seqera.io/nextflow/developer/diagram)
<https://docs.seqera.io/nextflow/developer/diagram>
- 32. [Configuration scopes](https://docs.seqera.io/nextflow/developer/config-scopes)
<https://docs.seqera.io/nextflow/developer/config-scopes>
- 33. [Packages](https://docs.seqera.io/nextflow/developer/packages)
<https://docs.seqera.io/nextflow/developer/packages>
- 34. [nextflow](https://docs.seqera.io/nextflow/developer/nextflow)
<https://docs.seqera.io/nextflow/developer/nextflow>
- 35. [nextflow.ast](https://docs.seqera.io/nextflow/developer/nextflow.ast)
<https://docs.seqera.io/nextflow/developer/nextflow.ast>
- 36. [nextflow.cache](https://docs.seqera.io/nextflow/developer/nextflow.cache)
<https://docs.seqera.io/nextflow/developer/nextflow.cache>
- 37. [nextflow.cli](https://docs.seqera.io/nextflow/developer/nextflow.cli)
<https://docs.seqera.io/nextflow/developer/nextflow.cli>
- 38. [nextflow.cloud.aws](https://docs.seqera.io/nextflow/developer/nextflow.cloud.aws)
<https://docs.seqera.io/nextflow/developer/nextflow.cloud.aws>
- 39. [nextflow.cloud.aws.nio](https://docs.seqera.io/nextflow/developer/nextflow.cloud.aws.nio)
<https://docs.seqera.io/nextflow/developer/nextflow.cloud.aws.nio>
- 40. [nextflow.cloud.azure](https://docs.seqera.io/nextflow/developer/nextflow.cloud.azure)
<https://docs.seqera.io/nextflow/developer/nextflow.cloud.azure>
- 41. [nextflow.cloud.google](https://docs.seqera.io/nextflow/developer/nextflow.cloud.google)
<https://docs.seqera.io/nextflow/developer/nextflow.cloud.google>
- 42. [nextflow.config](https://docs.seqera.io/nextflow/developer/nextflow.config)
<https://docs.seqera.io/nextflow/developer/nextflow.config>
- 43. [nextflow.container](https://docs.seqera.io/nextflow/developer/nextflow.container)
<https://docs.seqera.io/nextflow/developer/nextflow.container>
- 44. [nextflow.dag](https://docs.seqera.io/nextflow/developer/nextflow.dag)
<https://docs.seqera.io/nextflow/developer/nextflow.dag>

- 45. [nextflow.executor](https://docs.seqera.io/nextflow/developer/nextflow.executor)
<https://docs.seqera.io/nextflow/developer/nextflow.executor>
- 46. [nextflow.extension](https://docs.seqera.io/nextflow/developer/nextflow.extension)
<https://docs.seqera.io/nextflow/developer/nextflow.extension>
- 47. [nextflow.k8s](https://docs.seqera.io/nextflow/developer/nextflow.k8s)
<https://docs.seqera.io/nextflow/developer/nextflow.k8s>
- 48. [nextflow.plugin](https://docs.seqera.io/nextflow/developer/nextflow.plugin)
<https://docs.seqera.io/nextflow/developer/nextflow.plugin>
- 49. [nextflow.processor](https://docs.seqera.io/nextflow/developer/nextflow.processor)
<https://docs.seqera.io/nextflow/developer/nextflow.processor>
- 50. [nextflow.scm](https://docs.seqera.io/nextflow/developer/nextflow.scm)
<https://docs.seqera.io/nextflow/developer/nextflow.scm>
- 51. [nextflow.script](https://docs.seqera.io/nextflow/developer/nextflow.script)
<https://docs.seqera.io/nextflow/developer/nextflow.script>
- 52. [nextflow.secret](https://docs.seqera.io/nextflow/developer/nextflow.secret)
<https://docs.seqera.io/nextflow/developer/nextflow.secret>
- 53. [nextflow.trace](https://docs.seqera.io/nextflow/developer/nextflow.trace)
<https://docs.seqera.io/nextflow/developer/nextflow.trace>
- 54. [Using the Nextflow Gradle plugin](https://docs.seqera.io/nextflow/guides/gradle-plugin)
<https://docs.seqera.io/nextflow/guides/gradle-plugin>
- 55. [Migrating to the Nextflow plugin registry](https://docs.seqera.io/nextflow/guides/migrate-plugin)
<https://docs.seqera.io/nextflow/guides/migrate-plugin>
- 56. [Spot Instance failures and retries](https://docs.seqera.io/nextflow/guides/spot-retries)
<https://docs.seqera.io/nextflow/guides/spot-retries>
- 57. [Preparing for strict syntax](https://docs.seqera.io/nextflow/strict-syntax)
<https://docs.seqera.io/nextflow/strict-syntax>
- 58. [Migration notes](https://docs.seqera.io/nextflow/migrations/)
<https://docs.seqera.io/nextflow/migrations/>
- 59. [Migrating to 26.04](https://docs.seqera.io/nextflow/migrations/26-04)
<https://docs.seqera.io/nextflow/migrations/26-04>
- 60. [Migrating to 25.10](https://docs.seqera.io/nextflow/migrations/25-10)
<https://docs.seqera.io/nextflow/migrations/25-10>
- 61. [Migrating to 25.04](https://docs.seqera.io/nextflow/migrations/25-04)
<https://docs.seqera.io/nextflow/migrations/25-04>
- 62. [Migrating to 24.10](https://docs.seqera.io/nextflow/migrations/24-10)
<https://docs.seqera.io/nextflow/migrations/24-10>
- 63. [Migrating to 24.04](https://docs.seqera.io/nextflow/migrations/24-04)
<https://docs.seqera.io/nextflow/migrations/24-04>
- 64. [Migrating from DSL1](https://docs.seqera.io/nextflow/migrations/dsl1)
<https://docs.seqera.io/nextflow/migrations/dsl1>
- 65. [Using modules](https://docs.seqera.io/nextflow/modules/using-modules)
<https://docs.seqera.io/nextflow/modules/using-modules>
- 66. [Developing modules](https://docs.seqera.io/nextflow/modules/developing-modules)
<https://docs.seqera.io/nextflow/modules/developing-modules>
- 67. [Nextflow module registry](https://docs.seqera.io/nextflow/modules/module-registry)
<https://docs.seqera.io/nextflow/modules/module-registry>

- 68. [Working with files](https://docs.seqera.io/nextflow/working-with-files)
<https://docs.seqera.io/nextflow/working-with-files>
- 69. [Processes](https://docs.seqera.io/nextflow/process)
<https://docs.seqera.io/nextflow/process>
- 70. [Processes \(typed\)](https://docs.seqera.io/nextflow/process-typed)
<https://docs.seqera.io/nextflow/process-typed>
- 71. [Workflows](https://docs.seqera.io/nextflow/workflow)
<https://docs.seqera.io/nextflow/workflow>
- 72. [Workflows \(typed\)](https://docs.seqera.io/nextflow/workflow-typed)
<https://docs.seqera.io/nextflow/workflow-typed>
- 73. [Notifications](https://docs.seqera.io/nextflow/notifications)
<https://docs.seqera.io/nextflow/notifications>
- 74. [Secrets](https://docs.seqera.io/nextflow/secrets)
<https://docs.seqera.io/nextflow/secrets>
- 75. [Sharing pipelines](https://docs.seqera.io/nextflow/sharing)
<https://docs.seqera.io/nextflow/sharing>
- 76. [VS Code integration](https://docs.seqera.io/nextflow/vscode)
<https://docs.seqera.io/nextflow/vscode>
- 77. [Using plugins](https://docs.seqera.io/nextflow/plugins/using-plugins)
<https://docs.seqera.io/nextflow/plugins/using-plugins>
- 78. [Developing plugins](https://docs.seqera.io/nextflow/plugins/developing-plugins)
<https://docs.seqera.io/nextflow/plugins/developing-plugins>
- 79. [Nextflow plugin registry](https://docs.seqera.io/nextflow/plugins/plugin-registry)
<https://docs.seqera.io/nextflow/plugins/plugin-registry>
- 80. [Syntax](https://docs.seqera.io/nextflow/reference/syntax)
<https://docs.seqera.io/nextflow/reference/syntax>
- 81. [Semantics](https://docs.seqera.io/nextflow/reference/semantics)
<https://docs.seqera.io/nextflow/reference/semantics>
- 82. [Standard library](https://docs.seqera.io/nextflow/reference/stdlib)
<https://docs.seqera.io/nextflow/reference/stdlib>
- 83. [Process reference](https://docs.seqera.io/nextflow/reference/process)
<https://docs.seqera.io/nextflow/reference/process>
- 84. [Channel factories](https://docs.seqera.io/nextflow/reference/channel)
<https://docs.seqera.io/nextflow/reference/channel>
- 85. [Operators \(typed\)](https://docs.seqera.io/nextflow/reference/operator-typed)
<https://docs.seqera.io/nextflow/reference/operator-typed>
- 86. [Operators \(legacy\)](https://docs.seqera.io/nextflow/reference/operator)
<https://docs.seqera.io/nextflow/reference/operator>
- 87. [Configuration options](https://docs.seqera.io/nextflow/reference/config)
<https://docs.seqera.io/nextflow/reference/config>
- 88. [Environment variables](https://docs.seqera.io/nextflow/reference/env-vars)
<https://docs.seqera.io/nextflow/reference/env-vars>
- 89. [Namespaces](https://docs.seqera.io/nextflow/reference/stdlib-namespaces)
<https://docs.seqera.io/nextflow/reference/stdlib-namespaces>
- 90. [Types](https://docs.seqera.io/nextflow/reference/stdlib-types)
<https://docs.seqera.io/nextflow/reference/stdlib-types>

91. [Groovy and Java classes](#)

<https://docs.seqera.io/nextflow/reference/stdlib-groovy>

92. [Getting started with data lineage](#)

<https://docs.seqera.io/nextflow/tutorials/data-lineage>

93. [Migrating to workflow outputs](#)

<https://docs.seqera.io/nextflow/tutorials/workflow-outputs>

94. [Migrating to static typing](#)

<https://docs.seqera.io/nextflow/tutorials/static-types>

95. [Using operators with static typing](#)

<https://docs.seqera.io/nextflow/tutorials/static-types-operators>

96. [Understanding task resource metrics](#)

<https://docs.seqera.io/nextflow/tutorials/metrics>

97. [Using Nextflow with Flux](#)

<https://docs.seqera.io/nextflow/tutorials/flux>

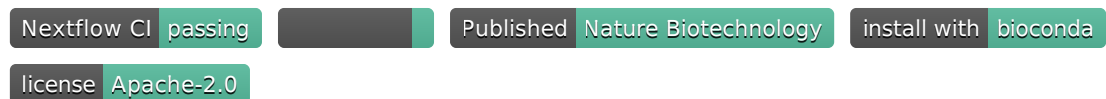
98. [Class Diagrams](#)

<https://docs.seqera.io/nextflow/developer/diagrams/>

Nextflow

"Dataflow variables are spectacularly expressive in concurrent programming"

[Henri E. Bal](#) , [Jennifer G. Steiner](#) , [Andrew S. Tanenbaum](#)



Nextflow is a workflow system for creating scalable, portable, and reproducible workflows. It uses a dataflow programming model that simplifies writing parallel and distributed pipelines by allowing you to focus on data flow and computation. Nextflow can deploy workflows on a variety of execution platforms, including your local machine, HPC schedulers, and cloud. Additionally, Nextflow supports a range of compute environments, software container runtimes, and package managers, allowing workflows to be executed in reproducible and isolated environments.

To get started with Nextflow:

1. See the Nextflow [overview](#) to learn key concepts
2. Download and [install](#) Nextflow
3. Set up an [environment](#) with the [Nextflow VS Code extension](#)
4. Run [your first script](#)

To continue learning about Nextflow, visit the [Nextflow community training portal](#) and find a training course that is right for you. Seqera, the company that develops Nextflow, also runs a variety of training events. See [Seqera Events](#) for more information.

You can post questions in the [Nextflow community forum](#) or the [Nextflow Slack](#). Bugs and feature requests should be reported as [GitHub issues](#).

The Nextflow community is highly active with regular community meetings, events, a podcast, and more. You can view this material on the [Nextflow](#) YouTube channel.

The [nf-core](#) project is a community effort aggregating high-quality Nextflow workflows that can be used by everyone.

Contributions to Nextflow are welcome. See [Contributing](#) for details.

Nextflow is released under the Apache 2.0 license. Nextflow is a [registered trademark](#).

If you use Nextflow in your work, please cite:

- P. Di Tommaso, et al. Nextflow enables reproducible computational workflows. Nature Biotechnology 35, 316–319 (2017) doi:[10.1038/nbt.3820](https://doi.org/10.1038/nbt.3820)

Overview

The rise of big data has made it increasingly necessary to be able to analyze and perform experiments on large datasets in a portable and reproducible manner. Parallelization and distributed computing are the best ways to tackle this challenge. However, the tools commonly available to computational scientists often lack good support for these techniques or provide a model that fits poorly with the needs of computational scientists and require knowledge of complex tools and APIs. Nextflow was created to address these challenges.

The Nextflow language is inspired by [the Unix philosophy](#), in which many simple command line tools can be composed into powerful workflows. Similarly, a Nextflow script consists of many simple processes composed into pipelines. Each process executes a given tool or script in any supported language, and by specifying the process inputs and outputs, Nextflow coordinates the execution of tasks for you.

The Nextflow runtime integrates with popular execution platforms (i.e., HPC schedulers and cloud providers) and software tools (e.g., Git, Docker, and Conda), allowing you to fully describe a computational pipeline with all of its dependencies and run it in nearly any environment - write once, run anywhere.

A Nextflow pipeline script is made by joining together different processes. Each process can be written in any scripting language that can be executed by the Linux platform (Bash, Perl, Ruby, Python, etc.).

A process can define one or more inputs and outputs. Data flows from process to process through asynchronous dataflow structures, known as channels and values. The data dependencies between these processes implicitly determines the flow of execution. For example:


```
// Script parameters
params.query = "/some/data/sample.fa"
params.db = "/some/path/pdb"

// Define the blast_search process
process blast_search {
  input:
  path query
  path db

  output:
  path "top_hits.txt"

  script:
  """
  blastp -db $db -query $query -outfmt 6 > blast_result
  cat blast_result | head -n 10 | cut -f 2 > top_hits.txt
  """
}

// Define the extract_top_hits process
process extract_top_hits {
  input:
  path top_hits
  path db

  output:
  path "sequences.txt"

  script:
  """
  blastdbcmd -db $db -entry_batch $top_hits > sequences.txt
  """
}

// Define the workflow
workflow {
  def query_ch = channel.fromPath(params.query)
  blast_search(query_ch, params.db)
  extract_top_hits(blast_search.out, params.db).view()
}
```

The above example defines two processes. Their execution order is not determined by the `blast_search` process comes before `extract_top_hits` in the script (it could be written the other way around). Instead, execution order is determined by their dependencies — `extract_top_hits` depends on the output of `blast_search`, so `blast_search` will be executed first, and then `extract_top_hits`.

When the workflow is executed, it creates two processes (`blast_search` and `extract_top_hits`) that are connected by the `query_ch` channel. Each process executes a task and emits a value for each input that it receives. Whenever `blast_search` emits a value, `extract_top_hits` receives it through the `query_ch` channel.

See [Dataflow](#) to learn more.

While a process defines what command or script has to be executed, the executor determines how that script is run on the target system.

If not otherwise specified, processes are executed on the local computer. The local executor is useful for pipeline development and testing purposes, but for real world computational pipelines an HPC or cloud platform is often required.

Nextflow provides an abstraction between the pipeline's functional logic and the underlying execution system. Thus, it is possible to write a pipeline once and to seamlessly run it on your computer, a grid platform, or the cloud, by defining the target execution platform in the configuration file.

The following batch schedulers are supported:

- [Open Grid Engine](#)
- [Univa Grid Engine](#)
- [Platform LSF](#)
- [SLURM](#)
- [Flux Framework](#)
- [PBS](#)
- [Torque](#)
- [HTCondor](#)

The following cloud platforms are supported:

- [Amazon Web Services \(AWS\)](#)
- [Microsoft Azure](#)
- [Google Cloud Platform \(GCP\)](#)
- [Kubernetes](#)

See [Executors](#) to learn more.

Nextflow is a workflow language based on [Java](#) and [Groovy](#). It is designed to simplify writing scalable and reproducible pipelines. In most cases, you can leverage your existing programming skills to develop Nextflow pipelines without the steep learning curve that usually comes with a new programming language.

See [Scripts](#) for more information about the Nextflow scripting language.

Pipeline configuration properties are defined in a file named `nextflow.config` in the pipeline execution directory.

This file can be used to define which executor to use, the process's environment variables, pipeline parameters etc. For example:

```
process {
  executor = 'sge'
  queue = 'cn-el6'
}
```

See [Configuration](#) to learn more.

Installation

Nextflow can be used on any POSIX-compatible system (Linux, macOS, etc), and on Windows through [WSL](#). This page describes how to install Nextflow.

note

New versions of Nextflow are released regularly. See [Updating Nextflow](#) for more information about Nextflow release cadence, how to update Nextflow, and how select your version of Nextflow.

Nextflow requires Bash 3.2 (or later) and [Java 17 \(or later, up to 26\)](#) to be installed. To see which version of Java you have, run the following command:

```
java -version
```

Deprecated in version 25.04

Support for Java versions prior to 17 was dropped.

If you don't have a compatible version of Java installed, it is recommended that you install it through [SDKMAN!](#), and that you use the latest Long-Term-Support (LTS) version of Temurin. See [Which version of JDK should I use?](#) for more information about different versions of Java.

To install Java with SDKMAN:

1. [Install SDKMAN:](#)

```
curl -s https://get.sdkman.io | bash
```

2. Open a new terminal.

3. Install Java:

```
sdk install java 17.0.10-tem
```

4. Confirm that Java is installed correctly:

```
java -version
```

Nextflow is distributed as an easy to use self-installing package. It is also distributed via Conda and as a standalone distribution.

To install Nextflow with the self-installing package:

1. Download Nextflow:

```
curl -s https://get.nextflow.io | bash
```

warning

Nextflow updates its executable during the self-install process. An update can fail if the executable is placed in a directory with restricted permissions.

2. Make Nextflow executable:

```
chmod +x nextflow
```

3. Move Nextflow into an executable path. For example:

```
mkdir -p $HOME/.local/bin/  
mv nextflow $HOME/.local/bin/
```

tip

Ensure the directory `$HOME/.local/bin/` is included in your `PATH` variable. Temporarily add this directory to `PATH` by setting `export PATH="$PATH:$HOME/.local/bin"`. Add the directory to `PATH` permanently by adding the export command to your shell configuration file, such as `~/.bashrc` or `~/.zshrc`. Alternatively, move the `nextflow` executable to a directory already in your `PATH`.

4. Confirm Nextflow is installed correctly:

```
nextflow info
```

To install Nextflow with Conda:

1. Create an environment with Nextflow:

```
conda create --name nf-env bioconda::nextflow
```

2. Activate the environment:

```
source activate nf-env
```

3. Confirm Nextflow is installed correctly:

```
nextflow info
```

warning

Installing Nextflow via Conda may lead to outdated versions, dependency conflicts, and Java compatibility issues. Using the self-installing package is recommended for a more reliable and up-to-date installation.

The Nextflow standalone distribution (i.e., the `dist` release) is a self-contained `nextflow` executable that can run without needing to download core dependencies at runtime. This distribution is useful for offline environments as well as building and testing Nextflow locally. The standalone distribution will still download core and third-party plugins at runtime.

To use the standalone distribution:

1. Download the standalone distribution from Assets section of the [GitHub releases page](#).
2. Grant execution permissions to the downloaded file. For example:

```
chmod +x nextflow-24.10.1-dist
```

3. Use it as a drop-in replacement for `nextflow` command. For example:

```
./nextflow-24.10.1-dist run info
```

You can launch workflows directly from [Seqera Platform](#) without installing Nextflow locally.

Launching from Seqera provides you with:

- User-friendly launch interfaces.
- Automated cloud infrastructure creation.
- Organizational user management.
- Advanced analytics with resource optimization.

Seqera Cloud Basic is free for small teams. Researchers at qualifying academic institutions can apply for free access to Seqera Cloud Pro. See [Seqera Platform Cloud](#) to get started.

If you have installed Nextflow locally, you can use the `nextflow auth` command to authenticate with Seqera and automatically configure workflow monitoring.

Environment setup

Setting up a Nextflow development environment is a prerequisite for creating, testing, and optimizing data analysis pipelines.

Recommended tools

- **VS Code**: A versatile code editor that enhances your Nextflow development with features like syntax highlighting and debugging.
- **Extensions**: The VS Code marketplace offers a variety of extensions to enhance development. The **Nextflow extension** is specifically designed to enhance Nextflow development with diagnostics, hover hints, code navigation, code completion, and more.
- **Docker**: A containerization platform that ensures your Nextflow workflows run consistently across different environments by packaging dependencies into isolated containers.
- **Git**: A version control system that helps manage and track changes in your Nextflow projects, making collaboration, and code management more efficient.

The sections below outline the steps for setting up these tools.

note

Nextflow must be installed separately. See [Installation](#) for Nextflow installation instructions.

note

If you are using a Windows computer, first install and configure the Windows Subsystem for Linux (WSL). See [Windows Subsystem for Linux](#) for installation instructions.

An Integrated Development Environment (IDE) provides a user-friendly interface for writing, editing, and managing code. Installing one is an essential step for setting up your environment.

Visual Studio Code (VS Code) is a popular lightweight IDE known for its versatility and extensibility. It offers features like syntax highlighting, intelligent code completion, and integrated debugging tools for various programming languages. VS Code supports Windows, macOS, and Linux, and is a good choice for both new and experienced Nextflow developers.

- Windows
- macOS
- Linux

To install VS Code on Windows:

1. Visit the [VS Code](#) website.
2. Download VS Code for Windows.
3. Double-click the installer executable (`.exe`) file and follow the setup steps.

Extensions are a key feature of IDEs and allow you to customize your development environment by adding support for various programming languages, tools, and features. The [VS Code Marketplace](#) offers thousands of extensions that can enhance your productivity and tailor the editor to your specific needs. Popular VS Code extensions for Nextflow developers are listed below:

The VS Code [Nextflow extension](#) adds Nextflow language support to the editor. The Nextflow extension enhances development with:

- Diagnostics
- Hover hints
- Code navigation
- Code completion
- Formatting
- Renaming
- Parameter schemas
- DAG previews

See [VS Code](#) for more information about the Nextflow extension features and how it enforces the Nextflow syntax.

[nf-core](#) is a community effort to collect a curated set of analysis pipelines built using Nextflow. The [nf-core extension pack](#) adds a selection of tools that support development. For example, it includes [Code Spell Checker](#), [Prettier](#), [Todo Tree](#), and [Markdown Extended](#).

See the [nf-core extension pack](#) for more information about the included tools.

The [Remote Development extension pack](#) enables you to run WSL, SSH, or a development container for editing and debugging with the full set of VS Code features.

The pack includes the [Remote - SSH](#), [Remote - Tunnels](#), [Dev Containers](#), and [WSL](#) extensions. See [Remote Development](#) for more information about the tools included in the remote development extension pack.

note

The Remote Development extension pack is required if you are developing using remote servers, Windows Subsystem for Linux, or Development Containers.

Installing VS Code extensions requires just a few clicks in the Extensions Marketplace.

- Windows
- macOS
- Linux

To install a VS Code extension on Windows:

1. Open VS Code.
2. Open the **Extensions** view in the left-hand menu.
3. Search for the extension.
4. Select **Install**.

Docker is an open-source platform that simplifies application development, deployment, and execution by packaging applications and their dependencies into containers. Containerization enables the creation of self-contained and fully reproducible computational pipelines by bundling a script's binary dependencies into a standardized and portable format. Containers can be executed on any platform that supports a container runtime and ensures consistency across different environments.

Docker Desktop provides a Graphical User Interface (GUI) for managing Docker containers. Installing Docker Desktop is a straightforward process that allows you to create, deploy, and manage applications within containers.

- Windows
- macOS
- Linux

To install Docker Desktop on Windows:

1. Go to [Install Docker Desktop on Windows](#).
2. Download the installer.
3. Double-click Docker Desktop `Installer.exe` to run the installer. By default, Docker Desktop is installed at `C:\Program Files\Docker\Docker`.
4. Depending on your choice of backend, select the **Use WSL 2 instead of Hyper-V** option on the Configuration page.

note

You won't be able to select which backend to use if your system only supports one of the two options.

5. Follow the instructions on the installation wizard to authorize the installer and proceed with the install.
6. When the installation is complete, select **Close**.
7. Start Docker Desktop.
8. Review the Docker Subscription Service Agreement and select **Accept** to continue.

note

Docker Desktop won't run if you do not agree to the terms. You can choose to accept the terms at a later date by opening Docker Desktop.

9. Docker Desktop starts after you accept the terms.

Nextflow supports multiple container technologies (e.g., Singularity and Podman) so you can choose the one that best fits your needs. See [Containers](#) for more information about other supported container engines.

Git provides powerful version control that helps track code changes. Git operates locally, meaning you don't need an internet connection to track changes, but it can also be used with remote platforms like GitHub, GitLab, or Bitbucket for collaborative development.

Nextflow seamlessly integrates with Git for source code management providers to manage pipelines as version-controlled Git repositories.

- Windows
- macOS
- Linux

Git is already installed on most WSL distributions. You can check if it is already installed by running `git version`.

To install the latest stable Git version on Linux Debian/Ubuntu distributions:

1. Open a terminal window and run `sudo apt-get install git-all`.
2. Once complete, run `git version` to verify Git was installed.

See [git-scm documentation](#) for more information about installing Git on other Linux distributions.

Developers can access the power of both Windows and Linux on a Windows machine. The Windows Subsystem for Linux (WSL) lets developers install a Linux distribution and use Linux applications, utilities, and Bash command-line tools directly on Windows without the overhead of a virtual machine or dual-boot setup.

WSL is an optional feature on Windows 10 version 2004 and higher (Build 19041 and higher) or Windows 11. You can enable it through PowerShell or Windows Command Prompt. The steps below outline the recommended setup.

To enable WSL on Windows using Powershell or Windows Command Prompt:

1. Right-click and select **Run as administrator** to use PowerShell or Windows Command Prompt in administrator mode.
2. Run `wsl --install`.

note

This command will enable the features necessary to run WSL and install the Ubuntu distribution.

3. When prompted, restart Windows.
4. After restarting Windows, open the Ubuntu distribution and create a new Linux **User Name** and **Password** when prompted.

note

The **User Name** and **Password** is specific to each Linux distribution that you install and has no bearing on your Windows user name.

See [Set up a WSL development environment](#) for more about installing WSL.

Your first script

This guide details fundamental skills to run a basic Nextflow pipeline. It includes:

- Running a pipeline
- Modifying and resuming a pipeline
- Configuring a pipeline parameter

Prerequisites

You will need the following to get started:

- Nextflow version 25.10 or later. See [Installation](#) for installation instructions. If you have an older version, see [Updating Nextflow](#) to update.

You will run a basic Nextflow pipeline that splits a string of text into two files and then converts lowercase letters to uppercase letters.

The pipeline is defined in the following script:

```

// Default parameter input
params.str = "Hello world!"

// split process
process split {
  publishDir "results/lower"

  input:
  val x

  output:
  path 'chunk_*'

  script:
  """
  printf '${x}' | split -b 6 - chunk_
  """
}

// convert_to_upper process
process convert_to_upper {
  tag "$y"

  input:
  path y

  output:
  path 'upper_*'

  script:
  """
  cat $y | tr '[a-z]' '[A-Z]' > upper_${y}
  """
}

// Workflow block
workflow {
  main:
  ch_str = channel.of(params.str) // Create a channel using parameter input
  ch_chunks = split(ch_str) // Split string into chunks and create a named channel
  ch_upper = convert_to_upper(ch_chunks.flatten()) // Convert lowercase letters to uppercase letters

  publish:
  lower = ch_chunks.flatten()
  upper = ch_upper
}

output {
  lower {
    path 'lower'
  }
  upper {
    path 'upper'
  }
}

```

This script defines two processes:

- `split`: takes a string input, splits it into 6-character chunks, and writes the chunks to files with the prefix `chunk_`.
- `convert_to_upper`: takes files as input, transforms their contents to uppercase letters, and writes the uppercase strings to files with the prefix `upper_`.

The `split` output is emitted as a single element. The `flatten` operator splits this combined element so that each file is treated as a sole element.

The workflow block is organized into two sections:

- `main`: defines the workflow logic and how processes are connected via channels

- `publish::` declares which channels should be published as workflow outputs

The `output` block (outside the workflow) defines where and how each output should be published. In this example, the outputs from both processes are published in subdirectories (`lower` and `upper`) in the default `results` out directory.

To run your pipeline:

1. Create a new file named `main.nf` in your current directory.
2. Copy and save the above pipeline to your new file.
3. Run your pipeline using the following command:

```
nextflow run main.nf
```

You will see output similar to the following:

```
N E X T F L O W ~ version 25.10.0

Launching `main.nf` [big_wegener] DSL2 - revision: 13a41a8946

executor > local (3)
[82/457482] split (1) | 1 of 1 ✓
[2f/056a98] convert_to_upper (chunk_aa) | 2 of 2 ✓
```

Nextflow creates a `work` directory to store files used during a pipeline run. Each execution of a process is run as a separate task. The `split` process is run as one task and the `convert_to_upper` process is run as two tasks. The hexadecimal string, for example, `82/457482`, is the beginning of a unique hash. It is a prefix used to identify the task directory where the script was executed.

When the pipeline completes, you can view the output files in the `results` directory:

```
ls -R results/
```

You will see the published output files organized in the `lower` and `upper` subdirectories.

tip

Run your pipeline with `-ansi-log false` to see each task printed on a separate line:

```
nextflow run main.nf -ansi-log false
```

You will see output similar to the following:

```
N E X T F L O W ~ version 25.10.0
Launching `main.nf` [peaceful_watson] DSL2 - revision: 13a41a8946
[43/f1f8b5] Submitted process > split (1)
[a2/5aa4b1] Submitted process > convert_to_upper (chunk_ab)
[30/ba7de0] Submitted process > convert_to_upper (chunk_aa)
```

Nextflow tracks task executions in a task cache, a key-value store of previously executed tasks. The task cache is used in conjunction with the work directory to recover cached tasks. If you modify and resume your pipeline, only the processes that are changed will be re-executed. The cached results will be used for tasks that don't change.

You can enable resumability using the `-resume` flag when running a pipeline. To modify and resume your pipeline:

1. Open `main.nf`.
2. Replace the `convert_to_upper` process with the following:

```
process convert_to_upper {
    tag "$y"

    input:
    path y

    output:
    path 'upper_*'

    script:
    """
    rev $y > upper_${y}
    """
}
```

3. Save your changes.
4. Run your updated pipeline using the following command:

```
nextflow run main.nf -resume
```

You will see output similar to the following:

```
N E X T F L O W  ~  version 25.10.0

Launching `main.nf` [furious_curie] DSL2 - revision: 5490f13c43

executor > local (2)
[82/457482] split (1) | 1 of 1, cached: 1 ✓
[02/9db40b] convert_to_upper (chunk_aa) | 2 of 2 ✓
```

Nextflow skips the execution of the `split` process and retrieves the results from the cache. The `convert_to_upper` process is executed twice.

See [Caching and resuming](#) for more information about Nextflow cache and resume functionality.

Parameters are used to control the inputs to a pipeline. They are declared by prepending a variable name to the prefix `params`, separated by dot character. Parameters can be specified on the command line by prefixing the parameter name with a double dash character, for example, `--paramName`. Parameters specified on the command line override parameters specified in a main script.

You can configure the `str` parameter in your pipeline. To modify your `str` parameter:

1. Run your pipeline using the following command:

```
nextflow run main.nf --str 'Bonjour le monde'
```

You will see output similar to the following:

```
N E X T F L O W ~ version 24.10.3

Launching `main.nf` [distracted_kalam] DSL2 - revision: 082867d4d6

executor > local (4)
[55/a3a700] process > split (1) [100%] 1 of 1 ✓
[f4/af5ddd] process > convert_to_upper (chunk_ac) [100%] 3 of 3 ✓
```

The input string is now longer and the `split` process splits it into three chunks. The `convert_to_upper` process is run three times.

See [Pipeline parameters](#) for more information about modifying pipeline parameters.

Your first script is a brief introduction to running pipelines, modifying and resuming pipelines, and pipeline parameters. See training.nextflow.io for further Nextflow training modules.

Command line interface

Nextflow provides a robust command line interface (CLI) for the management and execution of pipelines. This page explains the key concepts and common usage patterns for the CLI. For the complete reference of all commands, subcommands, and options, see [CLI reference](#).

note

Nextflow uses two types of command line flags:

- Nextflow options use a single dash (e.g., `-log`) and modify Nextflow's behavior.
- Pipeline parameters use a double dash (e.g., `--input`) and are passed to your pipeline script.

Pipeline execution is the core function of Nextflow. These commands run Nextflow workflows, either from local files or remote Git repositories. Nextflow handles downloading, caching, and executing pipelines with minimal user intervention.

The `run` command executes pipeline scripts from local files or remote repositories. It automatically manages repository downloads, caching, and execution, supporting various Git providers and authentication methods.

See [run](#) for more information.

Run a pipeline from your local filesystem:

```
nextflow run main.nf
```

Use the format `<organization>/<repository>` to run a pipeline directly from Git repositories:

```
nextflow run nextflow-io/hello
```

Nextflow automatically:

1. Downloads the repository to `$HOME/.nextflow/assets/`
2. Caches it for future runs
3. Executes the main script

If you omit the organization, Nextflow searches cached pipelines first, then attempts to download from the `NXF_ORG` organization (default: `nextflow-io`).

You can also use full repository URLs:

```
nextflow run https://github.com/nextflow-io/hello
```

Use the `-user` option to add credentials for private repositories:

```
nextflow run organization/private-repo -user my-username
```

Alternatively, configure Git authentication. See [Git configuration](#) for more information.

Use the `-hub` option specify Bitbucket, GitLab, or other Git providers:

```
nextflow run organization/repo -hub bitbucket
```

Use the `-r` option to specify Git branches, tags, or commits:

```
nextflow run nextflow-io/hello -r v1.1
nextflow run nextflow-io/hello -r dev-branch
nextflow run nextflow-io/hello -r a3f5c8e
```

Added in version 26.04

Nextflow downloads and stores each explicitly requested Git branch, tag, or commit ID in a separate directory path, enabling you to run multiple revisions of the same pipeline simultaneously. Downloaded revisions are stored in a subdirectory of the local project: `$NXF_ASSETS/.repos/<org>/<repo>/clones/<commitId>.`

tip

Use tags or commit IDs instead of branches for reproducible pipeline runs. Branch references change as development progresses over time.

Pipeline parameters are values defined with `params` in your script. Override them on the command line using the `--` prefix to customize pipeline behavior without modifying code:

```
nextflow run main.nf --input data.csv --output results
```

Parameter names support automatic conversion between kebab-case and camelCase:

```
nextflow run main.nf --input-file data.csv # Becomes params.inputFile
```

Parameters without values are set to `true`:

```
nextflow run main.nf --verbose # params.verbose = true
```

warning

Quote parameters containing wildcards to prevent shell expansion:

```
nextflow run main.nf --files "*.fasta"
```

Parameter files

For complex parameter sets, use YAML or JSON files with `-params-file`. This is cleaner than long command lines:


```
{
  "input": "data.csv",
  "output": "results/",
  "min_quality": 20
}
```

```
nextflow run main.nf -params-file params.json
```

Parameter precedence

Nextflow applies parameters defined in multiple places in the following order (lowest to highest priority):

1. Script parameters (`params.foo = 'default'`)
2. Configuration parameters (see [Parameters](#) for more information)
3. Parameter files (`-params-file`)
4. Command line parameters (`--foo bar`)

The `kuberun` command executes pipelines entirely within a Kubernetes cluster. This experimental feature runs the Nextflow driver itself inside Kubernetes.

Use this when you want both the Nextflow driver and tasks running in Kubernetes. This differs from using the Kubernetes executor with `run`, where only tasks run in Kubernetes while the driver runs externally:

```
nextflow kuberun nextflow-io/hello
```

See [kuberun](#) for more information.

Project management commands interact with Git-hosted pipelines. Nextflow integrates with Git providers (e.g., GitHub, GitLab, and Bitbucket) to treat pipelines as versioned projects and maintains a local cache in `$HOME/.nextflow/assets/`.

Use these commands to explore available pipelines, inspect their code, maintain your cache, and clone projects.

The `list` command shows all pipelines currently downloaded to your local cache. This helps you track which projects are available offline and manage your cache directory. Pipelines are stored in `$HOME/.nextflow/assets/` by default:

```
nextflow list
```

The `info` command displays detailed metadata about a downloaded project, including its repository location, local path, main script, and available revisions.

Use this to understand a project's structure, see available versions, or verify which revision is currently checked out:

```
nextflow info hello
```

This prints an output similar to the following:

```

project name: nextflow-io/hello
repository   : https://github.com/nextflow-io/hello
local path   : $HOME/.nextflow/assets/.repos/nextflow-io/hello
main script  : main.nf
revisions    :
> master (default)
  mybranch
> v1.1 [t]
  v1.2 [t]

```

This shows:

- The full project name and repository URL
- Where it's cached locally
- Which script runs by default
- Available revisions (branches and tags marked with [t])
- Which revisions are currently checked out (marked with >)

The `pull` command downloads a pipeline or updates an existing one to the latest version from its Git repository.

Use this to manually download pipelines before running them, update cached pipelines, or download pipelines for offline use:

```
nextflow pull nextflow-io/hello
```

You can specify a particular revision to download:

```
nextflow pull nextflow-io/hello -r mybranch
```

See [pull](#) for more information.

The `view` command displays the contents of a pipeline's main script or lists all files in the repository.

Use this to quickly inspect pipeline code without opening files or explore the project structure. Specify `-l` option lists all repository files instead of showing script contents:

```

nextflow view nextflow-io/hello
nextflow view nextflow-io/hello -l

```

See [view](#) for more information.

The `clone` command copies a pipeline from the cache to a local directory and creates a full Git repository you can modify.

Use this when you want to modify an existing pipeline, create a derivative pipeline, or study a pipeline's structure. If you omit the target directory it uses the pipeline name:

```
nextflow clone nextflow-io/hello my-hello
```

See [clone](#) for more information.

The `drop` command removes a downloaded pipeline from the local cache.

Use this to free disk space by removing pipelines you no longer need. The project is deleted from `$HOME/.nextflow/assets/`. The next `run` will download it again if needed:

```
nextflow drop nextflow-io/hello
```

See [drop](#) for more information.

Added in version 26.04

The `module` command allows you to discover, install, and run modules through a centralized module registry. See [Using modules](#) and the `module` command reference for more information.

The `secrets` command manages secure pipeline secrets.

Use this to store credentials securely, reference them in pipelines without exposing values, and manage sensitive data centrally across your organization:

```
nextflow secrets list
nextflow secrets set AWS_ACCESS_KEY_ID
nextflow secrets delete AWS_ACCESS_KEY_ID
```

See [secrets](#) for more information.

Configuration and validation options and commands help you control and verify pipeline settings. Configuration options supplement pipeline configuration at runtime, while validation commands inspect how Nextflow interprets your configuration files, process definitions, and scripts.

Use these to customize pipeline configuration, debug configuration issues, verify settings, and catch issues before execution.

The `-c` option adds your configuration on top of the defaults and merges them together.

Use this when you want to override specific settings while keeping other defaults intact. Multiple configuration files can be specified as a comma separated list:

```
nextflow -c my.config run nextflow-io/hello
```

See [Configuration](#) for more information.

The `-c` option replaces all default configuration with your custom configuration files. Multiple configuration files can be specified as a comma separated list.

Use this when you want to ensure no default configurations interfere with your custom settings. Unlike `-c` which merges configurations, `-C` ensures only your specified file is used:

```
nextflow -C my.config run nextflow-io/hello
```

See [Configuration](#) for more information.

The `config` command prints the resolved configuration for a pipeline.

Use this to debug configuration issues, verify which settings will be applied, understand configuration precedence, or inspect specific configuration properties:

```
nextflow config
nextflow config nextflow-io/hello
```

See [config](#) for more information.

The `inspect` command analyzes process settings in a pipeline without executing it. It outputs container information in JSON or Nextflow configuration format.

Use this to determine which container images will be used by each process before running the pipeline:

```
nextflow inspect nextflow-io/hello
nextflow inspect nextflow-io/hello -format json
```

See [inspect](#) for more information.

The `lint` command analyzes Nextflow scripts and configuration files for syntax errors and code issues. It can also automatically format your code to maintain consistent style across your project.

Use this to catch syntax errors before execution, enforce consistent code formatting, or validate entire directories of Nextflow code:

```
nextflow lint main.nf
```

See [lint](#) for more information.

Execution history and maintenance commands manage past runs and clean up cached files. Nextflow maintains metadata about all executions and stores intermediate files in work directories.

Use these commands to review past executions, free disk space, troubleshoot failures, or explore data lineage.

The `log` command displays execution history and details about past pipeline runs, such as run names, timestamps, and customizable output fields.

Use this to find run names for resuming, review execution history, or debug failed runs. The command shows recent executions by default, with options to view specific runs or customize output fields:

```
nextflow log
nextflow log dreamy_euler
nextflow log last -f name,status,duration
```

See [log](#) for more information.

The `clean` command removes work directories and cached intermediate files from past executions.

Use this to free disk space, clean up failed or test runs, or maintain your work directory. Use `-n` to perform a dry run and show what would be deleted. Use `-f` to delete files:

```
nextflow clean -n
nextflow clean dreamy_euler -f
```

See [clean](#) for more information.

Added in version 25.04

warning

Experimental: may change in a future release.

The `lineage` command explores data lineage and provenance for workflow executions and tracks relationships between inputs, outputs, and processing steps.

Use this to understand input/output relationships between tasks, trace data flow through the pipeline, or establish file provenance. Lineage tracking must be enabled in configuration:

```
nextflow lineage
```

See [Getting started with data lineage](#) to get started and [lineage](#) for more information.

[Seqera Platform](#) is a comprehensive workflow orchestration platform that extends Nextflow with features for workflow management, monitoring, and collaboration.

Use these commands to authenticate with Seqera Platform and launch workflows directly to the Platform's managed infrastructure.

Added in version 25.10

The `auth` command manages authentication credentials for Seqera Platform, saving access tokens for API interactions.

Use this to log in or out of the Platform, establishing or removing your authentication credentials:

```
nextflow auth login
```

Additional authentication operations include checking login status, viewing configuration details, and logging out:

```
nextflow auth status
nextflow auth config
nextflow auth logout
```

See [auth](#) for more information.

Added in version 25.10

The `launch` command submits a workflow to run on Seqera Platform's infrastructure instead of your local machine.

Use this to leverage Platform's cloud resources, monitoring capabilities, and execution management. The Platform handles resource provisioning, execution monitoring, and result storage:

```
nextflow launch nextflow-io/hello
```

See [launch](#) for more information.

System utilities provide administrative and development tools for managing Nextflow itself, interacting with remote filesystems, working with plugins, and debugging.

Use these commands for system administration, development, and testing.

The `console` command launches an interactive Groovy console with Nextflow's execution context loaded.

Use this to test Nextflow DSL code interactively, debug expressions, explore Nextflow's APIs, or experiment with syntax. It opens a GUI or REPL depending on your environment:

```
nextflow console
```

The `fs` command performs filesystem operations on remote storage systems supported by Nextflow, such as S3, Google Cloud Storage, and Azure Blob Storage.

Use this to manage remote data, test cloud storage access, or perform bulk file operations without additional tools:

```
nextflow fs list s3://my-bucket/data/  
nextflow fs cat s3://my-bucket/data/file.txt  
nextflow fs cp s3://my-bucket/data/file.txt s3://dest/  
nextflow fs delete s3://my-bucket/data/file.txt
```

See [fs](#) for more information.

The `plugin` command creates plugins, installs them, and executes plugin-specific operations.

See [plugin](#) for more information.

Plugin creation

Added in version 25.04

Use the `create` subcommand to create a new plugin scaffold for development:

```
nextflow plugin create
```

Plugin installation

Use the `install` subcommand to install a plugin and extend Nextflow functionality:

```
nextflow plugin install my-plugin
```

Plugin execution

Use the the format `plugin-name:command` to execute plugin-specific commands:

```
nextflow plugin my-plugin:hello --alpha --beta
```

See individual plugin documentation for plugin specific commands.

The `self-update` command updates Nextflow to a newer version. It downloads and installs the latest release or a specific version. By default, it updates to the latest stable release:

```
nextflow self-update
```

Specify a version or use the `NXF_EDGE` environment variable for development releases:

```
NXF_EDGE=1 nextflow self-update
```

The `help` command displays detailed usage information for any Nextflow command.

Use this to learn about command-specific options, refresh your memory about syntax, or discover available features for a particular command:

```
nextflow help run
```

The `-v` and `-version` options print Nextflow version information.

Use `-v` for minimal output showing version and build number:

```
nextflow -v
```

You will see output similar to the following:

```
nextflow version 24.04.0.5917
```

Use `-version` for detailed output showing creation date, citation, and website:

```
nextflow -version
```

You will see output similar to the following:

```
N E X T F L O W
version 24.04.0 build 5917
created 03-05-2024 15:07 UTC
cite doi:10.1038/nbt.3820
http://nextflow.io
```

Scripts

Nextflow is a workflow language that runs on the Java virtual machine (JVM). Nextflow's syntax is very similar to [Groovy](#), a scripting language for the JVM. However, Nextflow is specialized for writing computational pipelines in a declarative manner.

This page is a practical introduction to the Nextflow language. See [Syntax](#) for the language grammar, [Semantics](#) for the meaning and behavior of each language construct, and [Standard library](#) for the built-in types and functions available to every script.

warning

Nextflow uses UTF-8 as the default character encoding for source files. Make sure to use UTF-8 encoding when editing Nextflow scripts with your preferred text editor.

warning

Nextflow scripts have a maximum size of 64 KiB. To avoid this limit for large pipelines, consider moving pipeline components into separate files and including them in the main script.

You can use the `println` function to print to the console:

```
println 'Hello, World!'
```

Variables are declared using the `def` keyword:

```
def num = 1
def str = "Hi"
```

A variable can hold any type of value, and it can be reassigned:

```
def x = 1
x = 'hello'
```

Strings can be defined using single or double quotes (`'` or `"` characters):

```
println "he said 'cheese' once"
println 'he said "cheese!" again'
```

Strings can be concatenated with `+`:

```
def a = "world"
println "hello " + a
```

The important difference between single- and double-quoted strings is that double-quoted strings support interpolation: the value of a variable can be inserted by prefixing its name with `$`, and the value of any expression can be inserted with `${expression}`:


```
def foxttype = 'quick'
def foxcolor = ['b', 'r', 'o', 'w', 'n']
println "The $foxttype ${foxcolor.join()} fox"
// prints: The quick brown fox

println '$foxttype is not interpolated'
// prints: $foxttype is not interpolated
```

A block of text that spans multiple lines can be defined by delimiting it with triple single or double quotes:

```
def text = """
hello there James
how are you today?
"""
```

See [Strings](#) for more about string behavior, including multi-line strings and escaping.

Lists are defined using square brackets:

```
def myList = [1776, -1, 33, 99, 0, 928734928763]
```

You can access an item in a list with square-bracket notation (indexes start at 0), and get the length of a list with the `size` method:

```
println myList[0]
println myList.size()
```

See [List<E>](#) for the set of available list operations.

Maps are used to store key-value pairs. They are unordered collections of named values, which can each be of a different type:

```
def scores = ["Brett": 100, "Pete": "Did not finish", "Andrew": 86.87934]
```

You can access the values in a map by key in two ways:

```
println scores["Pete"]
println scores.Pete
```

To add or modify an entry, assign to a key:

```
scores["Pete"] = 3
scores["Cedric"] = 120
```

tip

You can also use the `+` operator to add two maps together, which produces a new map rather than modifying either input. This is a safer way to modify maps when passing them through channels, because any references to the original map won't be affected. See [Maps](#) for details.

See [Map<K,V>](#) for the set of available map operations.

Records store a set of related fields, where each field can have its own type. They are created with the `record` function and accessed by field name:

```
def person = record(name: 'Alice', age: 42, is_alive: true)
println person.name
```

Tuples store a fixed sequence of heterogeneous values. They are created with the `tuple` function and accessed by index, and can be destructured in an assignment:

```
def coords = tuple(1, 2)
def (x, y) = coords
println "x=$x, y=$y"
```

Records and tuples are both immutable -- once created, they cannot be modified. See [Records](#) and [Tuples](#) for more information.

Operators are symbols that perform specific functions on one or more values. This section highlights some of the most commonly used operators. See [Operators](#) for the full set.

note

Operators in this context are different from channel operators, which are the methods used to work with channels. See [Dataflow](#) for more information.

The `==` and `!=` operators test whether two values are equal (or not equal):

```
assert 2 + 2 == 4
assert [2, 2] != [4]
```

tip

The `assert` keyword tests a condition and raises an error if the condition is false. Every assert on this page succeeds when executed.

Comparison operators compare two values:

```
assert 3 < 3.14           // numbers are compared as numbers
assert 'hello' < 'world'  // strings are compared alphabetically
```

Logical operators perform Boolean logic:

```
assert (true && false) == false // logical AND
assert (true || false) == true  // logical OR
assert (!true) == false         // logical NOT
```

The `in` operator tests membership, i.e., whether a collection contains a value:

```
assert 2 in [1, 2, 3]
```

Arithmetic operators do math:

```

assert 2 + 2 == 4
assert 2 * 3 == 6
assert 2 ** 8 == 256    // exponent
assert 7 % 2 == 1       // modulo (division remainder)

```

Some arithmetic operators can be used with other types of values. For example, `+` can also concatenate lists, maps, and strings:

```

assert [1, 2, 3] + [4] == [1, 2, 3, 4]

```

If-else statements can be used to execute different code under different conditions:

```

def x = Math.random()
if( x < 0.5 ) {
  println 'You lost.'
}
else {
  println 'You won!'
}

```

In some cases, a conditional can be expressed more concisely as a conditional expression (also known as a ternary expression):

```

def message = Math.random() < 0.5
  ? 'You lost.'
  : 'You won!'
println message

```

A shortened version of the conditional expression, known as the Elvis operator, returns the first value if it is truthy, or falls back to a second value otherwise:

```

def scores = ['A': 1, 'B': 2]
assert (scores['C'] ?: 0) == 0

```

Regular expressions can be used to match and extract patterns from strings.

Use `==~` to check whether a pattern occurs anywhere in a string, and `===~` to check whether a string matches a pattern exactly:

```

assert 'hello world' ==~ /hello/
assert 'hello' ===~ /hello/

```

Use the `replaceFirst` and `replaceAll` methods to replace matching occurrences in a string:

```

assert 'colour'.replaceFirst(/ou/, 'o') == 'color'
assert 'cheesecheese'.replaceAll(/cheese/, 'nice') == 'nicenice'

```

You can use the matcher object returned by `==~` to extract capture groups from a string:

```
def version = '2.7.3-beta'
def m = version =~ /(\d+)\.(\d+)\.(\d+)-?(.+)/

assert m[0] == ['2.7.3-beta', '2', '7', '3', 'beta']
assert m[0][1] == '2'
assert m[0][2] == '7'
assert m[0][3] == '3'
assert m[0][4] == 'beta'
```

See [Regex operators](#) for more information.

A closure is a function that can be used like a regular value. Typically, closures are passed as arguments to higher-order functions to express computations in a declarative manner.

For example, the following closure takes one parameter named `v` and returns the square of `v`:

```
{ v -> v * v }
```

This closure can be used with the `collect` method of a list, which uses it to transform each value in the list and produce a new list:

```
assert [1, 2, 3, 4].collect { v -> v * v } == [1, 4, 9, 16]
```

Another example is the `each` method of a map, which takes a closure with two parameters corresponding to the key and value of each map entry:

```
["Yue": "Wu", "Mark": "Williams", "Sudha": "Kumari"].each { key, value ->
  println "$key $value"
}
```

Prints:

```
Yue Wu
Mark Williams
Sudha Kumari
```

See [Closures](#) and [Iteration](#) for more about closures and the higher-order functions used to process collections.

The previous sections covered the basic building blocks of Nextflow code: variables, strings, collections, and closures.

In practice, Nextflow scripts are composed of workflows, processes, functions, and types (collectively known as definitions), and can include definitions from other scripts.

To turn a code snippet into a workflow script, wrap it in a `workflow` block:

```
workflow {
  println 'Hello!'
}
```

This block is called the entry workflow. It is the entrypoint when the script is executed. Whenever a script contains only simple statements like `println 'Hello!'`, Nextflow treats it as an entry workflow.

You can break up code into functions, for example:

```
def sayHello() {
  println 'Hello!'
}

def add(a, b) {
  a + b
}

workflow {
  sayHello()
  println "2 + 2 = ${add(2, 2)}!"
}
```

You can organize these definitions further by moving them to a separate script and including them in the main script:

```
// functions.nf
def sayHello() {
  println 'Hello!'
}

def add(a, b) {
  a + b
}
```

```
// main.nf
include { sayHello; add } from './functions.nf'

workflow {
  sayHello()
  println "2 + 2 = ${add(2, 2)}!"
}
```

See [Processes](#), [Workflows](#), and [Scripts](#) for more information about how to use these features in your Nextflow scripts.

Overview

Nextflow scripts can include **definitions** (workflows, processes, and functions) from other scripts. A **module** is a self-contained process definition and corresponding [spec file](#). Modules can be re-used within a pipeline and can be shared across projects. By packaging process definitions as modules, you avoid duplicating code and benefit from community improvements.

There are two ways to use modules in Nextflow:

- [Local modules](#)
- [Registry modules](#)

See [Using modules](#) to learn how to install and use modules in a project. See [Developing modules](#) to learn how to create and publish your own modules. See [Nextflow module registry](#) to learn how to use the Nextflow module registry.

Local modules are stored and maintained directly in your project. You include definitions from local modules using the `include` keyword with a relative path:

```
include { CAT } from './modules/cat'

workflow {
  data = channel.fromPath('data/*.txt')
  CAT(data)
}
```

The above snippet imports a process named `CAT` from the included script into the including script. The include source `./modules/cat` refers to the script `./modules/cat.nf` relative to the including script path.

Local modules are well suited for project-specific components that are not intended for sharing.

Added in version 26.04

Registry modules are sourced from the [Nextflow registry](#) or a compatible module registry. Like local modules, registry modules are stored in your project, but they are installed and managed using the `nextflow module` command.

Key features of registry modules:

- **Discoverability:** Search for modules by keyword or name and browse available versions.
- **Version management:** Pin specific versions or use the latest release, with automatic integrity checking via checksums.
- **Direct execution:** Run modules as standalone workflows for ad-hoc tasks or testing without writing a wrapper script.
- **Standard structure:** Each module includes a script (`main.nf`), spec (`meta.yml`), and documentation (`README.md`), providing a consistent structure for tooling and automation.

note

Modules from the [nf-core](#) community are automatically mirrored to the Nextflow module registry under the `nf-core` namespace.

Install registry modules into your project and include them by name:

```
$ nextflow module install nf-core/fastqc
```

```
include { FASTQC } from 'nf-core/fastqc'
```

Git integration

Nextflow integrates with Git to manage pipeline versioning and to pull pipelines directly from source code repositories. This page describes how to configure credentials and other settings for Git providers.

The file `$HOME/.nextflow/scm` allows you to centralize the security credentials required to access private project repositories on Bitbucket, GitHub and GitLab source code management (SCM) platforms or to manage the configuration properties of private server installations (of the same platforms).

The configuration properties for each Git provider are defined inside the `providers` section. Properties for the same provider are grouped with a common name and delimited with curly brackets. For example:

```
providers {
  <provider-name> {
    property = value
    // ...
  }
}
```

In the above template replace `<provider-name>` with one of the "default" servers (i.e. `bitbucket`, `github` or `gitlab`) or a custom identifier representing a private SCM server installation.

The following configuration properties are supported for each provider configuration:

User name required to access private repositories on the SCM server.

User password required to access private repositories on the SCM server. For BitBucket, use your API token as the `token` value instead. App passwords are deprecated.

Required only for private Gitlab servers and supported for BitBucket.

Private API access token.

Required only for private SCM servers.

Git provider name, either: `github`, `gitlab`, or `bitbucket`.

Required only for private SCM servers.

SCM server name including the protocol prefix, e.g., `https://github.com`.

Required only for private SCM servers.

SCM API endpoint URL, e.g., `https://api.github.com` (default: the same as `providers.<provider>.server`).

Create a `bitbucket` entry in the [SCM configuration file](#) specifying your credentials and an API token, as shown below:

```
providers {
  bitbucket {
    user = 'me@my-email.com'
    token = 'my-api-token'
  }
}
```


warning

For the Bitbucket Cloud REST API, `user` must be your **Atlassian account email** (listed under Personal settings > Email aliases in Bitbucket), not your Bitbucket username. Using the username results in a `401 Not authorized` error.

When you create the [API token](#), select **Create API token with scopes** and grant at least `read:repository:bitbucket`. Add `write:repository:bitbucket` if you also push. A token without Bitbucket scopes (for example, one created only for Jira or Confluence) is rejected with `401 Not authorized`.

Added in version 25.10

API tokens are supported for BitBucket authentication. When both `token` and `password` are provided, the API token takes priority over the app password.

note

[App passwords](#) are deprecated and stop working on June 9, 2026. Use [API tokens](#) instead. An API token is a substitute password you can use in scripts and tools to avoid putting your real password into configuration files.

[Bitbucket Server](#) is a self-hosted Git repository and management platform.

note

Bitbucket Servers use a different API from the [Bitbucket](#) Cloud service. Make sure to use the right configuration whether you are using the cloud service or a self-hosted installation.

To access your local Bitbucket Server, create an entry in the [SCM configuration file](#) as shown below:

```
providers {
  mybitbucket {
    platform = 'bitbucketserver'
    server = 'https://your.bitbucket.host.com'
    endpoint = 'https://your.bitbucket.host.com'
    user = 'your-user'
    password = 'your-password or your-token'
  }
}
```

Create a `github` entry in the [SCM configuration file](#) specifying your user name and access token as shown below:

```
providers {
  github {
    user = 'your-user-name'
    password = 'your-personal-access-token'
  }
}
```

GitHub requires the use of a personal access token (PAT) in place of a password when accessing APIs. Learn more about PAT and how to create it at [this link](#).

Added in version 23.04

Nextflow automatically uses the `GITHUB_TOKEN` environment variable to authenticate access to the GitHub repository if no credentials are provided via the `scm` file. This is useful especially when accessing pipeline code from a GitHub Action. Read more about the token authentication in the [GitHub documentation](#).

Create a `gitlab` entry in the [SCM configuration file](#) specifying the user name, password and your API access token that can be found in your GitLab [account page](#) (sign in required). For example:

```
providers {
  gitlab {
    user = 'me'
    password = 'my-secret'
    token = 'YgpR8m7viH_ZYnC8YSe8'
  }
}
```

tip

The GitLab token string can be used as the `password` value in the above setting. When doing that the `token` field can be omitted.

[Gitea](#) is a Git repository server with GitHub-like GUI access. Since Gitea installation is quite easy, it is suitable for building a private development environment in your network. To access your Gitea server, you have to provide all the credential information below:

```
providers {
  mygitea {
    server = 'http://your-domain.org/gitea'
    endpoint = 'http://your-domain.org/gitea/api/v1'
    platform = 'gitea'
    user = 'your-user'
    password = 'your-password'
    token = 'your-api-token'
  }
}
```

See [Gitea documentation](#) about how to enable API access on your server and how to issue a token.

Nextflow has built-in support for [Azure Repos](#), a Git source code management service hosted in the Azure cloud. To access your Azure Repos with Nextflow provide the repository credentials using the configuration snippet shown below:

```
providers {
  azurerepos {
    user = 'your-user-name'
    password = 'your-personal-access-token'
  }
}
```

tip

The Personal access token can be generated in the repository `Clone Repository` dialog.

Nextflow supports [AWS CodeCommit](#) as a Git provider to access and to share pipelines code.

To access your project hosted on AWS CodeCommit with Nextflow provide the repository credentials using the configuration snippet shown below:

```
providers {
  my_aws_repo {
    platform = 'codecommit'
    user = '<AWS ACCESS KEY>'
    password = '<AWS SECRET KEY>'
  }
}
```

In the above snippet replace `<AWS ACCESS KEY>` and `<AWS SECRET KEY>` with your AWS credentials, and `my_aws_repo` with a name of your choice.

tip

The `user` and `password` settings are optional. If omitted, the [AWS default credentials provider chain](#) is used.

Then the pipeline can be accessed with Nextflow as shown below:

```
nextflow run https://git-codecommit.eu-west-1.amazonaws.com/v1/repos/my-repo
```

In the above example replace `my-repo` with your own repository. Note also that AWS CodeCommit has different URLs depending on the region in which you are working.

note

The support for protocols other than HTTPS is not available at this time.

Nextflow is able to access repositories hosted on private Bitbucket, GitHub, GitLab and Gitea server installations.

In order to use a private SCM installation you will need to set the server name and access credentials in your [SCM configuration file](#).

If, for example, the host name of your private GitLab server is `gitlab.acme.org`, you will need to have in the `$HOME/.nextflow/scm` file a configuration like the following:

```
providers {
  mygit {
    server = 'http://gitlab.acme.org'
    platform = 'gitlab'
    user = 'your-user'
    password = 'your-password'
    token = 'your-api-token'
  }
}
```

Then you will be able to run/pull a project with Nextflow using the following command line:

```
nextflow run acme/hello -hub mygit
```

Or, alternatively, using the Git clone URL:

```
nextflow run http://gitlab.acme.org/acme/hello.git
```

note

You must also specify the server API endpoint URL if it differs from the server base URL. For example, for GitHub Enterprise V3, add `endpoint = 'https://git.your-domain.com/api/v3'`.

warning

When accessing a private SCM installation over `https` from a server that uses a custom SSL certificate, you may need to import the certificate into your local Java keystore. See [Import the Certificate as a Trusted Certificate](#) for more information.

Amazon Web Services

tip

This page describes how to manually set up and use Nextflow with AWS Cloud. You may be interested in using [Batch Forge](#) in [Seqera Platform](#), which automatically creates the required AWS infrastructure for you with minimal intervention.

Nextflow uses the [AWS security credentials](#) to make programmatic calls to AWS services.

The AWS credentials are selected from the following sources, in order of descending priority:

1. Nextflow configuration file - `aws.accessKey` and `aws.secretKey`. See [aws](#) for more details.
2. A custom profile in `$HOME/.aws/credentials` and/or `$HOME/.aws/config`. The profile can be supplied from the `aws.profile` config option, or the `AWS_PROFILE` or `AWS_DEFAULT_PROFILE` environmental variables.
3. Environment variables - `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.
4. The `default` profile in `~/.aws/credentials` and/or `~/.aws/config`.
5. Single Sign-On (SSO) credentials. See the [AWS documentation](#) for more details.

Added in version 23.10

6. EC2 instance profile credentials. See the [AWS documentation](#) and [this blog post](#) for more details.

The AWS region is selected from the following sources, in order of descending priority:

1. Nextflow configuration file - `aws.region`
2. Environment variables - `AWS_REGION` or `AWS_DEFAULT_REGION`
3. EC2 instance metadata (if Nextflow is running in an EC2 instance)

SSO credentials and instance profile credentials are the most recommended because they don't require you to manage and distribute AWS keys explicitly. SSO credentials are ideal for launching pipelines from outside of AWS (e.g. your laptop), while instance profile credentials are ideal for launching pipelines within AWS (e.g. an EC2 instance).

[IAM policies](#) are the mechanism used by AWS to define permissions for IAM identities. In order to access certain AWS services, the proper policies must be attached to the identity associated to the AWS credentials.

Minimal permissions policies to be attached to the AWS account used by Nextflow are:

- To use AWS Batch:

```
"batch:CancelJob"
"batch:DescribeComputeEnvironments"
"batch:DescribeJobDefinitions"
"batch:DescribeJobQueues"
"batch:DescribeJobs"
"batch:ListJobs"
"batch:RegisterJobDefinition"
"batch:SubmitJob"
"batch:TagResource"
"batch:TerminateJob"
```

- To view [EC2](#) instances:

```
"ec2:DescribeInstanceAttribute"
"ec2:DescribeInstances"
"ec2:DescribeInstanceState"
"ec2:DescribeInstanceTypes"
"ecs:DescribeContainerInstances"
"ecs:DescribeTasks"
```

- To pull container images from [ECR](#) repositories:

```
"ecr:BatchCheckLayerAvailability"
"ecr:BatchGetImage"
"ecr:DescribeImages"
"ecr:DescribeImageScanFindings"
"ecr:DescribeRepositories"
"ecr:GetAuthorizationToken"
"ecr:GetDownloadUrlForLayer"
"ecr:GetLifecyclePolicy"
"ecr:GetLifecyclePolicyPreview"
"ecr:GetRepositoryPolicy"
"ecr:ListImages"
"ecr:ListTagsForResource"
```

Alternatively, you can use AWS provided `AmazonEC2ContainerRegistryReadOnly` managed policy.

note

If you are running Fargate or Fargate Spot, you may need the following policies in addition to the listed above:

```
"ec2:DescribeSubnets"
"ecs:CreateCluster"
"ecs:DeleteCluster"
"ecs:DescribeClusters"
"ecs:ListClusters"
```

Nextflow also requires policies to access [S3 buckets](#) in order to use the work directory, pull input data, and publish results.

Depending on the pipeline configuration, the above actions can be done all in a single bucket but, more likely, spread across multiple buckets. Once the list of buckets used by the pipeline is identified, there are two alternative ways to give Nextflow access to these buckets:

1. Grant access to all buckets by attaching the policy `"s3:*"` to the IAM identity. This works only if buckets do not set their own access policies (see point 2);

2. For more fine grained control, assign to each bucket the following policy (replace the placeholders with the actual values):

```
{
  "Version": "2012-10-17",
  "Id": "<policy-id>",
  "Statement": [
    {
      "Sid": "<statement-id>",
      "Effect": "Allow",
      "Principal": {
        "AWS": "<nextflow-identity-arn>"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:PutObjectTagging",
        "s3:AbortMultipartUpload"
      ],
      "Resource": "arn:aws:s3:::<bucket-name>/*"
    },
    {
      "Sid": "AllowSSLRequestsOnly",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::<bucket-name>",
        "arn:aws:s3:::<bucket-name>/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

See the [bucket policy documentation](#) for additional details.

AWS Batch is a managed computing service that enables the execution of containerized workloads on AWS cloud infrastructure. It dynamically provisions the optimal quantity and type of compute resources (e.g., CPU or memory optimized compute resources) based on the volume and specific resource requirements of the submitted jobs.

Nextflow natively supports AWS Batch, allowing you to run pipeline tasks as Batch jobs in the cloud.

See [AWS Batch](#) to learn more about the `awsbatch` executor.

tip

The recommended approach for running Nextflow pipelines with AWS Batch is to use [Wave](#) and [Fusion](#) for S3 access, rather than relying on the AWS CLI. This approach removes the requirement to install and configure the AWS CLI in containers or custom AMIs.

Nextflow uses the [AWS command line tool](#) (`aws`) to stage input files and output files between S3 and the task containers.

The `aws` command can be made available by either (1) installing it in the container image(s) or (2) installing it in a [custom AMI](#) to be used instead of the default AMI when configuring AWS Batch.

1. In the AWS Console, navigate to **AWS Batch** and create a [Compute environment](#) (CE).
 1. If you are using a custom AMI (see following sections), the AMI ID must be specified in the CE configuration
 2. Make sure to select an AMI (either custom or existing) with Docker installed (see following sections)
 3. Make sure the policy `AmazonS3FullAccess` (granting access to S3 buckets) is attached to the instance role configured for the CE
 4. If you plan to use Docker images from Amazon ECS container, make sure the `AmazonEC2ContainerServiceforEC2Role` policy is also attached to the instance role
2. In the AWS Console, create (at least) one [Job Queue](#) and bind it to the Compute environment.
3. In the AWS Console, create an S3 bucket for the work directory (see below). You can also create separate buckets for input data and results, as needed.
4. Make sure that every process in your pipeline specifies a Docker container with the [container](#) directive.
5. Make sure that all of your container images are published in a Docker registry that can be reached by AWS Batch, such as [Docker Hub](#), [Quay](#), or [Elastic Container Registry](#).

To configure your pipeline for AWS Batch:

1. Specify the AWS Batch [executor](#)
2. Specify the AWS Batch queue with the [queue](#) directive
3. Specify any Batch job container options with the [containerOptions](#) directive.

An example `nextflow.config` file is shown below:

```
process {
    executor = 'awsbatch'
    queue = 'my-batch-queue'
    container = 'quay.io/biocontainers/salmon'
    containerOptions = '--shm-size 16000000 --ulimit nfile=1280:2560 --ulimit nproc=16:32'
}

aws {
    batch {
        // NOTE: this setting is only required if the AWS CLI is installed in a custom AMI
        cliPath = '/home/ec2-user/miniconda/bin/aws'
    }
    region = 'us-east-1'
}
```

tip

Each process can be configured with its own queue by using the [queue](#) directive in the process definition or via [Process selectors](#) in your Nextflow configuration.

The [containerOptions](#) directive can be used to control the properties of the container execution associated with each Batch job.

The following container options are currently supported:


```
-e, --env string
    Set environment variables (format: <name> or <name>=<value>)
--init
    Run an init inside the container that forwards signals and reaps processes
--memory-swap int
    The total amount of swap memory (in MiB) the container can use: '-1' to enable unlimited swap
--memory-swappiness int
    Tune container memory swappiness (0 to 100) (default -1)
--privileged
    Give extended privileges to the container
--read-only
    Mount the container's root filesystem as read only
--shm-size int
    Size (in MiB) of /dev/shm
--tmpfs string
    Mount a tmpfs directory (format: <path>:<options>,size=<int>), size is in MiB
-u, --user string
    Username or UID (format: <name|uid>[:<group|gid>])
--ulimit string
    Ulimit options (format: <type>=<soft limit>[:<hard limit>])
```

Container options may be passed in long form (e.g. `--option value`) or short form (e.g. `-o value`) where available.

Few examples:

```
containerOptions '--tmpfs /run:rw,noexec,nosuid,size=128 --tmpfs /app:ro,size=64'

containerOptions '-e MYVAR1 --env MYVAR2=foo2 --env MYVAR3=foo3 --memory-swap 3240000 --memory-swappiness 20 --shm-size 16000000'

containerOptions '--ulimit nofile=1280:2560 --ulimit nproc=16:32 --privileged'
```

Check the [AWS documentation](#) for further details.

There are several reasons why you might need to create your own **AMI (Amazon Machine Image)** to use in your Compute Environments:

- You do not want to install the AWS CLI into each of your Docker images and would rather provide it through the AMI
- The existing AMI (selected from the marketplace) does not have Docker installed
- You need to attach more storage to your EC2 instance (the default ECS instance AMI has only a 30GB EBS volume which is not enough for most data pipelines)
- You need to install additional software that is not available in your Docker image

From the EC2 Dashboard, select **Launch Instance**, then select **Browse more AMIs**. In the new page, select **AWS Marketplace AMIs**, and then search for Amazon ECS-Optimized Amazon Linux 2023 (AL2023) x86_64 AMI. Select the AMI and continue as usual to configure and launch the instance.

note

The selected instance has a root volume of 30GB. Make sure to increase its size or add a second EBS volume with enough storage for real genomic workloads.

When the instance is running, SSH into it (or connect with the Session Manager service), install the AWS CLI, and install any other tool that may be required (see following sections).

Finally, select **Create Image** from the EC2 Dashboard to create a new AMI from the running instance (you can also do it through the AWS CLI).

The new AMI ID needs to be specified when creating the Batch Compute Environment.

warning

Any additional software must be installed on the EC2 instance before creating the AMI.

tip

The need for the AWS CLI is considered a legacy requirement for the deployment of Nextflow pipelines with AWS Batch. Instead, consider using [Wave](#) and [Fusion](#) to facilitate access to S3 without using the AWS CLI.

The [AWS CLI](#) should be installed in your custom AMI using a self-contained package manager such as [Conda](#). That way, you can control which version of Python is used by the AWS CLI (which is written in Python).

If you don't use Conda, the `aws` command will attempt to use the version of Python that is installed in the container, and it won't be able to find the necessary dependencies.

The following snippet shows how to install AWS CLI with [Miniconda](#) in the home folder:

```
cd $HOME
sudo yum install -y bzip2 wget
wget https://repo.continuum.io/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -f -p $HOME/miniconda
$HOME/miniconda/bin/conda install -c conda-forge --override-channels -y awscli
rm Miniconda3-latest-Linux-x86_64.sh
```

Afterwards, verify that the AWS CLI package works correctly:

```
$ ./miniconda/bin/aws --version
aws-cli/1.29.20 Python/3.11.4 Linux/4.14.318-241.531.amzn2.x86_64 botocore/1.31.20
```

note

The `aws` tool will be placed in a directory named `bin` in the main installation folder. Modifying this directory structure after the tool is installed will cause it to not work properly.

To configure Nextflow to use this installation, specify the `aws.batch.cliPath` option in the Nextflow configuration as shown below:

```
aws.batch.cliPath = '/home/ec2-user/miniconda/bin/aws'
```

Replace the path above with the one matching the location where the `aws` tool is installed in your AMI.

warning

The grandparent directory of the `aws` tool will be mounted into the container at the same path as the host, e.g. `/home/ec2-user/miniconda`, which will shadow existing files in the container. Make sure you use a path that is not already present in the container.

Docker is required by Nextflow to execute tasks on AWS Batch. The **Amazon ECS-optimized Amazon Linux 2023 AMI** has Docker installed, however, if you create your AMI from a different AMI that does not have Docker installed, you will need to install it manually.

The following snippet shows how to install Docker on an Amazon EC2 instance:

```
# install Docker
sudo yum update -y
sudo yum install docker

# start the Docker service
sudo service docker start

# empower your user to run Docker without sudo
sudo usermod -a -G docker ec2-user
```

You must log out and log back in again to use the new `ec2-user` permissions.

These steps must be done before creating the AMI from the current EC2 instance.

The [ECS container agent](#) is a component of Amazon Elastic Container Service (Amazon ECS) and is responsible for managing containers on behalf of ECS. AWS Batch uses ECS to execute containerized jobs, therefore it requires the agent to be installed on EC2 instances within your Compute Environments.

The ECS agent is included in the **Amazon ECS-optimized Amazon Linux 2023 AMI**. If you use a different base AMI, you can also install the agent on any EC2 instance that supports the Amazon ECS specification.

To install the agent, follow these steps:

```
sudo yum install ecs-init
sudo systemctl enable --now ecs
```

To test the installation:

```
curl -s http://localhost:51678/v1/metadata | python -mjson.tool
```

note

The `AmazonEC2ContainerServiceforEC2Role` policy must be attached to the instance role in order to be able to connect the EC2 instance created by the Compute Environment to the ECS container.

note

The `AmazonEC2ContainerRegistryReadOnly` policy should be attached to the instance role in order to get read-only access to Amazon EC2 Container Registry repositories.

Nextflow automatically creates the Batch [Job definitions](#) needed to execute tasks in your pipeline, so you don't need to define them beforehand.

However, sometimes you may still need to specify a custom **Job Definition** to fine tune the configuration of a specific job, for example to define custom mount paths.

To do that, first create a **Job Definition** in the AWS Console (or by other means). Note the name of the Job definition you created. You can then associate a process execution with this Job definition by using the [container](#) directive and specifying, in place of the container image name, the Job definition name prefixed by `job-definition://`, as shown below:

```
process.container = 'job-definition://your-job-definition-name'
```

Nextflow allows the use of multiple executors in the same workflow application. This feature enables the deployment of hybrid workloads in which some jobs are executed in the local computer or local computing cluster and some jobs are offloaded to AWS Batch.

To enable this feature, use one or more [Process selectors](#) in your Nextflow configuration to apply the AWS Batch configuration to the subset of processes that you want to offload. For example:

```
process {
  withLabel: bigTask {
    executor = 'awsbatch'
    queue = 'my-batch-queue'
    container = 'my/image:tag'
  }
}

aws {
  region = 'eu-west-1'
}
```

With the above configuration, processes with the `bigTask` [label](#) will run on AWS Batch, while the remaining processes will run in the local computer.

Then launch the pipeline with the `-bucket-dir` option to specify an AWS S3 path for the jobs computed with AWS Batch and, optionally, the `-work-dir` to specify the local storage for the jobs computed locally:

```
nextflow run <script-or-project-name> -bucket-dir s3://<bucket>
```

warning

The AWS S3 path needs to contain at least one sub-directory (e.g. `s3://<bucket>/work` rather than `s3://<bucket>`).

note

Nextflow will automatically manage the transfer of input and output files between the local and cloud environments when using hybrid workloads.

User provided container volume mounts can be provided as shown below:

```
aws {
  region = 'eu-west-1'
  batch {
    volumes = '/tmp'
  }
}
```

Multiple volumes can be specified as a comma-separated list of paths. The usual Docker volume mount syntax can be used to specify complex volumes where the container path is different from the host path or the volume should be read-only. For example:

```
aws {
  region = 'eu-west-1'
  batch {
    volumes = ['/tmp', '/host/path:/mnt/path:ro']
  }
}
```

The above snippet defines two volume mounts for the jobs executed in your pipeline. The first volume mounts the host path `/tmp` to the same path in the container, with the read-write access mode. The second volume mounts the host path `/host/path` to `/mnt/path` in the container, with the read-only access mode.

Problem: The Pipeline execution terminates with an AWS error message similar to the one shown below:

```
JobQueue <queue> not found
```

Make sure you have defined a AWS region in the Nextflow configuration file and it matches the region in which your Batch environment has been created.

Problem: A process execution fails reporting the following error message:

```
Process <task> terminated for an unknown reason -- Likely it has been terminated by the external system
```

This may happen when Batch is unable to execute the process script. A common cause of this problem is that the Docker container image you have specified uses a non standard [entrypoint](#) which does not allow the execution of the Bash launcher script required by Nextflow to run the job.

This may also happen if the AWS CLI doesn't run correctly.

Other places to check for error information:

- The `.nextflow.log` file.
- The Job execution log in the AWS Batch dashboard.
- The [CloudWatch](#) logs found in the `/aws/batch/job` log group.

Problem: A process execution is stalled in the `RUNNABLE` status and the pipeline output is similar to the one below:

```
executor > awsbatch (1)
process > <process> (1) [ 0%] 0 of ....
```

It may happen that the pipeline execution hangs indefinitely because one of the jobs is held in the queue and never gets executed. In AWS Console, the queue reports the job as `RUNNABLE` but it never moves from there.

There are multiple reasons why this can happen. They are mainly related to the Compute Environment workload/configuration, the docker service or container configuration, network status, etc.

This [AWS page](#) provides several resolutions and tips to investigate and work around the issue.

Added in version 24.04

Nextflow provides experimental support for the execution of [AWS Batch jobs with Fargate resources](#).

AWS Fargate is a technology that you can use with AWS Batch to run containers without having to manage servers or EC2 instances. With AWS Fargate, you no longer have to provision, configure, or scale clusters of virtual machines to run containers.

To enable the use of AWS Fargate in your pipeline use the following settings in your `nextflow.config` file:

```
process.executor = 'awsbatch'
process.queue = '<aws-batch-queue>'
aws.region = '<aws-region>'
aws.batch.platformType = 'fargate'
aws.batch.jobRole = 'JOB_ROLE_ARN'
aws.batch.executionRole = 'EXECUTION_ROLE_ARN'
wave.enabled = true
```

See the AWS documentation for details how to create the required AWS Batch queue for Fargate, the Batch Job Role and the Batch Execution Role.

note

Nextflow uses `s5cmd` to download the task input data and upload the task outputs. To enable this capability, you need to enable the Wave service in the Nextflow configuration, as shown in the above example. See [Wave][] for more details.

See [aws](#) to learn more about advanced configuration options.

Plugins

Nextflow plugins are extensions that enhance the functionality of Nextflow. They allow users to add new capabilities and integrate with external services without bloating their pipeline code or modifying Nextflow itself.

There are two types of Nextflow plugins: core plugins and third-party plugins. The main features of each plugin type are described below.

Core plugins

Core plugins do not require configuration. The latest versions of core plugins are automatically installed when a Nextflow pipeline requests them. Core plugins include:

- `nf-amazon`: Support for Amazon Web Services.
- `nf-azure`: Support for Microsoft Azure.
- `nf-cloudcache`: Support for the cloud cache.
- `nf-console`: Implementation of the Nextflow [REPL console](#).
- `nf-google`: Support for Google Cloud.
- `nf-seqera`: Support for [Seqera Platform](#) and Seqera Intelligent Compute.
- `nf-wave`: Support for [Wave containers service](#).

Specific versions of core plugins can be declared in Nextflow configuration files or by using the `-plugins` option. See [Using plugins](#) for more information.

note

The automatic retrieval of core plugins can be disabled by setting `NXF_PLUGINS_DEFAULT=false`. See [Environment variables](#) for more information.

Third-party plugins

Third-party plugins must be configured via Nextflow configuration files or at runtime. To configure a plugin via configuration files, use the `plugins` block. For example:

```
plugins {
  id 'nf-hello@0.5.0'
}
```

To configure plugins at runtime, use the `-plugins` option. For example:

```
nextflow run main.nf -plugins nf-hello@0.5.0
```

See [Using plugins](#) for more information.

Nextflow plugins are free to use any versioning convention. Many plugins, including all core plugins, use [Semantic Versioning](#), which helps developers communicate the kind of changes in a release.

Semantic versions have the form MAJOR.MINOR.PATCH (e.g., 0.5.0):

- MAJOR: Increment for backward-incompatible changes.
- MINOR: Increment for backward-compatible feature additions.
- PATCH: Increment for backward-compatible bug fixes.

Optional pre-release and build metadata can be added (e.g., 1.2.1-alpha+001) as extensions to the base version format.

Feature flags

Feature flags enable opt-in features. They must be specified in the pipeline script.

warning

Feature flags marked as preview can cause pipelines run with newer versions of Nextflow to fail due to breaking changes. Consult the [migration notes](#) before updating to a new Nextflow version.

Deprecated in version 25.10

Use the [strict parser](#) instead. It validates process selectors without producing false warnings.

When `true`, prints a warning for every `withName:` process selector that doesn't match a process in the pipeline (default `true`).

Deprecated in version 25.04

Defines the DSL version to use (`1` or `2`).

When `true`, enables the use of module-scoped executable scripts via [module resources](#).

Deprecated in version 26.04

When `true`, executes the pipeline in "strict" mode, which introduces the following rules:

- When reading a params file, Nextflow will fail if a dynamic param value references an undefined variable
- When merging params from a config file with params from the command line, Nextflow will fail if a param is specified from both sources but with different types
- When using the `join` operator, the `failOnDuplicate` option is `true` regardless of any user setting
- When using the `join` operator, the `failOnMismatch` option is `true` (unless `remainder` is also `true`) regardless of any user setting
- When using the `publishDir` process directive, the `failOnError` option is `true` regardless of any user setting
- In a process definition, Nextflow will fail if an input or output tuple has only one element
- In a process definition, Nextflow will fail if an output emit name is not a valid identifier (i.e. it should match the pattern `/[A-Za-z_][A-Za-z0-9_]* /`)
- During a process execution, Nextflow will fail if a received input tuple does not have the same number of elements as was declared
- During a process execution, Nextflow will fail if the `storeDir` directive is used with non-file outputs

- Nextflow will fail if a pipeline param is referenced before it is defined
- Nextflow will fail if multiple functions and/or processes with the same name are defined in a module script

Added in version 26.04

Preview feature: the syntax and behavior may change in future releases.

When `true`, enables the use of [typed processes](#) and [typed workflows](#). Must be enabled in every script that uses typed processes/workflows. Legacy processes/workflows cannot be defined in scripts with this flag enabled.

CLI reference

This page lists all of the available options and subcommands for the [Nextflow CLI](#).

The top-level options are meant to be invoked in relation to the core Nextflow application and are applied to all commands. For options specific to any command, refer the CLI Commands section.

Available options:

Comma-separated list of configuration files which are used as the configuration set. Any other default configuration files are ignored.

Set JVM properties (e.g. `-Dfile.encoding=UTF-8`). Equivalent to the `NXF_JVM_ARGS` environment variable.

Execute Nextflow in background.

Allows you to close your terminal without terminating the pipeline run.

Comma-separated list of configuration files which are added to the configuration set.

Deprecated in version 23.10

Launch Nextflow via Docker (experimental).

Print available commands and options.

Set Nextflow log file path (default: `.nextflow.log`). Must be a local path.

Do not print the Nextflow banner and execution progress to the console.

Does not affect error messages.

Enable JVM interactive remote debugging (experimental).

Send logs to a [Syslog](#) server (e.g. `localhost:514`).

Enable trace level logging for the specified packages. Multiple packages can be provided separating them with a comma, e.g. `-trace nextflow,io.seqera`.

Print the program version.

Added in version 25.10

Manage Seqera Platform authentication.

```
$ nextflow auth <subcommand> [options]
```

The `auth` command provides authentication and configuration management for Seqera. For Seqera Cloud, it uses an OAuth2 authentication flow to generate and save a Personal Access Token (PAT) locally. For Seqera Enterprise installations, it uses direct PAT authentication. Credentials are saved to `~/.nextflow/seqera-auth.config`. An `includeConfig` statement is automatically added to `~/.nextflow/config` or the configuration file set using `$NXF_HOME`.

Prints the command usage information.

Authenticates with Seqera and saves credentials. Sets Seqera primary compute environment, monitoring, and workspace.

Available options:

Specifies your Seqera API endpoint (default: `https://api.cloud.seqera.io`)

Removes Seqera authentication and revokes the Seqera Cloud access token (if applicable).

Sets Seqera primary compute environment, monitoring, and workspace.

Shows Seqera authentication status and configuration.

Authenticate with Seqera Cloud:

```
$ nextflow auth login
```

Authenticate with an Enterprise installation:

```
$ nextflow auth login -u https://tower.example.com/api
```

View current authentication status:

```
$ nextflow auth status
```

Configure Seqera settings:

```
$ nextflow auth config
```

Remove authentication:

```
$ nextflow auth logout
```

Clean up cache and work directories.

```
$ nextflow clean [run_name|session_id] [options]
```

Upon invocation within a directory, `nextflow` creates a project specific `.nextflow.log` file, `.nextflow` cache directory as well as a `work` directory. The `clean` command is designed to facilitate removal of these files from previous executions. A list of run names and session ids can be generated by invoking `nextflow log -q`.

If no run name or session id is provided, it will clean the latest run.

Clean up runs executed after the specified one.

Clean up runs executed before the specified one.

Clean up all runs except the specified one.

Print names of files to be removed without deleting them.

Force clean command.

Print the command usage.

Removes only temporary files but retains execution log entries and metadata.

Do not print names of files removed.

Dry run to remove work directories for the run name `boring_euler`:

```
$ nextflow clean boring_euler -n

Would remove work/92/c1a9cd9a96e0531d81ca69f5dc3bb7
Would remove work/3f/70944c7a549b6221e1ccc7b4b21b62
Would remove work/0e/2ebdba85f76f6068b21a1bcbf10cab
```

Remove work directories for the run name `boring_euler`:

```
$ nextflow clean boring_euler -f

Removed work/92/c1a9cd9a96e0531d81ca69f5dc3bb7
Removed work/3f/70944c7a549b6221e1ccc7b4b21b62
Removed work/0e/2ebdba85f76f6068b21a1bcbf10cab
```

Remove the execution entries except for a specific execution:

```
$ nextflow clean -but tiny_leavitt -f

Removed work/1f/f1ea9158fb23b53d5083953121d6b6
Removed work/bf/334115deec60929dc18edf0010032a
Removed work/a3/06521d75da296d4dd7f4f8caaddad8
```

Dry run to remove the execution data before a specific execution:

```
$ nextflow clean -before tiny_leavitt -n

Would remove work/5d/ad76f7b7ab3500cf616814ef644b61
Would remove work/c4/69a82b080a477612ba8d8e4c27b579
Would remove work/be/a4fa2aa38f76fd324958c81c2e4603
Would remove work/54/39116773891c47a91e3c1733aad4de
```

Dry run to remove the execution data after a specific execution:

```
$ nextflow clean -after focused_payne -n

Would remove work/1f/f1ea9158fb23b53d5083953121d6b6
Would remove work/bf/334115deec60929dc18edf0010032a
Would remove work/a3/06521d75da296d4dd7f4f8caaddad8
```

Dry run to remove the temporary execution data for a specific execution, while keeping the log files:

```
$ nextflow clean -keep-logs tiny_leavitt -n

Would remove temp files from work/1f/f1ea9158fb23b53d5083953121d6b6
Would remove temp files from work/bf/334115deec60929dc18edf0010032a
Would remove temp files from work/a3/06521d75da296d4dd7f4f8caaddad8
```

Clone a remote project into a folder.

```
$ nextflow clone [options] [project]
```

The `clone` command downloads a pipeline from a Git-hosting platform into the current directory and modifies it accordingly. For downloading a pipeline into the global cache `~/.nextflow/assets`, please refer to the `nextflow pull` command.

Create a shallow clone of the specified depth.

Print the command usage.

Service hub where the project is hosted. Options: `gitlab` or `bitbucket`.

Revision to clone. It can be a git branch, tag, or commit SHA number.

Private repository user name.

Clone the latest revision of a pipeline:

```
$ nextflow clone nextflow-io/hello
nextflow-io/hello cloned to: hello
```

Clone a specific revision of a pipeline:

```
$ nextflow clone nextflow-io/hello -r v1.1
nextflow-io/hello cloned to: hello
```

Print the resolved pipeline configuration.

```
$ nextflow config [options] [project name or path]
```

The `config` command is used for printing the project's configuration i.e. the `nextflow.config` and is especially useful for understanding the resolved profiles and parameters that Nextflow will use run a pipeline. For in-depth information, see the [Config profiles](#) section.

Print config using flat notation.

Print the command usage.

Choose a configuration profile.

Print config using Java properties notation.

Added in version 26.04

Project revision. Can be a git branch, tag, or commit SHA number.

Show all configuration profiles.

Sort config attributes.

Added in version 23.10

Print the value of a config option, or fail if the option is not defined.

Print out the inferred config using a the default group key-value notation:

```
$ nextflow config

docker {
    enabled = true
}

process {
    executor = 'local'
}
```

Print out the config using a flat notation:

```
$ nextflow config -flat

docker.enabled = true
process.executor = 'local'
```

Print out the config using the Java properties notation:

```
$ nextflow config -properties

docker.enabled = true
process.executor = local
```

Print out the value of a specific configuration property:

```
$ nextflow config -value process.executor
local
```

Print out all profiles from the project's configuration:

```
$ nextflow config -show-profiles

docker {
    enabled = true
}

profiles {
    standard {
        process {
            executor = 'local'
        }
    }
    cloud {
        process {
            executor = 'cirrus'
            container = 'cbcr/imagex'
        }
    }
}
```

Launch the Nextflow interactive console.

```
$ nextflow console
```

The `console` command provides a Graphical User Interface (GUI) and an interactive REPL (Read-Eval-Print-Loop) for quick experimentation.

None available.

Launch the `console` GUI:

```
$ nextflow console
```

Delete the local copy of a project.

```
$ nextflow drop [options] [project]
```

The `drop` command is used to remove the projects which have been downloaded into the global cache. Please refer the `list` command for generating a list of downloaded pipelines.

Delete the repository without taking care of local changes.

Print the command usage.

Added in version 26.04

Project revision to drop. Can be a git branch, tag, or commit SHA number.

Drop the `nextflow-io/hello` project:

```
$ nextflow drop nextflow-io/hello
```

Forcefully drop the `nextflow-io/hello` pipeline, ignoring any local changes:

```
$ nextflow drop nextflow-io/hello -f
```

Perform basic filesystem operations.

```
$ nextflow fs [subcommands]
```

The `fs` command is used to perform filesystem operations like copy, move, delete, list directory, etc. Like the `file()` method, it can work with local files, remote URLs, and remote object storage. Storage credentials can be provided through the same manner as launching a pipeline (Nextflow config, environment vars, etc).

Print the command usage.

List a directory:

```
$ nextflow fs list <directory>
```

Print the contents of a file to standard output:

```
$ nextflow fs cat <file>
```

Copy a file or directory:

```
$ nextflow fs cp <source> <target>
```

Move a file or directory:

```
$ nextflow fs mv <source> <target>
```

Delete a file or directory:

```
$ nextflow fs rm <path>
```

Added in version 23.10

Print file or directory attributes:

```
$ nextflow fs stat <path>
```

Print the top-level help or specific help for a command.

```
$ nextflow help [options] [command]
```

The `help` command prints out the overview of the CLI interface and enumerates the top-level options and commands. Note that this command is equivalent to simply invoking `nextflow` at the command line.

Print the command usage.

Invoke the `help` option for the `drop` command:

```
$ nextflow help drop

Delete the local copy of a project
Usage: drop [options] name of the project to drop
Options:
  -f          Delete the repository without taking care of local changes
              Default: false
  -h, -help   Print the command usage
              Default: false
```

Print project or system runtime information.

```
$ nextflow info [options] [project]
```

The `info` command prints out the nextflow runtime information about the hardware as well as the software versions of the Nextflow version and build, operating system, and Groovy and Java runtime. It can also be used to display information about a specific project.

If no run name or session id is provided, it will clean the latest run.

Check for remote updates.

Show detailed information.

Print the command usage.

Output format, either `text`, `json`, or `yaml`.

Display Nextflow runtime and system info:

```
$ nextflow info

Version: 20.07.1 build 5412
Created: 24-07-2020 15:18 UTC (20:48 IDT)
System: Mac OS X 10.15.6
Runtime: Groovy 2.5.11 on OpenJDK 64-Bit Server VM 1.8.0_192-b01
Encoding: UTF-8 (UTF-8)
```

Display information about a specific project:

```
$ nextflow info nextflow-io/hello

project name: nextflow-io/hello
repository  : https://github.com/nextflow-io/hello
local path  : /Users/evanfloden/.nextflow/assets/.repos/nextflow-io/hello
main script : main.nf
revisions   :
* master (default)
  mybranch
  testing
* v1.1 [t]
  v1.2 [t]
```

Added in version 23.10

Inspect process settings in a pipeline project. Currently only supports the `container` directive.

```
$ nextflow inspect [options] [project]
```

The `inspect` command allows you to determine the container for each process in a pipeline without running the pipeline. It prints to stdout a listing of containers for each process, formatted either as JSON or Nextflow configuration.

Build the container images resolved by the inspect command.

Inspect output format. Can be `json` or `config`.

Ignore errors while inspecting the pipeline.

Load script parameters from a JSON/YAML file.

Use the given configuration profile(s).

Revision of the project to inspect (either a git branch, tag or commit SHA number).

Get the list of containers used by a pipeline:

```
$ nextflow inspect nextflow-io/hello
```

Specify parameters as with the `run` command:

```
$ nextflow inspect main.nf --alpha 1 --beta hello
```

Launch a Nextflow pipeline on a Kubernetes cluster.

```
$ nextflow kuberun [options] [project]
```

The `kuberun` command builds upon the `run` command and offers a deep integration with the Kubernetes execution environment. This command deploys the Nextflow runtime as a Kubernetes pod and assumes that you've already installed the `kubectl` CLI. The `kuberun` command does not allow the execution of local Nextflow scripts. See [Kubernetes](#) for more information.

The `kuberun` command supports the following options from `run`:

- `-cache`
- `-disable-jobs-cancellation`
- `-dsl1`
- `-dsl2`
- `-dump-channels`
- `-dump-hashes`
- `-e.<key>=<value>`
- `-entry`
- `-h, -help`
- `-hub`
- `-latest`
- `-main-script`
- `-name`
- `-offline`
- `-params-file`
- `-plugins`
- `-preview`
- `-process.<key>=<value>`
- `-profile`
- `-qs, -queue-size`
- `-resume`
- `-r, -revision`
- `-stub, -stub-run`
- `-user`
- `-with-conda`
- `-with-dag`
- `-N, -with-notification`
- `-with-report`
- `-with-spack`
- `-with-timeline`
- `-with-tower`
- `-with-trace`
- `-with-wave`
- `-with-weblog`
- `-without-spack`
- `-without-wave`
- `-w, -work-dir`

The following new options are also available:

Specify number of CPUs requested for the Nextflow pod.

Specify the container image for the Nextflow driver pod.

Specify amount of memory requested for the Nextflow pod.

Specify script to be run before the Nextflow pod starts.

Specify the K8s namespace to use.

Add the specified file from the K8s cluster to configuration set.

Choose a configuration profile in the remoteConfig.

Volume claim mounts. For example, `my-pvc:/mnt/path`.

Execute a pipeline into a Kubernetes cluster:

```
$ nextflow kuberun nextflow-io/hello
```

Added in version 25.10

Launch a workflow in [Seqera Platform](#).

```
$ nextflow launch [options] [project]
```

The `launch` command launches a pipeline run in Seqera Platform. To log in and configure the execution environment, use the `auth` command.

The compute environment for workflow execution.

A configuration file to add to the configuration set.

The entry workflow to be executed.

Prints command usage information.

Whether to pull the latest changes before execution.

A mnemonic name to assign to the run.

The script file to execute when launching a project directory or repository. Should be a path relative to the project root. For example, `-main-script subproject/main.nf`.

A JSON or YAML file to load parameters from.

A configuration profile.

Whether to resume an execution using cached results from a previous run.

The project revision to run. Can be a git branch, tag, or commit hash.

Whether to replace scripts with command stubs when executing the run.

Added in version 26.04

Name of user secret to use in the pipeline. Can be specified multiple times.

The directory where intermediate result files are stored.

The Seqera Platform workspace name.

Added in version 26.04

Name of workspace secret to use in the pipeline. Can be specified multiple times.

Execute a pipeline in Seqera Platform:

```
$ nextflow launch nextflow-io/hello
```

Added in version 25.04

warning

Experimental: may change in a future release.

Inspect lineage metadata for Nextflow runs.

```
$ nextflow lineage SUBCOMMAND [arg ..]
```

The `lineage` command is used to inspect lineage metadata.

See [Getting started with data lineage](#) to learn how to get started with data lineage.

Print the command usage.

Validate the checksum of output lineage record.

Display a git-style diff between two lineage records.

Find all lineage records that match the given field values.

List the Nextflow runs with lineage enabled, printing the corresponding lineage ID (LID) for each run.

Render the lineage graph for a lineage record as an HTML file (default output path: `./lineage.html`).

The lineage record should be of type `FileOutput`, `TaskRun`, or `WorkflowRun`.

View a lineage record.

Lint Nextflow scripts and config files.

```
$ nextflow lint [options] [paths]
```

The `lint` command parses and analyzes the given Nextflow scripts and config files, formats them if specified, and prints any errors. Directories are recursively scanned for scripts and config files to lint.

File pattern to exclude from linting (default: `.git`, `.lineage`, `.nextflow`, `.nf-test`, `nf-test.config`, `work`).

Can be specified multiple times.

Added in version 26.04

Read list of paths to lint from a text file. Use `-` to read from standard input.

Format scripts and config files that have no errors.

Output mode for reporting errors: `full`, `extended`, `concise`, `json`, `markdown` (default: `full`).

Added in version 26.04

Path to project directory (default: `.'`). Used to locate project-level assets such as the `lib` directory and modules directory.

Sort script declarations in Nextflow scripts (default: `false`).

Number of spaces to indent (default: `4`).

Indent with tabs (default: `false`).

Lint a specific file:

```
$ nextflow lint main.nf
```

Lint all files in the current directory (and subdirectories) with concise output:

```
$ nextflow lint -o concise .
```

Lint and format all files in the current directory (and subdirectories) and use two spaces per indent:

```
$ nextflow lint -format -spaces 2 .
```

note

Formatting code with the `lint` command in Nextflow 25.10 or later may make your code incompatible with previous versions of Nextflow. If you need your code to remain compatible with versions prior to 25.10, run the formatter with Nextflow 25.04:

```
NXF_VER=25.04.8 nextflow lint -format .
```

List all downloaded projects.

```
$ nextflow list [options]
```

The `list` commands prints a list of the projects which are already downloaded into the global cache `~/.nextflow/assets`.

Print the command usage.

List the downloaded pipelines:

```
$ nextflow list

nextflow-io/hello
nextflow-hub/fastqc
```

Print the execution history and log information.

```
$ nextflow log [options] [run_name | session_id]
```

The `log` command is used to query the execution metadata associated with pipelines executed by Nextflow. The list of executed pipelines can be generated by running `nextflow log`. Instead of run name, it's also possible to use a session id. Moreover, this command contains multiple options to facilitate the queries and is especially useful while debugging a pipeline and while inspecting pipeline execution metadata.

Show log entries for runs executed after the specified one.

Show log entries for runs executed before the specified one.

Show log entries for runs executed but the specified one.

Comma-separated list of fields to include in the printed log.

The same fields as the `trace.fields` option can be specified here, as well as `stdout` and `stderr`. The trace fields `%cpu` and `%mem` must be specified as `pcpu` and `pmem`, respectively.

Use the `-l` option to see the complete list of available fields.

Filter log entries by a custom expression, e.g. `process =~ /hello.*/ && status == 'COMPLETED'`.

Print the command usage.

Show all available fields.

Show only run names.

Character used to separate column values.

Text template used to each record in the log.

Listing the execution logs of previous invocations of all pipelines in a project:

```
$ nextflow log
```

TIMESTAMP	DURATION	RUN NAME	STATUS	REVISION ID	SESSION
2020-10-07 11:52:24	2.1s	focused_payne	OK	96eb04d6a4	af6adaaa-ad4f-48a2-9f6a-b121e789adf5
2020-10-07 11:53:00	3.1s	tiny_leavitt	OK	e3b475a61b	4d3b95c5-4385-42b6-b430-c865a70d56a4
2020-10-07 11:53:29	2.5s	boring_euler	OK	e3b475a61b	a6276975-7173-4208-ae09-ab9d6dce8737

Listing only the run names of the execution logs of all pipelines invocations in a project:

```
$ nextflow log -quiet
```

focused_payne
tiny_leavitt
boring_euler

List the execution entries only a specific execution:


```
$ nextflow log tiny_leavitt

work/1f/f1ea9158fb23b53d5083953121d6b6
work/bf/334115deec60929dc18edf0010032a
work/a3/06521d75da296d4dd7f4f8caaddad8
```

List the execution entries after a specific execution:

```
$ nextflow log -after tiny_leavitt

work/92/c1a9cd9a96e0531d81ca69f5dc3bb7
work/3f/70944c7a549b6221e1ccc7b4b21b62
work/0e/2ebdba85f76f6068b21a1bcbf10cab
```

List the execution entries before a specific execution:

```
$ nextflow log -before tiny_leavitt

work/5d/ad76f7b7ab3500cf616814ef644b61
work/c4/69a82b080a477612ba8d8e4c27b579
work/be/a4fa2aa38f76fd324958c81c2e4603
work/54/39116773891c47a91e3c1733aad4de
```

List the execution entries except for a specific execution:

```
$ nextflow log -but tiny_leavitt

work/5d/ad76f7b7ab3500cf616814ef644b61
work/c4/69a82b080a477612ba8d8e4c27b579
work/be/a4fa2aa38f76fd324958c81c2e4603
work/54/39116773891c47a91e3c1733aad4de
```

Filter specific fields from the execution log of a process:

```
$ nextflow log tiny_leavitt -f 'process,exit,hash,duration'

split_letters      0      1f/f1ea91      112ms
convert_to_upper    0      bf/334115      144ms
convert_to_upper    0      a3/06521d      139ms
```

Filter fields from the execution log of a process based on a criteria:

```
$ nextflow log tiny_leavitt -F 'process =~ /split_letters/'

work/1f/f1ea9158fb23b53d5083953121d6b6
```

Added in version 26.05.0-edge

Print the contents of a Nextflow log file, with optional level filtering and follow mode.

```
$ nextflow logfile [options] [run_name | session_id | path]
```

The `logfile` command prints the contents of a `.nextflow.log` file written by Nextflow during a pipeline run. The argument can be:

- A run name (e.g. `kickass_rutherford`) -- resolved via the local execution history and matched to the corresponding `.nextflow.log[.N]` file in the current directory by session id.
- A session id (or unique prefix), or the literal `last` for the most recent run.
- A direct path to a log file (e.g. `.nextflow.log.4` or an absolute path).

If no argument is given, the most recent run is used.

Continuously print new lines as the file grows (similar to `tail -f`). Stop with Ctrl+C.

Print the command usage.

Preserve any ANSI escape sequences found inside log content. By default these are stripped so the output is clean text.

Minimum log level to print: `TRACE`, `DEBUG`, `INFO`, `WARN`, or `ERROR`. Case-insensitive. When set, only entries at this level or higher are printed. Multi-line entries (e.g. stack traces) inherit the level of their parent entry. No filtering is applied by default.

Print only the last N log entries. Counts entries (one or more lines each), not raw lines.

Do not pipe the output through a pager. By default, when standard output is a terminal, the output is piped through `$NXF_PAGER`, falling back to `$PAGER`, falling back to `less -FR`. A bare `less` invocation is augmented with `-FR` so colors render. Short outputs exit without entering the pager. Pager is automatically disabled with `-f` / `-follow`.

Disable colored output. By default, when the output destination is a terminal:

- Every line is prefixed with a coloured 2-character level indicator.
- `TRACE` and `DEBUG` entries are dimmed.
- `DEBUG`, `INFO`, `WARN` and `ERROR` entries are syntax-highlighted to make them easier to read.

Styling is auto-disabled when output goes to a non-terminal destination (file or pipe with `-no-pager`) or when the `NO_COLOR` environment variable is set.

Print the log of the most recent run:

```
$ nextflow logfile
```

Print the log of a specific run, looked up by name:

```
$ nextflow logfile kickass_rutherford
```

Print a specific log file directly:

```
$ nextflow logfile .nextflow.log.4
```

Show only warnings and errors from the most recent run:

```
$ nextflow logfile -level WARN
```

Show the last 10 entries of an ongoing run and continue streaming new lines:

```
$ nextflow logfile -n 10 -f
```

Added in version 26.04

Manage Nextflow modules.

```
$ nextflow module <subcommand> [options]
```

The `module` command provides a comprehensive system for managing registry-based modules. It enables installing modules from registries, running them directly, searching for available modules, and publishing your own modules to a registry.

Create a new module with a basic `main.nf`, `meta.yml`, and `README.md`.

Install a module from the registry into your project. Downloaded modules are stored in the `modules/` directory. The `.module-info` file is created in the module directory to store additional information of the installed module.

The following options are available:

Specify the module version to install (e.g., `1.0.0`). If not specified, installs the latest version.

Force reinstall even if the module exists locally with modifications. Without this flag, Nextflow prevents overwriting locally modified modules.

List all modules currently installed in your project. Shows each module's name, version, and integrity status (whether it has been modified locally).

The following options are available:

Output mode for list results. Options: `table` (default), `json`.

Publish a module to the registry, making it available for others to install. The argument can be either a `namespace/name` reference (for an already-installed module) or a local directory path containing the module files. Requires authentication via the `NXF_REGISTRY_TOKEN` environment variable or the `registry.apiKey` config option. The module directory must contain `main.nf`, `meta.yml`, and `README.md`.

The following options are available:

Validate the module structure and metadata without uploading to the registry. Useful for testing before publishing.

Specify the registry to publish the module (default: `https://registry.nextflow.io`)

Remove a module from your project. By default, removes both local files and configuration entries. Use options to control what gets removed.

The following options are available:

Force removal even if the module has no `.module-info` file (i.e. not installed from a registry) or has local modifications.

Remove the `.module-info` but keep local files in the `modules/` directory.

Execute a module directly. Can be a remote module (`namespace/name`) or a local module path (beginning with `./`, `../`, or `/`). Automatically downloads the module if not already installed.

Accepts all standard Nextflow run options. For example, `-profile`, `-resume`, and `-c`. Command-line params (i.e., `--<input_name>`) are inferred from the module's declared inputs.

The following additional options are available:

Specify the module version to run (e.g., `1.0.0`). If not specified, uses the latest version.

Search for modules in the registry by keyword or name. Returns modules matching the query with their names, versions, descriptions, and download statistics.

The following options are available:

Maximum number of results to return (default: varies by registry).

Output mode for search results. Options: `simple` (default), `json`.

Generate the `meta.yml` for a local module from the source code (`main.nf`).

The generated file includes `TODO` placeholders for fields that were not specified.

If a spec file already exists, it is incorporated into the new file.

The following options are available:

Display detailed information about a module from the registry. Shows module name, version, description, and other metadata, as well as example usage.

The following options are available:

Specify the module version to query (e.g., `1.0.0`). If not specified, displays information for the latest version.

Output mode for info results. Options: `text` (default), `json`.

Module namespace, used to construct the module name. Required when the argument is a path; ignored when it is a module name.

Module version string (e.g. `1.0.0`). Defaults to `TODO: Add version`.

Short description of what the module does. Defaults to `TODO: Add description`.

SPDX license identifier (e.g. `MIT`, `Apache-2.0`). Defaults to `TODO: Add license (e.g., MIT)`.

Module author. Can be specified multiple times, once per author. Defaults to `[TODO: Add author]`.

Print the generated spec to stdout without writing any file.

Validate a module before publishing to the registry.

Verifies that all required files are present (`main.nf`, `meta.yml`, `README.md`) and that the module spec contains all required fields (name, version, description, license).

Install the latest module version:

```
$ nextflow module install nf-core/fastqc
```

Display installed modules in formatted table:

```
$ nextflow module list
```

Validate module structure without publishing:

```
$ nextflow module publish myorg/my-module -dry-run
```

Publish module to nextflow registry:

```
$ export NXF_REGISTRY_TOKEN=your-token  
$ nextflow module publish myorg/my-module
```

Display information for latest module version:

```
$ nextflow module view nf-core/fastqc
```

Display information for specific version:

```
$ nextflow module view nf-core/fastqc -version 1.0.0
```

Create a module:

```
$ nextflow module create
```

Remove module completely:

```
$ nextflow module remove nf-core/fastqc
```

Run a remote module:

```
$ nextflow module run nf-core/fastqc \  
  --input 'data/*.fastq.gz'
```

Run a remote module with specific version and run options:

```
$ nextflow module run nf-core/fastqc \  
  -version 1.0.0 \  
  --input 'data/*.fastq.gz' \  
  -with-conda \  
  -resume
```

Run a local module:

```
$ nextflow module run ./modules/nf-core/fastqc/main.nf \  
  --input 'data/*.fastq.gz'
```

Search for alignment-related modules:

```
$ nextflow module search alignment
```

Generate spec for local module by name:

```
$ nextflow module spec nf-core/fastqc
```

Generate spec for local module by path (namespace required):

```
$ nextflow module spec -namespace nf-core ./modules/my-module
```

Provide additional fields to avoid TODO placeholders:

```
$ nextflow module spec \  
-namespace nf-core \  
-version 1.0.0 \  
-description "Quality control of raw sequencing reads" \  
-license MIT \  
-author "@drpatelh" \  
-author "@joseespinoza" \  
./modules/nf-core/fastqc
```

Validate module by name:

```
$ nextflow module validate myorg/my-module
```

Validate module by path:

```
$ nextflow module validate ./modules/myorg/my-module
```

Manage plugins and run plugin-specific commands:

```
$ nextflow plugin <subcommand> [options]
```

Added in version 25.04

Create a plugin scaffold using the [Nextflow plugin template](#). See [Creating a plugin](#) for more information.

Install a plugin. Multiple plugins can be specified as a comma-separated list. Each plugin id consists of a name and optional version separated by a @.

Execute a plugin-specific command.

Download or update a project.

```
$ nextflow pull [options] [project]
```

The `pull` command downloads a pipeline from a Git-hosting platform into the global cache `~/.nextflow/assets` and modifies it accordingly. For downloading a pipeline into a local directory, please refer to the `nextflow clone` command.

Update all downloaded projects.

Deprecated in version 26.04

Ignored for new multi-revision asset management strategy. Still used in legacy assets.

Create a shallow clone of the specified depth.

Print the command usage.

Service hub where the project is hosted. Options: `gitlab` or `bitbucket`

Added in version 26.04

Update the project asset to new multi-revision strategy.

Project revision to run. Can be a git branch, tag, or commit SHA number.

When passing a git tag or branch, the `workflow.revision` and `workflow.commitId` fields are populated. When passing only the commit hash, `workflow.revision` is not defined.

Private repository user name.

Download a new pipeline or pull the latest revision for a specific project:

```
$ nextflow pull nextflow-io/hello

Checking nextflow-io/hello ...
done - revision: 96eb04d6a4 [master]
```

Pull the latest revision for all downloaded projects:

```
$ nextflow pull -all

Checking nextflow-io/hello ...
done - revision: 96eb04d6a4 [master]
Checking nextflow-hub/fastqc ...
done - revision: 087659b18e [master]
```

Download a specific revision of a new project or pull the latest revision for a specific project:

```
$ nextflow pull nextflow-io/hello -r v1.1

Checking nextflow-io/hello ...
checkout-out at AnyObjectId[1c3e9e7404127514d69369cd87f8036830f5cf64] - revision: 1c3e9e7404 [v1.1]
```

Execute a pipeline.

```
$ nextflow run [options] [project]
```

The `run` command is used to execute a local pipeline script or remote pipeline project.

Exports all current system environment.

Enable/disable ANSI console logging.

Remote bucket where intermediate result files are stored. When running a hybrid workflow, `-bucket-dir` and `-work-dir` should define separate work directories for remote tasks and local tasks, respectively.

Enable/disable processes caching.

Deprecated in version 26.04

Ignored for new multi-revision asset management strategy. Still used in legacy assets.

Create a shallow clone of the specified depth.

Prevent the cancellation of child jobs on execution termination

Deprecated in version 23.10

Execute the workflow using DSL1 syntax.

Deprecated in version 23.10

Execute the workflow using DSL2 syntax.

Dump channels for debugging purpose. Optionally accepts a comma-separated list of channel tags to filter which channels are printed.

Dump task hash keys for debugging purposes.

Added in version 23.10

You can use `-dump-hashes json` to dump the task hash keys as JSON for easier post-processing. See the [caching and resuming tips](#) for more details.

Add the specified variable to execution environment.

Deprecated in version 24.10

Use params in the entry workflow to call different workflows from the command line.

Entry workflow to be executed.

Print the command usage.

Service hub where the project is hosted. Options: `gitlab` or `bitbucket`

Pull latest changes before run.

Library extension path.

The script file to execute when launching a project directory or repository. Should be a path relative to the project root. For example, `-main-script subproject/main.nf`.

The project `nextflow.config` is always read from the project root regardless of the main script location.

Assign a mnemonic name to the a pipeline run.

Do not check for remote project updates.

Added in version 24.10

Directory where workflow outputs are stored.

-output-format

Added in version 26.04

Output format for printing workflow outputs. Options: `text` (default), `json`, `none`.

Load script parameters from a JSON/YAML file.

Comma separated list of plugin ids to be applied in the pipeline execution.

Run the workflow script skipping the execution of all processes.

Set process config options.

Choose a configuration profile.

Max number of processes that can be executed in parallel by each executor.

Execute the script using the cached results, useful to continue executions that was stopped by an error.

Project revision to run. Can be a git branch, tag, or commit SHA number.

When passing a git tag or branch, the `workflow.revision` and `workflow.commitId` fields are populated. When passing only the commit hash, `workflow.revision` is not defined.

Execute the workflow replacing process scripts with command stubs

Test a script function with the name specified.

Private repository user name.

Enable process execution in an Apptainer container.

Enable process execution in a Charliecloud container.

Enable the use of the Cloud cache plugin for storing cache metadata to an object storage bucket.

Use the specified Conda environment package or file (must end with `.yaml` or `.yml`)

Create pipeline DAG file.

Changed in version 23.10

The default format was changed from `dot` to `html`.

Enable process execution in a Docker container.

Send a notification email on workflow completion to the specified recipients.

Enable process execution in a Podman container.

Create workflow execution HTML report.

Enable process execution in a Singularity container.

Use the specified Spack environment package or file (must end with `.yaml`)

Create workflow execution timeline.

Monitor workflow execution with [Sequera Platform](#) (formerly Tower Cloud).

Create workflow execution trace file.

Enable the use of Wave containers.

Send workflow status messages via HTTP to target URL.

Disable process execution with Conda.

Disable process execution with Docker.

Disable process execution in a Podman container.

Disable process execution with Spack.

Disable the use of Wave containers.

Directory where intermediate result files are stored.

Run a specific revision of a remote pipeline:

```
$ nextflow run nextflow-io/hello -r v1.1

N E X T F L O W ~ version 20.07.1
Launching `nextflow-io/hello` [grave_cajal] - revision: 1c3e9e7404 [v1.1]
```

Choose a `profile` for running the project. Assumes that a profile named `docker` has already been defined in the config file:

```
$ nextflow run main.nf -profile docker
```

Execute a pipeline and generate the summary HTML report. For more information on the metrics, see [Reports](#):

```
$ nextflow run main.nf -with-report
```

Execute a pipeline with a custom queue size. By default, the queue size is the number of available CPUs:

```
$ nextflow run nextflow-io/hello -qs 4
```

Invoke the pipeline with a specific workflow as the entry-point:

```
$ nextflow run main.nf -entry workflow_A
```

Execute a pipeline with integrated monitoring in [Sequera Platform](#):

```
$ nextflow run nextflow-io/hello -with-tower
```

Execute a pipeline with a custom parameters file (YAML or JSON):

```
$ nextflow run main.nf -params-file pipeline_params.yml
```

See [Pipeline parameters](#) for more information about writing custom parameters files.

Manage [pipeline secrets](#).

```
$ nextflow secrets <subcommand> [options]
```

Print the command usage.

List secrets available in the current store.

Retrieve a secret value.

Create or update a secret.

Delete a secret.

Set a secret:

```
$ nextflow secrets set F00 "Hello world"
```

List secrets:

```
$ nextflow secrets list
```

Get a secret:

```
$ nextflow secrets get F00
```

Delete a secret:

```
$ nextflow secrets delete F00
```

Update the nextflow runtime to the latest available version.

```
$ nextflow self-update
```

The `self-update` command directs the `nextflow` CLI to update itself to the latest stable release.

Update Nextflow:

```
$ nextflow self-update

N E X T F L O W
version 20.07.1 build 5412
created 24-07-2020 15:18 UTC (20:48 IDT)
cite doi:10.1038/nbt.3820
http://nextflow.io

Nextflow installation completed. Please note:
- the executable file `nextflow` has been created in the folder: /usr/local/bin
```

View a project's script file(s).

```
$ nextflow view [options] [project]
```

The `view` command is used to inspect the pipelines that are already stored in the global nextflow cache. For downloading a pipeline into the global cache `~/.nextflow/assets`, refer to the `pull` command.

Print the command usage.

List repository content.

Hide header line.

Added in version 26.04

Project revision. Can be a git branch, tag, or commit SHA number.

Viewing the contents of a downloaded pipeline:

```
$ nextflow view nextflow-io/hello

== content of file: .nextflow/assets/.repos/nextflow-io/hello/main.nf
#!/usr/bin/env nextflow

process sayHello {
  input:
    val x
  output:
    stdout
  script:
    """
    echo '$x world!'
    """
}

workflow {
  channel.of('Bonjour', 'Ciao', 'Hello', 'Hola') | sayHello | view
}
```

List the folder structure of the downloaded pipeline:

```
$ nextflow view -l nextflow-io/hello

== content of path: .nextflow/assets/.repos/nextflow-io/hello
.git
.gitignore
LICENSE
README.md
main.nf
nextflow.config
```

View the contents of a downloaded pipeline without omitting the header:

```
$ nextflow view -q nextflow-io/hello

#!/usr/bin/env nextflow

process sayHello {
  input:
    val x
  output:
    stdout
  script:
    """
    echo '$x world!'
    """
}

workflow {
  channel.of('Bonjour', 'Ciao', 'Hello', 'Hola') | sayHello | view
}
```

Updating Nextflow

This page describes Nextflow release cadence, how to self-update Nextflow, and how to select your version of Nextflow.

A stable version of Nextflow is released in the 4th and 10th month of each year. An edge version of Nextflow is released on a monthly basis. The edge version can be used to access the latest updates and experimental features.

Nextflow uses [Calendar Versioning](#). Versions are numbered as `<year>.<month>.<patch>`. For example, `23.10.1` corresponds to the first patch of the October 2023 stable release.

See [Migration notes](#) for a list of migration guides for each stable release. See the [changelog](#) for an exhaustive list of changes across all releases.

To update to the latest stable release of Nextflow, run the `self-update` command:

```
nextflow self-update
```

To use the latest edge release, set `NXF_EDGE=1` when updating Nextflow:

```
NXF_EDGE=1 nextflow self-update
```

note

When updating from an edge release to a stable release, you must explicitly set `NXF_EDGE=0`.

warning

Nextflow will update its executable during the self-update process. The update can fail if the Nextflow executable is in a directory with restricted permissions.

The `NXF_VER` environment variable can be used to define which version of Nextflow to use. To switch to a specific version of Nextflow for a single run, set the `NXF_VER` environment variable in your execution command. For example:

```
NXF_VER=23.10.0 nextflow info
```

To set a specific version of Nextflow for a terminal session, export the `NXF_VER` environment variable. For example:

```
export NXF_VER=23.10.0
```

To set a specific version of Nextflow for your user profile, add the above `NXF_VER` export command to your shell configuration file, such as `~/.bashrc` or `~/.zshrc`, and restart your session.

tip

You can use `NXF_VER` to switch to an edge release. For example:

```
NXF_VER=24.06.0-edge nextflow info
```

warning

Nextflow will update its executable during the self-update process. The update can fail if the Nextflow executable is in a directory with restricted permissions.

Overview

This section provides a high-level overview of the Nextflow source code for users who want to understand or contribute to it. Rather than a comprehensive API documentation, these docs simply provide a conceptual map to help you understand the key concepts of the Nextflow implementation, and to quickly find code sections of interest for further investigation.

Before you dive into code, be sure to check out the [CONTRIBUTING.md](#) for Nextflow to learn about the many ways to contribute to the project.

The suggested development environment is [IntelliJ IDEA](#). Nextflow development with IntelliJ IDEA requires a recent version of the IDE (2019.1.2 or later).

After installing IntelliJ IDEA, use the following steps to use it with Nextflow:

1. Clone the Nextflow repository to a directory in your computer.
2. Open IntelliJ IDEA and go to **File > New > Project from Existing Sources...**
3. Select the Nextflow project root directory in your computer and click **OK**.
4. Select **Import project from external model > Gradle** and click **Finish**.
5. After the import process completes, select **File > Project Structure...**
6. Select **Project**, and make sure that the **SDK** field contains Java 11 (or later).
7. Go to **File > Settings > Editor > Code Style > Groovy > Imports** and apply the following settings:
 - Use single class import
 - Class count to use import with '*': 99
 - Names count to use static import with '*': 99
 - Imports layout:
 - import java.*
 - import javax.*
 - blank line
 - all other imports
 - all other static imports

New files must include the appropriate license header boilerplate and the author name(s) and contact email(s) ([see for example](#)).

Nextflow is written in [Groovy](#), which is itself a programming language based on [Java](#). Groovy is designed to be highly interoperable with Java -- Groovy programs compile to Java bytecode, and nearly any Java program is also a valid Groovy program. However, Groovy adds several language features (e.g. closures, list and map literals, optional typing, optional semicolons, meta-programming) and standard libraries (e.g. JSON and XML parsing) that greatly improve the overall experience of developing for the Java virtual machine.

Recommended resources for Groovy, from most reference-complete to most user-friendly, are listed below:

- [Groovy documentation](#)

- [Groovy in Action](#)
- [Groovy: The Awesome Parts](#)
- [Groovy cheat sheet](#)

Nextflow depends on a variety of libraries and frameworks, the most prominent of which are listed below:

- [AWS SDK for Java 1.x](#): AWS integration
- [Azure SDK for Java](#): Azure integration
- [Google Cloud Client Libraries for Java](#): Google Cloud integration
- [GPar](#)s: dataflow concurrency
- [Gradle](#): build automation
- [JCommander](#): command line interface
- [JGit](#): Git integration
- [Kryo](#): serialization
- [LevelDB](#): key-value store for the cache database
- [Logback](#): application logging
- [PF4J](#): plugin extensions
- [Spock](#): unit testing framework

Any other integrations are likely implemented using a CLI (e.g. Conda, Docker, HPC schedulers) or REST API (e.g. Kubernetes).

Each package has a class diagram, abridged and annotated for relevance and ease of use.

Each node is a class. Fields are selectively documented in order to show only the core data structures and the classes that "own" them. Methods are not explicitly documented, but they are mentioned in certain links where appropriate. Links are selectively documented in order to show only the most important classes and relationships.

Links between classes denote one of the following relationships:

- Inheritance (A <|-- B): B is a subclass of A
- Composition (A --* B): A contains B
- Instantiation (A --> B : f): A creates instance(s) of B at runtime via A::f()

See [packages-page](#) for the list of Nextflow packages.

Class diagrams are manually curated, so they might not always reflect the latest version of the source code.

If you are interested in modifying the source code, you only need Java 11 or later to build Nextflow from source. Nextflow uses the [Gradle](#) build automation system, but you do not need to install Gradle to build Nextflow. In other words, if you can run Nextflow, then you can probably build it too!

To build locally from a branch (useful for testing PRs):

```
git clone -b <branch> git@github.com:nextflow-io/nextflow.git
cd nextflow
make compile
```

The build system will automatically download all of the necessary dependencies on the first run, which may take several minutes.

Once complete, you can run your local build of Nextflow using the `launch.sh` script in place of the `nextflow` command:

```
./launch.sh run <script> ...
```

Alternatively, you can build a self-contained executable with the following command:

```
make pack
```

It will create a binary in the `build/releases` directory which can be used in place of `nextflow`. This approach is useful when testing a pipeline that uses third-party plugins, which is not supported by `launch.sh`.

To run the unit tests:

```
# run all tests
make test

# run individual test
make test module=<nextflow|plugins:nf-amazon|...> class=<package>.<class>.<method>

# refer to the Makefile for all build rules
```

When a test fails, it will give you a report that you can open in your browser to view the reason for each failed test. The **Standard output** tab is particularly useful as it shows the console output of each test.

To run the integration tests:

```
cd tests/checks
./qrun.sh
```

To run a specific integration test:

```
cd tests/checks
./qrun.sh <FOLDER>
```

To test the documentation snippets:

```
cd docs/snippets
./test.sh
```

Refer to the [build.yml](#) configuration to see how to run other end-to-end tests locally.

The `groovysh` command provides a command-line REPL that you can use to play around with Groovy code independently of Nextflow. The `groovyConsole` command provides a graphical REPL similar to `nextflow console`. These commands require a standalone Groovy distribution, which can be installed as described for Java on the [Installation](#) page.

note

If you are using WSL, you must also install an X server for Windows, such as [VcXsrv](#) or [Xming](#), in order to use these commands.

Added in version 23.10

You can perform limited breakpoint debugging on a Nextflow script using IntelliJ IDEA.

1. Set a breakpoint in your Nextflow script by clicking on a line number.
2. Run `nextflow -remote-debug run <script>`
3. Select the **Run / Debug Configurations** dropdown, select **Edit Configurations...**, and create a new configuration of type **Remote JVM Debug**. Set the port that appeared in the terminal when you launched your Nextflow script. Click **OK**.
4. Select the green bug icon to begin the remote debug session. The Debug window will appear and allow you to step through and inspect your script as it runs.

Note that this approach can only be used to debug the script execution, which does not include the pipeline execution.

Getting started with rnaseq-nf

`rnaseq-nf` is a basic Nextflow pipeline for RNA-Seq analysis that performs quality control, transcript quantification, and result aggregation. The pipeline processes paired-end FASTQ files, generates quality control reports with `FastQC`, quantifies transcripts with `Salmon`, and produces a unified report with `MultiQC`.

This tutorial describes the architecture of the `rnaseq-nf` pipeline and provides instructions on how to run it.

The pipeline is organized into modular workflows and processes that coordinate data flow from input files through analysis steps to final outputs.

The `entry workflow` orchestrates the entire pipeline by coordinating input parameters and data flow:

Data flow:

- The `transcriptome` and `reads` parameters are passed to the `RNASEQ` subworkflow, which performs indexing, quality control, and quantification.
- The outputs from `RNASEQ`, along with the `MultiQC` configuration (`multiqc`), are passed to the `MULTIQC` module, which aggregates results into a unified HTML report.
- The `outdir` parameter defines where all results are published.

The `RNASEQ` subworkflow coordinates three processes that run in parallel and sequence:

Inputs (`take:`):

- `read_pairs_ch`: A channel of paired-end read files
- `transcriptome`: A reference transcriptome file

Data flow (`main:`):

- `INDEX` creates a `Salmon` index from the `transcriptome` input (runs once).
- `FASTQC` analyzes the samples in the `read_pairs_ch` channel in parallel (runs independently for each sample).
- `QUANT` quantifies transcripts using the index from `INDEX` and the samples in the `read_pairs_ch` channel (runs for each sample after `INDEX` completes).

Outputs (`emit:`):

- `fastqc`: The results from `FASTQC`
- `quant`: The results from `QUANT`

The `MULTIQC` process aggregates all quality control and quantification outputs into a comprehensive HTML report.

Inputs:

- Input files: All collected outputs from the `RNASEQ` subworkflow (FastQC reports and Salmon quantification files).

- `config`: MultiQC configuration files and branding (logo, styling).

Process execution:

- `MULTIQC` scans all input files, extracts metrics and statistics, and generates a unified report.

Outputs:

- `multiqc_report.html`: A single consolidated HTML report providing an overview of:
 - General stats
 - Salmon fragment length distribution
 - FastQC quality control
 - Software versions

The pipeline behavior can be customized using command-line parameters to specify input data, output locations, and configuration files.

The pipeline accepts the following command-line parameters:

- `--reads`: Path to paired-end FASTQ files (default: `data/ggal/ggal_gut_{1,2}.fq`).
- `--transcriptome`: Path to reference transcriptome FASTA (default: `data/ggal/ggal_1_48850000_49020000.Ggal71.500bpflank.fa`).
- `--outdir`: Output directory for results (default: `results`).
- `--multiqc`: Path to MultiQC configuration directory (default: `multiqc`).

Configuration profiles allow you to customize how and where the pipeline runs by specifying the `-profile` flag. Multiple profiles can be specified as a comma-separated list. Profiles are defined in the `nextflow.config` file in the base directory.

Software profiles

Software profiles specify how software dependencies for processes should be provisioned:

- `conda`: Provision a Conda environment for each process based on its required Conda packages
- `docker`: Use a Docker container which contains all required dependencies
- `singularity`: Use a Singularity container which contains all required dependencies
- `wave`: Provision a Wave container for each process based on its required Conda packages

note

The respective container runtime or package manager must be installed to use these profiles.

Execution profiles

Execution profiles specify the compute and storage environment used by the pipeline:

- `slurm`: Run on a SLURM HPC cluster
- `batch`: Run on AWS Batch
- `google-batch`: Run on Google Cloud Batch
- `azure-batch`: Run on Azure Batch

note

Depending on your environment, you may need to configure underlying infrastructure such as resource pools, storage, and credentials.

The pipeline includes test data in the `data/ggal/` directory for demonstration and validation purposes:

- Paired-end FASTQ files from four tissue samples (gut, liver, lung, spleen):
 - `ggal_gut_{1,2}.fq`
 - `ggal_liver_{1,2}.fq`
 - `ggal_lung_{1,2}.fq`
 - `ggal_spleen_{1,2}.fq`
- Reference transcriptome:
 - `ggal_1_48850000_49020000.Ggal71.500bpflank.fa`

By default, only the `gut` sample is processed. You can use the `all-reads` profile to process all four tissue samples.

The `rnaseq-nf` pipeline is executable out-of-the-box. This section provides examples for running the pipeline with different configurations.

Run the pipeline with default parameters using Docker:

```
nextflow run nextflow-io/rnaseq-nf -profile docker
```

Override default parameters to use custom input files and output locations:

```
nextflow run nextflow-io/rnaseq-nf \
  --reads '/path/to/reads/*_{1,2}.fastq.gz' \
  --transcriptome '/path/to/transcriptome.fa' \
  --outdir 'my_results' \
  -profile docker
```

Specify configuration profiles to customize runtime environments and data sources:

```
# Use Conda to provision software dependencies
nextflow run nextflow-io/rnaseq-nf -profile conda

# Run on a SLURM cluster
nextflow run nextflow-io/rnaseq-nf -profile slurm

# Combine multiple profiles: process all reads using Docker
nextflow run nextflow-io/rnaseq-nf -profile all-reads,docker
```

tip

See [Configuration profiles](#) for more information about profiles.

The `rnaseq-nf` pipeline produces the following outputs in the `results` directory:

```
results/
├── fastqc_<SAMPLE_ID>_logs/      # FastQC quality reports per sample
│   ├── <SAMPLE_ID>_1_fastqc.html
│   ├── <SAMPLE_ID>_1_fastqc.zip
│   ├── <SAMPLE_ID>_2_fastqc.html
│   └── <SAMPLE_ID>_2_fastqc.zip
└── multiqc_report.html          # Aggregated QC and Salmon report
```

The MultiQC report (`multiqc_report.html`) can be viewed in a web browser.

AWS Java SDK v2

AWS Java SDK v1 will reach end of life at the end of 2025. Starting in Nextflow 25.10, Nextflow uses AWS Java SDK v2 in the `nf-amazon` plugin.

This migration introduces several breaking changes to the `aws.client` config scope, including new and removed options. This page describes these changes and how they affect your Nextflow configuration.

The HTTP client in SDK v2 does not support overriding certain advanced HTTP options. As a result, the following config options are no longer supported:

- `aws.client.protocol`
- `aws.client.signerOverride`
- `aws.client.socketRecvBufferSizeHint`
- `aws.client.socketSendBufferSizeHint`
- `aws.client.userAgent`

Nextflow manages S3 transfers, including uploads, downloads, and S3-to-S3 copies, separately from other S3 API calls such as listing a directory or retrieving object metadata.

Use the `aws.client.targetThroughputInGbps` option to control the concurrency of S3 transfers based on the available network bandwidth. This setting defaults to `10`, which allows Nextflow to perform concurrent S3 transfers up to 10 Gbps of network throughput.

Use the `aws.client.maxConnections` config option to control the maximum number of concurrent HTTP connections for all other S3 API calls.

Nextflow transfers large files to and from S3 as multipart transfers. Use the `aws.client.minimumPartSize` and `aws.client.multipartThreshold` configuration options to control when and how multipart transfers are performed.

Concurrent multipart downloads can consume a large amount of heap memory due to the buffer allocated by each transfer. To avoid out-of-memory errors, the size consumed by these buffers is limited to 400 MB by default. Use the `aws.client.maxDownloadHeapMemory` option to control this value.

The following multipart upload config options are no longer supported:

- `aws.client.uploadChunkSize`
- `aws.client.uploadMaxAttempts`
- `aws.client.uploadMaxThreads`
- `aws.client.uploadRetrySleep`

Amazon S3

Nextflow includes support for AWS S3 storage. Files stored in an S3 bucket can be accessed transparently in your pipeline script like any other file in the local file system.

In order to access an S3 file, you only need to prefix the file path with the `s3` schema and the `bucket` name where it is stored.

For example, if you need to access the file `/data/sequences.fa` stored in a bucket named `my-bucket`, that file can be accessed using the following fully qualified path:

```
s3://my-bucket/data/sequences.fa
```

The usual file operations can be applied to a path handle with the above notation. For example, the content of an S3 file can be printed as follows:

```
println file('s3://my-bucket/data/sequences.fa').text
```

See [Working with files](#) and the [Path](#) reference to learn more about available file operations.

AWS access credentials can be provided in two ways:

1. Using AWS access and secret keys in your pipeline configuration.
2. Using IAM roles to grant access to S3 storage on AWS EC2 instances.

The AWS access and secret keys can be specified by using the `aws` section in the `nextflow.config` configuration file as shown below:

```
aws {
  accessKey = '<aws-access-key>'
  secretKey = '<aws-secret-key>'
  region = '<aws-region-identifier>'
}
```

If the access credentials are not found in the above file, Nextflow looks for AWS credentials in the following order:

1. The `nextflow.config` file in the pipeline execution directory
2. The environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`
3. The environment variables `AWS_ACCESS_KEY` and `AWS_SECRET_KEY`
4. The profile in the AWS credentials file located at `~/.aws/credentials`
 - Uses the `default` profile or the environment variable `AWS_PROFILE` if set
5. The profile in the AWS client configuration file located at `~/.aws/config`
 - Uses the `default` profile or the environment variable `AWS_PROFILE` if set
6. The temporary AWS credentials provided by an IAM instance role. See [IAM Roles](#) documentation for details.

More information regarding [AWS Security Credentials](#) are available in the AWS documentation.

When running your pipeline in an EC2 instance, IAM roles can be used to grant access to AWS resources.

In this scenario, you only need to launch the EC2 instance with an IAM role which includes the `AmazonS3FullAccess` policy. Nextflow will detect and automatically acquire the permission to access S3 storage, without any further configuration.

Learn more about [Using IAM Roles to Delegate Permissions to Applications that Run on AWS EC2](#) in the AWS documentation.

To use an AWS China region, make sure to specify the corresponding AWS API S3 endpoint in the Nextflow configuration file as shown below:

```
aws {
  client {
    endpoint = "https://s3.cn-north-1.amazonaws.com.cn"
  }
}
```

Read more about AWS API endpoints in the [AWS documentation](#)

To use S3-compatible object storage such as [Ceph](#) or [Minio](#) specify the endpoint of your storage provider and enable the [S3 path style access](#) in your Nextflow configuration as shown below:

```
aws {
  accessKey = '<aws-access-key>'
  secretKey = '<aws-secret-key>'
  client {
    endpoint = '<storage-endpoint-url>'
    s3PathStyleAccess = true
  }
}
```

See [AWS configuration](#) for more information about advanced S3 client configuration options.

Azure

Nextflow provides built-in support for Azure Cloud Services. It enables you to store and access data using Azure Blob Storage and Azure file shares, and to execute workflows using Azure Batch.

tip

For automated Azure infrastructure setup, consider using [Batch Forge](#) in [Seqera Platform](#).

To run pipelines with Azure Batch:

1. Create an Azure Batch account in the Azure portal.
2. Increase the quotas in your Azure Batch account to the pipeline's needs. Quotas impact the number of pools, CPUs, and jobs you can create.
3. Create a storage account and Azure Blob Storage in the same region as the Batch account.
4. Add the following settings to your Nextflow configuration file:
 - Add authentication and account details. See [Authentication](#) for configuration examples.
 - Set `process.executor` to `azurebatch` to make Nextflow submit tasks to Azure Batch.
 - Set `workDir` to a working directory on Azure Blob Storage. For example, `az://<blob-container>/work`, where `blob-container` is a blob container in your storage account.
5. Launch your pipeline with the above configuration:

```
nextflow run <pipeline-name>
```

tip

You can list Azure regions with:

```
az account list-locations -o table
```

Nextflow supports three authentication methods for Azure services, listed in order of security and recommended usage:

- [Managed identities](#): The most secure option, available only within Azure. Uses Azure-managed credentials without storing secrets.
- [Service principals](#): A secure and flexible method that works across environments. Uses Microsoft Entra credentials for authentication.
- [Access keys](#): The most basic and least secure method. Relies on direct access keys for authentication.

The following role assignments are required to use Azure services:

For Azure Storage:

- Storage Blob Data Reader
- Storage Blob Data Contributor

For Azure Batch:

- Batch Data Contributor

To assign roles to a managed identity or service principal, see the [Azure documentation](#).

Added in version 24.10

Managed identity is the recommended authentication method when running Nextflow within Azure. It automatically manages credentials without requiring you to store secrets.

When using a managed identity, an Azure resource can authenticate based on what it is rather than using access keys. For example, Nextflow can submit jobs to Azure Batch and access private storage accounts without additional credentials if it is running on Azure Virtual Machines with a managed identity and the necessary permissions.

There are two types of managed identities:

- System-assigned managed identity
- User-assigned managed identity

System-assigned managed identity

A system-assigned identity is tied to a specific Azure resource. To use it:

1. Enable [system-assigned managed identity](#) on your Azure resource.
2. Configure the required role assignments. See [Required roles](#) for more information.
3. Add the following configuration:

```
azure {
  managedIdentity {
    system = true
  }
  storage {
    accountName = '<storage-account-name>'
  }
  batch {
    accountName = '<batch-account-name>'
    location = '<batch-account-location>'
  }
}
```

Replace the following:

- `storage-account-name`: your Azure storage account name
- `batch-account-name`: your Azure Batch account name
- `batch-account-location`: your Azure Batch account location

tip

Nextflow uses the `AZURE_MANAGED_IDENTITY_SYSTEM` environment variable if the managed identity is not set in the Nextflow configuration file. Set this to `true` to enable a system-assigned managed identity.

User-assigned managed identity

A user-assigned identity can be shared across multiple Azure resources and its lifecycle is not tied to any specific resource. See the [Azure documentation](#) for more information.

To use user-assigned managed identity:

1. Create a [managed identity](#).
2. Configure the required role assignments.
3. [Assign the identity](#) to your Azure resource.
4. Add the following configuration:

```
azure {
  managedIdentity {
    clientId = '<user-assigned-managed-identity-client-id>'
  }
  storage {
    accountName = '<storage-account-name>'
  }
  batch {
    accountName = '<batch-account-name>'
    location = '<batch-account-location>'
  }
}
```

Replace the following:

- `user-assigned-managed-identity-client-id`: your user assigned managed identity client ID
- `storage-account-name`: your Azure Storage account name
- `batch-account-name`: your Azure Batch account name
- `batch-account-location`: your Azure Batch account location

tip

Nextflow uses the `AZURE_MANAGED_IDENTITY_USER` environment variable if the managed identity client ID is not provided in the Nextflow configuration file.

[Service principal](#) credentials can be used to access Azure Batch and Storage accounts.

Similar to a managed identity, a service principal is an application identity that can have specific permissions and role-based access. Unlike managed identities, service principals require a secret key for authentication. The secret key is less secure, but it allows you to authenticate with Azure resources when operating outside of the Azure environment, such as from your local machine or a non-Azure cloud.

To create a service principal, follow the steps in the [Register a Microsoft Entra app and create a service principal](#) guide and add the credentials to your Nextflow configuration file. For example:

```

azure {
  activeDirectory {
    servicePrincipalId = '<service-principal-client-id>'
    servicePrincipalSecret = '<service-principal-client-secret>'
    tenantId = '<tenant-id>'
  }
  storage {
    accountName = '<storage-account-name>'
  }
  batch {
    accountName = '<batch-account-name>'
    location = '<batch-account-location>'
  }
}

```

tip

Nextflow uses the following environment variables if storage settings are not set in the Nextflow config file:

- `AZURE_CLIENT_ID`: your Azure service principal application ID
- `AZURE_CLIENT_SECRET`: your Azure service principal secret
- `AZURE_TENANT_ID`: your Azure Entra account tenant ID

The basic authentication method uses direct access keys. While easy to set up, it's less secure than managed identities or service principals because it provides full access to your resources. This approach requires you to obtain and manage the access keys for your Azure Batch and Storage accounts.

You can find your access keys in the Azure portal:

- For Storage accounts, navigate to your Storage account and select **Access keys** in the left-hand navigation menu.
- For Batch accounts, navigate to your Batch account and select **Keys** in the left-hand navigation menu.

Keep access keys secure and rotate them regularly. Use environment variables instead of hardcoding keys in configuration files, especially in shared or version-controlled environments:

```

azure {
  storage {
    accountName = '<storage-account-name>'
    accountKey = '<storage-account-key>'
  }
  batch {
    accountName = '<batch-account-name>'
    accountKey = '<batch-account-key>'
    location = '<batch-account-location>'
  }
}

```

You can also use a Shared Access Token (SAS) instead of an account key to provide time-limited access to your storage account:

```

azure.storage.sasToken = '<sas-token>'

```

tip

When creating a SAS token, make sure to enable `Read`, `Write`, `Delete`, `List`, `Add`, `Create` permissions for the `Container` and `Object` resource types. The value of `sasToken` should be stripped of the leading `?` character.

tip

Nextflow uses the following environment variables if storage settings are not provided in the Nextflow configuration file:

- `AZURE_STORAGE_ACCOUNT_NAME`: your Azure Storage account name
- `AZURE_STORAGE_ACCOUNT_KEY`: your Azure Storage account key
- `AZURE_BATCH_ACCOUNT_NAME`: your Azure Batch account name
- `AZURE_BATCH_ACCOUNT_KEY`: your Azure Batch account key
- `AZURE_STORAGE_SAS_TOKEN`: your Azure Storage account SAS token

Azure Storage provides several services for storing and accessing data in Azure. Nextflow primarily supports Azure Blob Storage for most workloads. Limited support is available for Azure file shares for specific use cases.

Nextflow uses the `az://` URI scheme to enable transparent access to files stored in Azure Blob Storage. Once Azure Storage is configured, you can use this scheme to reference files in any blob container as if they were local. For example, the path `/path/to/file.txt` in a container named `my-container` can be accessed as follows:

```
file('az://<container-path>/file.txt')
```

The Blob storage account name and key need to be provided in the Nextflow configuration file as shown below:

```
azure {
  storage {
    accountName = '<blob-account-name>'
  }
}
```

tip

Nextflow uses the following environment variables if storage settings are not provided in the Nextflow configuration file:

- `AZURE_STORAGE_ACCOUNT_NAME`: your Azure Storage account name
- `AZURE_STORAGE_ACCOUNT_KEY`: your Azure Storage account access key
- `AZURE_STORAGE_SAS_TOKEN`: a shared access signature (SAS) token for Azure Storage access

Azure Batch is a managed computing service that enables large-scale parallel and high-performance computing (HPC) batch jobs in Azure. Nextflow provides a built-in executor for Azure Batch, allowing you to offload workflow tasks to the cloud.

Nextflow integrates seamlessly with Azure Batch to:

- Dynamically create and manage compute pools based on workflow requirements.
- Automatically scale nodes up and down as needed during workflow execution.
- Support both regular and low-priority VMs for cost optimization.
- Manage task dependencies and data transfers between Azure Storage and compute nodes.

This section describes how to configure and use Azure Batch with Nextflow for efficient cloud-based workflow execution. For comprehensive information about Azure Batch features and capabilities, refer to the [official Azure Batch documentation](#).

Nextflow integrates with Azure Batch by mapping its execution model to Azure Batch's structure. A Nextflow process corresponds to an Azure Batch job, and every execution of that process (a Nextflow task) becomes an Azure Batch task. These Azure Batch tasks are executed on compute nodes within an Azure Batch pool, which is a collection of virtual machines that can scale up or down based on an autoscale formula.

Nextflow manages these pools dynamically. You can assign processes to specific, pre-existing pools using the `process queue` directive. Nextflow will create it if it doesn't exist and `azure.batch.allowPoolCreation` is set to `true`. Alternatively, `autoPoolMode` enables Nextflow to automatically create multiple pools based on the CPU and memory requirements defined in your processes.

An Azure Batch task is created for each Nextflow task. This task first downloads the necessary input files from Azure Blob Storage to its assigned compute node. It then runs the process script. Finally, it uploads any output files back to Azure Blob Storage.

Nextflow ensures that the corresponding Azure Batch jobs are marked as terminated when the pipeline completes. The compute pools, if configured for auto-scaling, automatically scale down according to their defined formula to minimize costs. Nextflow can also be configured to delete these node pools after pipeline completion.

To use Azure Batch, set the `executor` directive to `azurebatch` in your Nextflow configuration file and add a work directory on Azure Blob Storage:

```
process {
  executor = 'azurebatch'
}

workDir = 'az://<blob-container>/work'

azure {
  storage {
    accountName = '<storage-account-name>'
  }
  batch {
    location = '<batch-account-location>'
    accountName = '<batch-account-name>'
    autoPoolMode = true
    allowPoolCreation = true
  }
}
```

note

The work directory must be a subdirectory, for example, `az://container/work`, not `az://container`.

Finally, launch your pipeline with the above configuration:

```
nextflow run <pipeline-name>
```

Azure Batch enforces quotas on resources like pools, jobs, and VMs.

- **Pools:** There is a limit on the number of pools that can exist at one time. Nextflow throws an error if it attempts to create a pool when this quota is exhausted. To proceed, you must delete existing pools.
- **Active jobs:** There is a limit on the number of active jobs. Completed jobs should be terminated. If this limit is reached, Nextflow throws an error. You must terminate or delete jobs before you run additional pipelines.
- **VM cores:** Each Azure VM family has a maximum core quota. If you request a VM family size for which you lack sufficient core quota, any jobs and tasks created by Nextflow will remain pending, as the pool cannot scale due to the quota. This can cause your pipeline to hang indefinitely.

You can increase your quota on the Azure Portal by opening your Batch account and selecting **Quotas** in the left-hand menu, then request an increase in quota numbers.

Nextflow supports two approaches for managing Batch pools: auto pools and named pools. These approaches are described in the following sections.

warning

Clean up the Batch pools or use auto-scaling to avoid additional charges in the Batch account.

When using the `autoPoolMode` mode, Nextflow automatically creates pools of compute nodes appropriate for your pipeline.

The `cpus` and `memory` directives are used to find the smallest machine type in the Azure machine family, specified by the `machineType` directive, that satisfies the requested resources. If memory is not specified, 1 GB of memory is requested per CPU. When no options are specified, it uses one compute node of type `Standard_D4_v3`.

To use `autoPoolMode`, enable it in the `azure.batch` config scope:

```
azure {
  batch {
    autoPoolMode = true
    allowPoolCreation = true
  }
}

process {
  machineType = 'Standard_D4_v3'
  cpus = 4
  memory = 16.GB
}
```

You can specify multiple machine families per process using glob patterns in the `machineType` directive:

```
// D or E v5 machines with data disk
process.machineType = "Standard_D*d_v5,Standard_E*d_v5"
```

For example, the following process creates a pool of `Standard_E8d_v5` machines when using `autoPoolMode`:

```
process EXAMPLE_PROCESS {
  machineType "Standard_E*d_v5"
  cpus 8
  memory 8.GB

  script:
  """
  echo "cpus: ${task.cpus}"
  """
}
```

Additional notes:

- For tasks using fewer than 4 CPUs, Nextflow creates pools with 4x CPUs to enable task packing.
- For tasks using fewer than 8 CPUs, Nextflow uses 2x CPUs.
- This behavior can be overridden by specifying an exact machine type (e.g., `Standard_E2d_v5`).
- Regexes can be used to avoid certain machine types (e.g., `Standard_*[^p]_v*` avoids ARM-based machines).
- The pool is not removed when the pipeline terminates unless `azure.batch.deletePoolsOnCompletion` is enabled in your Nextflow configuration file.

The `queue` directive can be used to specify the ID of the Azure Batch compute pool that should be used for a process.

When `azure.batch.allowPoolCreation = true` is set in your configuration, Nextflow creates the pools on-demand. Otherwise, the pool must be available in the Batch environment when Nextflow submits tasks.

Each pool can be configured separately in the Nextflow configuration. For example:

```
azure {
  batch {
    pools {
      small {
        vmType = 'Standard_D2_v2'
        vmCount = 5
      }
      large {
        vmType = 'Standard_D8_v3'
        vmCount = 2
      }
    }
  }
}

process {
  withLabel: small_jobs {
    queue = 'small'
  }
  withLabel: large_jobs {
    queue = 'large'
  }
}
```

The above example defines the configuration for two node pools, `small` and `large`. The `small` pool provisions 10 compute nodes of type `Standard_D2_v2`, and the `large` pool provisions 5 nodes of type `Standard_E2_v3`. See the [azure](#) configuration scope for the list of available configuration options.

warning

Pool names may contain alphanumeric characters, hyphens, and underscores. Hyphenated names must be quoted, e.g., 'pool-1'.

Requirements for pre-existing named pools

The target pool must satisfy the following requirements when Nextflow is configured to use a pool that is already available in the Batch account:

- The pool must be declared as `dockerCompatible` (Container Type property).
- The task slots per node must match the number of cores for the selected VM. Otherwise, Nextflow returns an error of the form "Azure Batch pool 'ID' slots per node does not match the VM num cores (slots: N, cores: Y)".
- Unless you are using Fusion, all tasks must have the `azcopy` command line tool available in the path. If `azure.batch.copyToolInstallMode = 'node'`, every node must have `azcopy` available at `$AZ_BATCH_NODE_SHARED_DIR/bin/`.

Azure Batch can automatically scale pools based on the parameters you define. With automatic scaling, Batch dynamically adds nodes to a pool as task demands increase and removes nodes as task demands decrease.

To enable this feature for pools created by Nextflow, add `autoScale = true` to the corresponding pool configuration scope. For example, when using the `autoPoolMode`:

```
azure {
  batch {
    pools {
      <pool-name> {
        autoScale = true
        vmCount = 5 // Initial count
        maxVmCount = 50 // Maximum count
      }
    }
  }
}
```

The default auto-scaling formula initializes the pool with the number of VMs specified by the `vmCount` option. It dynamically scales up the pool based on the number of pending tasks. The pool limit is defined by `maxVmCount`. The formula automatically scales the pool down to zero nodes to minimize costs when no jobs are submitted for execution. The default formula is shown below:

```
// Get pool lifetime since creation.
lifespan = time() - time("${poolCreationTime}");
interval = TimeInterval_Minute * {{scaleInterval}};

// Compute the target nodes based on pending tasks.
$samples = $PendingTasks.GetSamplePercent(interval);
$tasks = $samples < 70 ? max(0, $PendingTasks.GetSample(1)) : max( $PendingTasks.GetSample(1),
avg($PendingTasks.GetSample(interval)));
$targetVMs = $tasks > 0 ? $tasks : max(0, $TargetDedicatedNodes/2);
targetPoolSize = max(0, min($targetVMs, {{maxVmCount}}));

// For first interval deploy 1 node, for other intervals scale up/down as per tasks.
$TargetDedicatedNodes = lifespan < interval ? {{vmCount}} : targetPoolSize;
$NodeDeallocationOption = taskcompletion;
```

The `azure.batch.pools.<pool-name>.scaleFormula` setting can be used to specify custom formulas.

By default, Nextflow creates SAS tokens for specific containers and passes them to tasks to enable file operations with Azure Storage. SAS tokens expire after a set period of time. The expiration time is 48 hours by default and can be configured using `azure.storage.tokenDuration` in your configuration.

Added in version 25.10

warning

Only available when using [Fusion filesystem](#).

You can also authenticate to Azure Storage using a managed identity when using Fusion.

To do this:

1. Create a user-assigned managed identity with the Azure Storage Blob Data Contributor role for your storage account.
2. Attach the user-assigned managed identity to the node pool manually.
3. Set `azure.batch.poolIdentityClientId` to this identity in your configuration.

```
azure {
  batch {
    poolIdentityClientId = '<managed-identity-client-id>'
  }
}
```

Each task authenticates as the managed identity, and downloads and uploads files to Azure Storage using these credentials. It is possible to attach more than one managed identity to a pool. Fusion uses the identity specified by `azure.batch.poolIdentityClientId`.

Each Azure Batch node is allocated a specific number of task slots that determine how many tasks can run concurrently on that node. The number of slots is calculated based on the virtual machine's available CPUs. Nextflow intelligently assigns task slots to each process based on the proportion of total node resources requested through the `cpus`, `memory`, and `disk` directives.

For example, consider a `Standard_D4d_v5` machine with 4 vCPUs, 16 GB of memory, and 150 GB local disk:

- If a process requests `cpus 2`, `memory 8.GB`, or `disk 75.GB`, two task slots are allocated (50% of resources), allowing two tasks to run concurrently on the node.
- If a process requests `cpus 4`, `memory 16.GB`, or `disk 150.GB`, four task slots are allocated (100% of resources), allowing one task to run on the node.

Resource overprovisioning can occur if tasks consume more than their allocated share of resources. For instance, the node described above may become overloaded and fail if a task with `cpus 2` uses more than 8 GB of memory or 75 GB of disk space. Make sure to accurately specify resource requirements to ensure optimal performance and prevent task failures.

warning

Azure virtual machines come with fixed storage disks that are not expandable. Tasks will fail if the tasks running concurrently on a node use more storage than the machine has available.

By default, Docker and containerd store image layers on the OS disk, which can fill up quickly when pulling large container images. On VMs with an attached data disk (e.g., `Standard_E16ds_v5`), you can use a pool [start task](#) to relocate Docker and containerd storage to `$AZ_BATCH_NODE_ROOT_DIR`, which uses the larger data disk.

Every process in your pipeline must specify a container in order to be executed on Azure Batch.

Nextflow supports both public and private container images. You can pull public images from repositories such as <https://community.wave.sequera.io>. You must authenticate for any private images that you use. Ensure the Batch pool has access when using a private registry like Azure Container Registry (ACR). Provide ACR credentials in the `azure.registry` configuration scope:

```
azure {
  registry {
    server = '<registry-server>' // e.g., 'myregistry.azurecr.io'
    userName = '<registry-user>'
    password = '<registry-password>'
  }
}
```

tip

Nextflow uses the following environment variables if the registry credentials are not provided in the Nextflow configuration file:

- `AZURE_REGISTRY_USER_NAME`: the username for Azure Container Registry authentication
- `AZURE_REGISTRY_PASSWORD`: the password for Azure Container Registry authentication

Container images from public registries such as Docker Hub can be used without additional configuration, even when a private registry is specified in the `azure.registry` scope. The Docker runtime on Azure Batch VMs automatically uses the appropriate registry based on the container image specified for the task.

When Nextflow creates a pool of compute nodes, it selects:

- The virtual machine image reference (defines the operating system (OS) and software to be installed on the node).
- The Batch node agent SKU (a program that runs on each node and provides the interface between the node and the Azure Batch service).

These settings determine the operating system, version, and available features for each compute node in your pool.

By default, Nextflow creates pool nodes suitable for most bioinformatics workloads based on Ubuntu 24.04. You can customize the OS in your pool configuration to meet specific requirements.

Below are example configurations for common operating systems used in scientific computing:

Ubuntu 24.04 (default)

```

azure {
  batch {
    pools {
      <pool-name> {
        sku = "batch.node.ubuntu 24.04"
        offer = "ubuntu-hpc"
        publisher = "microsoft-dsvm"
      }
    }
  }
}

```

CentOS 8

```

azure {
  batch {
    pools {
      <pool-name> {
        sku = "batch.node.centos 8"
        offer = "centos-container"
        publisher = "microsoft-azure-batch"
      }
    }
  }
}

```

Virtual networks

Pools can be configured to use virtual networks to connect to your existing network infrastructure.

```

azure.batch.pools.<pool-name>.virtualNetwork = '<subnet-id>'

```

The subnet ID must be in the following format:

```

/subscriptions/<subscription>/resourceGroups/<group>/providers/Microsoft.Network/virtualNetworks/<vnet>/subnets/<subnet>

```

Nextflow can submit tasks to existing pools that are already attached to a virtual network without requiring additional configuration.

warning

Virtual networks require Microsoft Entra authentication (service principal or managed identity).

Start tasks

Start tasks are optional commands that run when a node joins the pool. They are useful for setting up the node environment or for running other commands that need to be executed when the node is created.

```

azure {
  batch {
    pools {
      <pool-name> {
        startTask {
          script = 'echo "Hello, world!"'
          privileged = true // optional, defaults to false
        }
      }
    }
  }
}

```

Azure file shares

Azure file shares provide fully managed file shares that can be mounted to compute nodes. Files become immediately available in the file system and can be accessed as local files within processes.

The Azure file share must exist in the storage account configured for Blob Storage. You must provide the name of the source Azure file share and mount path (the path where the files are mounted). Additional mount options can also be set for further customization of the mounting process. See the [Azure Files documentation](#) for more information.

For example:

```

azure {
  storage {
    fileShares {
      references { // any name of your choice
        mountPath = '/mnt/references'
        mountOptions = '-o vers=3.0,dir_mode=0777,file_mode=0777,sec=ntlmssp' // optional
      }
    }
  }
}

```

warning

File shares must be authenticated using a storage account key. Managed identity and service principal are not supported.

Hybrid workloads

It is possible to use multiple executors in the same Nextflow pipeline. This feature enables the deployment of hybrid workloads in which some tasks are executed locally (or on an HPC cluster) and some tasks are offloaded to Azure Batch.

Use process selectors in your Nextflow configuration to execute specific processes with Azure Batch. For example:

```

process {
  withLabel: bigTask {
    executor = 'azurebatch'
    queue = 'my-batch-pool'
    container = 'my/image:tag'
  }
}

bucketDir = 'az://my-container/some/path'

azure {
  storage {
    accountName = '<storage-account-name>'
    accountKey = '<storage-account-key>'
  }
  batch {
    location = '<batch-account-location>'
    accountName = '<batch-account-name>'
    accountKey = '<batch-account-key>'
  }
}

```

With the above configuration, processes labeled as `bigTask` run on Azure Batch and use `bucketDir` as the work directory, while all other processes run locally and use the local work directory specified by `workDir`. Nextflow handles file transfers between the local and remote work directories.

Launch the pipeline with the above configuration:

```
nextflow run <pipeline-name>
```

note

When using Fusion, `bucketDir` is not needed as all tasks will use the remote work directory. Specify `workDir` instead.

See the [azure](#) configuration scope for the list of available configuration options.

Google Cloud

Nextflow integrates with Google Cloud to enable scalable pipeline execution using Google Cloud Batch and data storage using Google Cloud Storage. This page describes how to configure credentials and set up your environment for running Nextflow on Google Cloud.

Credentials for submitting requests to the Google Cloud Batch API are picked up from your environment using [Application Default Credentials](#). Application Default Credentials are designed to use the credentials most natural to the environment in which a tool runs.

The most common case will be to pick up your end-user Google credentials from your workstation. You can create these by running the following command and running through the authentication flow:

```
gcloud auth application-default login
```

This will write a credential file to your gcloud configuration directory that will be used for any tool you run on your workstation that picks up default credentials.

The next most common case would be when running on a Compute Engine VM. In this case, Application Default Credentials will pick up the Compute Engine Service Account credentials for that VM.

See the [Application Default Credentials](#) documentation for how to enable other use cases.

Finally, the `GOOGLE_APPLICATION_CREDENTIALS` environment variable can be used to specify location of the Google credentials file.

If you don't have it, the credentials file can be downloaded from the Google Cloud Console following these steps:

- Open the [Google Cloud Console](#)
- Go to APIs & Services → Credentials
- Click on the Create credentials (blue) drop-down and choose Service account key, in the following page
- Select an existing Service account or create a new one if needed
- Select JSON as Key type
- Click the Create button and download the JSON file giving a name of your choice e.g. `creds.json`.

Then, define the following variable replacing the path in the example with the one of your credentials file just downloaded:

```
export GOOGLE_APPLICATION_CREDENTIALS="/path/your/file/creds.json"
```

See [Get started with Nextflow on Google Cloud Batch](#) for more information on how to use Google Cloud Batch, including how to set the required roles for your service account.

[Google Cloud Batch](#) is a managed computing service that allows the execution of containerized workloads in the Google Cloud Platform infrastructure.

Nextflow provides built-in support for Google Cloud Batch, allowing the seamless deployment of Nextflow pipelines in the cloud, in which tasks are offloaded to the Cloud Batch service.

Read the [Google Cloud Batch executor](#) section to learn more about the `google-batch` executor in Nextflow.

Make sure to have defined in your environment the `GOOGLE_APPLICATION_CREDENTIALS` variable. See the [Credentials](#) section for details.

note

Make sure your Google account is allowed to access the Google Cloud Batch service by checking the [APIs & Services](#) dashboard.

Create or edit the file `nextflow.config` in your project root directory. The config must specify the following parameters:

- Google Cloud Batch as Nextflow executor
- The Docker container image(s) for pipeline tasks
- The Google Cloud project ID and location

Example:

```
process {
  executor = 'google-batch'
  container = 'your/container:latest'
}

google {
  project = 'your-project-id'
  location = 'us-central1'
}
```

Notes:

- A container image must be specified to execute processes. You can use a different Docker image for each process using one or more [process selectors](#).
- Make sure to specify the project ID, not the project name.
- Make sure to specify a location where Google Batch is available. Refer to the [Google Batch documentation](#) for region availability.

Read the [Google configuration](#) section to learn more about advanced configuration options.

By default, the `cpus` and `memory` directives are used to find the cheapest machine type that is available at the current location and that fits the requested resources. If `memory` is not specified, 1 GB of memory is allocated per CPU.

The `machineType` directive can be used to request a specific VM instance type. It can be any predefined Google Compute Platform [machine type](#) or [custom machine type](#).

```

process my_task {
  cpus 8
  memory '40 GB'

  script:
  """
  your_command --here
  """
}

process other_task {
  machineType 'n1-highmem-8'

  script:
  """
  your_command --here
  """
}

```

Added in version 23.04

The `machineType` directive can also be a comma-separated list of patterns. The pattern can contain a `*` to match any number of characters and `?` to match any single character. Examples of valid patterns: `c2-*`, `m?-standard*`, `n*`.

```

process my_task {
  cpus 8
  memory '20 GB'
  machineType 'n2-*,c2-*,m3-*'

  script:
  """
  your_command --here
  """
}

```

Added in version 24.04

The `machineType` directive can also be an [Instance Template](#), specified as `template://<instance-template>`. For example:

```

process my_task {
  cpus 8
  memory '20 GB'
  machineType 'template://my-template'

  script:
  """
  your_command --here
  """
}

```

note

Using an instance template will overwrite the `accelerator` and `disk` directives, as well as the following Google Batch config options: `bootDiskImage`, `cpuPlatform`, `preemptible`, and `spot`.

To use an instance template with GPUs, you must also set the `google.batch.installGpuDrivers` config option to `true`.

To use an instance template with Fusion, the instance template must include a `local-ssd` disk named `fusion` with 375 GB. See the [Google Batch documentation](#) for more details about local SSDs.

Added in version 23.10

The `disk` directive can be used to set the boot disk size or provision a disk for scratch storage. If the disk type is specified with the `type` option, a new disk will be mounted to the task VM at `/tmp` with the requested size and type. Otherwise, it will set the boot disk size, overriding the `google.batch.bootDiskSize` config option. See the [Google Batch documentation](#) for more information about the available disk types.

Examples:

```
// set the boot disk size
disk 100.GB

// mount a persistent disk at '/tmp'
disk 100.GB, type: 'pd-standard'

// mount a local SSD disk at '/tmp' (should be a multiple of 375 GB)
disk 375.GB, type: 'local-ssd'
```

The pipeline can be launched either on a local computer or a cloud instance. Pipeline input data can be stored either locally or in a Google Storage bucket.

The pipeline execution must specify a Google Storage bucket where the workflow's intermediate results are stored using the `-work-dir` command line options. For example:

```
nextflow run <script or project name> -work-dir gs://my-bucket/some/path
```

tip

Any input data **not** stored in a Google Storage bucket will automatically be transferred to the pipeline work bucket. Use this feature with caution being careful to avoid unnecessary data transfers.

warning

The Google Storage path needs to contain at least sub-directory. Do not use only the bucket name e.g. `gs://my-bucket`.

Spot Instances are supported by adding the following setting in the Nextflow config file:

```
google {
  batch.spot = true
}
```

Added in version 24.04

Since this type of virtual machines can be retired by the provider before the job completion, it is advisable to add the following retry strategy to your config file to instruct Nextflow to automatically re-execute a job if the virtual machine was terminated preemptively:

```
process {
  errorStrategy = { task.exitStatus==50001 ? 'retry' : 'terminate' }
  maxRetries = 5
}
```

Added in version 23.04

The Google Batch executor supports the use of [Fusion](#). Fusion allows the use of Google Cloud Storage as a virtual distributed file system, optimizing the data transfer and speeding up most job I/O operations.

See [Google Cloud Batch](#) for more information about configuring Fusion for Google Cloud Batch.

Currently, the following Nextflow directives are supported by the Google Batch executor:

- [accelerator](#)
- [container](#)
- [containerOptions](#)
- [cpus](#)
- [disk](#)
- [executor](#)
- [machineType](#)
- [memory](#)
- [time](#)

Nextflow allows the use of multiple executors in the same workflow. This feature enables the deployment of hybrid workloads, in which some jobs are executed in the local computer or local computing cluster, and some jobs are offloaded to Google Cloud.

To enable this feature, use one or more [process selectors](#) in your Nextflow configuration file to apply the Google Cloud executor to the subset of processes that you want to offload. For example:

```
process {
  withLabel: bigTask {
    executor = 'google-batch'
    container = 'my/image:tag'
  }
}

google {
  project = 'your-project-id'
  location = 'us-central1' // for Google Batch
}
```

Then launch the pipeline with the `-bucket-dir` option to specify a Google Storage path for the jobs computed with Google Cloud and, optionally, the `-work-dir` to specify the local storage for the jobs computed locally:

```
nextflow run <script or project name> -bucket-dir gs://my-bucket/some/path
```

warning

The Google Storage path needs to contain at least one subdirectory (e.g. `gs://my-bucket/work` rather than `gs://my-bucket`).

note

Nextflow will automatically manage the transfer of input and output files between the local and cloud environments when using hybrid workloads.

- Compute resources in Google Cloud are subject to [resource quotas](#), which may affect your ability to run pipelines at scale. You can request quota increases, and your quotas may automatically increase over time as you use the platform. In particular, GPU quotas are initially set to 0, so you must explicitly request a quota increase in order to use GPUs. You can initially request an increase to 1 GPU at a time, and after one billing cycle you may be able to increase it further.
- Currently, it's not possible to specify a disk type different from the default one assigned by the service depending on the chosen instance type.

Kubernetes

Kubernetes is a cloud-native open-source system for deployment, scaling, and management of containerized applications.

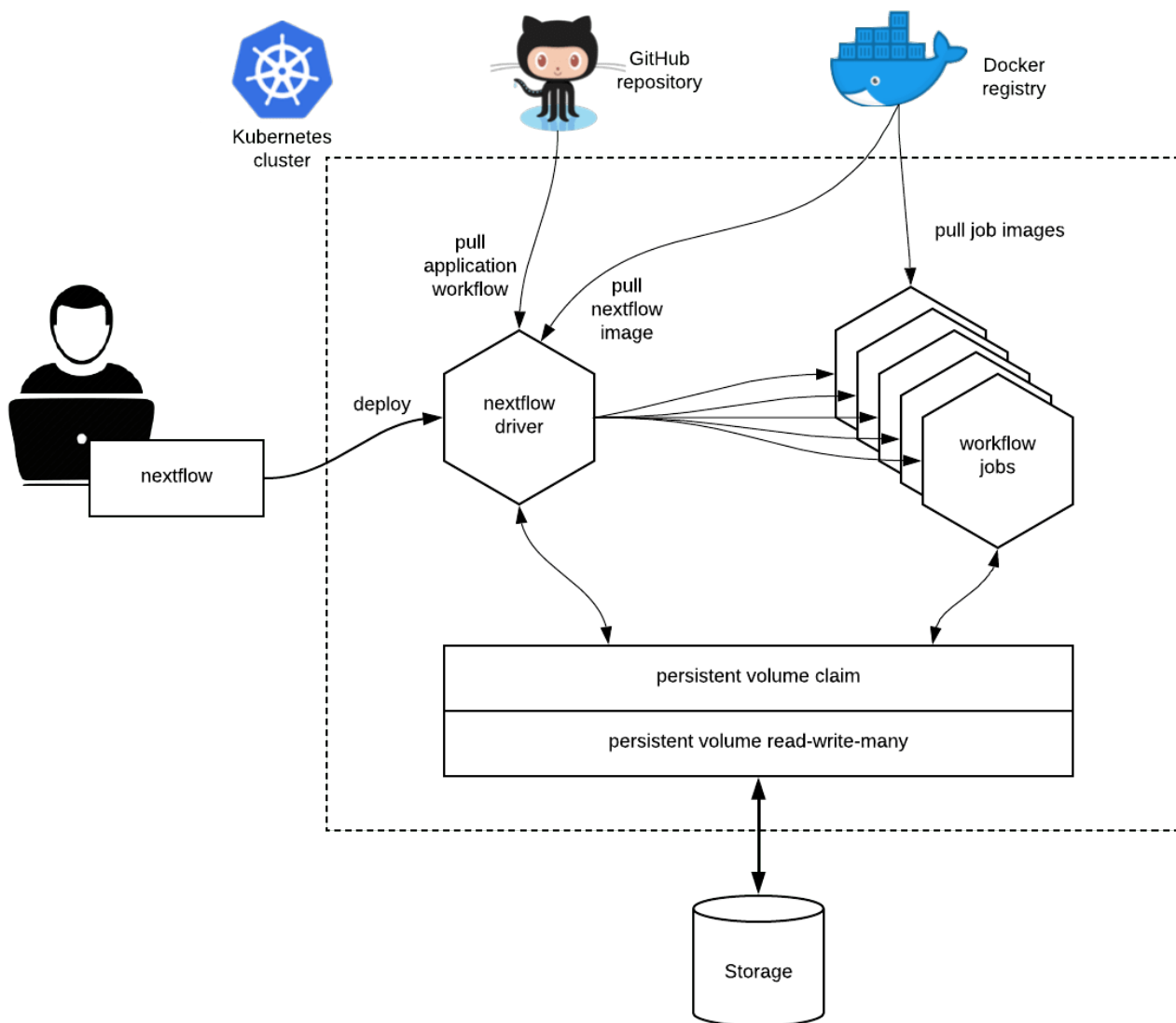
It provides clustering and file system abstractions that allows the execution of containerized workloads across different cloud platforms and on-premises installations.

The built-in support for Kubernetes provided by Nextflow streamlines the execution of containerized workflows in Kubernetes clusters.

Kubernetes main abstraction is the `pod`. A `pod` defines the (desired) state of one or more containers i.e. required computing resources, storage, network configuration.

Kubernetes abstracts also the storage provisioning through the definition of one more persistent volumes that allow containers to access to the underlying storage systems in a transparent and portable manner.

When using the `k8s` executor Nextflow deploys the workflow execution as a Kubernetes pod. This pod orchestrates the workflow execution and submits a separate pod execution for each job that needs to be carried out by the workflow application.



At least a [Persistent Volume](#) with `ReadWriteMany` access mode has to be defined in the Kubernetes cluster (check the supported storage systems at [this link](#)).

Such volume needs to be accessible through a [Persistent Volume Claim](#), which will be used by Nextflow to run the application and store the scratch data and the pipeline final result.

The workflow application has to be containerized using the usual Nextflow [container](#) directive.

tip

When using [Wave](#) and [Fusion](#) there is no need to use a shared file system and configure a persistent volume claim for the deployment of Nextflow pipeline with Kubernetes. You can ignore this requirement when using the Fusion file system. See the [Fusion](#) for further information.

The workflow execution needs to be submitted from a computer able to connect to the Kubernetes cluster.

Nextflow uses the Kubernetes configuration file available at the path `$HOME/.kube/config` or the file specified by the environment variable `KUBECONFIG`.

You can verify such configuration with the following command:

```
$ kubectl cluster-info
Kubernetes master is running at https://your-host:6443
KubeDNS is running at https://your-host:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

warning

The `kuberun` is considered an obsolete approach for the deployment of Nextflow pipeline with Kubernetes and is no longer maintained. For a better alternative, consider using [Launch with Fusion](#).

To deploy and launch the workflow execution use the Nextflow command `kuberun` as shown below:

```
nextflow kuberun <pipeline_name> -v vol-claim:/mount/path
```

This command will create and execute a pod running the nextflow orchestrator for the specified workflow. In the above example replace `<pipeline_name>` with an existing nextflow project or the absolute path of a workflow already deployed in the Kubernetes cluster.

The `-v` command line option is required to specify the volume claim name and mount path to use for the workflow execution. In the above example replace `vol-claim` with the name of an existing persistent volume claim and `/mount/path` with the path where the volume is required to be mount in the container. Volume claims can also be specified in the Nextflow configuration file. See [k8s](#) for details.

Once the pod execution starts, the application in the foreground prints the console output produced by the running workflow pod.

For debugging purpose it's possible to execute a Nextflow pod and launch an interactive shell using the following command:

```
nextflow kuberun login -v vol-claim:/mount/path
```

This command creates a pod, sets up the volume claim(s), configures the Nextflow environment and finally launches a Bash login session.

warning

The pod is automatically destroyed once the shell session terminates. Do not use it to launch long-running workflows in the background.

The use of [Fusion](#) allows deploying a Nextflow pipeline to a remote (or local) cluster without the need to use a shared file system and configure a persistent volume claim for the deployment of Nextflow pipeline with Kubernetes.

This also makes unnecessary the use of the special `kuberun` command for the pipeline execution.

See [Amazon EKS](#) for a guide to set up Fusion in Amazon EKS compute environments and [Google Kubernetes Engine \(GKE\)](#) for a guide to set up Fusion in GKE compute environments.

Nextflow can be executed directly from a pod running in a Kubernetes cluster. In these cases you will need to use the plain Nextflow `run` command and specify the `k8s` executor and the required persistent volume claim in the `nextflow.config` file as shown below:

```
process {
    executor = 'k8s'
}

k8s {
    storageClaimName = 'vol-claim'
    storageMountPath = '/mount/path'
    storageSubPath = '/my-data'
}
```

In the above snippet replace `vol-claim` with the name of an existing persistent volume claim and replace `/mount/path` with the actual desired mount path (default: `/workspace`) and `storageSubPath` with the directory in the volume to be mounted (default: `/`).

warning

The running pod must have been created with the same persistent volume claim name and mount as the one specified in your Nextflow configuration file. Note also that the `run` command does not support the `-v` option.

tip

It is also possible to mount multiple volumes using the `pod` directive, for example:

```
k8s.pod = [ [volumeClaim: "other-pvc", mountPath: "/other" ] ]
```

The process `pod` directive allows the definition of pods specific settings, such as environment variables, secrets and config maps when using the [Kubernetes](#) executor. See the `pod` directive for more details.

The `kuberun` command does not allow the execution of local Nextflow scripts. It is only intended as a convenient way to test the deployment of pipelines to a Kubernetes cluster.

See [k8s](#) and [Kubernetes](#) to learn more about advanced configuration options.

Fusion file system

Added in version 23.04

Cloud object stores such as AWS S3 are scalable and cost-effective, but they don't present a POSIX (Portable Operating System Interface). This means containerized applications must copy data to and from cloud storage for every task — a slow and inefficient process.

Fusion is a virtual, lightweight, distributed file system that bridges the gap between pipelines and cloud-native storage. Fusion enables seamless filesystem I/O to cloud object stores via a standard POSIX interface, resulting in simpler pipeline logic and faster, more efficient pipeline execution.

See [Fusion file system](#) for more information about Fusion features.

note

Fusion requires a license for use in Seqera Platform compute environments or directly in Nextflow. Fusion can be trialed at no cost. See the [Fusion licensing documentation](#) for more information.

Fusion integrates with Nextflow directly and does not require any installation or change in pipeline code. It only requires use of a container runtime or a container computing service, such as Kubernetes, AWS Batch, or Google Cloud Batch.

To enable Fusion in your Nextflow pipeline, add the following to your `nextflow.config` file:

```
fusion.enabled = true
wave.enabled = true
tower.accessToken = '<PLATFORM_ACCESS_TOKEN>'
```

Replace `<PLATFORM_ACCESS_TOKEN>` with your Platform access token.

See [Get started](#) for more information and guides to get started with Fusion.

Configuration

Nextflow configuration lets you control how your pipeline runs without modifying the pipeline code. You can set execution parameters, define resource requirements, configure executors and containers, and create profiles for different environments.

When you launch a pipeline script, Nextflow detects configuration files from multiple sources and applies them in the following order (from lowest to highest priority):

1. `$NXF_HOME/config` (defaults to `$HOME/.nextflow/config`)
2. `nextflow.config` in the project directory
3. `nextflow.config` in the launch directory
4. Config files specified with `-c <config-files>`

tip

You can use the `-C <config-file>` option to specify a fixed set of configuration files and ignore all other files.

Nextflow configuration uses a similar syntax as Nextflow scripts. Configuration options must be set declaratively, but the value of a config option can be an arbitrary expression.

A config file may contain any number of [assignments](#), [blocks](#), and [includes](#). You can add comments just like in scripts.

A config assignment sets a config option to a value:

```
workDir = 'work'
docker.enabled = true
process.maxErrors = 10
```

The config option consists of an option name prefixed by any number of scopes. Scopes group related config options. See [Configuration options](#) for the full set of config options.

The value is typically a literal value, such as a number, boolean, or string. However, you can use any [expression](#):

```
params.helper_file = "${projectDir}/assets/helper.txt"
```

You can specify config options in the same scope as a block:

```
// dot syntax
docker.enabled = true
docker.runOptions = '-u ${id -u}:${id -g}'

// block syntax
docker {
    enabled = true
    runOptions = '-u ${id -u}:${id -g}'
}
```

As a result, deeply nested config options can be assigned in multiple ways. The following three assignments are equivalent:

```
executor.retry.maxAttempt = 5

executor {
  retry.maxAttempt = 5
}

executor {
  retry {
    maxAttempt = 5
  }
}
```

You can include configuration files in other configuration files with the `includeConfig` keyword:

```
process.executor = 'sge'
process.queue = 'long'
process.memory = '10G'

includeConfig 'path/extra.config'
```

Relative paths are resolved against the location of the including file.

note

Config files should only be included at the top level or in a [profile](#). This ensures the included config file is valid on its own and in the context in which it is included.

The following constants are globally available in a Nextflow configuration file:

The directory where the workflow was launched.

The directory where the main script is located.

The directory where the workflow work directory is located.

Map of pipeline secrets. See [Secrets](#) for more information.

The following functions are globally available in a Nextflow configuration file:

Added in version 25.04

Get the value of the environment variable with the specified name in the Nextflow launch environment.

You can define pipeline parameters in config files using the `params` scope:

```
// dot syntax
params.max_cpus = 64
params.publish_mode = 'copy'

// block syntax
params {
  max_cpus = 64
  publish_mode = 'copy'
}
```

note

When including a config file, the included config is evaluated with the parameters defined before the include. The included config cannot see parameters defined after the include.

You should declare parameters in the config file only when other config options use them. When you use a parameter in the script, you should declare it there and override it in config profiles as needed:

```
params.input = null

params {
  publish_mode = 'copy'
}

workflow.output.mode = params.publish_mode

profiles {
  test {
    params.input = "${projectDir}/test/input.txt"
  }
}
```

See [Pipeline parameters](#) for information.

You can use the `process` scope to specify [process directives](#) separately from the pipeline code.

For example:

```
process {
  executor = 'sge'
  queue = 'long'
  clusterOptions = '-pe smp 10 -l virtual_free=64G,h_rt=30:00:00'
}
```

This configuration executes all processes using the SGE executor with the given settings.

You can use the `withLabel` selector to configure all processes annotated with a [label](#):

```
process {
  withLabel: big_mem {
    cpus = 16
    memory = 64.GB
    queue = 'long'
  }
}
```

This configuration assigns 16 CPUs, 64 GB of memory, and the `long` queue to all processes with the `big_mem` label.

You can use the `withName` selector to configure a specific process by its name:

```
process {
  withName: hello {
    cpus = 4
    memory = 8.GB
    queue = 'short'
  }
}
```

The `withName` selector matches both:

- Processes defined with that name
- Processes included under that alias

When you include a process with an alias, selectors for the alias take priority over selectors for the original name. For example, if you define a process as `hello` and include it as `sayHello`, both `withName: hello` and `withName: sayHello` apply, with `withName: sayHello` taking priority.

tip

You don't need to enclose label and process names in quotes unless they contain special characters (`-`, `!`, etc.) or are keywords or built-in type identifiers.

You can use regular expressions in process selectors to apply the same configuration to all processes matching the pattern:

```
process {
  withLabel: 'hello|bye' {
    cpus = 2
    memory = 4.GB
  }
}
```

This configuration requests 2 CPUs and 4 GB of memory for processes labeled as `hello` or `bye`.

You can negate a selector expression by prefixing it with the special character `!`:

```
process {
  withLabel: 'hello' { cpus = 2 }
  withLabel: '!hello' { cpus = 4 }
  withName: '!align.*' { queue = 'long' }
}
```

This configuration sets 2 CPUs for processes labeled as `hello` and 4 CPUs for all processes not labeled as `hello`. It also specifies the `long` queue for processes whose name does not start with `align`.

Nextflow applies process configuration settings in the following order (from lowest to highest priority):

1. Process configuration settings (without a selector)
2. Process directives in the process definition
3. `withLabel` selectors matching any of the process labels
4. `withName` selectors matching the process name
5. `withName` selectors matching the process included alias
6. `withName` selectors matching the process fully qualified name

For example:

```
process {
  cpus = 4
  withLabel: hello { cpus = 8 }
  withName: bye { cpus = 16 }
  withName: 'aloha:bye' { cpus = 32 }
}
```

This configuration:

- Sets 4 CPUs for all processes (unless otherwise specified in their process definition)
- Sets 8 CPUs for processes annotated with the `hello` label
- Sets 16 CPUs for any process named `bye` (or imported as `bye`)
- Sets 32 CPUs for any process named `bye` (or imported as `bye`) invoked by a workflow named `aloha`

Configuration files can define one or more profiles. A profile is a set of configuration settings that can be selected at runtime using the `-profile` command line option.

Configuration profiles are defined in the `profiles` scope. For example:

```
profiles {
  standard {
    process.executor = 'local'
  }

  cluster {
    process.executor = 'sge'
    process.queue = 'long'
    process.memory = '10GB'
  }

  cloud {
    process.executor = 'cirrus'
    process.container = 'cbcr/g/imagemx'
    docker.enabled = true
  }
}
```

This configuration defines three profiles: `standard`, `cluster`, and `cloud`. Each profile provides a different configuration for a given execution environment. When you do not specify a profile, Nextflow uses the `standard` profile by default.

You can enable configuration profiles at runtime as a comma-separated list:

```
nextflow run main.nf -profile standard,cloud
```

Nextflow applies config profiles in the order in which they were defined in the config, regardless of the order you specify them on the command line.

Added in version 25.04

When using the [strict parser](#), Nextflow applies profiles in the order you specify them on the command line.

Use a trace observer in a plugin to add custom workflow handlers to a pipeline via configuration.

You can define workflow event handlers in the config file:


```
workflow.onComplete = {  
    println "Pipeline complete"  
    println "Command line: $workflow.commandLine"  
}  
  
workflow.onError = {  
    println "Error: something went wrong"  
}
```

This approach is useful for handling workflow events without modifying the pipeline code. See [Workflow handlers](#) for more information.

Executors

In the Nextflow framework architecture, the executor is the component that determines the system where a pipeline process is run and supervises its execution.

The executor provides an abstraction between the pipeline processes and the underlying execution system. This allows you to write the pipeline functional logic independently from the actual processing platform.

In other words, you can write your pipeline script once and have it running on your computer, a cluster resource manager, or the cloud — simply change the executor definition in the Nextflow configuration file.

Nextflow supports the [AWS Batch](#) service that allows job submission in the cloud without having to spin out and manage a cluster of virtual machines. AWS Batch uses Docker containers to run tasks, which greatly simplifies pipeline deployment.

The pipeline processes must specify the Docker image to use by defining the `container` directive, either in the pipeline script or the `nextflow.config` file.

To enable this executor, set `process.executor = 'awsbatch'` in the `nextflow.config` file.

The pipeline can be launched either on a local computer, or an EC2 instance. EC2 is suggested for heavy or long-running workloads. Additionally, an S3 bucket must be used as the pipeline work directory.

Resource requests and other job characteristics can be controlled via the following process directives:

- [accelerator](#)
- [arch](#) (only when using Fargate platform type for AWS Batch)
- [container](#)
- [containerOptions](#)
- [cpus](#)
- [disk](#) (only when using Fargate platform type for AWS Batch)
- [memory](#)
- [queue](#)
- [resourceLabels](#)
- [time](#)

The following [hints](#) are supported:

- `consumableResources`: Specify [AWS Batch consumable resources](#) as a list of name-value pairs. For example:

```
hints consumableResources: ['my-license-a': 1, 'my-license-b': 2]
```

See [AWS Batch](#) for more information.

Nextflow supports the [Azure Batch](#) service that allows job submission in the cloud without having to spin out and manage a cluster of virtual machines. Azure Batch uses Docker containers to run tasks, which greatly simplifies pipeline deployment.

The pipeline processes must specify the Docker image to use by defining the `container` directive, either in the pipeline script or the `nextflow.config` file.

To enable this executor, set `process.executor = 'azurebatch'` in the `nextflow.config` file.

The pipeline can be launched either on a local computer, or a cloud virtual machine. The cloud VM is suggested for heavy or long-running workloads. Additionally, an Azure Blob storage container must be used as the pipeline work directory.

Resource requests and other job characteristics can be controlled via the following process directives:

- `container`
- `containerOptions`
- `cpus`
- `disk`
- `machineType`
- `memory`
- `queue`
- `resourceLabels`
- `time`

See [Azure Batch](#) for more information.

Bridge is an abstraction layer to ease batch system and resource manager usage in heterogeneous HPC environments.

It is open source software that can be installed on top of existing classical job schedulers such as Slurm, LSF, or other schedulers. Bridge allows you to submit jobs, get information on running jobs, stop jobs, get information on the cluster system, etc.

For more details on how to install the Bridge system, see the [documentation](#).

To enable the Bridge executor, set `process.executor = 'bridge'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- `clusterOptions`
- `cpus`
- `memory`
- `queue`
- `time`

The `flux` executor allows you to run your pipeline script using the [Flux Framework](#).

Nextflow submits each process to the cluster as a separate job using the `flux submit` command.

To enable the Flux executor, set `process.executor = 'flux'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- `clusterOptions`
- `cpus`
- `queue`
- `time`

note

Flux does not support the `memory` directive.

note

By default, Flux will send all output to the `.command.log` file. To send this output to stdout and stderr instead, set `flux.terminalOutput = true` in your config file.

[Google Cloud Batch](#) is a managed computing service that allows the execution of containerized workloads in the Google Cloud Platform infrastructure.

Nextflow provides built-in support for the Cloud Batch API, which allows the seamless deployment of Nextflow pipelines in the cloud, offloading the pipeline process executions.

The pipeline processes must specify the Docker image to use by defining the `container` directive, either in the pipeline script or the `nextflow.config` file. Additionally, the pipeline work directory must be located in a Google Storage bucket.

To enable this executor, set `process.executor = 'google-batch'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [accelerator](#)
- [container](#)
- [containerOptions](#)
- [cpus](#)
- [disk](#)
- [machineType](#)
- [memory](#)
- [resourceLabels](#)
- [time](#)

See [Cloud Batch](#) for further configuration details.

Experimental: may change in a future release

The `condor` executor allows you to run your pipeline script by using the [HTCondor](#) resource manager.

Nextflow manages each process as a separate job that is submitted to the cluster using the `condor_submit` command.

The pipeline must be launched from a node where the `condor_submit` command is available, which is typically the cluster login node.

note

The HTCondor executor for Nextflow does not currently support HTCondor's ability to transfer input/output data to the corresponding job's compute node. Therefore, the data must be made accessible to the compute nodes through a shared file system directory from where the Nextflow workflow is executed (or specified via the `-w` option).

To enable the HTCondor executor, set `process.executor = 'condor'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [disk](#)
- [memory](#)
- [time](#)

Changed in version 24.10

HyperQueue 0.17.0 or later is required.

Changed in version 25.04

HyperQueue 0.20.0 or later is required.

The `hyperqueue` executor allows you to run your pipeline script by using the [HyperQueue](#) job scheduler.

Nextflow manages each process as a separate job that is submitted to the cluster using the `hq` command line tool.

The pipeline must be launched from a node where the `hq` command is available, which is typically the cluster login node.

To enable the HyperQueue executor, set `process.executor = 'hq'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [accelerator](#)
- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [time](#)

The `k8s` executor allows you to run a pipeline on a [Kubernetes](#) cluster.

Resource requests and other job characteristics can be controlled via the following process directives:

- [accelerator](#)
- [cpus](#)
- [disk](#)
- [memory](#)
- [pod](#)
- [resourcelabels](#)
- [time](#)

See the [Kubernetes](#) page to learn how to set up a Kubernetes cluster to run Nextflow pipelines.

The `local` executor is used by default. It runs the pipeline processes on the computer where Nextflow is launched. The processes are parallelized by spawning multiple threads, taking advantage of the multi-core architecture of the CPU.

The `local` executor is useful for developing and testing a pipeline script on your computer, before switching to a cluster or cloud environment with production data.

Resource requests and other job characteristics can be controlled via the following process directives:

- `accelerator`
- `cpus`
- `memory`
- `time`
- `container`
- `containerOptions`

note

While the `local` executor limits the number of concurrent tasks based on requested vs available resources, it does not enforce task resource requests. In other words, it is possible for a local task to use more CPUs and memory than it requested, in which case it may starve other tasks. An exception to this behavior is when using `Docker` or `Podman` containers, in which case the resource requests are enforced by the container runtime.

The local executor supports two types of tasks:

- Script tasks (processes with a `script` or `shell` block) - executed via a Bash wrapper
- Native tasks (processes with an `exec` block) - executed directly in the JVM.

Added in version 26.10

The local executor allocates accelerators, such as GPUs, to tasks that request them with the `accelerator` directive. Set the environment variable for your accelerator type:

- `CUDA_VISIBLE_DEVICES` for `NVIDIA CUDA` applications
- `HIP_VISIBLE_DEVICES` for `HIP` applications
- `ROCR_VISIBLE_DEVICES` for `AMD ROCm` applications

Set the variable to a comma-separated list of device IDs that Nextflow can use. The list must not be empty. If no devices are listed, no accelerators are available and any task that requests one fails.

For example, to use all GPUs on a node with four NVIDIA GPUs, set `CUDA_VISIBLE_DEVICES` to `0,1,2,3`. If four tasks each request one GPU, they run with `CUDA_VISIBLE_DEVICES` set to `0`, `1`, `2`, and `3`, respectively.

note

To run tasks in containers, propagate the device environment variable into the container and configure the container runtime for GPU access. For example, with Docker, add `CUDA_VISIBLE_DEVICES` to `docker.envWhitelist` and set GPU options such as `--gpus` via `docker.runOptions`. See your container runtime's documentation for GPU support.

The `lsf` executor allows you to run your pipeline script using a `Platform LSF` cluster.

Nextflow manages each process as a separate job that is submitted to the cluster using the `bsub` command.

The pipeline must be launched from a node where the `bsub` command is available, which is typically the cluster login node.

To enable the LSF executor, set `process.executor = 'lsf'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

note

LSF supports both per-core and per-job memory limits. Nextflow assumes that LSF works in the per-core mode, thus it divides the requested [memory](#) by the number of requested [cpus](#).

When LSF is configured to work in the per-job memory limit mode, you must specify this limit with the `perJobMemLimit` option in the [executor](#) scope of your Nextflow config file.

See also the [Platform LSF documentation](#).

Experimental: may change in a future release

The `moab` executor allows you to run your pipeline script using the [Moab](#) resource manager by [Adaptive Computing](#).

Nextflow manages each process as a separate job that is submitted to the cluster using the `msub` command provided by the resource manager.

The pipeline must be launched from a node where the `msub` command is available, which is typically the cluster login node.

To enable the `Moab` executor, set `process.executor = 'moab'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

The `nqsii` executor allows you to run your pipeline script using the [NQSII](#) resource manager.

Nextflow manages each process as a separate job that is submitted to the cluster using the `qsub` command provided by the scheduler.

The pipeline must be launched from a node where the `qsub` command is available, which is typically the cluster login node.

To enable the NQSI executor, set `process.executor = 'nqsii'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

The `oar` executor allows you to run your pipeline script using the [OAR](#) resource manager.

Nextflow manages each process as a separate job that is submitted to the cluster using the `oarsub` command.

The pipeline must be launched from a node where the `oarsub` command is available, which is typically the cluster login node.

To enable the OAR executor set `process.executor = 'oar'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

When specifying `clusterOptions` as a string, multiple options must be separated by semicolons to ensure that the job script is formatted correctly:

```
clusterOptions = '-t besteffort;--project myproject'
```

Added in version 24.04

The same behavior can now be achieved using a string list:

```
clusterOptions = [ '-t besteffort', '--project myproject' ]
```

See [clusterOptions](#) for details.

The `pbs` executor allows you to run your pipeline script using a resource manager from the [PBS/Torque](#) family of batch schedulers.

Nextflow manages each process as a separate job that is submitted to the cluster using the `qsub` command provided by the scheduler.

The pipeline must be launched from a node where the `qsub` command is available, which is typically the cluster login node.

To enable the PBS executor, set `process.executor = 'pbs'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)

- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

The `pbspro` executor allows you to run your pipeline script using the [PBS Pro](#) resource manager.

Nextflow manages each process as a separate job that is submitted to the cluster using the `qsub` command provided by the scheduler.

The pipeline must be launched from a node where the `qsub` command is available, which is typically the cluster login node.

To enable the PBS Pro executor, set `process.executor = 'pbspro'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

The `sge` executor allows you to run your pipeline script using a [Sun Grid Engine](#) cluster or a compatible platform ([Open Grid Engine](#), [Univa Grid Engine](#), etc).

Nextflow manages each process as a separate grid job that is submitted to the cluster using the `qsub` command.

The pipeline must be launched from a node where the `qsub` command is available, which is typically the cluster login node.

To enable the SGE executor, set `process.executor = 'sge'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [penv](#)
- [queue](#)
- [time](#)

Added in version 26.04

warning

Preview feature: may change in a future release.

The `sequera` executor allows you to run your pipeline using the [Seqera](#) cloud infrastructure. It enables the seamless execution of Nextflow pipelines by offloading process executions to the Seqera scheduler service.

The pipeline processes must specify the Docker image to use by defining the `container` directive, either in the pipeline script or the `nextflow.config` file. Additionally, an S3 bucket must be used as the pipeline work directory.

To enable this executor, set `process.executor = 'sequera'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- `arch`
- `container`
- `containerOptions`
- `cpus`
- `disk`
- `memory`
- `time`

The following `hints` are supported:

- `machineRequirement.capacityMode`
- `machineRequirement.diskAllocation`
- `machineRequirement.diskEncrypted`
- `machineRequirement.diskIops`
- `machineRequirement.diskMountPath`
- `machineRequirement.diskSize`
- `machineRequirement.diskThroughputMiBps`
- `machineRequirement.diskType`
- `machineRequirement.machineTypes`
- `machineRequirement.maxSpotAttempts`
- `machineRequirement.provisioning`
- `predictionModel`

The `machineRequirement.*` hints override the corresponding field of the `sequera.executor.machineRequirement` config scope on a per-process basis. The `predictionModel` hint overrides the run-level `sequera.executor.predictionModel` for a single process: set it to `qr/v1` or `qr/v2` to select a model, or to `none` to disable resource prediction for that process even when the run enables a model. When omitted, the process inherits the run-level prediction model. Keys may be used as-is or with the `sequera/` prefix to restrict them to this executor.

For example, to override the provisioning mode for a single process:

```
process hello {
  hints 'sequera/machineRequirement.provisioning': 'spotFirst'

  script:
  """
  your_command --here
  """
}
```

To disable resource prediction for a single process when the run sets a model:

```
process hello {
  hints 'seqlera/predictionModel': 'none'

  script:
  """
  your_command --here
  """
}
```

See the [seqlera scope](#) for the full config reference.

When the [disk](#) directive is specified, the Seqera executor provisions storage for the task container. There are two disk allocation strategies:

- **task** (default): A dedicated EBS volume is created for each task at launch time. This provides isolated, high-performance storage with configurable volume type, IOPS, throughput, and encryption.
- **node**: Uses the instance storage attached at the cluster level. This is shared across tasks running on the same node and does not support EBS-specific options.

By default, a gp3 volume with 325 MiB/s throughput is used (Fusion recommended settings). You can customize the EBS volume configuration:

```
seqlera {
  executor {
    machineRequirement {
      diskAllocation = 'task'      // Per-task EBS volume (default)
      diskType = 'ebs/io1'        // Use provisioned IOPS SSD
      diskIops = 10000            // Required for io1/io2
      diskThroughputMiBps = 500   // Throughput for gp3 volumes
      diskEncrypted = true        // Enable KMS encryption
      diskMountPath = '/data'     // Container mount path (default: /tmp)
    }
  }
}
```

Supported volume types: `ebs/gp3` (default), `ebs/gp2`, `ebs/io1`, `ebs/io2`, `ebs/st1`, `ebs/sc1`.

To use instance storage instead of per-task EBS volumes:

```
seqlera {
  executor {
    machineRequirement {
      diskAllocation = 'node'      // Use instance storage
    }
  }
}
```

note

When using `node` allocation, the EBS-specific options (`diskType`, `diskIops`, `diskThroughputMiBps`, `diskEncrypted`) are not applicable and will cause an error if specified.

See the [seqlera scope](#) for the available configuration options.

The `slurm` executor allows you to run your pipeline script using the [SLURM](#) resource manager.

Nextflow manages each process as a separate job that is submitted to the cluster using the `sbatch` command.

The pipeline must be launched from a node where the `sbatch` command is available, which is typically the cluster login node.

To enable the SLURM executor, set `process.executor = 'slurm'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- [clusterOptions](#)
- [cpus](#)
- [memory](#)
- [queue](#)
- [time](#)

note

SLURM partitions can be specified with the `queue` directive.

note

Nextflow does not provide direct support for SLURM multi-clusters. If you need to submit workflow executions to a cluster other than the current one, specify it with the `SLURM_CLUSTERS` variable in the launch environment.

Added in version 23.10

Some SLURM clusters require memory allocations to be specified with `--mem-per-cpu` instead of `--mem`. You can specify `executor.perCpuMemAllocation = true` in the Nextflow configuration to enable this behavior. Nextflow will automatically compute the memory per CPU for each task (by default 1 CPU is used).

Added in version 26.04

Since SLURM 24, `squeue` supports an `--only-job-state` option that ignores the partition (`-p`) or user (`-u`) filters. To enable this behavior, specify `executor.$slurm.onlyJobState = true` in your Nextflow configuration. If `SchedulerParameters=enable_job_state_cache` is enabled, you can expect improved Nextflow performance and reduced load on the SLURM controller. See [enable_job_state_cache](#) and [--only-job-state](#) for more information.

The `tcs` executor allows you to run your pipeline script using a [Fujitsu Technical Computing Suite \(TCS\)](#).

Nextflow manages each process as a separate job that is submitted to the cluster using the `pjsub` command.

The pipeline must be launched from a node where the `pjsub` command is available, which is typically the login node.

To enable the TCS executor, set `process.executor = 'tcs'` in the `nextflow.config` file.

Resource requests and other job characteristics can be controlled via the following process directives:

- `clusterOptions`
- `time`

note

Use `clusterOptions` to specify system-dependent options such as queue (resource group), CPU, and node. These options vary across target systems and are not standardized. They correspond to `-L` options in the arguments of the `pjsub` command and should be configured according to the requirements of the specific cluster environment.

For example:

```
process {  
  executor = 'tcs'  
  time = '00:30:00'  
  clusterOptions = '-L rscgrp=a-batch -L vnode-core=4'  
}
```

Caching and resuming

One of the core features of Nextflow is the ability to cache task executions and re-use them in subsequent runs to minimize duplicate work. Resumability is useful both for recovering from errors and for iteratively developing a pipeline. It is similar to [checkpointing](#), a common practice used by HPC applications.

You can enable resumability in Nextflow with the `-resume` flag when launching a pipeline with `nextflow run`. In most cases, resumability will just work. This page describes Nextflow's caching behavior in detail to help users understand how the cache works and troubleshoot it when it doesn't work.

All task executions are automatically saved to the task cache, regardless of the `-resume` option (so that you always have the option to resume later). The task cache is a key-value store, where each key-value pair corresponds to a previously-executed task.

The task cache is used in conjunction with the [work directory](#) to recover cached tasks in a resumed run. It is also used by the `log` sub-command to query task metadata.

The task hash is computed from the following metadata:

- Session ID (see `workflow.sessionId` in the [workflow](#) namespace)
- Task name (see `name` in [Trace file](#))
- Task container image (if applicable)
- Task [environment modules](#) (if applicable)
- Task [Conda environment](#) (if applicable)
- Task [Spack environment](#) and [CPU architecture](#) (if applicable)
- Task [inputs](#)
- New in 25.10: Task [eval commands](#)
- Task [script](#)
- Global variables referenced in the task script
- New in 23.10: Task [ext](#) properties referenced in the task script
- Any [bundled scripts](#) used in the task script
- Whether the task is a [stub run](#)

note

Nextflow also includes an incrementing component in the hash generation process, which allows it to iterate through multiple hash values until it finds one that does not match an existing execution directory. This mechanism typically aligns with task retries (i.e., task attempts), however this is not guaranteed.

Nextflow computes this hash for every task before it is executed. If resumability is enabled, Nextflow checks whether the task cache contains a matching hash and whether the task outputs are still present in the [work directory](#). If both conditions are met, the task is resumed. Otherwise, it is re-executed.

Files are hashed differently depending on the caching mode. See the [cache](#) directive for more details.

The task entry is a serialized blob of the task metadata required to resume a task, including the fields used by the [Trace file](#) and the task input variables.

The default cache store uses the `.nextflow/cache` directory, relative to the launch directory (i.e. `workflow.launchDir`), to store the task cache, with a separate subdirectory for each session ID backed by [LevelDB](#).

Due to the limitations of LevelDB, the database for a given session ID can only be accessed by one reader/writer at a time. This means, for example, that you cannot use `nextflow log` to query the task metadata for a pipeline run while it is still running.

The cloud cache is an alternative cache store that uses cloud storage instead of the local cache directory. You can use it by setting the `NXF_CLOUDCACHE_PATH` environment variable to the desired cache path (e.g. `s3://my-bucket/cache`) and providing the necessary credentials.

The cloud cache is particularly useful when launching Nextflow from within the cloud, where the default cache would be lost once the pipeline completes and the VM instance is terminated. Furthermore, because it is backed by cloud storage, it can support multiple readers and writers.

While the [task cache](#) stores the task metadata for subsequent runs, the work directory stores various files used during a pipeline run.

Each task uses a unique directory based on its hash. When a task is created, Nextflow stages the task input files, script, and other helper files into the task directory. The task writes any output files to this directory during its execution, and Nextflow uses these output files for downstream tasks and/or publishing.

When a previous task is retrieved from the task cache on a resumed run, Nextflow then checks the corresponding task directory in the work directory. If all the required outputs are present and the exit code is valid, then the task is successfully cached; otherwise, the task is re-executed.

For this reason, it is important to preserve both the task cache (`.nextflow/cache`) and work directories in order to resume runs successfully. You can use the [clean](#) command to delete specific runs from the cache.

Cache failures occur when a task that was supposed to be cached was re-executed or a task that was supposed to be re-executed was cached.

Common causes of cache failures include:

- [Resume not enabled](#)
- [Cache directive disabled](#)
- [Modified inputs](#)
- [Inconsistent file attributes](#)
- [Race condition on a global variable](#)
- [Non-deterministic process inputs](#)

The `-resume` option is required to resume a pipeline. Ensure you enable `-resume` in your run command or your Nextflow configuration file.

The `cache` directive is enabled by default. However, you can disable or modify its behavior for a specific process. For example:

```
process F00 {
  cache false
  // ...
}
```

Ensure that the `cache` directive has not been disabled. See [cache](#) for more information.

Modifying inputs that are used in the task hash invalidates the cache. Common causes of modified inputs include:

- Changing input files
- Resuming from a different session ID
- Changing the process name
- Changing the calling workflow name
- Changing the task container image or Conda environment
- Changing the task script
- Changing a bundled script used by the task

Nextflow calculates a hash for an input file using its full path, last modified timestamp, and file size. If any of these attributes change, Nextflow re-executes the task.

warning

If a process modifies its input files, it cannot be resumed. Avoid processes that modify their own input files as this is considered an anti-pattern.

Some shared file systems, such as NFS, may report inconsistent file timestamps, which can invalidate the cache when using the standard caching mode.

To resolve this issue, use the `'lenient'` [caching mode](#) to ignore the last modified timestamp and use only the file path and size.

Race conditions can disrupt the caching behavior of your pipeline. For example:

```
channel.of(1,2,3).map { v -> X=v; X+=2 }.view { v -> "ch1 = $v" }  
channel.of(1,2,3).map { v -> X=v; X*=2 }.view { v -> "ch2 = $v" }
```

In the above example, `X` is declared in each `map` closure. Without the `def` keyword, the variable `X` is global to the entire script. Because operators are executed concurrently and `X` is global, there is a race condition that causes the emitted values to vary depending on the order of the concurrent operations. If these values were passed to a process as inputs, the process would execute different tasks during each run due to the race condition.

To resolve this issue, avoid declaring global variables in closures:

```
channel.of(1,2,3).map { v -> def X=v; X+=2 }.view { v -> "ch1 = $v" }
```

Added in version 25.04

The [strict parser](#) does not allow global variables to be declared in closures.

A process that merges inputs from different sources non-deterministically may invalidate the cache. For example:


```

workflow {
  ch_bam = channel.of( ['1', '1.bam'], ['2', '2.bam'] )
  ch_bai = channel.of( ['2', '2.bai'], ['1', '1.bai'] )
  check_bam_bai(ch_bam, ch_bai)
}

process check_bam_bai {
  input:
  tuple val(id), file(bam)
  tuple val(id), file(bai)

  script:
  """
  check_bam_bai $bam $bai
  """
}

```

In the above example, the inputs will be merged without matching on `id`, in a similar manner as the `merge` operator. As a result, the inputs are incorrect and non-deterministic.

To resolve this issue, use the `join` operator to join the channels into a single input channel before invoking the process:

```

workflow {
  ch_bam = channel.of( ['1', '1.bam'], ['2', '2.bam'] )
  ch_bai = channel.of( ['2', '2.bai'], ['1', '1.bai'] )
  check_bam_bai(ch_bam.join(ch_bai))
}

process check_bam_bai {
  input:
  tuple val(id), file(bam), file(bai)

  script:
  """
  check_bam_bai $bam $bai
  """
}

```

Nextflow resumes from the previous run by default. If you want to resume from an earlier run, simply specify the session ID for that run with the `-resume` option:

```

nextflow run rnaseq-nf -resume 4dc656d2-c410-44c8-bc32-7dd0ea87bebf

```

You can use the `log` command to view all previous runs as well as the task executions for each run.

One way to debug a resumed run is to compare the task hashes of each run using the `-dump-hashes` option.

1. Perform an initial run: `nextflow -log run_initial.log run <pipeline> -dump-hashes`
2. Perform a resumed run: `nextflow -log run_resumed.log run <pipeline> -dump-hashes -resume`
3. Extract the task hash lines from each log (search for `cache hash:`)
4. Compare the runs with a diff viewer

While some manual effort is required, the final diff can often reveal the exact change that caused a task to be re-executed.

When using `-dump-hashes json`, the task hashes can be more easily extracted into a diff. Here is an example Bash script to perform two runs and produce a diff:

```
nextflow -log run_1.log run $pipeline -dump-hashes json
nextflow -log run_2.log run $pipeline -dump-hashes json -resume

get_hashes() {
  cat $1 \
  | grep 'cache hash:' \
  | cut -d ' ' -f 10- \
  | sort \
  | awk '{ print; print ""; }'
}

get_hashes run_1.log > run_1.tasks.log
get_hashes run_2.log > run_2.tasks.log

diff run_1.tasks.log run_2.tasks.log
```

You can then view the `diff` output or use a graphical diff viewer to compare `run_1.tasks.log` and `run_2.tasks.log`.

Added in version 25.04

Nextflow now has a built-in way to compare two task runs. See the [Data lineage](#) guide for details.

Reports

Nextflow provides several reporting mechanisms to analyze and debug pipeline runs:

- [Execution log](#): Command to query information about past runs
- [Execution report](#): Comprehensive HTML report for a pipeline run
- [Execution timeline](#): HTML timeline of tasks executed in a pipeline run
- [Trace file](#): CSV file with detailed task metrics for a pipeline run
- [Workflow diagram](#): Graph visualization of pipeline structure

The `nextflow log` command shows information about past pipeline runs and task executions. You can query by run name or session ID, and the command provides multiple options to filter and format the output.

By default, `nextflow log` prints the list of pipeline runs:

```
$ nextflow log
TIMESTAMP          DURATION      RUN NAME      STATUS  REVISION ID  SESSION
ID                COMMAND
2025-12-09 11:43:18  1s           naughty_heisenberg  OK      2ce0b0e294    bae65ec6-a2b9-49bf-b63e-2fec91945e48
2025-12-09 11:43:28  931ms        thirsty_swanson    OK      2ce0b0e294    bae65ec6-a2b9-49bf-b63e-2fec91945e48
2025-12-09 11:43:37  8.5s         goofy_kilby       OK      ca20a6dfd2     d0a60572-076e-451a-b10e-16059ed77e36
nextflow run rnaseq-nf -profile conda
```

Providing a run name or session ID prints the tasks executed by that pipeline run:

```
$ nextflow log goofy_kilby
/Users/user/workspace/work/e1/fc2d06782f8263476426c576888033
/Users/user/workspace/work/72/f0542cfa81ad6abeaf1adf02e5cc2b
/Users/user/workspace/work/83/009346a3958358bb704e996d935c7a
/Users/user/workspace/work/84/15e4cd16df1db6f9bc0e22cc05316d
```

The `-f` (`-fields`) option specifies which fields to include in the log.

```
nextflow log <run-name> -f [fields]
```

You can set a custom list of fields to focus on specific information. For example, `hash`, `name`, `exit`, and `status`:

```
$ nextflow log goofy_kilby -f hash,name,exit,status
e1/fc2d06      RNASEQ:INDEX (gga1_1_48850000_49020000) 0      COMPLETED
72/f0542c      RNASEQ:FASTQC (gga1_gut)          0      COMPLETED
83/009346      RNASEQ:QUANT (gga1_gut)          0      COMPLETED
84/15e4cd      MULTIQC      0      COMPLETED
```

Run the `log` command with the `-l` option to see the full list of fields. See [log](#) for more information.

The `-t` option specifies a template (string or file) to use when inspecting a single run:

```
nextflow log <run-name> -t <template> > <report>
```

You can create a template. For example, in Markdown:

```
## $name

script:

    $script

exit status: $exit
task status: $status
task folder: $folder
```

Then, use it to create a report:

```
nextflow log goofy_kilby -t my-template.md > execution-report.md
```

The `filter` option selects which entries to include in the log.

```
nextflow log <run-name> -filter '<filters>'
```

Any valid Groovy boolean expression on the log fields can be used to define the filter condition. For example, you can filter for task names matching a pattern:

```
nextflow log goofy_kilby -filter 'name =~ /hello.*/ && status == "FAILED"'
```

The execution report is an HTML report that includes metrics about a pipeline run. The report is organized into three sections: **Summary**, **Resources**, and **Tasks**.

To enable the execution report, add the `-with-report` option when launching a pipeline run:

```
nextflow run <pipeline> -with-report [file_name]
```

You can also enable the report in your Nextflow configuration:

```
report {
    enabled = true
    file = 'report.html'
}
```

See [report](#) for the list of available config options.

The **Summary** section reports the execution status, the launch command, overall execution time, and other workflow metadata:

Nextflow workflow report

[deadly_baekeland]

Workflow execution completed successfully!

Run times

09-Dec-2025 02:43:21 - 09-Dec-2025 02:46:58 (duration: 3m 37s)

4 succeeded

Nextflow command

```
nextflow run rnaseq-nf -profile conda -with-report report.html
```

CPU-Hours	(a few seconds)
Launch directory	/workspaces/sandbox
Work directory	/workspaces/sandbox/work
Project directory	/home/gitpod/.nextflow/assets/nextflow-io/rnaseq-nf
Script name	main.nf
Script ID	295b10c8d2badd387ed3f6fc5e56e45e
Workflow session	61a4b8bc-9051-4108-b0b2-00f8ccd83e55
Workflow repository	https://github.com/nextflow-io/rnaseq-nf , revision <code>master</code> (commit hash <code>ca20a6dfd2d799b903e557dc2e736d55419c7a1a</code>)
Workflow profile	conda
Nextflow version	version 25.04.6, build 5954 (01-07-2025 11:27 UTC)

The **Resources** section plots the distribution of resource usage for each workflow process using the interactive [plotly.js](#) plotting library.

Plots are shown for CPU, memory, job duration, and disk I/O. Each usage metric is shown both in raw form and as a percentage of the requested resources. These plots are helpful to check that task resources are used efficiently.

Resource Usage

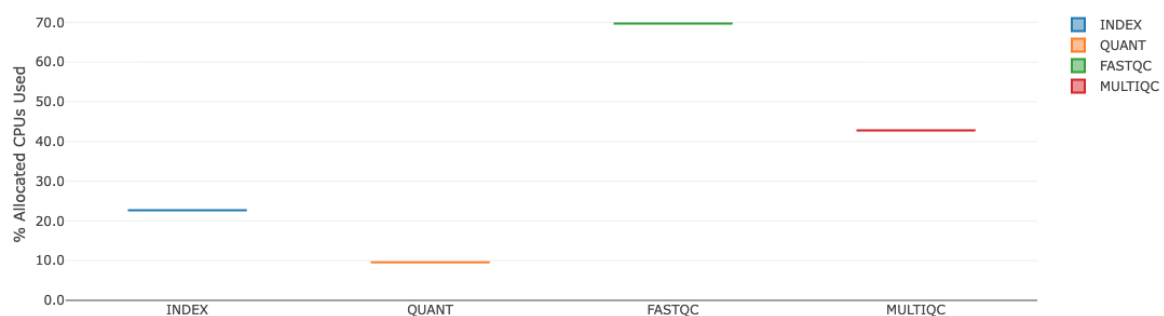
These plots give an overview of the distribution of resource usage for each process.

CPU

Raw Usage

% Allocated

% Requested CPU Used



For information about how resource usage is computed, see [Understanding task resource metrics](#).

The **Tasks** section lists all executed tasks, reporting the status, command script, and other metrics for each task:

Tasks

This table shows information about each task in the workflow. Use the search box on the right to filter rows for specific values. Clicking headers will sort the table by that value and scrolling side to side will reveal more columns.

Values shown as: Human readable

Show 25 entries

Filter: Metrics Metadata All Search:

task_id	process	tag	status	hash	allocated cpus	%cpu	allocated memory	%mem	vmem
1	RNASEQ:INDEX	ggal_1_48850000_49020000	COMPLETED	54/12b239	1	22.7	-	0.2	52.016 MB
2	RNASEQ:FASTQC	ggal_gut	COMPLETED	c7/e107f8	1	69.7	-	2.4	3.002
3	RNASEQ:QUANT	ggal_gut	COMPLETED	83/9c395b	1	9.6	-	0.6	368.9 MB
4	MULTIQC	-	COMPLETED	94/c235e1	1	42.8	-	1.4	571.56 MB

The same caveats for the [trace file](#) also apply to the task table.

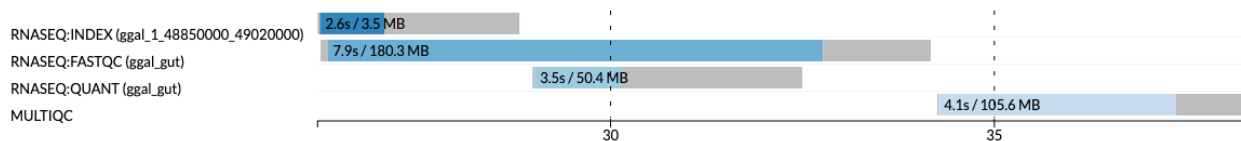
The execution timeline is an HTML report that shows a timeline of the tasks executed in a pipeline run:

Processes execution timeline

Launch time: 09 Dec 2025 16:07

Elapsed time: 15.8s

Legend: job wall time / memory usage (RAM)



Created with Nextflow - <http://nextflow.io>

The x-axis shows elapsed time relative to workflow launch. The y-axis shows a bar for each task execution. Each bar has the following properties (with the equivalent trace fields in parentheses):

- Bar length: Task duration (duration)
- Bar color: Identifies tasks from the same process
- Colored area: Real execution time (realtime)
- Gray area (left): Task scheduling time (start - submit)
- Gray area (right): Task termination time (complete - start - realtime)
- Label: Task duration (duration) and peak memory usage (peak_rss)

To enable the execution timeline, add the `-with-timeline` option when launching a pipeline run:

```
nextflow run <pipeline> -with-timeline [file_name]
```

You can also enable the execution timeline in your Nextflow configuration:

```
timeline {
  enabled = true
  file = 'timeline.html'
}
```

See [timeline](#) for the list of available config options.

The trace file is a CSV file containing information about each task executed in a pipeline run, including submission time, start time, completion time, CPU usage, and memory usage.

To create the trace file, add the `-with-trace` option when launching a pipeline:

```
nextflow run <pipeline> -with-trace
```

The above command creates a file named `trace.txt` in the current directory. For example:

task_id	hash	native_id	name	status	exit	submit	duration	realtime	%cpu	peak_rss
19	45/ ab752a	2032	blast (1)	COMPLETED	0	2014-10-23 16:33:16.288	1m	5s	0.0%	29.8 MB
20	72/ db873d	2033	blast (2)	COMPLETED	0	2014-10-23 16:34:17.211	30s	10s	35.7%	152.8 MB
21	53/ d13188	2034	blast (3)	COMPLETED	0	2014-10-23 16:34:17.518	29s	20s	4.5%	289.5 MB
28	b4/0f9613	2041	exonerate (1)	COMPLETED	0	2014-10-23 16:38:19.657	1m 30s	1m 11s	94.3%	611.6 MB
31	d7/ eabe51	2042	exonerate (3)	COMPLETED	0	2014-10-23 16:45:50.846	3m 1s	2m 6s	130.1%	703.3 MB
30	4f/1ad1f0	2043	exonerate (2)	COMPLETED	0	2014-10-23 16:45:50.961	10m 2s	9m 16s	95.5%	706.2 MB
52	72/41d0c6	2055	similarity (1)	COMPLETED	0	2014-10-23 17:13:23.543	30s	352ms	0.0%	35.6 MB
53	3e/bca30f	2061	similarity (2)	COMPLETED	0	2014-10-23 17:13:23.770	30s	238ms	0.0%	6.7 MB
54	8b/ d45b47	2062	similarity (3)	COMPLETED	0	2014-10-23 17:13:23.808	30s	442ms	0.0%	108.1 MB
98	de/d6c0a6	2099	matrix (1)	COMPLETED	0	2014-10-23 17:14:27.139	30s	1s	0.0%	4.8 MB

The following caveats apply to trace metrics:

- Nextflow collects trace metrics through a background process for each job in the target environment. Make sure the following tools are available in the environment where tasks are executed: `awk`, `date`, `grep`, `ps`, `sed`, `tail`, `tee`. Container images that do not include all of these tools may lead to missing data.
- Trace metrics provide an estimation of the resources used by running tasks. They are not an alternative to low-level performance analysis tools, and they may not be accurate for very short-lived tasks (e.g., less than a few seconds).
- Some trace metrics are not reported when running on macOS.

You can also enable the trace file in your Nextflow configuration:

```
trace {  
  enabled = true  
  file = 'trace.txt'  
}
```

See [trace](#) for the list of available config options.

A Nextflow pipeline can be represented as a directed acyclic graph (DAG). The vertices in the graph represent the pipeline's processes and operators, while the edges represent the data dependencies (i.e., channels) between them.

To render the workflow DAG, run your pipeline with the `-with-dag` option. By default, it creates a file named `dag-<timestamp>.html` with the workflow DAG rendered as a [Mermaid](#) diagram.

The workflow DAG can be rendered in a different format by specifying an output file name with a different extension based on the desired format. For example:

```
nextflow run main.nf -with-dag flowchart.png
```

tip

Use the `-preview` option with `-with-dag` to render the workflow DAG without executing any tasks.

The following example shows the Mermaid diagram for the [rnaseq-nf](#) pipeline (using the [Mermaid Live Editor](#) with the `default` theme):

```
nextflow run rnaseq-nf -preview -with-dag
```

See [dag](#) for the list of available config options.

Containers

Nextflow supports a variety of container runtimes. Containerization allows you to write self-contained and truly reproducible computational pipelines, by packaging the binary dependencies of a script into a standard and portable format that can be executed on any platform that supports a container runtime. Furthermore, the same pipeline can be transparently executed with any of the supported container runtimes, depending on which runtimes are available in the target compute environment.

note

When creating a container image to use with Nextflow, make sure that Bash (3.x or later) and `ps` are installed in the image, along with other tools required for collecting metrics (see [Tasks](#) for more information). Bash should be available on the path `/bin/bash` and it should be the container entrypoint.

Added in version 26.04

[Apple container](#) is a lightweight container runtime that uses the [Virtualization framework](#) to run each container inside its own virtual machine on macOS.

[Apple container](#) requires Apple silicon (M1 or newer) and a recent macOS release.

Start the system service before running your pipeline:

```
container system start
```

You don't need to modify your Nextflow script to run it with Apple container. Enable it in the Nextflow configuration file:

```
process.container = 'nextflow/examples:latest'
appleContainer.enabled = true
```

Each time your script launches a process, Nextflow wraps it and runs it inside a container created from the specified image using the `container run` command.

note

Each container runs in its own lightweight VM, so tasks benefit from strong isolation without the overhead of a full VM per task. Images are standard OCI images and can be pulled from any registry.

note

Apple container only supports Linux images. To run an `amd64` image on Apple silicon, pass `--rosetta` via `appleContainer.runOptions` and set the platform explicitly, for example, `process.arch = 'linux/amd64'`.

You can specify a different image for each process in your pipeline. For example, the following script defines two processes, `hello` and `bye`, each using a different image:

```
process hello {
  container 'image_name_1'

  script:
  """
  do this
  """
}

process bye {
  container 'image_name_2'

  script:
  """
  do that
  """
}
```

You can also define the same container assignments in the configuration file:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

appleContainer {
  enabled = true
}
```

See [Process configuration](#) for more information about process configuration.

See [appleContainer](#) on the Nextflow configuration page for advanced Apple container settings.

[Apptainer](#) is an alternative container runtime to Docker and an open source fork of Singularity. The main advantages of Apptainer are that it can be used without root privileges and it doesn't require a separate daemon process. These, along with other features such as support for autoofs mounts, makes Apptainer better suited to the requirements of HPC workloads. Apptainer is able to use existing Docker images and can pull from Docker registries.

You will need Apptainer installed on your execution environment e.g. your computer or a distributed cluster, depending on where you want to run your pipeline.

Apptainer makes use of a container image file, which physically contains the container. Refer to the [Apptainer documentation](#) to learn how to create Apptainer images.

Apptainer allows paths that do not currently exist within the container to be created and mounted dynamically by specifying them on the command line. However this feature is only supported on hosts that support the [Overlay file system](#) and is not enabled by default.

note

Nextflow expects that data paths are defined system wide, and your Apptainer images need to be created having the mount paths defined in the container file system.

If your Apptainer installation support the "user bind control" feature, enable the Nextflow support for that by defining the `apptainer.autoMounts = true` setting in the Nextflow configuration file.

The integration for Apptainer follows the same execution model implemented for Docker. You won't need to modify your Nextflow script in order to run it with Apptainer. Simply specify the Apptainer image file from where the containers are started by using the `-with-apptainer` command line option. For example:

```
nextflow run main.nf -with-apptainer [apptainer image file]
```

Every time your script launches a process execution, Nextflow will run it into an Apptainer container created by using the specified image. In practice Nextflow will automatically wrap your processes and launch them by running the `apptainer exec` command with the image you have provided.

note

An Apptainer image can contain any tool or piece of software you may need to carry out a process execution. Moreover, the container is run in such a way that the process result files are created in the host file system, thus it behaves in a completely transparent manner without requiring extra steps or affecting the flow in your pipeline.

If you want to avoid entering the Apptainer image as a command line parameter, you can define it in the Nextflow configuration file. For example you can add the following lines in the configuration file:

```
process.container = '/path/to/apptainer.img'  
apptainer.enabled = true
```

In the above example replace `/path/to/apptainer.img` with any Apptainer image of your choice.

See [Configuration](#) to learn more about the configuration file and how to use it to configure your pipeline execution.

note

Unlike Docker, Nextflow does not automatically mount host paths in the container when using Apptainer. It expects that the paths are configured and mounted system wide by the Apptainer runtime. If your Apptainer installation allows user defined bind points. See [Apptainer](#) to learn how to enable Nextflow auto mounts.

warning

When a process input is a symbolic link file, make sure the linked file is stored in a host folder that is accessible from a bind path defined in your Apptainer installation. Otherwise the process execution will fail because the launched container won't be able to access the linked file.

Changed in version 23.10

Nextflow no longer mounts the home directory when launching an Apptainer container. To re-enable the old behavior, set the environment variable `NXF_APPTAINER_HOME_MOUNT` to `true`.

It is possible to specify a different Apptainer image for each process definition in your pipeline script. For example, suppose you have two processes named `hello` and `bye`. You can specify two different Apptainer images in the configuration file as shown below:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}
apptainer {
  enabled = true
}
```

See [Process configuration](#) to learn more about processes configuration.

Nextflow is able to transparently pull remote container images stored in any Docker compatible registry.

By default, when a container name is specified, Nextflow checks if an image file with that name exists in the local file system. If that image file exists, it's used to execute the container. If a matching file does not exist, Nextflow automatically tries to pull an image with the specified name from the container registry.

If you want Nextflow to check only for local file images, prefix the container name with the `file://` pseudo-protocol. For example:

```
process.container = 'file:///path/to/apptainer.img'
apptainer.enabled = true
```

warning

Use three `/` slashes to specify an **absolute** file path, otherwise the path will be interpreted as relative to the workflow launch directory.

To pull images from Apptainer Hub or a third party Docker registry, prefix the image name with the `shub://`, `docker://` or `docker-daemon://` pseudo-protocol as required by Apptainer. For example:

```
process.container = 'docker://quay.io/biocontainers/multiqc:1.3--py35_2'
apptainer.enabled = true
```

You do not need to specify `docker://` to pull from a Docker repository. Nextflow will automatically prepend it to your image name when Apptainer is enabled. Additionally, the Docker engine will not work with containers specified as `docker://`.

note

This feature requires the `apptainer` tool to be installed where the workflow execution is launched (as opposed to the compute nodes).

Nextflow caches Apptainer images in the `apptainer` directory, in the pipeline work directory, by default. However, it is recommended to provide a centralized cache directory using the `NXF_APPTAINER_CACHEDIR` environment variable or the `apptainer.cacheDir` setting in the Nextflow config file.

Nextflow uses the library directory to determine the location of Apptainer containers. The library directory can be defined using the `apptainer.libraryDir` configuration setting or the `NXF_APPTAINER_LIBRARYDIR` environment variable. The configuration file option overrides the environment variable if both are set.

Nextflow first checks the library directory when searching for the image. If the image is not found it then checks the cache directory. The main difference between the library directory and the cache directory is that the first is assumed to be a read-only container repository, while the latter is expected to be writable path where container images can be added for caching purposes.

warning

When using a compute cluster, the Apptainer cache directory must reside in a shared filesystem accessible to all compute nodes.

danger

When pulling Docker images, Apptainer may be unable to determine the container size if the image was stored using an old Docker format, resulting in a pipeline execution error. See Apptainer documentation for details.

Apptainer advanced configuration settings are described in [Apptainer](#) section in the Nextflow configuration page.

Requires Charliecloud 0.28 or later.

Experimental: not recommended for production environments.

[Charliecloud](#) is an alternative container runtime to Docker, that is better suited for use in HPC environments. Its main advantage is that it can be used without root privileges, making use of user namespaces in the Linux kernel. Charliecloud is able to pull from Docker registries.

You will need Charliecloud installed in your execution environment e.g. on your computer or a distributed cluster, depending on where you want to run your pipeline.

You won't need to modify your Nextflow script in order to run it with Charliecloud. Simply specify the docker image from where the containers are started by using the `-with-charliecloud` command line option. For example:

```
nextflow run main.nf -with-charliecloud [container]
```

Every time your script launches a process execution, Nextflow will run it into a charliecloud container created by using the specified container image. In practice Nextflow will automatically wrap your processes and run them by executing the `ch-run` command with the container you have provided.

note

A container image can contain any tool or piece of software you may need to carry out a process execution. Moreover, the container is run in such a way that the process result files are created in the host file system, thus it behaves in a completely transparent manner without requiring extra steps or affecting the flow in your pipeline.

If you want to avoid entering the Container image as a command line parameter, you can define it in the Nextflow configuration file. For example you can add the following lines in the configuration file:

```
process.container = '/path/to/container'
charliecloud.enabled = true
```

warning

If an absolute path is provided, the container needs to be in the Charliecloud flat directory format. See below on compatibility with Docker registries.

See [Configuration](#) to learn more about the configuration file and how to use it to configure your pipeline execution.

warning

Nextflow automatically manages the file system mounts whenever a container is launched depending on the process input files. However, when a process input is a symbolic link, the linked file **must** be stored in the same folder where the symlink is located, or a sub-folder of it. Otherwise the process execution will fail because the launched container won't be able to access the linked file.

Nextflow is able to transparently pull remote container images stored in any Docker compatible registry and converts them to the Charliecloud compatible format.

By default when a container name is specified, Nextflow checks if a container with that name exists in the local file system. If it exists, it's used to execute the container. If a matching file does not exist, Nextflow automatically tries to pull an image with the specified name from the Docker Hub.

To pull images from a third party Docker registry simply provide the URL to the image. If no URL is provided, Docker Hub is assumed. For example this can be used to pull an image from quay.io and convert it automatically to the Charliecloud container format:

```
process.container = 'https://quay.io/biocontainers/multiqc:1.3--py35_2'
charliecloud.enabled = true
```

Whereas this would pull from Docker Hub:

```
process.container = 'nextflow/examples:latest'
charliecloud.enabled = true
```

It is possible to specify a different Docker image for each process definition in your pipeline script. For example, suppose you have two processes named `hello` and `bye`. You can specify two different Docker images for them in the config file as shown below:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

charliecloud {
  enabled = true
}
```

See [Process configuration](#) to learn more about processes configuration.

After running your pipeline, you can easily query the container image that each process used with the following command:

```
nextflow log last -f name,container
```

Charliecloud advanced configuration settings are described in the [Charliecloud](#) section in the Nextflow configuration page.

[Docker](#) is the industry standard container runtime.

You will need Docker installed on your execution environment e.g. your computer or a distributed cluster, depending on where you want to run your pipeline.

If you are running Docker on Mac OSX make sure you are mounting your local `/Users` directory into the Docker VM as explained in this excellent tutorial: [How to use Docker on OSX](#).

You won't need to modify your Nextflow script in order to run it with Docker. Simply specify the Docker image from where the containers are started by using the `-with-docker` command line option. For example:

```
nextflow run main.nf -with-docker [docker image]
```

Every time your script launches a process execution, Nextflow will run it into a Docker container created by using the specified image. In practice Nextflow will automatically wrap your processes and run them by executing the `docker run` command with the image you have provided.

note

A Docker image can contain any tool or piece of software you may need to carry out a process execution. Moreover, the container is run in such a way that the process result files are created in the host file system, thus it behaves in a completely transparent manner without requiring extra steps or affecting the flow in your pipeline.

If you want to avoid entering the Docker image as a command line parameter, you can define it in the Nextflow configuration file. For example you can add the following lines in the configuration file:


```
process.container = 'nextflow/examples:latest'
docker.enabled = true
```

In the above example replace `nextflow/examples:latest` with any Docker image of your choice.

See [Configuration](#) to learn more about the configuration file and how to use it to configure your pipeline execution.

warning

Nextflow automatically manages the file system mounts whenever a container is launched depending on the process input files. However, when a process input is a symbolic link, the linked file **must** be stored in the same folder where the symlink is located, or a sub-folder of it. Otherwise the process execution will fail because the launched container won't be able to access the linked file.

It is possible to specify a different Docker image for each process definition in your pipeline script. Suppose you have two processes named `hello` and `bye`. You can specify two different Docker images for them in the Nextflow script as shown below:

```
process hello {
  container 'image_name_1'

  script:
  """
  do this
  """
}

process bye {
  container 'image_name_2'

  script:
  """
  do that
  """
}
```

Alternatively, the same containers definitions can be provided by using the configuration file as shown below:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

docker {
  enabled = true
}
```

See [Process configuration](#) to learn more.

Docker advanced configuration settings are described in the [docker](#) section in the Nextflow configuration page.

Podman is a drop-in replacement for Docker that can run containers with or without root privileges.

You will need Podman installed on your execution environment e.g. your computer or a distributed cluster, depending on where you want to run your pipeline. Running in rootless mode requires appropriate OS configuration. Due to current Podman limits using cgroup for cpus and memory such is only possible using sudo.

You won't need to modify your Nextflow script in order to run it with Podman. Simply specify the Podman image from where the containers are started by using the `-with-podman` command line option. For example:

```
nextflow run main.nf -with-podman [OCI container image]
```

Every time your script launches a process execution, Nextflow will run it into a Podman container created by using the specified image. In practice Nextflow will automatically wrap your processes and run them by executing the `podman run` command with the image you have provided.

note

An OCI container image can contain any tool or piece of software you may need to carry out a process execution. Moreover, the container is run in such a way that the process result files are created in the host file system, thus it behaves in a completely transparent manner without requiring extra steps or affecting the flow in your pipeline.

If you want to avoid entering the Podman image as a command line parameter, you can define it in the Nextflow configuration file. For example you can add the following lines in the configuration file:

```
process.container = 'nextflow/examples:latest'
podman.enabled = true
```

In the above example replace `nextflow/examples:latest` with any Podman image of your choice.

See [Configuration](#) to learn more about the configuration file and how to use it to configure your pipeline execution.

warning

Nextflow automatically manages the file system mounts whenever a container is launched depending on the process input files. However, when a process input is a symbolic link, the linked file **must** be stored in the same folder where the symlink is located, or a sub-folder of it. Otherwise the process execution will fail because the launched container won't be able to access the linked file.

It is possible to specify a different container image for each process definition in your pipeline script. Let's suppose you have two processes named `hello` and `bye`. You can specify two different container images for them in the Nextflow script as shown below:

```

process hello {
  container 'image_name_1'

  script:
  """
  do this
  """
}

process bye {
  container 'image_name_2'

  script:
  """
  do that
  """
}

```

Alternatively, the same containers definitions can be provided by using the configuration file as shown below:

```

process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

podman {
  enabled = true
}

```

See [Process configuration](#) to learn more about processes configuration.

Podman advanced configuration settings are described in the [podman](#) section in the Nextflow configuration page.

[Sarus](#) is an alternative container runtime to Docker. Sarus works by converting Docker images to a common format that can then be distributed and launched on HPC systems. The user interface to Sarus enables a user to select an image from [Docker Hub](#) and then submit jobs which run entirely within the container.

You need Sarus installed in your execution environment, i.e. your personal computer or a distributed cluster, depending on where you want to run your pipeline.

Sarus converts a docker image to Squashfs layers which are distributed and launched in the cluster. For more information on how to build Sarus images see the [official documentation](#).

The integration for Sarus, at this time, requires you to set up the following parameters in your config file:

```

process.container = "dockerhub_user/image_name:image_tag"
sarus.enabled = true

```

and it will always try to search the Docker Hub registry for the images.

note

if you do not specify an image tag, the `latest` tag will be fetched by default.

It is possible to specify a different Sarus image for each process definition in your pipeline script. For example, suppose you have two processes named `hello` and `bye`. You can specify two different Sarus images specifying them in the configuration file as shown below:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

sarus {
  enabled = true
}
```

See [Process configuration](#) to learn more about processes configuration.

[Shifter](#) is an alternative container runtime to Docker. Shifter works by converting Docker images to a common format that can then be distributed and launched on HPC systems. The user interface to Shifter enables a user to select an image from [Docker Hub](#) and then submit jobs which run entirely within the container.

You need Shifter and Shifter image gateway installed in your execution environment, i.e: your personal computer or the entry node of a distributed cluster. In the case of a distributed cluster, you should have Shifter installed on all of the compute nodes and the `shifterimg` command should also be available and Shifter properly setup to access the Image gateway, for more information see the [official documentation](#).

Shifter converts a Docker image to squashfs layers which are distributed and launched in the cluster. For more information on how to build Shifter images see the [official documentation](#).

The integration for Shifter requires you to set up the following parameters in your config file:

```
process.container = "dockerhub_user/image_name:image_tag"
shifter.enabled = true
```

Shifter will search the Docker Hub registry for the images. If you do not specify an image tag, the `latest` tag will be fetched by default.

It is possible to specify a different Shifter image for each process definition in your pipeline script. For example, suppose you have two processes named `hello` and `bye`. You can specify two different Shifter images specifying them in the configuration file as shown below:

```

process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

shifter {
  enabled = true
}

```

See [Process configuration](#) to learn more about processes configuration.

[Singularity](#) is an alternative container runtime to Docker. The main advantages of Singularity are that it can be used without root privileges and it doesn't require a separate daemon process. These, along with other features such as support for autofs mounts, makes Singularity better suited to the requirements of HPC workloads. Singularity is able to use existing Docker images and can pull from Docker registries.

You will need Singularity installed on your execution environment e.g. your computer or a distributed cluster, depending on where you want to run your pipeline.

Singularity makes use of a container image file, which physically contains the container. Refer to the [Singularity documentation](#) to learn how create Singularity images.

Singularity allows paths that do not currently exist within the container to be created and mounted dynamically by specifying them on the command line. However this feature is only supported on hosts that support the [Overlay file system](#) and is not enabled by default.

note

Nextflow expects that data paths are defined system wide, and your Singularity images need to be created having the mount paths defined in the container file system.

If your Singularity installation support the "user bind control" feature, enable the Nextflow support for that by defining the `singularity.autoMounts = true` setting in the Nextflow configuration file.

The integration for Singularity follows the same execution model implemented for Docker. You won't need to modify your Nextflow script in order to run it with Singularity. Simply specify the Singularity image file from where the containers are started by using the `-with-singularity` command line option. For example:

```

nextflow run main.nf -with-singularity [singularity image file]

```

Every time your script launches a process execution, Nextflow will run it into a Singularity container created by using the specified image. In practice Nextflow will automatically wrap your processes and launch them by running the `singularity exec` command with the image you have provided.

note

A Singularity image can contain any tool or piece of software you may need to carry out a process execution. Moreover, the container is run in such a way that the process result files are created in the host file system, thus it behaves in a completely transparent manner without requiring extra steps or affecting the flow in your pipeline.

If you want to avoid entering the Singularity image as a command line parameter, you can define it in the Nextflow configuration file. For example you can add the following lines in the configuration file:

```
process.container = '/path/to/singularity.img'  
singularity.enabled = true
```

In the above example replace `/path/to/singularity.img` with any Singularity image of your choice.

See [Configuration](#) to learn more about the configuration file and how to use it to configure your pipeline execution.

note

Unlike Docker, Nextflow does not automatically mount host paths in the container when using Singularity. It expects that the paths are configured and mounted system wide by the Singularity runtime. If your Singularity installation allows user defined bind points, see [Singularity](#) to learn how to enable Nextflow auto mounts.

warning

When a process input is a symbolic link file, make sure the linked file is stored in a host folder that is accessible from a bind path defined in your Singularity installation. Otherwise the process execution will fail because the launched container won't be able to access the linked file.

Changed in version 23.10

Nextflow no longer mounts the home directory when launching a Singularity container. To re-enable the old behavior, set the environment variable `NXF_SINGULARITY_HOME_MOUNT` to `true`.

Changed in version 23.10

Nextflow automatically mounts the required host paths in the container. To re-enable the old behavior, set the environment variable `NXF_SINGULARITY_AUTO_MOUNTS` to `false` or set `singularity.autoMounts=false` in the Nextflow configuration file.

Changed in version 23.10

The execution command for Singularity/Apptainer containers can be set to `run` by means of the environment variable `NXF_SINGULARITY_RUN_COMMAND` (default command is `exec`).

It is possible to specify a different Singularity image for each process definition in your pipeline script. For example, suppose you have two processes named `hello` and `bye`. You can specify two different Singularity images specifying them in the configuration file as shown below:

```
process {
  withName:hello {
    container = 'image_name_1'
  }
  withName:bye {
    container = 'image_name_2'
  }
}

singularity {
  enabled = true
}
```

See [Process configuration](#) to learn more about processes configuration.

Requires Singularity 2.3 or later

Nextflow is able to transparently pull remote container images stored in the [Singularity-Hub](#), [Singularity Library](#), or any Docker compatible registry.

By default when a container name is specified, Nextflow checks if an image file with that name exists in the local file system. If that image file exists, it's used to execute the container. If a matching file does not exist, Nextflow automatically tries to pull an image with the specified name from the Docker Hub.

If you want Nextflow to check only for local file images, prefix the container name with the `file://` pseudo-protocol. For example:

```
process.container = 'file:///path/to/singularity.img'
singularity.enabled = true
```

warning

Use three `/` slashes to specify an **absolute** file path, otherwise the path will be interpreted as relative to the workflow launch directory.

To pull images from the Singularity Hub or a third party Docker registry simply prefix the image name with the `shub://`, `docker://` or `docker-daemon://` pseudo-protocol as required by the Singularity tool. For example:

```
process.container = 'docker://quay.io/biocontainers/multiqc:1.3--py35_2'
singularity.enabled = true
```

You do not need to specify `docker://` to pull from a Docker repository. Nextflow will automatically prepend it to your image name when Singularity is enabled. Additionally, the Docker engine will not work with containers specified as `docker://`.

Nextflow supports the [Singularity Library](#) repository:

```
process.container = 'library://library/default/alpine:3.8'
```

The `library://` pseudo-protocol allows you to import Singularity images from a local Docker installation instead of downloading them from a Docker registry. This feature requires the `singularity` tool to be installed where the workflow execution is launched (as opposed to the compute nodes).

Nextflow caches Singularity images in the `singularity` directory, in the pipeline work directory, by default. However, it is recommended to provide a centralized cache directory using the `NXF_SINGULARITY_CACHEDIR` environment variable or the `singularity.cacheDir` setting in the Nextflow config file.

Nextflow uses the library directory to determine the location of Singularity images. The library directory can be defined using the `singularity.libraryDir` configuration setting or the `NXF_SINGULARITY_LIBRARYDIR` environment variable. The configuration file option overrides the environment variable if both are set.

Nextflow first checks the library directory when searching for the image. If the image is not found it then checks the cache directory. The main difference between the library directory and the cache directory is that the first is assumed to be a read-only container repository, while the latter is expected to be writable path where container images can be added for caching purposes.

warning

When using a compute cluster, the Singularity cache directory must reside in a shared filesystem accessible to all compute nodes.

danger

When pulling Docker images, Singularity may be unable to determine the container size if the image was stored using an old Docker format, resulting in a pipeline execution error. See Singularity documentation for details.

Singularity advanced configuration settings are described in the [singularity](#) section in the Nextflow configuration page.

Conda environments

[Conda](#) is an open source package and environment management system that simplifies the installation and the configuration of complex software packages in a platform agnostic manner.

Nextflow has built-in support for Conda that allows the configuration of workflow dependencies using Conda recipes and environment files.

This allows Nextflow applications to use popular tool collections such as [Bioconda](#) and the [Python Package index](#), while taking advantage of the configuration flexibility provided by Nextflow.

This feature requires the Conda or [Miniconda](#) package manager to be installed on your system.

Nextflow automatically creates and activates the Conda environment(s) given the dependencies specified by each process.

Dependencies are specified by using the `conda` directive, providing either the names of the required Conda packages, the path of a Conda environment yaml file, or the path of an existing Conda environment directory.

note

Conda environments are stored on the file system. By default, Nextflow instructs Conda to save the required environments in the pipeline work directory. The same environment may be created/saved multiple times across multiple executions when using different work directories.

You can specify the directory where the Conda environments are stored using the `conda.cacheDir` configuration property. When using a computing cluster, make sure to use a shared file system path accessible from all compute nodes. See [Configuration](#) for details about Conda configuration.

warning

The Conda environment feature is not supported by executors that use remote object storage as a work directory. For example, AWS Batch.

The use of Conda recipes specified using the `conda` directive needs to be enabled explicitly in the pipeline configuration file (i.e., `nextflow.config`):

```
conda.enabled = true
```

Alternatively, it can be specified by setting the variable `NXF_CONDA_ENABLED=true` in your environment or by using the `-with-conda` command line option.

Conda package names can be specified using the `conda` directive. Multiple package names can be specified by separating them with a blank space. For example:

```
process hello {
  conda 'bwa samtools multiqc'

  script:
  """
  your_command --here
  """
}
```

Using the above definition, a Conda environment that includes BWA, Samtools and MultiQC tools is created and activated when the process is executed.

The usual Conda package syntax and naming conventions can be used. The version of a package can be specified after the package name as shown here `bwa=0.7.15`.

The name of the channel where a package is located can be specified prefixing the package with the channel name as shown here `bioconda::bwa=0.7.15`.

Changed in version 26.04

By default, Nextflow uses the `conda-forge` and `bioconda` channels to resolve Conda packages. You can override this using the `conda.channels` configuration option.

Conda environments can also be defined using one or more Conda environment files. This is a file that lists the required packages and channels structured using the YAML format. For example:

```
name: my-env
channels:
- conda-forge
- bioconda
dependencies:
- star=2.5.4a
- bwa=0.7.15
```

Read the Conda documentation for more details about how to create [environment files](#).

The path of an environment file can be specified using the `conda` directive:

```
process hello {
  conda '/some/path/my-env.yaml'

  script:
  """
  your_command --here
  """
}
```

warning

The environment file name **must** have a `.yaml` or `.yml` extension or else it won't be properly recognized.

Conda environment files can also be used to install Python packages from the [PyPI repository](#), through the `pip` package manager (which must also be explicitly listed as a required package):

```
name: my-env-2
channels:
  - conda-forge
  - bioconda
dependencies:
  - pip
  - pip:
    - numpy
    - pandas
    - matplotlib
```

It is possible to provide dependencies by listing each package name as a separate line in a plain text file. For example:

```
bioconda::star=2.5.4a
bioconda::bwa=0.7.15
bioconda::multiqc=1.4
```

note

Dependency files must be a text file with the `.txt` extension.

The final method for providing packages to Conda is by using [Conda lock files](#).

To generate a lock file from an existing Conda environment, run the following command:

```
conda list --explicit > spec-file.txt
```

If you're using Mamba or Micromamba, use this command instead:

```
micromamba env export --explicit > spec-file.txt
```

You can also download Conda lock files from [Wave](#) container build pages.

These files list every package and its dependencies, so Conda doesn't need to perform dependency resolution. This makes environment setup faster and more reproducible.

Each file includes package URLs and, optionally, an MD5 hash for verifying file integrity:

```
# micromamba env export --explicit
# This file may be used to create an environment using:
# $ conda create --name <env> --file <file>
# platform: linux-64
@EXPLICIT
https://conda.anaconda.org/conda-forge/linux-64/_libgcc_mutex-0.1-
conda_forge.tar.bz2#d7c89558ba9fa0495403155b64376d81
https://conda.anaconda.org/conda-forge/linux-64/libgomp-13.2.0-
h77fa898_7.conda#abf3fec87c2563697defa759dec3d639
https://conda.anaconda.org/conda-forge/linux-64/
_openmp_mutex-4.5-2_gnu.tar.bz2#73aaf86a425cc6e73fcf236a5a46396d
https://conda.anaconda.org/conda-forge/linux-64/libgcc-ng-13.2.0-
h77fa898_7.conda#72ec1b1b04c4d15d4204ecea5978
# .. and so on
```

To use a Conda lock file with Nextflow, set the `conda` directive to the path of the lock file.

note

Conda lock files must be a text file with the `.txt` extension.

If you already have a local Conda environment, you can use it in your workflow specifying the installation directory of such environment by using the `conda` directive:

```
process hello {  
  conda '/path/to/an/existing/env/directory'  
  
  script:  
    """  
    your_command --here  
    """  
}
```

Experimental: may change in a future release.

It is also possible to use [mamba](#) to speed up the creation of Conda environments. See [conda](#) for more information.

When a `conda` directive is used in any `process` definition within the workflow script, Conda tool is required for the workflow execution.

Specifying the Conda environments in a separate configuration [profile](#) is therefore recommended to allow the execution via a command line option and to enhance the workflow portability. For example:

```
profiles {  
  conda {  
    process.conda = 'samtools'  
  }  
  
  docker {  
    process.container = 'biocontainers/samtools'  
    docker.enabled = true  
  }  
}
```

The above configuration snippet allows the execution either with Conda or Docker specifying `-profile conda` or `-profile docker` when running the workflow script.

Conda advanced configuration settings are described in the [conda](#) section on the Nextflow configuration page.

Spack environments

Added in version 23.04

Spack is a package manager for supercomputers, Linux, and macOS. It makes installing scientific software easy. Spack is not tied to a particular language; you can build a software stack in Python or R, link to libraries written in C, C++, or Fortran, and easily swap compilers or target specific CPU microarchitectures.

Nextflow has built-in support for Spack that allows the configuration of workflow dependencies using Spack recipes and environment files.

This allows Nextflow applications to build packages from source on the compute infrastructure in use, while taking advantage of the configuration flexibility provided by Nextflow. At shared compute facilities where Spack has been configured by the administrators, this may result in optimized builds without user intervention. With appropriate options, this also permits end users to customize binary optimizations by themselves.

This feature requires the **Spack** package manager to be installed on your system.

Nextflow automatically creates and activates the Spack environment(s) given the dependencies specified by each process.

Dependencies are specified by using the **spack** directive, providing either the names of the required Spack packages, the path of a Spack environment yaml file or the path of an existing Spack environment directory.

note

Spack always installs the software packages in its own directories, regardless of the Nextflow specifications. The Spack environment created by Nextflow only contains symbolic links pointing to the appropriate package locations, and therefore it is relatively small in size.

You can specify the directory where the Spack environment is stored using the `spack.cacheDir` configuration property (see **spack** for more information). When using a computing cluster, make sure to use a shared file system path accessible from all compute nodes.

warning

The Spack environment feature is not supported by executors that use remote object storage as the work directory, e.g. AWS Batch.

The use of Spack recipes specified using the **spack** directive needs to be enabled explicitly by setting the option shown below in the pipeline configuration file (i.e. `nextflow.config`):

```
spack.enabled = true
```

Alternatively, it can be specified by setting the variable `NXF_SPACK_ENABLED=true` in your environment or by using the `-with-spack` command line option.

Spack package names can be specified using the `spack` directive. Multiple package names can be specified by separating them with a blank space. For example:

```
process hello {  
  spack 'bwa samtools py-multiqc'  
  
  script:  
  '''  
  your_command --here  
  '''  
}
```

Using the above definition, a Spack environment that includes BWA, Samtools, and MultiQC tools is created and activated when the process is executed.

The usual Spack package syntax and naming conventions can be used. The version of a package can be specified after the package name like so: `bwa@0.7.15`.

Spack is able to infer the local CPU microarchitecture and optimize the build accordingly. If you really need to customize this option, you can use the `arch` directive.

Read the Spack documentation for more details about [package specifications](#).

Spack environments can also be defined using one or more Spack environment files. A Spack environment file is a YAML file that lists the required packages and channels. For example:

```
spack:  
  specs:  
    - star@2.5.4a  
    - bwa@0.7.15  
  
  concretizer:  
    unify: true
```

Here, the `concretizer` option is a sensible default for Spack environments.

note

When creating a Spack environment, Nextflow always enables the corresponding Spack view. This is required by Nextflow to locate executables at pipeline runtime.

As mentioned above, Spack is able to guess the target CPU microarchitecture and optimize the build accordingly. If you really need to customize this option, we advise to use the `arch` directive rather than the available options for the Spack environment file.

Read the Spack documentation for more details about how to create [environment files](#).

The path of an environment file can be specified using the `spack` directive:

```
process hello {
  spack '/some/path/my-env.yaml'

  script:
  ...
  your_command --here
  ...
}
```

warning

The environment file name **must** have a `.yaml` extension or else it won't be properly recognized.

If you already have a local Spack environment, you can use it in your workflow specifying the installation directory of such environment by using the `spack` directive:

```
process hello {
  spack '/path/to/an/existing/env/directory'

  script:
  ...
  your_command --here
  ...
}
```

Spack builds most software packages from their source codes, and it does this for a requested package and for all its required dependencies. As a result, Spack builds can last for long, even several hours. This can represent an inconvenience, in that it can significantly lengthen the duration of Nextflow processes. Here we briefly discuss two strategies to mitigate this aspect, and render the usage of Spack more effective.

1. Use a Spack yaml file, and pre-build the environment outside of Nextflow, prior to running the pipeline. Building packages outside of the Nextflow pipeline will work since Spack always installs packages in its own directories, and only creates symbolic links in the environment. This sequence of commands will do the trick in most cases:

```
spack env create myenv /path/to/spack.yaml
spack env activate myenv
spack env view enable
spack concretize -f
spack install -y
spack env deactivate
```

2. Use the Nextflow stub functionality prior to running the pipeline for production. Nextflow will run the stub pipeline, skipping process executions but still setting up the required software packages. This option is useful if it is not possible to write a Spack yaml file for the environment. The stub functionality is described in the [Stub](#) section of the Processes page.

When the `spack` directive is used in any `process` definition within the pipeline script, Spack is required for the pipeline execution.

Specifying the Spack environments in a separate configuration [profile](#) is therefore recommended to allow the execution via a command line option and to enhance the pipeline portability. For example:

```
profiles {  
  spack {  
    process.spack = 'samtools'  
  }  
  
  docker {  
    process.container = 'biocontainers/samtools'  
    docker.enabled = true  
  }  
}
```

The above configuration snippet allows the execution either with Spack or Docker by specifying `-profile spack` or `-profile docker` when running the pipeline script.

note

Nextflow does not allow for fine-grained configuration of the Spack package manager. Instead, this has to be performed directly on the host Spack installation. For more information see the [Spack documentation](#).

Spack advanced configuration settings are described in the [spack](#) section on the Nextflow configuration page.

Wave containers

Wave is a container provisioning service integrated with Nextflow. With Wave, you can build, upload, and manage the container images required by your data analysis workflows automatically and on-demand during pipeline execution.

Wave can be used in any Nextflow pipeline by adding the following snippet to your `nextflow.config` file:

```
wave {
  enabled = true
}

tower {
  accessToken = '<your access token>'
}
```

note

The Seqera Platform access token is not mandatory, but it is recommended in order to access private container repositories and pull public containers without being affected by service rate limits. Credentials should be made available to Wave using the [credentials manager](#) in Seqera Platform.

Wave allows the use of private repositories in your Nextflow pipelines. The repository access keys must be provided in the form of [Seqera Platform credentials](#).

Once the credentials have been created, simply specify your [personal access token](#) in your pipeline configuration file. If the credentials were created in a Seqera Platform organization workspace, specify the workspace ID as well in the config file as shown below:

```
tower {
  accessToken = '<your access token>'
  workspaceId = '<your workspace id>'
}
```

Wave can build and provision container images on-demand for your Nextflow pipelines.

To enable this feature, add the Dockerfile of the container to be built in the [module directory](#) where the pipeline process is defined. When Wave is enabled, it automatically uses the Dockerfile to build the required container, upload to the registry, and it uses the container to carry out the tasks defined in the module.

tip

Make sure the process does not declare a `container` directive, otherwise it will take precedence over the Dockerfile definition.

If a process uses a `container` directive and you still want to build the container using the Dockerfile provided in the module directory, add the following setting to the pipeline config file:

```
wave.strategy = ['dockerfile','container']
```

This setting instructs Wave to prioritize the module Dockerfile over process `container` directives.

note

When building containers, Wave does not support `ADD`, `COPY`, or any other Dockerfile commands that access files in the host file system. To include custom scripts or files, use the module `resources/` directory instead. Wave automatically includes its contents in the container build context.

Wave allows the provisioning of containers based on the `conda` directive used by the processes in your pipeline. This is a quick alternative to building Conda packages in the local computer. Moreover, this enables the use of Conda packages in your pipeline when deploying to cloud-native platforms such as AWS Batch and Kubernetes, which do not allow the easy use of the Conda package manager.

With Wave enabled in your pipeline, simply define the `conda` requirements in the pipeline processes, provided the same process does not also specify a `container` directive or a Dockerfile.

In the latter case, add the following setting to your pipeline configuration:

```
wave.strategy = ['conda']
```

The above setting instructs Wave to use the `conda` directive to provision the pipeline containers and ignore the `container` directive and any Dockerfile(s).

tip

Some configuration options in the `conda` scope are used when Wave is used to build Conda-based containers. For example, the Conda channels and their priority can be set with `conda.channels`:

```
wave.strategy = 'conda'  
conda.channels = 'conda-forge,bioconda'
```

Packages from the [Python Package Index](#) can also be added to a Conda `environment.yml` file. See [Conda and PyPI](#) for more information.

Added in version 26.04

Wave supports different build templates for creating container images from Conda packages. Build templates control how packages are installed and how the final container image is structured.

Multi-stage build templates (`conda/pixi:v1` and `conda/micromamba:v2`) offer several advantages:

- **Smaller images:** Build tools and package managers are excluded from the final image (30-50% size reduction typical).
- **Reproducibility:** Lock files are generated for exact package version tracking.
- **Security:** Fewer binaries in the final image reduces the attack surface.

To use a specific build template, add the following to your configuration:

```

wave.enabled = true
wave.strategy = ['conda']
wave.build.template = 'conda/pixi:v1'

```

Available templates:

Template	Description
conda/ micromamba:v1	Standard Micromamba 1.x single-stage build. The final image includes the package manager. Default when unspecified.
conda/ micromamba:v2	Multi-stage build using Micromamba 2.x. Produces smaller images without the package manager.
conda/pixi:v1	Multi-stage build using Pixi package manager. Produces smaller images with faster dependency resolution.
cran/installr:v1	Build template for R/CRAN packages using installr.

When `wave.build.template` is not specified, Wave uses the standard `conda/micromamba:v1` template.

Added in version 23.10

Nextflow can build Singularity native images on-demand either using `Singularityfile`, Conda packages or Spack packages. The Singularity images are automatically uploaded in a container registry OCI compliant of your choice and stored as a [ORAS artifact](#).

note

This feature requires Singularity (or Apptainer) version supporting the pull of images using the `oras: pseudo-protocol`.

For example to enable the provisioning of Singularity images in your pipeline use the following configuration snippet:

```

singularity.enabled = true
wave.enabled = true
wave.freeze = true
wave.strategy = ['conda']
wave.build.repository = 'docker.io/user/repo'

```

In the above configuration replace `docker.io/user/repo` with a repository of your choice where Singularity image files should be uploaded.

note

When using a private repository, the repository access keys must be provided via the Sequera Platform credentials manager (see [Use cases](#)).

Moreover the access to the repository must be granted in the compute nodes by using the command `singularity remote login`. See Singularity documentation for further details.

note

In order to build Singularity native images, both `singularity.ociAutoPull` and `singularity.ociMode` need to be disabled in the configuration. See [singularity](#) for more information.

Containers built by Wave are uploaded to the Wave default repository hosted on AWS ECR at `195996028523.dkr.ecr.eu-west-1.amazonaws.com/wave/build`. The images in this repository are automatically deleted 1 week after the date of their push.

If you want to store Wave containers in your own container repository use the following settings in the Nextflow configuration file:

```
wave.build.repository = 'example.com/your/build-repo'
wave.build.cacheRepository = 'example.com/your/cache-repo'
```

The first repository is used to store the built container images. The second one is used to store the individual image layers for caching purposes.

The repository access keys must be provided as Seqera Platform credentials (see [Authenticate private repositories](#) above).

Wave allows mirroring, i.e., copying containers used by your pipeline into a container registry of your choice. This allows the pipeline to pull containers from the target registry rather than the original registry.

Mirroring is useful to create an on-demand cache of container images that are co-located in the same region where the pipeline is executed, and therefore optimizing cost and network efficiency.

Include the following settings in your Nextflow configuration to enable this capability:

```
wave.enabled = true
wave.mirror = true
wave.build.repository = '<YOUR REGISTRY>'
tower.accessToken = '<YOUR ACCESS TOKEN>'
```

In the above snippet, replace `<YOUR REGISTRY>` with a container registry of your choice. For example, `quay.io` (no prefix or suffix is needed). The container will be copied with the same name, tag, and checksum in the specified registry. For example, if the source container is `quay.io/biocontainers/bwa:0.7.13--1` and the build repository setting is `example.com`, the resulting container name is `example.com/biocontainers/bwa:0.7.13--1`.

tip

When using a path prefix in the target registry name, it will be prepended to the resulting container name. For example, having `quay.io/biocontainers/bwa:0.7.13--1` as source container and `example.com/library` as build repository, the resulting container will be named `example.com/library/biocontainers/bwa:0.7.13--1`.

The credentials to allow the push of containers to the target repository need to be provided via the Seqera Platform credentials manager. The account used for this is specified by the `tower.accessToken` in the configuration above.

Wave enables the scanning of containers used in your pipelines for security vulnerabilities. If any issues are detected, it will trigger an execution error and provide a report.

To enable this capability add the following settings to your Nextflow configuration file:

```
wave.enabled = true
wave.scan.mode = 'required'
tower.accessToken = '<YOUR ACCESS TOKEN>'
```

Nextflow will only allow the use of containers with no security vulnerabilities when using these settings. You can define the level of accepted vulnerabilities using `wave.scan.allowedLevels`. For example:

```
wave.scan.allowedLevels = 'low,medium'
```

The above setting will allow the use of containers with low and medium vulnerabilities. Accepted values are `low`, `medium`, `high`, and `critical`. See [common vulnerabilities scoring system](#) for more information about these levels.

note

Wave's security scanning applies to any container used in your pipeline, whether it was built by Wave or simply accessed through it. The security scan automatically expires after one week. If a container is accessed again after 7 days or more, the scan will be re-executed.

Wave containers allow you to run your containerized workflow with [Fusion](#).

This enables the use of an object storage bucket such as AWS S3 or Google Cloud Storage as your pipeline work directory, simplifying and speeding up many operations on local, AWS Batch, Google Batch or Kubernetes executions.

See [Fusion](#) for more information.

Wave advanced configuration settings are described in the [Wave](#) section on the Nextflow configuration page.

Workflow diagram

The following diagram is a high-level overview of the Nextflow source code in a similar style as the [workflow diagram](#) visualization for Nextflow pipelines. Each node and subgraph is a class. Arrows depict the flow of data and/or communication between classes.

In general, nodes with sharp corners are "record" classes that simply hold information, while nodes with rounded edges are "function" classes that transform some input into an output. Subgraphs are either long-running classes, i.e. "places where things happen", or one of the other two types for which it was useful to expand and show internal details.

Source: <https://docs.seqera.io/nextflow/developer/diagram>

Configuration scopes

This page provides guidance on defining configuration scopes in the Nextflow runtime.

The Nextflow configuration is defined as a collection of scope classes. Each scope class defines the set of available options, including their name, type, and an optional description for a specific configuration scope.

Scope classes are used to generate a configuration spec, which is in turn used for several purposes:

- Validating config options at runtime (`nextflow run` and `nextflow config`)
- Providing code intelligence in the language server (validation, hover hints, code completion)
- Generating reference documentation (in progress)

Scope classes are also used by the runtime itself as type-safe domain objects. This way, the construction of domain objects from the configuration map is isolated from the rest of the runtime.

A config scope is defined as a class that implements the `ConfigScope` interface. Top-level scope classes must have the `@ScopeName` annotation, which defines the name of the config scope.

For example:

```
package nextflow.hello

import nextflow.config.spec.ConfigScope
import nextflow.config.spec.ScopeName

@ScopeName('hello')
class HelloConfig implements ConfigScope {
}
```

A scope class must provide a default constructor, so that it can be instantiated as an extension point. If no such constructor is defined, the config scope will not be detected by Nextflow. In the above example, this constructor is implicitly defined because no constructors were declared.

The fully-qualified class name (in this case, `nextflow.hello.HelloConfig`) must be included in the list of extension points.

A config option is defined as a field with the `@ConfigOption` annotation. The field name determines the name of the config option.

For example:

```
@ConfigOption
String createMessage
```

The `@ConfigOption` annotation can specify an optional set of types that are valid in addition to the field type. For example, the `fusion.tags` option, which accepts either a `String` or `Boolean`, is declared as follows:

```
@ConfigOption(types=[Boolean])
String tags
```

The field type and any additional types are included in the config spec, allowing them to be used for validation.

The field type can be any Java or Groovy class, but in practice it should be a class that can be constructed from primitive values (numbers, booleans, strings). For example, `Duration` and `MemoryUnit` are standard Nextflow types that can each be constructed from an integer or string.

A nested scope is defined as a field whose type is an implementation of `ConfigScope`. The field name determines the name of the nested scope.

The scope class referenced by the field type defines config options and scopes in the same manner as top-level scope classes. Unlike top-level scopes, nested scope classes do not need to use the `@ScopeName` annotation or provide a default constructor.

See `ExecutorConfig` and `ExecutorRetryConfig` for an example of how a nested scope is defined and constructed.

A placeholder scope is a config scope that applies to a collection of user-defined names.

For example, the `azure.batch.pools` scope allows the user to define a set of named pools, where each pool is configured with a standard set of options such as `autoScale`, `lowPriority`, `maxVmCount`, etc. These options are defined in a placeholder scope with a placeholder name of `<name>`. Thus, the generic name for the `autoScale` option is `azure.batch.pools.<name>.autoScale`.

A placeholder scope is defined as a field with type `Map<String, P>`, where `P` is a nested scope class which defines the scope options. The field should have the `@PlaceholderName` annotation which defines the placeholder name (e.g. `<name>`).

See `AzBatchOpts` and `AzPoolOpts` for an example of how placeholder scopes are defined and constructed.

Top-level scope classes and config options should use the `@Description` annotation to provide a description of the scope or option. This description is included in the config spec, which is in turn used by the language server to provide hover hints.

For example:

```
@ScopeName('hello')
@Description(''
    The `hello` scope controls the behavior of the `nf-hello` plugin.
'')
class HelloConfig implements ConfigScope {

    @ConfigOption
    @Description(''
        Message to print to standard output when a run is initialized.
    '')
    String createMessage
}
```

Nested scopes and placeholder scopes may also use this annotation, but will inherit the description of the top-level scope by default.

The Nextflow runtime adheres the following best practices where appropriate:

- Config options should be declared as public and final, so that the scope class can be used as an immutable domain object.
- Scope classes should define a constructor that initializes each field from a map, casting each map property to the required type and providing default values as needed.

- In cases where an option defaults to an environment variable, the environment map should be provided as an additional constructor argument rather than accessing the system environment directly.
- In cases where an option with a primitive type (e.g., `int`, `float`, `boolean`) can be unspecified without a default value, it should be declared with the equivalent reference type (e.g. `Integer`, `Float`, `Boolean`), otherwise it should use the primitive type.
- In cases where an option represents a path, it should be declared as a `String` and allow clients to construct paths as needed, since path construction may depend on plugins which aren't yet loaded.

For example:

```
import nextflow.config.spec.ConfigOption
import nextflow.config.spec.ConfigScope
import nextflow.config.spec.ScopeName

@ScopeName('hello')
class HelloConfig implements ConfigScope {

    @ConfigOption
    final String createMessage

    @ConfigOption
    final boolean verbose

    HelloConfig() {}

    HelloConfig(Map opts, Map env) {
        this.createMessage = opts.createMessage ?: env.get('NXF_HELLO_CREATE_MESSAGE')
        this.verbose = opts.verbose as boolean
    }
}
```

Nextflow validates the config map after it is loaded. Top-level config scopes are loaded by the plugin system as extension points and converted into a config spec, which is used to validate the config map.

Plugins are loaded after the config is loaded and before it is validated, since plugins can also define config scopes. If a third-party plugin declares a config scope, it must be explicitly enabled in order to validate config options from the plugin. Otherwise, Nextflow will report these options as unrecognized.

Core plugins are loaded automatically based on other config options. Therefore, Nextflow only validates config from a core plugin when that plugin is loaded. Otherwise, any config options from the plugin are ignored -- they are neither validated nor reported as unrecognized.

For example, when the `process.executor` config option is set to `'awsbatch'`, the `nf-amazon` is automatically loaded. In this case, all options in the `aws` config scope will be validated. If the executor is not set to `'awsbatch'`, all `aws` options will be ignored. This way, config files can be validated appropriately without loading additional core plugins that won't be used by the run.

The scope classes themselves can be used to construct domain objects on-demand from the config map. For example, an `ExecutorConfig` can be constructed from the `executor` config scope as follows:

```
new ExecutorConfig( Global.session.config.executor as Map ?: Collections.emptyMap() )
```

note

In practice, it is better to avoid the use of `Global` and provide an instance of `Session` to the client class instead.

Config scope classes can be converted into a config spec with the `SpecNode` class, which uses reflection to extract metadata such as scope names, option names, types, and descriptions. This spec is rendered to JSON and used by the language server at build-time to provide code intelligence such as code completion and hover hints.

The config spec described above can be rendered to Markdown using the `MarkdownRenderer` class. It produces a Markdown document approximating the [Configuration options](#) page.

This approach to docs generation is not yet complete, and has not been incorporated into the build process yet. However, it can be used to check for discrepancies between the source code and docs when making changes. The documentation should match the `@Description` annotations as closely as possible, but may contain additional details such as version notes and extra paragraphs.

Packages

The following pages correspond to packages in the Nextflow source code.

nextflow

The `nextflow` package contains various top-level classes.

nextflow.ast

The `nextflow.ast` package implements the Nextflow language extensions as AST transforms.

nextflow.cache

The `nextflow.cache` package implements the cache database of previously executed tasks.

nextflow.cli

The `nextflow.cli` package implements the command line interface.

nextflow.cloud.aws

The `nextflow.cloud.aws` package implements the AWS Batch executor.

nextflow.cloud.aws.nio

The `nextflow.cloud.aws.nio` package implements the S3 filesystem.

nextflow.cloud.azure

The `nextflow.cloud.azure` package implements the Azure Batch executor.

nextflow.cloud.google

The `nextflow.cloud.google` package implements the Google Batch executor.

nextflow.config

The `nextflow.config` package contains the implementation of the Nextflow configuration.

nextflow.container

The `nextflow.container` package implements the integration with container runtimes.

nextflow.dag

The `nextflow.dag` package implements the workflow DAG and renderers for several diagram formats.

nextflow.executor

The `nextflow.executor` package defines the executor interface and implements several built-in executors.

nextflow.extension

The `nextflow.extension` package implements the channel operators and other extension methods.

nextflow.k8s

The `nextflow.k8s` package implements the Kubernetes executor and the `kuberun` command.

nextflow.plugin

The `nextflow.plugin` package implements the plugin manager.

nextflow.processor

The `nextflow.processor` package implements the execution and monitoring of tasks.

nextflow.scm

The `nextflow.scm` package defines the Git provider interface and implements several built-in Git providers. It also manages local pipeline repositories using a Strategy pattern to support different repository management approaches.

nextflow.script

The `nextflow.script` package implements the parsing and execution of Nextflow scripts.

nextflow.secret

The `nextflow.secret` package defines the secrets provider interface and implements the built-in local secrets store.

nextflow.trace

The `nextflow.trace` package defines the trace observer interface and implements several built-in trace observers.

Source: <https://docs.seqera.io/nextflow/developer/packages>

`nextflow`

The `nextflow` package contains various top-level classes.

Some classes may be excluded from the above diagram for brevity.

The `Nextflow` class implements several methods that are exposed to Nextflow scripts. See [Global namespace](#) for details.

The `Channel` class implements the channel factory methods, and it is exposed directly to Nextflow scripts. See [Channel factories](#) for details.

The `Session` class is the top-level representation of a Nextflow run, or "session". See [nextflow.script](#) for more details about how a `Session` is created.

nextflow.ast

The `nextflow.ast` package implements the Nextflow language extensions as AST transforms.

note

Some classes may be excluded from the above diagram for brevity.

The Nextflow scripting language is essentially Groovy with some extensions, implemented as transformations to the abstract syntax tree (AST). Every Nextflow script is syntactically (but not semantically) valid Groovy.

You can see the effect of Nextflow's AST transforms by using the Nextflow console:

1. Run `nextflow console` to open the console
2. Enter a Nextflow script
3. Execute the script
4. Go to **Script > Inspect AST**

Here is an example:

```
params.str = 'Hello world!'

process split_letters {
  output:
    path 'chunk_*'

  script:
    """
    printf '${params.str}' | split -b 6 - chunk_
    """
}

process convert_to_upper {
  input:
    path x
  output:
    stdout

  script:
    """
    cat $x | tr '[a-z]' '[A-Z]'
    """
}

workflow {
  split_letters | flatten | convert_to_upper | view { it.trim() }
}
```

Here it is after being parsed and de-sugared by Groovy:

```

params.str = 'Hello world!'

process( split_letters( {
  output:
    path('chunk_*')

  script:
    """
    printf '${params.str}' | split -b 6 - chunk_
    """
} ))

process( convert_to_upper( {
  input:
    path(x)
  output:
    stdout

  script:
    """
    cat $x | tr '[a-z]' '[A-Z]'
    """
} ))

workflow({
  split_letters | flatten | convert_to_upper | view { it.trim() }
})

```

Here it is after being transformed by Nextflow (whitespace edited for readability):


```

import static nextflow.Nextflow.*

import org.apache.commons.lang.StringUtils as StringUtils
import groovy.transform.Field as Field
import java.nio.file.Path as Path
import nextflow.Channel as Channel
import nextflow.util.Duration as Duration
import nextflow.util.MemoryUnit as MemoryUnit
import nextflow.io.ValueObject as ValueObject
import nextflow.Channel as channel

@groovy.transform.BaseScript
public class script1677225313239 extends nextflow.script.BaseScript {

    public script1677225313239() {
        nextflow.script.ScriptMeta.get(this).setDsl1ProcessNames(['split_letters', 'convert_to_upper'])
    }

    public script1677225313239(final groovy.lang.Binding context) {
        super.setBinding(context)
        nextflow.script.ScriptMeta.get(this).setDsl1ProcessNames(['split_letters', 'convert_to_upper'])
    }

    public static void main(final java.lang.String[] args) {
        org.codehaus.groovy.runtime.InvokerHelper.runScript(script1677225313239, args)
    }

    @groovy.transform.Generated
    protected java.lang.Object runScript() {
        params.str = 'Hello world!'

        this.process('split_letters', {
            this._out_path('chunk *')
            new nextflow.script.BodyDef(
                {
                    "printf '$params.str' | split -b 6 - chunk_"
                },
                """"\n printf '\${params.str}\` | split -b 6 - chunk_\n """"\n',
                'script',
                [
                    new nextflow.script.TokenValRef('params.str', 8, 13)
                ]
            )
        })

        this.process('convert_to_upper', {
            this._in_path(new nextflow.script.TokenVar('x'))
            this._out_stdout()
            new nextflow.script.BodyDef(
                {
                    "cat $x | tr '[a-z]' '[A-Z]'"
                },
                """"\n cat $x | tr '\[a-z]\` \[A-Z]\`_\n """"\n',
                'script',
                [
                    new nextflow.script.TokenValRef('x', 19, 8)
                ]
            )
        })

        this.workflow({
            new nextflow.script.BodyDef(
                {
                    split_letters | flatten | convert_to_upper | this.view({
                        it.trim()
                    })
                },
                ' split_letters | flatten | convert_to_upper | view { it.trim() }\n',
                'workflow',
                [
                    new nextflow.script.TokenValRef('flatten', 24, 18),
                    new nextflow.script.TokenValRef('split_letters', 24, 3),
                    new nextflow.script.TokenValRef('convert_to_upper', 24, 28)
                ]
            )
        })
    }
}

```

```
}  
    }  
    })  
    )  
    ]  
}
```

`nextflow.cache`

The `nextflow.cache` package implements the cache database of previously executed tasks.

note

Some classes may be excluded from the above diagram for brevity.

The cache database uses [Kryo](#) to serialize and deserialize task data. Each key-value pair in the cache database corresponds to a task. The key is the task hash, and the value consists of (1) the task `TraceRecord`, (2) the `TaskContext`, and (3) the task reference count.

The default cache store is backed by [LevelDB](#) and is stored in `.nextflow/cache/<session-id>` relative to the launch directory.

The cloud cache store is backed by remote object storage such as Amazon S3, Azure Blob Storage, and Google Cloud Storage. It stores each task entry as a separate object.

`nextflow.cli`

The `nextflow.cli` package implements the command line interface.

note

Some classes may be excluded from the above diagram for brevity.

The `Launcher` class is the entrypoint for Nextflow. It uses `JCommander` to parse the command-line arguments. Additionally, there is a class for each subcommand which implements the application logic of that command. By far the most complex command is `CmdRun`.

`nextflow.cloud.aws`

The `nextflow.cloud.aws` package implements the AWS Batch executor.

note

Some classes may be excluded from the above diagrams for brevity.

`nextflow.cloud.aws.nio`

The `nextflow.cloud.aws.nio` package implements the S3 filesystem.

note

Some classes may be excluded from the above diagrams for brevity.

The S3 filesystem translates Java Path API calls into S3 API calls, which allows Nextflow to interact with S3 objects through the same interface for local files.

nextflow.cloud.azure

The `nextflow.cloud.azure` package implements the Azure Batch executor.

note

Some classes may be excluded from the above diagram for brevity.

`nextflow.cloud.google`

The `nextflow.cloud.google` package implements the Google Batch executor.

note

Some classes may be excluded from the above diagram for brevity.

`nextflow.config`

The `nextflow.config` package contains the implementation of the Nextflow configuration.

note

Some classes may be excluded from the above diagram for brevity.

Any command that parses Nextflow config files (`config`, `run`, etc) uses the `ConfigBuilder` to build a `ConfigMap` from a set of config files. The `ConfigBuilder` itself uses a `ConfigParser` to parse the config files.

The Nextflow configuration language is essentially Groovy with some extensions. These extensions are implemented in `ConfigBase` and `ConfigTransformImpl`.

`nextflow.container`

The `nextflow.container` package implements the integration with container runtimes.

note

Some classes may be excluded from the above diagram for brevity.

The `ContainerBuilder` class is the base class for all container runtimes supported by Nextflow. It produces the container wrapper command for a given task run.

Executors that support containerized tasks insert this wrapper command into the task wrapper script (`.command.run`). Executors that are container-native, i.e. that launch the task wrapper itself inside a container, don't need to generate a container wrapper command.

`nextflow.dag`

The `nextflow.dag` package implements the workflow DAG and renderers for several diagram formats.

note

Some classes may be excluded from the above diagram for brevity.

The workflow DAG defines the network of processes, channels, and operators that comprise a workflow. It is produced by the execution of the Nextflow script. See [nextflow.script](#) for more details.

Implementations of the `DagRenderer` interface define how to render the workflow DAG to a particular diagram format. See [Workflow diagram](#) for more details.

Separately, `DagSerializer` produces a faithful machine-readable map of the DAG (every vertex and edge, with no reduction) for transport to external consumers such as Seqera Platform, as opposed to the file-oriented `DagRenderer` implementations.

`nextflow.executor`

The `nextflow.executor` package defines the executor interface and implements several built-in executors.

note

Some classes may be excluded from the above diagram for brevity.

The `Executor` class is the base class for all Nextflow executors. The main purpose of an `Executor` is to submit tasks to an underlying compute environment, such as an HPC scheduler or cloud batch executor. It uses a `TaskMonitor` to manage the lifecycle of all tasks and a `TaskHandler` to manage each individual task. Most executors use the same polling monitor, but each executor implements its own task handler to customize it for a particular compute environment. See [nextflow.processor](#) for more details about these classes.

The built-in executors include the local executor (`LocalExecutor`) and the various grid executors (SLURM, PBS, LSF, etc), all of which extend `AbstractGridExecutor`. The `LocalExecutor` implements both "script" tasks (processes with a `script` or `shell` block) and "native" tasks (processes with an `exec` block).

The `BashWrapperBuilder` is used by executors to generate the wrapper script (`.command.run`) for a task, from a template script called `command-run.txt`, as well as the task configuration and the execution environment.

`nextflow.extension`

The `nextflow.extension` package implements the channel operators and other extension methods.

note

Some classes may be excluded from the above diagram for brevity.

Operators are implemented using the [GPars](#) dataflow library. In general, an operator consumes one or more `DataflowReadChannel`s and produces one or more `DataflowWriteChannel`s. See [Operators](#) for details about each operator.

Other notable classes include `Bolts` and `FilesEx`, which implement various extension methods used throughout the Nextflow codebase. If you see a method that doesn't appear to be implemented by the calling object, it may be implemented in one of these extension classes.

`nextflow.k8s`

The `nextflow.k8s` package implements the Kubernetes executor and the `kuberun` command.

note

Some classes may be excluded from the above diagram for brevity.

The Kubernetes integration uses the K8s HTTP API to interact with K8s clusters, and relies on the `kubectl` command and `~/.kube/config` file for authentication.

`nextflow.plugin`

The `nextflow.plugin` package implements the plugin manager.

note

Some classes may be excluded from the above diagram for brevity.

The plugin system uses the [PF4J](#) library, which allows for extension classes to be loaded at runtime. Each plugin includes a manifest of extension classes, all of which extend or implement some base class in Nextflow. The `Plugins` class can be used to query the available extensions for a given base class. Extensions can be assigned a priority using the `@Priority` annotation, to ensure that certain extensions are used over others when available.

`nextflow.processor`

The `nextflow.processor` package implements the execution and monitoring of tasks.

note

Some classes may be excluded from the above diagram for brevity.

While the `executor` package defines how tasks are submitted to a particular execution backend (such as an HPC scheduler), the `processor` package defines how tasks are created and executed. As such, these packages work closely together, and in fact several components of the `Executor` interface, specifically the `TaskHandler` and `TaskMonitor`, are defined in this package.

The `TaskProcessor` is by far the largest and most complex class in this package. It implements both the dataflow operator for a given process as well as the task execution logic. In other words, it defines the mapping from an abstract process definition with input and output channels into concrete task executions.

A `TaskRun` represents a particular task execution. There is also `TaskBean`, which is a serializable representation of a task. Legends say that `TaskBean` was originally created to support a "daemon" mode in which Nextflow would run on both the head node and the worker nodes, so the Nextflow "head" would need to send tasks to the Nextflow "workers". This daemon mode was never completed, but echoes of it remain (see `CmdNode`, `DaemonLauncher`, and the `nf-ignite` plugin).

When a `TaskProcessor` receives a set of input values, it creates a `TaskRun` and submits it to an `Executor`, which in turn submits the task to a underlying execution backend. The executor's `TaskMonitor` then monitors the status of the task, and when it is completed, returns it to the task processor for finalization. If the task completed successfully, the task processor collects the task outputs and emits them on the corresponding output channels. If the task failed, the task processor will retry it if possible, or else return a task error to the workflow run.

nextflow.scm

The `nextflow.scm` package defines the Git provider interface and implements several built-in Git providers. It also manages local pipeline repositories using a Strategy pattern to support different repository management approaches.

Some classes may be excluded from the above diagram for brevity.

The `AssetManager` uses the **Strategy pattern** to support different ways of managing local pipeline installations:

- **LegacyRepositoryStrategy**: Traditional approach where each project gets a full git clone in `$NXF_HOME/{project}` directory. Only one revision can exist at a time per project.
- **MultiRevisionRepositoryStrategy**: Modern approach that allows multiple revisions to coexist efficiently by:
 - Maintaining a shared bare repository in `$NXF_HOME/.repos/{project}/bare/`
 - Creating lightweight clones for each commit in `$NXF_HOME/.repos/{project}/clones/{commitId}/`
 - Sharing git objects between revisions to minimize disk space

The `AssetManager` automatically selects the appropriate strategy based on:

1. **Environment variable**: `NXF_SCM_LEGACY=true` forces legacy mode
2. **Repository status**: Detected by checking existing repository structure:
 - `UNINITIALIZED`: No repository exists, use Multi-Revision (default)
 - `LEGACY_ONLY`: Only legacy `.git` directory exists, use Legacy
 - `BARE_ONLY`: Only bare repository exists, use Multi-Revision
 - `HYBRID`: Both exist, prefer Multi-Revision

The `RepositoryProvider` class is the base class for all Git providers. It defines how to authenticate with the provider, clone a Git repository, inspect branches and tags, etc. The provider is used by repository strategies to interact with remote Git services.

Central class that manages pipeline assets. Key responsibilities include:

- Project name resolution and validation
- Strategy selection and initialization
- Provider configuration and authentication
- Repository download and checkout operations
- Coordination between strategy and provider

Interface defining for repository management operations:

- `download()`: Download or update a revision
- `checkout()`: Switch to a specific revision
- `drop()`: Delete local copies
- `getLocalPath()`: Get path to working directory
- `getGit()`: Access JGit repository instance

`nextflow.script`

The `nextflow.script` package implements the parsing and execution of Nextflow scripts.

note

Some classes may be excluded from the above diagram for brevity.

The execution of a Nextflow pipeline occurs in two phases. In the first phase, Nextflow parses and runs the script (using the language extensions in [nextflow.ast](#) and [nextflow.extension](#)), which produces the workflow DAG. In the second phase, Nextflow executes the workflow.

note

In DSL1, there was no separation between workflow construction and execution -- dataflow operators were executed as soon as they were constructed. DSL2 introduced lazy execution in order to separate process definition from execution, and thereby facilitate subworkflows and modules.

nextflow.secret

The `nextflow.secret` package defines the secrets provider interface and implements the built-in local secrets store.

note

Some classes may be excluded from the above diagram for brevity.

The default secrets provider simply stores key-value pairs in a local JSON file.

`nextflow.trace`

The `nextflow.trace` package defines the trace observer interface and implements several built-in trace observers.

note

Some classes may be excluded from the above diagram for brevity.

The `TraceObserver` interface defines a set of hooks into the workflow execution, such as when a workflow starts and completes, when a task starts and completes, and when an output file is published. The `Session` maintains a list of all observers and triggers each hook when the corresponding event occurs. Implementing classes can use these hooks to perform custom behaviors. In fact, this interface is used to implement several core features, including the various execution reports, DAG renderer, and the integration with Seqera Platform.

Using the Nextflow Gradle plugin

Nextflow provides a new `plugin create` command that simplifies the creation of Nextflow plugins. This command leverages the [nf-plugin-template](#) project, which uses the [Nextflow Gradle plugin](#) to streamline plugin development.

The [Nextflow Gradle plugin](#) configures default dependencies needed for Nextflow integration and defines Gradle tasks for building, testing, and publishing Nextflow plugins. The Gradle plugin is versioned and published to the [Gradle Plugin Portal](#), allowing developers to manage it like any other dependency. As the plugin ecosystem evolves, the Gradle plugin will enable easier maintenance and adoption of improvements. This page introduces [Nextflow Gradle plugin](#) and how to use it.

Added in version 25.04

The best way to create a plugin with the Nextflow Gradle plugin is to use the `nextflow plugin create` sub-command and create a plugin project based on the [Nextflow plugin template](#). See [Nextflow plugin template](#) for more information about the Nextflow plugin template.

To create Nextflow plugins with the Gradle plugin:

1. Run `nextflow plugin create`.
2. Follow the prompts to add your plugin name, plugin provider, and project path.

note

Your plugin provider is usually your organization. This must match the provider specified when claiming your plugin.

3. Develop your plugin extension points. See [Extension points](#) for more information.
4. Develop your tests. See [Testing a plugin](#) for more information.
5. In the plugin root directory, run `make assemble`.

Plugins can be installed locally without being published.

To install plugins locally:

1. In the plugin root directory, run `make install`.

note

Running `make install` will add your plugin to your `$HOME/.nextflow/plugins` directory.

2. Run your pipeline:

```
nextflow run main.nf -plugins <plugin-name>@<version>
```

Replace `<plugin-name>@<version>` with your plugin name and version.

note

Plugins can also be configured via Nextflow configuration files. See [Using plugins](#) for more information.

Testing your Nextflow plugin requires unit tests and end-to-end tests.

Unit tests

Unit tests are small, focused tests designed to verify the behavior of individual plugin components.

To run unit tests:

1. Develop your unit tests. See [MyObserverTest.groovy](#) in the [plugin template](#) for an example unit test.
2. In the plugin root directory, run `make test`.

End-to-end tests

End-to-end tests are comprehensive tests that verify the behavior of an entire plugin as it would be used in Nextflow pipelines.

End-to-end tests should be tailored to the needs of your plugin, but generally take the form of a small Nextflow pipeline. See the `validation` directory in the [plugin template](#) for an example end-to-end test.

The Nextflow Gradle plugin allows you to publish plugins to the [Nextflow plugin registry](#) from the command line.

When publishing to the registry, your project must include a `README.md` file in the plugin project root directory. This file will be used as the plugin description in the registry.

Required sections:

1. **Summary** - Explain what the plugin does
2. **Get Started** - Setup and configuration instructions
3. **License** - Specify the plugin's license (e.g., Apache 2.0, MIT, GPL)

Optional sections:

- **What's new** - Recent changes or new features
- **Breaking changes** - Incompatible changes users should be aware of

Requirements:

- Content must be meaningful (no placeholder text like TODO, TBD, Lorem ipsum)
- Content must be in English

To publish plugins to the [Nextflow plugin registry](#):

1. [Claim your plugin](#) in the registry.

note

You can claim a plugin even if it doesn't exist in the registry yet.

2. Create a file named `$HOME/.gradle/gradle.properties`, where `$HOME` is your home directory.
3. Add your API key to the file:

```
npr.apiKey=<api-key>
```

Replace `<api-key>` with your plugin registry API key. See [Creating an access token](#) for more information.

4. Run `make release`.

For additional Nextflow Gradle plugin configuration options, see the [Nextflow Gradle plugin](#) repository.

Migrating to the Nextflow plugin registry

The [Nextflow plugin registry](#) is a central repository for Nextflow plugins. It hosts an index of plugin metadata that supports plugin discovery, accessibility, and version tracking.

The [legacy plugin index](#) is deprecated in favor of the Nextflow plugin registry. Starting with Nextflow 25.10, the plugin registry is the only method for downloading plugins, replacing the legacy plugin index.

This page describes the registry's impact on users and developers, and how to migrate existing plugins.

No immediate actions are required for plugin users. The plugin configuration has not changed.

Plugin developers will need to update their plugin to publish to the Nextflow plugin registry instead of the legacy plugin index. The best way to do this is to migrate to the [Nextflow Gradle plugin](#), which simplifies the development process and supports publishing to the plugin registry from the command line.

To migrate an existing Nextflow plugin to the [Nextflow Gradle plugin](#):

1. Remove the following files and folders:

- `buildSrc/`
- `launch.sh`
- `plugins/build.gradle`

2. If your plugin has a `plugins` directory, move the `src` directory to the project root.

note

Plugin sources should be in `src/main/groovy` or `src/main/java`.

3. Replace the contents of `settings.gradle` with the following:

```
rootProject.name = '<plugin-name>'
```

Replace `<plugin-name>` with your plugin name.

4. In the project root, create a new `build.gradle` file with the following configuration:


```
// Plugins
plugins {
    id 'io.nextflow.nextflow-plugin' version '1.0.0-beta.6'
}

// Dependencies (optional)
dependencies {
    <dependency>
}

// Plugin version
version = '<plugin-version>'

nextflowPlugin {
    // Minimum Nextflow version
    nextflowVersion = '<minimum-nextflow-version>'

    // Plugin metadata
    provider = '<provider>'
    className = '<class-name>'
    extensionPoints = [
        '<extension-point>'
    ]
}
```

Replace the following:

- `<dependency>`: (Optional) Your plugins dependency libraries. For example, `commons-io:commons-io:2.18.0`.
- `<plugin-version>`: Your plugin version. For example, `0.5.0`.
- `<minimum-nextflow-version>`: The minimum Nextflow version required to run your plugin. For example, `25.04.0`.
- `<provider>`: Your name or organization. For example, `acme`.
- `<class-name>`: Your plugin class name. For example, `acme.plugin.MyPlugin`.
- `<extension-point>`: Your extension point identifiers that the plugin will implement or expose. For example, `acme.plugin.MyFactory`.

5. Replace the contents of `Makefile` with the following:

```
# Build the plugin
assemble:
    ./gradlew assemble

clean:
    rm -rf .nextflow*
    rm -rf work
    rm -rf build
    ./gradlew clean

# Run plugin unit tests
test:
    ./gradlew test

# Install the plugin into local nextflow plugins dir
install:
    ./gradlew install

# Publish the plugin
release:
    ./gradlew releasePlugin
```

6. Update `README.md` with information about the structure of your plugin.

7. In the plugin root directory, run `make assemble`.

Alternatively, use the `nextflow plugin create` command to re-create your plugin with the plugin template and add your existing plugin code. See [Nextflow plugin template](#) for more information about the plugin template.

The Nextflow Gradle plugin supports publishing plugins from the command line. See [Publishing a plugin](#) for more information.

The legacy plugin index is **closed to new pull requests**. It will be kept up-to-date with the plugin registry and will remain available indefinitely to ensure continued support for older versions of Nextflow.

Spot Instance failures and retries

This page describes changes in how Nextflow handles Spot Instance failures and retries on AWS and Google Cloud, the impact of those changes, and how to configure spot retry behavior for your pipelines. These changes apply to Nextflow 24.10 and later.

Up to version 24.10, Nextflow would silently retry Spot Instance failures up to 5 times when using AWS Batch or Google Batch. These retries were controlled by cloud-specific configuration parameters (e.g., `aws.batch.maxSpotAttempts`) and happened in cloud infrastructure without explicit visibility to Nextflow.

By default, Nextflow would instruct AWS and Google to automatically retry jobs lost to Spot reclamation up to 5 times. Retries were handled by the cloud provider within a Nextflow task. It was often unclear that tasks were restarted as there was no explicit message. Task runtimes and associated cloud costs were increased because they included the runtime of the reclaimed and retried tasks. Due to the high likelihood of reclamation before completion, long-running tasks running on Spot Instances frequently required retries, leading to inefficient allocation of resources and higher costs.

The default Spot reclamation retry setting changed to 0 on AWS and Google. By default, no internal retries are attempted on these platforms. Spot reclamations lead to an immediate failure, exposed to Nextflow in the same way as other generic failures (for example, returning `exit code 1` on AWS). Nextflow treats these failures like any other job failure, unless a retry strategy is configured.

If you rely on silent Spot retries (the previous default behavior), you may now see more tasks fail with the following characteristics:

- **AWS:** Generic failure with `exit code 1`. You may see messages indicating the host machine was terminated.
- **Google:** Spot reclamation typically produces a specific code, but is now surfaced as a recognizable task failure in Nextflow logs.

Since the default for Spot retries is now 0, you must actively enable a retry strategy if you want Nextflow to handle reclaimed Spot Instances automatically.

There are four possible actions.

If you do not configure anything, you will observe more pipeline failures when Spot Instances are reclaimed. This approach provides clearer visibility into failures. Failed tasks can be re-run with the `resume` option. However, frequent task reclamation may lead to a higher failure rate and each retry requires manual intervention.

note

If you resume the pipeline using the `resume` option, it will pick up at the point the pipeline was interrupted and start with a retry of that task.

You can re-enable Spot retries for each provider in your Nextflow config:

```
aws.batch.maxSpotAttempts = 5
google.batch.maxSpotAttempts = 5
```

The above example sets the maximum number of Spot retries to 5 for both AWS and Google.

You can set `maxRetries` to enable Nextflow-level retries for any failure:

```
process.maxRetries = 5
```

The above example sets retries to 5 for any failures across all providers.

This option is only available for AWS Batch.

If you have long-running tasks where progress lost due to Spot reclamation is costly, consider [Fusion Snapshots](#) (if supported by your environment). Fusion Snapshots allow you to resume a partially completed task on a new machine if a Spot Instance is reclaimed, thereby reducing wasted compute time.

Key features of Fusion Snapshots:

- Jobs do not need to start from scratch after reclamation.
- Especially useful for tasks that take many hours or days to complete.
- May significantly reduce costs and run times in high-reclamation environments.

See [Fusion Snapshots for AWS Batch](#) for more information.

Best practices for Spot Instance failures and retries:

- **Evaluate job duration:** If your tasks are very long (multiple hours or more), Spot Instances can cause repeated interruptions. Consider using on-demand instances or Fusion Snapshots.
- **Set sensible retry limits:** If you enable Spot retries, choose a retry count that balances the cost savings of Spot usage against the overhead of restarting tasks.
- **Monitor logs and exit codes:** Failures due to Spot reclamation will now appear in Nextflow logs. Monitor failures and fine-tune your strategy.
- **Consider partial usage of Spot:** Some workflows may mix on-demand instances for critical or long tasks and Spot Instances for shorter, less critical tasks. This can optimize cost while minimizing wasted compute time.

Preparing for strict syntax

This page explains how to update Nextflow scripts and config files to adhere to the [Nextflow language specification](#), which is enforced by the strict syntax parser.

The strict syntax parser is a strict implementation of DSL2. While the legacy DSL2 parser allows any Groovy syntax, the strict parser allows only a subset of Groovy syntax for Nextflow scripts and config files. It is used by the language server and `nextflow lint` to provide more specific error messages when checking Nextflow code.

In Nextflow 25.04 and 25.10, the strict parser is disabled by default when running Nextflow code. You can enable it by setting the `NXF_SYNTAX_PARSER` environment variable to `v2`:

```
export NXF_SYNTAX_PARSER=v2
```

In Nextflow 26.04 and later, the strict parser is enabled by default. You can disable it by setting `NXF_SYNTAX_PARSER` to `v1`:

```
export NXF_SYNTAX_PARSER=v1
```

In the future, the legacy parser will be removed and the strict parser will become the only way to run Nextflow code. You can prepare for the strict parser by rewriting unsupported code with supported patterns -- code that runs with the strict parser will also run with the legacy parser.

Starting in Nextflow 25.10, new language features can only be used with the strict parser. Therefore, it is also important to prepare for the strict parser in order to use new language features (e.g., static typing).

This page describes how to migrate the most common unsupported patterns to comply with the strict parser. The extent of required changes will vary depending on the amount of custom Groovy code used within your scripts and config files.

In Groovy, the `import` declaration can be used to import external classes:

```
import groovy.json.JsonSlurper

def json = new JsonSlurper().parseText(json_file.text)
```

In Nextflow, use the fully qualified name to reference the class:

```
def json = new groovy.json.JsonSlurper().parseText(json_file.text)
```

Some users use classes in Nextflow to define helper functions or custom types. Helper functions should be defined as standalone functions in Nextflow. Custom types should be moved to the `lib` directory.

note

Enums, a special type of class, are supported, but they cannot be included across modules at this time.

note

Record types will be addressed in a future version of the Nextflow language specification.

A Nextflow script may contain any of the following top-level declarations:

- Feature flags
- Include declarations
- Parameter declarations
- Workflows
- Processes
- Functions
- Type definitions
- Output block

Alternatively, a script may contain only statements, also known as a code snippet:

```
println 'Hello world!'
```

Code snippets are treated as an implicit entry workflow:

```
workflow {  
    println 'Hello world!'  
}
```

Script declarations and statements cannot be mixed at the same level. All statements must reside within script declarations unless the script is a code snippet:

```
process hello {  
    // ...  
}  
  
// incorrect -- move into entry workflow  
// println 'Hello world!'  
  
// correct  
workflow {  
    println 'Hello world!'  
}
```

note

Mixing statements and script declarations was necessary in DSL1 and optional in DSL2. However, this pattern is not supported by the strict parser in order to ensure that top-level statements are not executed when the script is included by another script.

In Groovy, variables can be assigned in an expression:

```
hello(x = 1, y = 2)
```

In Nextflow, assignments are allowed only as statements:

```
x = 1
y = 2
hello(x, y)
```

In Groovy, variables can be incremented and decremented in an expression:

```
hello(x++, y--)
```

In Nextflow, use `+=` and `-=` instead:

```
x += 1
y -= 1
hello(x, y)
```

In Groovy, loop statements, such as `for` and `while`, are supported:

```
for (rseqc_module in ['read_distribution', 'inner_distance', 'tin']) {
    if (rseqc_modules.contains(rseqc_module))
        rseqc_modules.remove(rseqc_module)
}
```

In Nextflow, use higher-order functions, such as the `each` method, instead:

```
['read_distribution', 'inner_distance', 'tin'].each { rseqc_module ->
    if (rseqc_modules.contains(rseqc_module))
        rseqc_modules.remove(rseqc_module)
}
```

Lists, maps, and sets provide several functions (e.g., `collect`, `find`, `findAll`, `inject`) for iteration. See [Groovy standard library](#) for more information.

In Groovy, switch statements are used for pattern matching on a value:

```
switch (aligner) {
case 'bowtie2':
    // ...
    break
case 'bwamem':
    // ...
    break
case 'dragmap':
    // ...
    break
case 'snap':
    // ...
    break
default:
    // ...
}
```

In Nextflow, use if-else statements instead:

```

if (aligner == 'bowtie2') {
    // ...
} else if (aligner == 'bwamem') {
    // ...
} else if (aligner == 'dragmap') {
    // ...
} else if (aligner == 'snap') {
    // ...
} else {
    // ...
}

```

In Groovy, the spread operator can be used to flatten a nested list:

```

ch.map { meta, bambai -> [meta, *bambai] }

```

In Nextflow, enumerate the list elements explicitly:

```

// alternative 1
ch.map { meta, bambai -> [meta, bambai[0], bambai[1]] }

// alternative 2
ch.map { meta, bambai ->
    def (bam, bai) = bambai
    [meta, bam, bai]
}

```

In Nextflow DSL1 and DSL2, environment variables can be referenced directly in strings:

```

println "PWD = ${PWD}"

```

Use `System.getenv()` instead:

```

println "PWD = ${System.getenv('PWD')}"

```

Added in version 24.04

The `env()` function should be used instead of `System.getenv()` :

```

println "PWD = ${env('PWD')}"

```

The following patterns are still supported but have been restricted. That is, some syntax variants have been removed.

In Nextflow DSL2, include declarations can have an `addParams` or `params` clause:

```

params.message = 'Hola'
params.target = 'Mundo'

include { sayHello } from './some/module' addParams(message: 'Ciao')

workflow {
    sayHello()
}

```


These clauses are no longer supported by the strict parser. Params should be passed to workflows, processes, and functions as explicit inputs:

```
include { sayHello } from './some/module'

params.message = 'Hola'
params.target = 'Mundo'

workflow {
    sayHello('Ciao', params.target)
}
```

Where the `sayHello` workflow is defined as follows:

```
workflow sayHello {
    take:
        message
        target

    main:
        // ...
}
```

In Groovy, variables can be declared in many different ways:

```
def a = 1
final b = 2
def c = 3, d = 4
def (e, f) = [5, 6]
String str = 'hello'
def Map meta = [:]
```

In Nextflow, variables must be declared with `def` and must not specify a type:

```
def a = 1
def b = 2
def (c, d) = [3, 4]
def (e, f) = [5, 6]
def str = 'hello'
def meta = [:]
```

Added in version 25.10

Local variables can be declared with a type annotation:

```
def a: Integer = 1
def b: Integer = 2
def (c: Integer, d: Integer) = [3, 4]
def (e: Integer, f: Integer) = [5, 6]
def str: String = 'hello'
def meta: Map = [:]
```

Groovy-style type annotations are still supported. However, the language server and `nextflow lint` will automatically convert them to Nextflow-style type annotations when formatting code. Groovy-style type annotations will not be supported in a future version.

Groovy supports a wide variety of strings, including multi-line strings, dynamic strings, slashy strings, multi-line dynamic slashy strings, and more.

Nextflow supports single- and double-quoted strings, multi-line strings, and slashy strings.

Slashy strings cannot be interpolated:

```
def id = 'SRA001'
assert 'SRA001.fastq' =~ /${id}\.f(?:ast)?q/
```

Use a double-quoted string instead:

```
def id = 'SRA001'
assert 'SRA001.fastq' =~ "${id}\\.f(?:ast)?q"
```

Slashy strings cannot span multiple lines:

```
/
Patterns in the code,
Symbols dance to match and find,
Logic unconfined.
/
```

Use a multi-line string instead:

```
"""
Patterns in the code,
Symbols dance to match and find,
Logic unconfined.
"""
```

Dollar slashy strings are not supported:

```
$/
echo "Hello world!"
/$
```

Use a multi-line string instead:

```
"""
echo "Hello world!"
"""
```

In Groovy, there are two ways to perform type conversions or casts:

```
def map = (Map) readJson(json) // soft cast
def map = readJson(json) as Map // hard cast
```

In Nextflow, only hard casts are supported. Use an explicit method to cast a value to a different type if one is available. For example, to parse a string as a number:

```
def x = '42' as Integer
def x = '42'.toInteger() // preferred
```

In Nextflow DSL2, the name of a process `env` input/output can be specified with or without quotes:

```
process my_task {
  input:
  env F00
  env 'BAR'

  // ...
}
```

The strict parser requires the name to be specified with quotes:

```
process my_task {
  input:
  env 'F00'
  env 'BAR'

  // ...
}
```

In Nextflow DSL1 and DSL2, the process `script:` section label can almost always be omitted:

```
process greet {
  input:
  val greeting

  """
  echo '${greeting}!'
  """
}
```

The strict parser requires the `script:` label to be specified unless there are no other sections:

```
process hello {
  """
  echo 'Hello world!'
  """
}

process greet {
  input:
  val greeting

  script:
  """
  echo '${greeting}!'
  """
}
```

Workflow handlers (i.e. `workflow.onComplete` and `workflow.onError`) can be defined in several different ways in a script, but are typically defined as top-level statements and without an equals sign:

```
workflow.onComplete {
  println "Pipeline completed at: $workflow.complete"
  println "Execution status: ${ workflow.success ? 'OK' : 'failed' }"
}
```

The strict parser does not allow statements to be mixed with script declarations, so workflow handlers must be defined in the entry workflow:

```

workflow {
    // ...

    workflow.onComplete = {
        println "Pipeline completed at: $workflow.complete"
        println "Execution status: ${ workflow.success ? 'OK' : 'failed' }"
    }
}

```

Added in version 25.10

Workflow handlers can be specified as sections in the entry workflow:

```

workflow {
    main:
    // ...

    onComplete:
    println "Pipeline completed at: $workflow.complete"
    println "Execution status: ${ workflow.success ? 'OK' : 'failed' }"
}

```

See [Workflow handlers](#) for details.

The following patterns are deprecated, and the strict parser reports warnings for them. These warnings will become errors in the future.

Channel factories should be accessed using the `channel` namespace instead of the `Channel` type:

```

Channel.of(1, 2, 3) // incorrect
channel.of(1, 2, 3) // correct

```

See [channel](#) and [Channel<E>](#) for more information.

In Groovy, a closure with no parameters is assumed to have a single parameter named `it`:

```

ch.map { it * 2 }

```

In Nextflow, the closure parameter should be explicitly declared:

```

ch.map { v -> v * 2 } // correct
ch.map { it -> it * 2 } // also correct

```

The process `shell` section is deprecated. Use the `script` section instead. The strict parser provides error checking to help distinguish between Nextflow variables and Bash variables.

The following patterns are discouraged and may become warnings or errors in future Nextflow versions. The language server can detect these patterns, but does not report them by default.

To enable these checks, set **Nextflow > Error reporting mode** to **paranoid** in the extension settings.

Added in version 25.10

[Legacy parameters](#) can automatically cast CLI parameters to numbers and booleans:

```
params.save_intermeds = true

workflow {
  println "save_intermeds = ${params.save_intermeds ? 'true' : 'false'}"
}
```

```
$ NXF_SYNTAX_PARSER=v1 nextflow run main.nf --save_intermeds false
save_intermeds = false
```

However, this type detection is disabled when using the strict parser. In the above example, `params.save_intermeds` will be set to `'false'` instead of `false`, causing it to be truthy:

```
$ NXF_SYNTAX_PARSER=v2 nextflow run main.nf --save_intermeds false
save_intermeds = true
```

Legacy parameters should not rely on CLI type detection when using the strict parser. Parameters that may be supplied on the command line should be treated as strings:

```
params.save_intermeds = 'true'

workflow {
  println "save_intermeds = ${params.save_intermeds.toBoolean() ? 'true' : 'false'}"
}
```

```
$ NXF_SYNTAX_PARSER=v2 nextflow run main.nf --save_intermeds false
save_intermeds = false
```

Alternatively, use the `params` block to convert CLI parameters based on their type annotations:

```
params {
  save_intermeds: Boolean = true
}

workflow {
  println "save_intermeds = ${params.save_intermeds ? 'true' : 'false'}"
}
```

See [Typed parameters](#) for details.

While `params` can be used anywhere in the pipeline code, they are only intended to be used in the entry workflow and the `output` block.

As a best practice, processes and workflows should receive `params` as explicit inputs:

```

process myproc {
  input:
  val myproc_args

  // ...
}

workflow myflow {
  take:
  myflow_args

  // ...
}

workflow {
  myproc(params.myproc_args)
  myflow(params.myflow_args)
}

```

The process [when](#) section is discouraged. As a best practice, conditional logic should be implemented in the calling workflow (e.g. using an [if](#) statement or [filter](#) operator) instead of the process definition.

See [Configuration](#) for a comprehensive description of the configuration language.

The legacy parser treats config files as Groovy scripts, allowing the use of scripting constructs like variables, helper functions, try-catch blocks, and conditional logic for dynamic configuration:

```

def getHostname() {
  // ...
}

def hostname = getHostname()
if (hostname == 'small') {
  params.max_memory = 32.GB
  params.max_cpus = 8
}
else if (hostname == 'large') {
  params.max_memory = 128.GB
  params.max_cpus = 32
}
}

```

The strict parser only allows config assignments, config blocks, and config includes. Function declarations are not supported. Statements (e.g., variables and if statements) can only be used within closures. The same dynamic configuration can be achieved using a dynamic include:

```

includeConfig ({
  def hostname = // ...
  if (hostname == 'small')
    return 'small.config'
  else if (hostname == 'large')
    return 'large.config'
  else
    return '/dev/null'
}())

```

The include source is a closure that is immediately invoked. It includes a different config file based on the return value of the closure. Including `/dev/null` is equivalent to including nothing.

Each conditional configuration is defined in a separate config file:

```

// small.config
params.max_memory = 32.GB
params.max_cpus = 8

```

```
// large.config
params.max_memory = 128.GB
params.max_cpus = 32
```

The legacy parser allows config settings to be referenced like variables:

```
google.location = "us-west1"
google.batch.subnetwork = "regions/${google.location}/subnetworks/default"
```

The strict parser does not support this. Only params can be referenced as variables:

```
params.location = "us-west1"

google.location = params.location
google.batch.subnetwork = "regions/${params.location}/subnetworks/default"
```

There are two ways to preserve Groovy code:

- Move the code to the `lib` directory
- Create a plugin

Any Groovy code can be moved into the `lib` directory, which supports the full Groovy language. This approach is useful for temporarily preserving some Groovy code until it can be updated later and incorporated into a Nextflow script. See [The `lib` directory](#) for more information.

For Groovy code that is complicated or if it depends on third-party libraries, it may be better to create a plugin. Plugins can define custom functions that can be included by Nextflow scripts like a script definition. Furthermore, plugins can be easily re-used across different pipelines. See [Developing plugins](#) for more information on how to develop plugins.

Migration notes

This section covers important information for migrating between Nextflow versions. It describes new features, enhancements, breaking changes, deprecations, and other miscellaneous updates that may require action during a migration.

Migrating to 26.04

[Review new features, breaking changes, and deprecations introduced in Nextflow 26.04.](#)

Migrating to 25.10

[Review new features, breaking changes, and deprecations introduced in Nextflow 25.10.](#)

Migrating to 25.04

[Review new features, breaking changes, and deprecations introduced in Nextflow 25.04.](#)

Migrating to 24.10

[Review new features, breaking changes, and deprecations introduced in Nextflow 24.10.](#)

Migrating to 24.04

[Review new features, breaking changes, and deprecations introduced in Nextflow 24.04.](#)

Migrating from DSL1

[Migrate Nextflow pipelines from DSL1 to DSL2.](#)

Source: <https://docs.seqera.io/nextflow/migrations/>

Migrating to 26.04

[Nextflow 26.04](#) was released on April 29, 2026.

Nextflow now has native support for publishing, installing, running, and including remote modules using the [Nextflow registry](#).

The new `module` command can be used to work with remote modules:

- Run a module directly:

```
nextflow module run nf-core/fastqc --meta.id 1 --reads sample1.fastq
```

- Search for modules in registry:

```
nextflow module search bwa
```

- View module information:

```
nextflow module view nf-core/bwa/mem
```

- Install a module:

```
nextflow module install nf-core/bwa/mem
```

Remote modules are installed into the `modules` directory of your project. They can be included in scripts by name instead of a relative path.

- Before:

```
include { BWA_MEM } from '.././../modules/nf-core/bwa/mem'
```

- After:

```
include { BWA_MEM } from 'nf-core/bwa/mem'
```

note

All [nf-core modules](#) are automatically synced to the registry from GitHub. They are available as `nf-core/<name>` as shown above.

See the [Modules overview](#) and the `module` command reference for details.

Records are a new data structure for modeling composite data. They serve as an alternative to tuples. While tuple elements are accessed by index, record fields are accessed by name. This allows you to model data with meaningful names rather than keeping track of the order of tuple elements.

A record can be created using the `record()` function:

```
sample = record(
  id: '1',
  fastq_1: file('1_1.fastq'),
  fastq_2: file('1_2.fastq')
)

println sample.id
println sample.foo // error: unrecognized property `foo`
```

Record types are a way to define custom types for validating records:

```
record Sample {
  id: String
  fastq_1: Path
  fastq_2: Path?
}
```

While records created with the `record()` function can have any set of fields, record types can be used to specify a minimum set of requirements in a particular context, such as a process, function, or workflow:

```
def hello(sample: Sample) {
  println "Hello sample ${sample.id}!"
}

workflow {
  sample1 = record(id: '1', fastq_2: file('1_2.fastq'))
  hello(sample1) // error: `sample1` is missing `fastq_1` field required by Sample

  sample2 = record(id: '1', fastq_1: file('1_1.fastq'))
  hello(sample2) // ok

  sample3 = record(id: '1', single_end: true, fastq_1: file('1_1.fastq'))
  hello(sample3) // also ok
}
```

See [Records](#) for details. See [Migrating to static typing](#) for more information about migrating from tuples to records.

note

Typed processes and typed workflows require the `nextflow.enable.types` feature flag to be enabled in every script that uses them.

Nextflow 26.04 brings full support for static typing with [typed processes](#) and [typed workflows](#).

Typed processes can now declare record inputs and outputs:

```

nextflow.enable.types = true

process FASTQC {
  input:
  record(
    id: String,
    fastq_1: Path,
    fastq_2: Path
  )

  output:
  record(
    id: id,
    fastqc: file('fastqc_logs')
  )

  script:
  // ...
}

```

Typed workflows can declare typed inputs and outputs, and provide first-class support for static typing in dataflow logic:

```

nextflow.enable.types = true

workflow RNASEQ {
  take:
  read_pairs_ch: Channel<Sample>
  transcriptome: Path

  main:
  index = INDEX(transcriptome)
  fastqc_ch = FASTQC(read_pairs_ch)
  quant_ch = QUANT(read_pairs_ch, index)
  samples_ch = fastqc_ch.join(quant_ch, by: 'id')

  emit:
  samples: Channel<AlignedSample> = samples_ch
}

record Sample { /* ... */ }
record AlignedSample { /* ... */ }

```

Several operators have been extended to support static typing and records:

- `combine` can now combine records or tuples
- `groupBy` is a statically-typed alternative to `groupTuple`
- `join` can now join records by matching field

Breaking changes from the [first preview](#):

- The `nextflow.enable.types` feature flag has been replaced with `nextflow.enable.types`.
- The `nextflow.enable.types` feature flag must be enabled in order to use type annotations in workflow takes/emits.
- The syntax for process tuple inputs has been updated:

Before (25.10):

```

input:
(id, fastq_1, fastq_2): Tuple<String,Path,Path>

```

After (26.04):

```
input:
tuple(id: String, fastq_1: Path, fastq_2: Path)
```

- The method signature for the `stageAs` directive was changed from `(filePattern, value)` to `(value, filePattern)`.

See [Migrating to static typing](#) for more information about migrating to static typing. See [Using operators with static typing](#) for best practices on using operators with static typing.

The `hints` process directive allows you to specify executor-specific hints for individual processes.

For example, the AWS Batch executor uses the `consumableResources` hint for [resource-aware scheduling](#):

```
process hello {
  hints 'awsbatch/consumableResources': ['my-license': 1]

  // ...
}
```

Custom executors can use hints to control scheduling behavior in a fine-grained manner over configuration options.

See [hints](#) for details.

The [strict syntax parser](#) is now enabled by default. This change adds the rich error checking from `nextflow lint` to `nextflow run`, and makes it easier to adopt new language features.

The legacy parser can be re-enabled by setting the `NXF_SYNTAX_PARSER` environment variable to `v1`.

Nextflow prints logs in an agent-friendly format when enabled via `NXF_AGENT_MODE=1`:

```
$ NXF_AGENT_MODE=1 nextflow run [...]
[PIPELINE] nf-core/rnaseq 3.14.0 | profile=test,docker
[WORKDIR] /path/to/work
[PROCESS b8/623fc3] sayHello (3)
[PROCESS 5f/761e51] sayHello (2)

[WARN] Task runtime metrics are not reported when using macOS

[ERROR] FASTQC (sample_1)
exit: 127
cmd: fastqc --version
stderr: bash: fastqc: command not found
workdir: /path/to/work/ab/123456

[SUCCESS] completed=10 failed=0 cached=5
```

Agent mode provides minimal structured logging for efficient token usage. It is ideal when using agents to run and debug Nextflow runs.

Nextflow now checks out pipelines separately by revision when running remote pipelines, allowing you to run multiple revisions of a pipeline at the same time.

Nextflow uses the default branch specified by the Git repository when no revision is specified, rather than defaulting to `master`. Since each revision is checked out separately, using a specific revision doesn't prevent the default branch from being used on subsequent runs.

The `manifest.defaultBranch` config option is no longer needed because of this update.

See [Typed parameters](#) for details. See [Migrating to static typing](#) for a migration example.

When the `output` block is defined, Nextflow prints a summary of the workflow outputs at the end of a run.

For example, given the following output block:

```
workflow {
    // ...
}

output {
    samples {
        path { sample -> "${sample.id}" }
        index {
            path 'samples.csv'
        }
    }

    multiqc_report {
        path 'multiqc_report.html'
    }
}
```

Nextflow prints the following summary:

```
$ nextflow -q run main.nf
Outputs:

/path/to/results

samples: samples.csv

multiqc_report: multiqc_report.html
```

You can specify `-output-format json` to produce JSON output instead:

```
$ nextflow -q run main.nf -output-format json
{
  "samples": "/path/to/results/samples.csv",
  "multiqc_report": "/path/to/results/multiqc_report.html"
}
```

The `module run` command prints a summary of the module outputs:

```
$ nextflow -q module run nf-core/fastqc --meta.id 1 --reads sample1.fastq -output-format json
{
  "html": [
    {
      "id": "SRR6357070_1"
    },
    "/path/to/results/SRR6357070_1_fastqc.html"
  ],
  "zip": [
    {
      "id": "SRR6357070_1"
    },
    "/path/to/results/SRR6357070_1_fastqc.zip"
  ]
}
```

See the `run` command reference for details.

The `lint` command now has a `-project-dir` option, which can be used to specify the project root when linting files.

For example, given the following project structure:

```
├── lib
│   └── Utils.groovy
├── modules
│   └── nf-core
│       └── fastqc
│           └── main.nf
│               └── ...
├── workflows
│   └── rnaseq.nf
├── main.nf
└── nextflow.config
```

You can specify `-project-dir` when linting from a location other than the project root:

```
cd workflows
nextflow lint -project-dir ../ rnaseq.nf
```

This option allows the linter to resolve remote module includes and Groovy code in the `lib` directory:

```
// workflows/rnaseq.nf

include { FASTQC } from 'nf-core/fastqc'

workflow RNASEQ {
    // ...

    Utils.validateInput( /* ... */ )

    // ...
}
```

See the [lint](#) command reference for details.

Nextflow can run containers on macOS using [Apple container](#), a lightweight container runtime for Apple silicon machines:

```
process.container = 'nextflow/examples:latest'
appleContainer.enabled = true
```

See [Apple container](#) for details.

Nextflow can download [Seqera Platform datasets](#) using the `seqera` URI scheme.

Seqera dataset URIs are specified as `seqera://<organization>/<workspace>/datasets/<name>[@version]`. For example:

```
params.dataset = 'seqera://seqeralabs/showcase/datasets/sarek_samples'

workflow {
    println file(params.dataset).text
}
```

A Platform access token with appropriate permissions is required to download non-public datasets. It can be specified using the `TOWER_ACCESS_TOKEN` environment variable or the `tower.accessToken` config option.

- The [strict syntax parser](#) is now enabled by default. The legacy parser can be enabled by setting the `NXF_SYNTAX_PARSER` environment variable to `v1`.
- The `nextflow.enable.strict` feature flag is deprecated. It is not needed when using the [strict parser](#).
- The `manifest.defaultBranch` config option is deprecated. Nextflow can automatically detect the default branch when using a remote pipeline.
- The Path `listFiles()` method is deprecated. Use `listDirectory()` instead.
- Several operators have been deprecated. See [Operators](#) for details.
- New config option: `aws.batch.forceGlacierTransfer`
- New config option: `executor.onlyJobState`
- New config option: `google.batch.installOpsAgent`
- New config option: `google.batch.logsPath`
- New config option: `wave.build.conda.baseImage`
- New config option: `wave.build.template`
- New standard library function: `listDirectory()`
- Support [Java 26](#)

Migrating to 25.10

Nextflow 25.10 was released on October 22, 2025.

note

Typed parameters require the [strict parser](#). Set the `NXF_SYNTAX_PARSER` environment variable to `v2` to enable:

```
export NXF_SYNTAX_PARSER=v2
```

The `params` block is a new way to declare pipeline parameters in a Nextflow script:

```
params {
    // Path to input data.
    input: Path

    // Whether to save intermediate files.
    save_intermeds: Boolean = false
}

workflow {
    println "params.input = ${params.input}"
    println "params.save_intermeds = ${params.save_intermeds}"
}
```

This syntax allows you to declare all parameters in one place with explicit type annotations, and it allows Nextflow to validate parameters at runtime.

See [Parameters](#) for details.

[Workflow outputs](#), introduced in Nextflow 24.04 as a preview feature, have been brought out of preview.

If you were using a preview version of workflow outputs, you must remove the `nextflow.preview.output` feature flag when upgrading to Nextflow 25.10. Setting this feature flag in 25.10 will cause your pipeline to fail.

The final version adds the ability to specify a type annotation for each output declaration:

```
workflow {
    main:
        ch_samples = // ...

    publish:
        samples = ch_samples
}

output {
    samples: Channel<Map> {
        path '.'
        index {
            path 'samples.csv'
        }
    }
}
```


See [Migrating to workflow outputs](#) to get started.

note

Static typing requires the [strict parser](#). Set the `NXF_SYNTAX_PARSER` environment variable to `v2` to enable:

```
export NXF_SYNTAX_PARSER=v2
```

Static typing is a new way to write Nextflow pipelines using type annotations. Type annotations help document and validate pipeline code.

```
workflow RNASEQ {
  take:
  reads: Channel<Path>
  index: Value<Path>

  main:
  samples_ch = QUANT( reads, index )

  emit:
  samples: Channel<Path> = samples_ch
}

def isSraId(id: String) -> Boolean {
  return id.startsWith('SRA')
}

// feature flag required for typed processes
nextflow.enable.types = true

process fastqc {
  input:
  (id, fastq_1, fastq_2): Tuple<String,Path,Path>

  output:
  logs = tuple(id, file('fastqc_logs'))

  script:
  """
  mkdir fastqc_logs
  fastqc -o fastqc_logs -f fastq -q ${fastq_1} ${fastq_2}
  """
}
```

The following declarations can be annotated with types:

- Pipeline parameters (the `params` block)
- Workflow takes and emits
- Process inputs and outputs
- Function parameters and returns
- Local variables
- Closure parameters
- Workflow outputs (the `output` block)

Type annotations can refer to any of the [standard types](#).

Some types use generic type parameters to work with different data types in a type-safe way. For example:

- `List<E>` and `Channel<E>` use the generic type `E` to specify the element type in the list or channel

- `Map<K,V>` uses the generic types `K` for the key type and `V` for the value type in the map

The following examples show types with type parameters:

```
// List<E> where E is String
def sequences: List<String> = ['ATCG', 'GCTA', 'TTAG']

// List<E> where E is Path
def fastqs: List<Path> = [file('sample1.fastq'), file('sample2.fastq')]

// Map<K,V> where K is String and V is Integer
def readCounts: Map<String,Integer> = [sample1: 1000, sample2: 1500]

// Channel<E> where E is Path
def ch_bams: Channel<Path> = channel.fromPath('*.bam')
```

Type annotations can be appended with `?` to denote values can be `null`:

```
def x_opt: String? = null
```

In the type system, queue channels are represented as `Channel`, while value channels are represented as `Value`. To make the terminology clearer and more concise, queue channels are now called dataflow channels (or simply channels), and value channels are now called dataflow values.

warning

Since Nextflow-style type annotations are new in Nextflow 25.10, formatting code with Groovy-style type annotations will make it incompatible with previous versions of Nextflow. If you need your code to remain compatible with versions prior to 25.10, run the formatter with Nextflow 25.04:

```
NXF_VER=25.04.8 nextflow lint -format .
```

See [Migrating to static typing](#) for details.

The Nextflow CLI has two new commands: `nextflow auth` and `nextflow launch`. Together, they allow you to authenticate with Seqera Platform and launch pipelines from the command line. This approach streamlines the previous workflow of using `nextflow run` with the `-with-tower` option.

```
# Authenticate with Seqera Cloud
nextflow auth login

# Check authentication status
nextflow auth status

# Configure Seqera Platform settings
nextflow auth config

# Launch a pipeline in Seqera Platform
nextflow launch nextflow-io/hello
```

See [auth](#) and [launch](#) for details.

Nextflow now uses the [Nextflow plugin registry](#) to download plugins in a more efficient and scalable manner.

The legacy plugin index can still be used by setting the `NXF_PLUGINS_REGISTRY_URL` environment variable:

```
export NXF_PLUGINS_REGISTRY_URL="https://raw.githubusercontent.com/nextflow-io/plugins/main/plugins.json"
```

note

Plugin developers will not be able to submit PRs to the legacy plugin index once the plugin registry is generally available. Plugins should be updated to publish to the Nextflow plugin registry using the [Nextflow Gradle plugin](#) instead of the legacy plugin index. See [plugin registry](#) for details.

note

The new syntax for workflow handlers requires the [strict parser](#). Set the `NXF_SYNTAX_PARSER` environment variable to `v2` to enable:

```
export NXF_SYNTAX_PARSER=v2
```

The workflow `onComplete` and `onError` handlers were previously defined by calling `workflow.onComplete` and `workflow.onError` in the pipeline script. You can now define handlers as `onComplete` and `onError` sections in an entry workflow:

```
workflow {
  main:
  // ...

  onComplete:
  println "workflow complete"

  onError:
  println "error: ${workflow.errorMessage}"
}
```

This syntax is simpler and easier to use with the [strict parser](#). See [Workflow handlers](#) for details.

The [strict parser](#) allows dynamic process directives to be specified without a closure:

```
process hello {
  queue (entries > 100 ? 'long' : 'short')

  input:
  tuple val(entries), path('data.txt')

  script:
  """
  your_command --here
  """
}
```

Dynamic process settings in configuration files must still be specified with closures.

See [Dynamic directives](#) for details.

You can now customize the date and time format used in notifications, reports, and console output using the `NXF_DATE_FORMAT` environment variable:

```
# Default format
# e.g., 11-Aug-2016 09:40:20

# Use ISO format with timezone
export NXF_DATE_FORMAT="iso"
# e.g., 2016-08-11T09:40:20+02:00

# Use custom format
export NXF_DATE_FORMAT="yyyy-MM-dd HH:mm"
# e.g., 2016-08-11 09:40
```

This feature addresses previous inconsistencies in timestamp representations.

- The `google-lifesciences` executor is no longer supported because the Google Cloud Life Sciences API was discontinued in July 2025. Use the `google-batch` executor instead.
- The AWS Java SDK used by Nextflow was upgraded from v1 to v2, which introduced some breaking changes to the `aws.client` config options. See [AWS Java SDK v2](#) for details.
- The `nextflow.config.schema` package was renamed to `nextflow.config.spec`. Plugin developers that define custom [configuration scopes](#) will need to update their imports accordingly.
- The legacy type detection of CLI parameters is disabled when using the strict parser (`NXF_SYNTAX_PARSER=v2`). [Legacy parameters](#) should not rely on legacy type detection. Alternatively, use the new `params` block to convert CLI parameters based on their type annotations. Legacy type detection can be disabled globally by setting the environment variable `NXF_DISABLE_PARAMS_TYPE_DETECTION=true`.
- The use of workflow handlers in the configuration file has been deprecated. You should define workflow handlers in the pipeline script or a plugin instead. See [Workflow handlers](#) for details.
- The `nextflow.enable.configProcessNamesValidation` feature flag is deprecated. It was originally introduced to suppress false warnings for process selectors targeting conditional processes. The [strict parser](#) now validates process selectors without producing false warnings.
- New config option: `docker.registryOverride`
- New config options: `wave.build.compression.*`
- New executor: Fujitsu Technical Computing Suite (`tcs`)
- Support Java 25

Migrating to 25.04

[Nextflow 25.04](#) was released on May 8, 2025.

The strict syntax parser is a strict implementation of Nextflow DSL2 based on the [Nextflow language specification](#). Originally introduced by the [Nextflow language server](#) alongside Nextflow 24.10, the strict parser is now available in the Nextflow CLI.

The strict parser is disabled by default. Enable it by setting the environment variable `NXF_SYNTAX_PARSER=v2`. See [strict parser](#) for details.

The `nextflow lint` command checks Nextflow scripts and config files for errors using the [strict parser](#). It can also format Nextflow files using the same formatter as the Nextflow language server. See [lint](#) for details.

The third preview of workflow outputs introduces the following breaking changes from the [previous version](#):

- The `publish:` section can only be specified in the entry workflow.
- Workflow outputs in the `publish:` section are assigned instead of using the `>>` operator. Output names must be valid identifiers.
- By default, output files are published to the base output directory, rather than a subdirectory corresponding to the output name.
- The syntax for dynamic publish paths has changed. Instead of defining a closure that returns a closure with the `path` directive, the outer closure should use the `>>` operator to publish individual files. See [Publishing files](#) for details.
- The `mapper` index directive has been removed. Use a `map` operator in the workflow body instead.

See [Workflow outputs](#) to get started.

[Topic channels](#), introduced in Nextflow 24.04 as a preview feature, have been brought out of preview, which means that they can be used without the `nextflow.preview.topic` feature flag.

This release introduces built-in provenance tracking, also known as data lineage. When `lineage.enabled` is set to `true` in your configuration, Nextflow will record every workflow run, task execution, output file, and the links between them.

You can explore this lineage from the command line using the [lineage](#) command. Additionally, you can refer to files in the lineage store from a Nextflow script using the `lid://` path prefix as well as the [fromLineage](#) channel factory.

See the [Data lineage](#) guide to get started.

Simplified plugin development

The `nextflow plugin create` sub-command creates the scaffold for a Nextflow plugin using the official [Nextflow plugin template](#). It is simpler and easier to maintain than the previous plugin boilerplate.

See the [Using the Nextflow Gradle plugin](#) guide for details.

Previously, the `nextflow inspect` command included all processes that were invoked in a preview run. Now, the `inspect` command includes all processes that are included by the entry workflow (directly or indirectly), which has several benefits:

- It includes all processes that could potentially be invoked by the workflow, not just the processes that are invoked for a particular run configuration.
- It's faster, as it doesn't need to evaluate the entry workflow -- only the includes.
- It can be run as `nextflow inspect <pipeline>`, without specifying any parameters or config profiles.

See [inspect](#) to learn more about the `inspect` command.

When specifying a plugin, the plugin version can now be prefixed with `~` to pin the major and minor version while allowing the latest patch release. This syntax makes it possible to pin the plugin version while automatically using new patch releases.

See [Plugins](#) for details.

Nextflow will terminate the run if the thread pool responsible for publishing files takes too long. Previously, this timeout was reported as a warning. Now, it is reported as an error.

The previous behavior can be restored with the following config setting:

```
workflow.output.ignoreErrors = true
```

- Nextflow now requires Java 17 or newer. See [Requirements](#) for instructions to install Java 17.
- The [HyperQueue](#) executor now requires HyperQueue 0.20.0 or later.
- The process `shell` section has been deprecated. See [Shell](#) for details.
- Nextflow will report a warning if it encounters a process `shell` directive that contains newlines. This warning will become an error in a future release.
- The `-with-weblog` CLI option has been deprecated. See [nextflow-io/nf-weblog](#) to learn how to use the `nf-weblog` plugin.
- New config option: `aws.batch.terminateUnschedulableJobs`
- New config option: `azure.batch.jobMaxWallClockTime`
- New config option: `fusion.snapshots`
- New config option: `google.batch.gcsfuseOptions`
- New config option: `google.batch.networkTags`
- New config option: `workflow.output.copyAttributes`
- New environment variable: `NXF_PLUGINS_ALLOWED`
- New plugin extension point: `TraceObserverFactoryV2`
- New standard library function: `env()`
- Support `disk` directive for Azure Batch

Migrating to 24.10

[Nextflow 24.10](#) was released on October 27, 2024.

The [Nextflow language server](#) implements the Language Server Protocol (LSP) for Nextflow scripts and config files. It can be integrated with any editor that functions as an LSP client, and is used by the official [Nextflow VS Code extension](#). See [VS Code integration](#) for details.

The language server parses scripts and config files according to the [Nextflow language specification](#), which is more strict than the Nextflow CLI. See [Preparing for strict syntax](#) for details.

The second preview of workflow outputs introduces the following breaking changes from the [previous version](#):

- The `publish:` section within processes has been removed. Channels should now be published only from workflows, ideally the entry workflow.
- The `directory` output directive has been replaced with the `outputDir` config option and `-output-dir` command line option, which defaults to `results`. Other directives, such as `mode`, have been replaced with config options under `workflow.output.*`.

In other words, only target blocks can be specified in the output block, but target blocks can still specify directives such as `mode`.

- Target names cannot begin or end with a slash (`/`).

See [Workflow outputs](#) to learn more about the workflow output definition.

Nextflow now supports managed identities for the Azure Batch executor. See [Managed identities](#) for details.

The `task` variable in the process definition has two new properties, `task.previousTrace` and `task.previousException`, which allows a task to access the runtime metadata of the previous attempt. See [Dynamic task resources with previous execution trace](#) for details.

- The `all` release of Nextflow has been replaced with the `dist` release, which includes fewer core plugins by default and supports third-party plugins. See [Standalone distribution](#) for details.
- The `nf-ga4gh` plugin has been moved into its own repository, [nextflow-io/nf-ga4gh](#). To continue using the TES executor, you must explicitly declare the `nf-ga4gh` plugin in your Nextflow configuration. Refer to the plugin repository for documentation and updates on GA4GH integrations.
- The [HyperQueue](#) executor now requires HyperQueue 0.17.0 or later.
- The use of [Grape](#) dependencies with the `@Grab` annotation is no longer supported. Code that requires third-party libraries should be migrated to a plugin.

- Automatic spot retries for AWS Batch and Google Batch have been disabled by default. See [Spot Instance failures and retries](#) for details.
- The use of `addParams` and `params` clauses in include declarations is deprecated. See [Preparing for strict syntax](#) for details.
- New config option: `aws.client.requesterPays`
- New config option: `google.batch.autoRetryExitCodes`
- New config option: `google.batch.bootDiskImage`
- New config option: `manifest.contributors`
- New config option: `wave.mirror`
- New config option: `wave.scan.mode`
- New config option: `wave.scan.allowedLevels`
- New config option: `workflow.failOnIgnore`
- Rename `nextflow.publish.retryPolicy` config scope to `workflow.output.retryPolicy`

Migrating to 24.04

[Nextflow 24.04](#) was released on May 20, 2024.

A topic channel is a channel that can receive values from multiple sources, based on a matching name or topic.

Topic channels are particularly useful for collecting metadata from various points in a pipeline without writing all of the channel logic that is normally required (e.g., using the `mix` operator).

See [topic](#) for details.

A process `eval` output is a new type of process output that captures the standard output of an arbitrary shell command.

This output type is a simpler way to capture command outputs, such as the version of a command-line tool, instead of using a `path` or `env` output.

See [Eval output \(eval\)](#) for details.

The `resourceLimits` process directive allows you to define global limits on the resources requested by individual tasks.

This directive is useful for ensuring that a task's resource requirements never exceed system-wide limits. It is especially important for preventing tasks from becoming unschedulable when using retry with [dynamic resources](#).

See [resourceLimits](#) for details.

The `array` directive allows a process to submit jobs as job arrays, a feature used by many batch schedulers to submit many similar jobs in an efficient manner.

tip

On Google Batch, the `array` directive can be used in conjunction with the `cpus`, `memory`, and `machineType` directives to pack multiple tasks onto the same VM.

See [array](#) for details.

Workflow outputs are a new way to publish outputs at the workflow level:

```

nextflow.preview.output = true

workflow {
  main:
  ch_fastqc = fastqc(data)
  multiqc(ch_fastqc.collect())

  publish:
  ch_fastqc >> 'qc'
  multiqc.out >> 'summary'
}

output {
  directory 'results'
  mode 'copy'
}

```

This syntax allows you to publish channels in a workflow, instead of publishing file patterns in a process using `publishDir`.

See [Workflow outputs](#) for details.

Nextflow's console output now features colored logs and improved organization to highlight the most important information about a pipeline while it's running.

Nextflow now supports AWS Fargate for the AWS Batch executor. See [AWS Fargate](#) for details.

The `singularity.ociAutoPull` (`apptainer.ociAutoPull`) config option enables OCI auto-pull mode for Singularity (Apptainer). When enabled, this setting allows Singularity (Apptainer) to download and convert Docker images during task execution rather than on the Nextflow head node.

See [singularity](#) and [apptainer](#) for details.

The publishing logic used by `publishDir` and workflow outputs will automatically retry failed publish operations with exponential backoff, providing better resilience against transient errors. The retry behavior can be controlled using the `nextflow.publish.retryPolicy.*` config settings.

Additionally, the `failOnError` option of the `publishDir` directive is now `true` by default. This means that, by default, publish errors (after retries) will cause the pipeline to fail. To restore the previous behavior, set `failOnError` to `false` for each `publishDir` directive.

note

The `nextflow.publish` config settings have since been moved to `workflow.output`. See [workflow](#) for details.

The TES executor has been updated to be compliant with version 1.1 of the [TES API specification](#). Notable improvements include support for directory outputs, glob outputs, and the `bin` directory.

note

The `nf-ga4gh` plugin has since been moved into its own repository, [nextflow-io/nf-ga4gh](#). Refer to this repository for documentation and the latest updates on GA4GH integrations for Nextflow.

- Support for [Glacier auto-retrieval](#) was removed. Consider using the AWS CLI to restore Glacier objects before or at the beginning of your pipeline.

- The `sequera` and `defaults` channels were removed from the list of default conda channels used by Wave. They can be restored with the following setting:

```
conda.channels = ['sequera', 'conda-forge', 'bioconda', 'defaults']
```

- New config option: `azure.batch.pools.<name>.lowPriority`
- New config option: `azure.batch.pools.<name>.startTask.script`
- New config option: `azure.batch.pools.<name>.startTask.privileged`
- New config option: `executor.account`
- New config option: `fusion.cacheSize`
- New config option: `google.batch.maxSpotAttempts`
- New config option: `k8s.cpuLimits`
- New config option: `k8s.fuseDevicePlugin`
- New environment variable: `NXF_CLOUDINFO_ENABLED`
- New environment variable: `NXF_CACHE_DIR`
- New plugin extension point: `TaskTipProvider`
- New `pod` directive option: `schedulerName`
- New `pod` directive option: `ttlSecondsAfterFinished`
- New workflow metadata: `workflow.fusion.enabled`
- New workflow metadata: `workflow.fusion.version`
- New workflow metadata: `workflow.preview`
- New workflow metadata: `workflow.wave.enabled`
- Support Google Batch [instance templates](#)
- Support the use of [secrets in the pipeline script](#)
- Upgrade to Groovy 4

Migrating from DSL1

In Nextflow version 22.03.0-edge, DSL2 became the default DSL version. In version 22.12.0-edge, DSL1 support was removed and the Nextflow documentation was updated to use DSL2 by default. Users who are still using DSL1 should migrate their pipelines to DSL2 to use the latest versions of Nextflow. This page describes the differences between DSL1 and DSL2 and how to migrate to DSL2.

In DSL1, a process definition is also the process invocation. Process inputs and outputs are connected to channels using `from` and `into`. You can see a basic Nextflow script written in DSL1 here:

```
nextflow.enable.dsl=1

params.str = 'Hello world!'

process split_letters {
    output:
    file 'chunk_*' into letters

    script:
    """
    printf '${params.str}' | split -b 6 - chunk_
    """
}

process convert_to_upper {
    input:
    file x from letters.flatten()

    output:
    stdout result

    script:
    """
    cat $x | tr '[a-z]' '[A-Z]'
    """
}

result.view { it.trim() }
```

To migrate this code to DSL2, you need to move all of your channel logic throughout the script into a `workflow` definition. Additionally, you must call each process explicitly, passing any input channels as arguments (instead of `from ...`) and receiving any output channels as return values (instead of `into ...`).

See [Workflows](#) to learn how to define a workflow.

You can see the DSL1 Nextflow script from above written in DSL2 here:

```

params.str = 'Hello world!'

process split_letters {
  output:
  path 'chunk_*'

  script:
  """
  printf '${params.str}' | split -b 6 - chunk_
  """
}

process convert_to_upper {
  input:
  path x

  output:
  stdout

  script:
  """
  cat $x | tr '[a-z]' '[A-Z]'
  """
}

workflow {
  split_letters | flatten | convert_to_upper | view { v -> v.trim() }
}

```

In DSL1, a channel can be used as an input only once; to use a channel multiple times, the channel must be forked using the `into` operator. In DSL2, channels are automatically forked when connecting two or more consumers. For example:

```

Channel
  .of('Hello', 'Hola', 'Ciao')
  .set { cheers }

cheers
  .map { v -> v.toUpperCase() }
  .view()

cheers
  .map { v -> v.reverse() }
  .view()

```

Similarly, in DSL2, process outputs can be consumed by multiple consumers automatically, which makes workflow scripts much easier to read and write.

In DSL1, the entire Nextflow pipeline must be defined in a single file. For example, `main.nf`. This restriction becomes cumbersome as a pipeline grows and hinders the sharing and reuse of pipeline components.

DSL2 introduces the ability for scripts to include other scripts. While splitting a pipeline into multiple scripts is not mandatory in DSL2, it is useful for organizing a large pipeline and re-using modules created by others. See [Modules](#) for more information.

note

DSL2 scripts cannot exceed 64 KB in size. Split large DSL1 scripts into multiple smaller scripts to avoid this limit.

- The `set` process input type is no longer supported, use [tuple](#) instead.

- The `set` process output type is no longer supported, use `tuple` instead.
- The `mode flatten` option for process outputs is no longer available. Use the `flatten` operator on the corresponding output channel instead.
- Unqualified value and file elements in a tuple declaration are no longer allowed. Use an explicit `val` or `path` qualifier. For example:

```
process sam2bam {
  input:
  tuple X, 'some-file.sam'

  output:
  tuple X, 'some-file.bam'

  script:
  """
  your_command --in $X some-file.sam > some-file.bam
  """
}
```

Use:

```
process sam2bam {
  input:
  tuple val(X), path('some-file.sam')

  output:
  tuple val(X), path('some-file.bam')

  script:
  """
  your_command --in $X some-file.sam > some-file.bam
  """
}
```

- Channel factories should be accessed through the `channel` namespace instead of the `Channel` type, although they can still be accessed through either method.
- Channel method `bind` has been deprecated in DSL2.
- Channel method `<<` has been deprecated in DSL2.
- Channel factory `create` has been deprecated in DSL2.
- Operator `choice` has been deprecated in DSL2. Use `branch` instead.
- Operator `close` has been deprecated in DSL2.
- Operator `countBy` has been deprecated in DSL2.
- Operator `into` has been deprecated in DSL2, as it is no longer needed.
- Operator `fork` has been renamed to `multiMap`.
- Operator `groupBy` has been deprecated in DSL2. Use `groupTuple` instead.
- Operators `print` and `println` have been deprecated in DSL2. Use `view` instead.
- Operator `route` has been deprecated in DSL2.
- Operator `separate` has been deprecated in DSL2.
- Operator `spread` has been deprecated in DSL2. Use `combine` instead.

An early preview of DSL2 was available in 2020. Note that some of that early DSL2 syntax has since changed.

- The `nextflow.preview.dsl=2` (and `nextflow.enable.dsl=1`) feature flags are no longer needed.
- Anonymous and unwrapped includes are no longer supported. Include each definition by name instead.

For example:

```
include './modules/hello'
include bye from './modules/bye'

workflow {
  hello()
  bye()
}
```

Should be replaced with:

```
include { hello } from './modules/hello'
include { bye } from './modules/bye'

workflow {
  hello()
  bye()
}
```

Using modules

Added in version 26.04

The Nextflow module system allows you to discover, install, and manage reusable modules from centralized registries. This page describes how to use modules in your pipelines.

Search for available modules using the `module search` command:

```
$ nextflow module search alignment
$ nextflow module search "quality control" -limit 10
```

Results include module names and descriptions. Use `-output json` for machine-readable output.

See [module search](#) for the full command reference.

Use the `module install` command to download modules from a registry into your project:

```
$ nextflow module install nf-core/fastqc
$ nextflow module install nf-core/fastqc -version 0.0.0-0c7146d
```

note

Modules mirrored from nf-core do not follow standard semantic versioning. Instead, they use the format `0.0.0-<hash>`, where the suffix is a short portion of the nf-core module's commit hash.

Nextflow stores installed modules in the `modules/` directory and creates a `.module-info` file alongside the module to record installation metadata such as the module checksum and registry URL.

tip

Commit the `modules/` directory to your Git repository to ensure reproducibility.

See [module install](#) for the full command reference.

View all modules installed in your project with the `module list` command:

```
$ nextflow module list
```

The output shows each module's name, installed version, and whether it has been modified locally. Use `-output json` for machine-readable output.

See [module list](#) for the full command reference.

Use the `module view` command to view metadata and a usage template for a module:

```
$ nextflow module view nf-core/fastqc
$ nextflow module view nf-core/fastqc -version 0.0.0-0c7146d
```


The output includes the module's version, URL, description, authors, maintainers, keywords, tools, input/output channels, and a generated usage template. Use `-output json` for machine-readable output.

See [module view](#) for the full command reference.

Modules installed from a registry can be included by name:

```
include { FASTQC } from 'nf-core/fastqc'

workflow {
  reads = channel.fromPath('data/*.fastq')
  reads = reads.map { fastq -> tuple([id: fastq.baseName], fastq) }
  FASTQC(reads)
}
```

Local modules must be included by relative path:

```
include { FASTQC } from './modules/local/fastqc'
```

See [include declarations](#) for a full description of the include syntax.

For ad-hoc tasks or testing, run a module directly without creating a wrapper workflow:

```
$ nextflow module run nf-core/fastqc --meta.id test_sample --reads sample1_R1.fastq.gz
```

tip

Run `nextflow module view` to see the available inputs for a module.

The command automatically downloads the module if it is not already installed. It accepts all standard Nextflow run options (`-profile`, `-resume`, etc.):

```
$ nextflow module run nf-core/fastqc \
  --meta.id test_sample \
  --reads sample1_R1.fastq.gz \
  -with-docker
```

Run a local module by specifying a path starting with `./` or `../`:

```
$ nextflow module run ./modules/local/fastqc/main.nf \
  --meta.id test_sample \
  --reads sample1_R1.fastq.gz \
  -with-docker
```

note

The local module should define a single process, or else the command will fail.

See [module run](#) for the full command reference.

To update a module to a newer version, reinstall it with the desired version:

```
$ nextflow module install nf-core/fastqc -version 0.0.0-c9h0bv4
```

Nextflow automatically verifies module integrity using a checksum stored in the `.module-info` file. If the module has local modifications, it will not be updated. Use the `-force` flag to overwrite local changes:

```
$ nextflow module install nf-core/fastqc -version 0.0.0-c9h0bv4 -force
```

Use the `module remove` command to uninstall a module from your project:

```
$ nextflow module remove nf-core/fastqc
```

By default, Nextflow removes both the module files and the `.module-info` file. Use flags to control this behavior:

- `-keep-files`: Remove the `.module-info` file but keep the module files in the `modules/` directory.
- `-force`: Remove the module directory even if it has no `.module-info` file or has local modifications.

See [module remove](#) for the full command reference.

Developing modules

Learn how to create modules and share them through the Nextflow module registry.

Use the `module create` command to scaffold a new module with the required files:

```
$ nextflow module create myorg/my-module
```

If you omit the name, the command prompts you for details:

```
$ nextflow module create
```

The command creates a module directory with the following files:

- `main.nf`: The module script containing your process definition.
- `meta.yml`: The module spec describing metadata, inputs, and outputs.
- `README.md`: Documentation for the module.

See [module create](#) for the full command reference.

Registry modules follow a standard directory structure:

```
modules/
├── myorg/
│   └── my-module/
│       ├── .module-info      # Integrity checksum
│       ├── README.md        # Documentation
│       ├── main.nf          # Module script
│       ├── meta.yml         # Module spec
│       ├── resources/       # Optional: Module resources
│       └── templates/       # Optional: Process script templates
```

Local modules that are not intended for publishing do not need to follow this structure, although it is recommended as a best practice.

The `main.nf` file contains the process definition. For example, a simple module wrapping FastQC:

```
process FASTQC {
    tag "$meta.id"
    label 'process_medium'

    conda 'bioconda::fastqc=0.12.1'
    container 'biocontainers/fastqc:0.12.1--hdfd78af_0'

    input:
    tuple val(meta), path(reads)

    output:
    tuple val(meta), path("*.html"), emit: html
    tuple val(meta), path("*.zip") , emit: zip

    script:
    """
    fastqc $reads --threads $task.cpus
    """
}
```

Registry modules are subject to the following constraints:

- The module must contain a single script called `main.nf`
- The module must define a single process
- The module must not define any named workflows
- The module may define an entry workflow to override the default behavior of `module run`.
- The module may define any number of functions

Local scripts can define any number of processes, workflows, and functions. As a best practice, each process and named workflow should be defined in its own script.

The `meta.yml` file contains the module's metadata, including its name, version, description, authors, and input/output specifications. The registry uses this file to display module information and generate usage templates.

The `README.md` file provides documentation for the module. It should describe what the module does, the tools it wraps, and any configuration requirements.

Modules can include resource files in the `resources/` directory.

When running the module with [Wave](#), the contents of `resources/` are mounted into the root directory of the task container.

For example, given a module with the following structure:

```
my-module/
├── main.nf
└── resources/
    ├── data/
    │   └── file.txt
    ├── usr/
    │   └── bin/
    │       └── hello.sh
```

The process script can use these files as follows:

```
process hello {
  container 'quay.io/nextflow/bash'

  script:
  """
  cat /data/file.txt
  hello.sh
  """
}
```

Module resources can be used without Wave or containerization, with the following limitations:

- The `nextflow.enable.moduleBinaries` feature flag must be enabled in the pipeline script.
- The pipeline work directory must be in a local or shared file system. Remote object storage is not supported without Wave.
- Only executable scripts in `resources/usr/bin/` are made accessible to the process script.

Modules can include process script [templates](#) in the `templates/` directory.

For example, given a module with the following structure:

```
my-module/
├─ main.nf
└─ templates/
   └─ hello.sh
```

The module's process can use the script template as follows:

```
process hello {
  input:
  val STR

  script:
  template 'hello.sh'
}
```

Use the `module spec` command to generate or update the `meta.yml` file from the module's `main.nf`:

```
$ nextflow module spec myorg/my-module
```

Use `-dry-run` to preview the generated spec without writing to disk:

```
$ nextflow module spec -dry-run myorg/my-module
```

When generating the module spec for the first time, provide required fields directly to avoid `TODO` placeholders in the generated file:

```
$ nextflow module spec \
  -namespace myorg \
  -version 1.0.0 \
  -description "Quality control of raw sequencing reads" \
  -license MIT \
  -author "@myname" \
  ./modules/myorg/my-module
```

When updating an existing module spec, it is incorporated into the new file.

See [module spec](#) for the full command reference.

Use the `module validate` command to check that a module is ready for publishing:

```
$ nextflow module validate myorg/my-module
```

The command verifies that:

- All required files are present (`main.nf`, `meta.yml`, `README.md`).
- The module spec contains all required fields (name, version, description, and license).

See [module validate](#) for the full command reference.

Before publishing, test your module by running it directly:

```
$ nextflow module run myorg/my-module --input 'test-data/*.fastq.gz'
```

The command executes the module as a standalone run, allowing you to verify that inputs are correctly declared, the process runs successfully, and the correct outputs are produced.

For more thorough testing, create a small wrapper workflow that exercises the module:

```
include { MY_MODULE } from './modules/myorg/my-module'

workflow {
  input_ch = channel.fromPath('test-data/*.fastq.gz')
  results_ch = MY_MODULE(input_ch)
  results_ch.view()
}
```

Nextflow module registry

Added in version 26.04

The [Nextflow module registry](#) is a central repository for sharing Nextflow modules. It provides module discovery, version tracking, and integrity verification.

The module registry shares its login, access tokens, and web interface with the [Nextflow plugin registry](#).

A namespace is an organizational prefix (e.g., the namespace `nf-core` owns modules like `nf-core/fastqc` and `nf-core/samtools`). As a namespace member, you can publish new modules and release new versions under that prefix. Each namespace can have multiple members who share publishing rights.

To claim a namespace:

1. Open the [Nextflow module registry](#) in a browser.
2. Log in to [Seqera](#) with your GitHub or Google account, or by providing an email address.

note

If you are logging in for the first time, Seqera sends an authentication link to your email address to complete the login process.

3. Go to **My Resources** and select the **My Namespaces** tab.
4. Select **Claim Namespace**.
5. Enter your **Namespace name**, **Organization**, **Project URL**, and a **Description**.
6. Select **Submit Request**.

The namespace shows as **PENDING REVIEW** until an administrator approves the request. Admin approval is required only once. Once approved, you can publish modules using the `nextflow module publish` command.

An API access token is required to publish modules to the registry.

To create an API access token:

1. Open the [Nextflow module registry](#) in a browser.
2. Log in to [Seqera](#) with your GitHub or Google account, or by providing an email address.

note

If you are logging in for the first time, Seqera sends an authentication link to your email address to complete the login process.

3. Go to **My Resources** and select the **My access tokens** tab.

4. Under **Create New Access Token**, enter a descriptive name for the **Token name** and select a token duration from the **Expiry** drop down.
5. Select **Generate Token**.
6. Copy the token and store it somewhere safe. You cannot view it again after this step.

Once you have your token, see [Publishing modules](#) for instructions on how to use it.

View the modules you have published in the registry:

1. Open the [Nextflow module registry](#) in a browser.
2. Log in and go to **My Resources**.
3. Select the **My Modules** tab.

The page lists all modules published under your namespaces, along with their versions and status.

Publishing requires a [claimed namespace](#) and an [access token](#).

Provide your registry token in one of two ways:

Environment variable:

```
$ export NXF_REGISTRY_TOKEN=<token>
```

Nextflow configuration:

```
registry {  
  apiKey = '${MYORG_TOKEN}'  
}
```

Use the `module publish` command to upload a module to the registry:

```
$ nextflow module publish myorg/my-module
```

The module directory must include the following files:

- `main.nf`: The module script.
- `meta.yml`: The module spec with name, version, description, and license.
- `README.md`: Module documentation.

Provide a `namespace/name` reference or a relative path to the module directory.

tip

Before publishing, verify your module is ready:

- Run `module spec` to generate or update `meta.yml` from your `main.nf`.
- Run `module validate` to check that all required files and spec fields are present.

See [Developing modules](#) for a complete guide on creating modules.

See [module publish](#) for the full command reference.

By default, Nextflow uses the public registry at `https://registry.nextflow.io`. Configure alternative or additional registries using the `registry` scope in your Nextflow configuration.

Replace the default registry with your organization's private registry:

```
registry {
  url = 'https://registry.myorg.com'
  apiKey = '${MYORG_TOKEN}'
}
```

Nextflow uses this registry for all module operations (search, install, and publish).

Specify multiple registries as a list. Nextflow queries them in order when searching for or installing a module:

```
registry {
  url = [
    'https://registry.myorg.com',
    'https://private.registry.nextflow.io'
  ]
  apiKey = '${MYORG_TOKEN}'
}
```

In this example, Nextflow searches the private registry first and falls back to the public registry.

note

The `apiKey` authenticates with the primary (first) registry.

tip

Override the target registry with the `-registry` flag:

```
$ export NXF_REGISTRY_TOKEN=<token>
$ nextflow module publish myorg/my-module -registry 'https://private.registry.myorg.com'
```

Working with files

Use the `file()` function to obtain a reference to a file by name:

```
myFile = file('some/path/to/my_file.file')
```

The `file()` function can reference both files and directories.

Use the `files()` function to obtain a list of files. When using the wildcard characters `*`, `?`, `[]` and `{}`, the file name is treated as a [glob](#) pattern, returning all files that match the given pattern, or an empty list if no matching files are found:

```
listOfFiles = files('some/path/*.fa')
```

note

The `file()` function can also be called with a glob pattern, as long as the pattern is intended to match exactly one file.

A double asterisk (`**`) in a glob pattern works like `*` but also searches through subdirectories:

```
deeplyNestedFiles = files('some/path/**/*.fa')
```

By default, wildcard characters do not match directories or hidden files. Use the `hidden` option to include hidden files:

```
listWithHidden = file('some/path/*.fa', hidden: true)
```

Given a file reference, you can use the `resolve()` method or the `/` operator to obtain files relative to that path:

```
def dir = file('s3://bucket/some/data/path')

dir.resolve('sample.bam')           // correct
dir / 'sample.bam'                  // correct (but verbose)
file("$dir/sample.bam")              // correct (but verbose)
"$dir/sample.bam"                    // incorrect
```

The `file()` function returns a [Path](#) object, which has several methods for retrieving metadata about the file:

```
def path = file('/some/path/file.txt')

assert path.baseName == 'file'
assert path.extension == '.txt'
assert path.name == 'file.txt'
assert path.parent == '/some/path'
```

See the [Path](#) reference for the list of available methods.

Reading a file is as easy as using the file's `text` property, which returns the file contents as a string:

```
print myFile.text
```

Similarly, you can write text to a file by assigning it to the file's `text` property:

```
myFile.text = 'Hello world!'
```

This approach overwrites any existing file contents, and implicitly creates the file if it doesn't exist.

tip

The `text` property is shorthand for the `getText()` and `setText()` methods:

```
println myFile.getText()
myFile.setText('Hello world!')
```

warning

The above methods read and write the entire file contents at once, requiring the entire file to be loaded into memory. Consider using a more memory-efficient approach for large files, such as reading/writing the file line by line.

You can use the `readLines()` method to read a text file line by line:

```
file('some/my_file.txt')
  .readLines()
  .each { line ->
    println line
  }
```

The `readLines()` method loads the entire file into memory, so it is not ideal for large files.

You can use the `eachLine()` method to read line by line while only loading one line at a time into memory:

```
count = 0
myFile.eachLine { line ->
  println "line ${count++}: $line"
}
```

The `withReader()` method creates a [Reader](#) that you can use to read the file line by line, or even character by character. It is useful when you don't need to read the entire file.

For example, to read only the first line of a file:

```
myFile.withReader { r ->
  def firstLine = r.readLine()
  println firstLine
}
```

You can use the `append()` method or left shift (`<<`) operator to append text to a file without erasing the existing contents:

```
myFile.append('Add this line\n')
myFile << 'Add a line more\n'
```

For example, the following snippet copies the contents of a source file into a target file, replacing all `u` characters with `x`:

```
sourceFile.eachLine { line ->
  targetFile << line.replaceAll('u', 'x')
}
```

See the [Path](#) reference for the complete list of methods for performing filesystem operations.

You can use the `listFiles()` method to list the contents of a directory:

```
children = file('any/path').list()
children.each { file ->
  println file
}
```

Changed in version 26.04

The `listFiles()` method is deprecated -- use `listDirectory()` instead.

You can use the `eachFile()` method to iterate through the contents of a directory:

```
myDir.eachFile { item ->
  if( item.isFile() ) {
    println "${item.getName()} - size: ${item.size()}"
  }
  else if( item.isDirectory() ) {
    println "${item.getName()} - DIR"
  }
}
```

In general, you should not need to manually copy files, because Nextflow will automatically stage files in and out of the task environment based on the definition of process inputs and outputs. Ideally, any operation which transforms files should be encapsulated in a process, in order to leverage Nextflow's staging capabilities as much as possible.

Nextflow works with many types of remote files and objects using the same interface as for local files. The following protocols are supported:

- HTTP(S)/FTP (`http://`, `https://`, `ftp://`)
- Amazon S3 (`s3://`)
- Azure Blob Storage (`az://`)
- Google Cloud Storage (`gs://`)

To reference a remote file, simply specify the URL when opening the file:

```
pdb = file('http://files.rcsb.org/header/5FID.pdb')
```

It can then be used in the same way as a local file:

```
println pdb.text
```

note

Not all operations are supported for all protocols. For example, writing and directory listing is not supported for HTTP(S) and FTP paths.

note

Additional configuration may be necessary for cloud object storage, such as authenticating with a private bucket. See the documentation for each cloud storage provider for further details.

When a process input file resides on a different file system than the work directory, Nextflow copies the file into the work directory using an appropriate Java SDK.

Remote files are staged in a subdirectory of the work directory with the form `stage-<session-id>/<hash>/<filename>`, where `<hash>` is determined by the remote file path. If multiple tasks request the same remote file, the file will be downloaded once and reused by each task. These files can be reused by resumed runs with the same session ID.

note

Remote file staging can be a bottleneck during large-scale runs, particularly when input files are stored in object storage but need to be staged in a shared filesystem work directory. This bottleneck occurs because Nextflow handles all of these file transfers.

To mitigate this, you can implement a custom process to download the required files, allowing you to stage multiple files efficiently through parallel jobs. Files should be given as a `val` input instead of a `path` input to bypass Nextflow's built-in remote file staging.

Alternatively, use [Fusion](#) with the work directory set to object storage. In this case, tasks can access remote files directly without any prior staging, eliminating the bottleneck.

Processes

In Nextflow, a **process** is a specialized function for executing scripts in a scalable and portable manner.

Here is an example process definition:

```
process hello {
  output:
  path 'hello.txt'

  script:
  """
  echo 'Hello world!' > hello.txt
  """
}
```

See [Process](#) for a full description of the process syntax.

The `script` section defines, as a string expression, the script that is executed by the process.

A process may contain only one script, and if the `script` guard is not explicitly declared, the script must be the final statement in the process definition.

The script string is executed as a [Bash](#) script in the host environment. It can be any command or script that you would normally execute on the command line or in a Bash script. Naturally, the script may only use commands that are available in the host environment.

The script section can be a simple string or a multi-line string. The latter approach makes it easier to write scripts with multiple commands spanning multiple lines. For example:

```
process blast {
  """
  blastp -db $db -query query.fa -outfmt 6 > blast_result
  cat blast_result | head -n 10 | cut -f 2 > top_hits
  blastdbcmd -db $db -entry_batch top_hits > sequences
  """
}
```

As explained in the script tutorial section, strings can be defined using single-quotes or double-quotes, and multi-line strings are defined by three single-quote or three double-quote characters.

There is a subtle but important difference between them. Like in Bash, strings delimited by a `"` character support variable substitutions, while strings delimited by `'` do not.

In the above code fragment, the `$db` variable is replaced by the actual value defined elsewhere in the pipeline script.

warning

Since Nextflow uses the same Bash syntax for variable substitutions in strings, you must manage them carefully depending on whether you want to evaluate a Nextflow variable or a Bash variable.

When you need to access a system environment variable in your script, you have two options.

If you don't need to access any Nextflow variables, you can define your script section with single-quotes:

```
process echo_path {
    '''
    echo "The path is: $PATH"
    '''
}
```

Otherwise, you can define your script with double-quotes and escape the system environment variables by prefixing them with a back-slash `\` character, as shown in the following example:

```
process blast {
    """
    blastp -db \${DB} -query query.fa -outfmt 6 > blast_result
    cat blast_result | head -n $MAX | cut -f 2 > top_hits
    blastdbcmd -db \${DB} -entry_batch top_hits > sequences
    """
}
```

In this example, `$MAX` is a Nextflow variable that must be defined elsewhere in the pipeline script. Nextflow replaces it with the actual value before executing the script. Meanwhile, `$DB` is a Bash variable that must exist in the execution environment, and Bash will replace it with the actual value during execution.

The process script is interpreted by Nextflow as a Bash script by default, but you are not limited to Bash.

You can use your favourite scripting language (Perl, Python, R, etc), or even mix them in the same pipeline.

A pipeline may be composed of processes that execute very different tasks. With Nextflow, you can choose the scripting language that best fits the task performed by a given process. For example, for some processes R might be more useful than Perl, whereas for others you may need to use Python because it provides better access to a library or an API, etc.

To use a language other than Bash, simply start your process script with the corresponding [shebang](#). For example:

```
process perl_task {
    """
    #!/usr/bin/perl

    print 'Hi there!' . '\n';
    """
}

process python_task {
    """
    #!/usr/bin/python

    x = 'Hello'
    y = 'world!'
    print "%s - %s" % (x,y)
    """
}

workflow {
    perl_task()
    python_task()
}
```

tip

Since the actual location of the interpreter binary file can differ across platforms, it is wise to use the `env` command followed by the interpreter name, e.g. `#!/usr/bin/env perl`, instead of the absolute path, in order to make your script more portable.

The `script` section is like a function that returns a string. This means that you can write arbitrary code to determine the script, as long as the final statement is a string.

If-else statements based on task inputs can be used to produce a different script. For example:

```
mode = 'tcoffee'

process align {
  input:
  path sequences

  script:
  if( mode == 'tcoffee' )
    """
    t_coffee -in $sequences > out_file
    """

  else if( mode == 'mafft' )
    """
    mafft --anysymbol --parttree --quiet $sequences > out_file
    """

  else if( mode == 'clustalo' )
    """
    clustalo -i $sequences -o out_file
    """

  else
    error "Invalid alignment mode: ${mode}"
}
```

In the above example, the process will execute one of several scripts depending on the value of the `mode` parameter. By default it will execute the `tcoffee` command.

Process scripts can be externalized to **template** files, which allows them to be reused across different processes and tested independently from the pipeline execution.

A template can be used in place of an embedded script using the `template` function in the script section:

```
process hello {
  input:
  val STR

  output:
  stdout

  script:
  template 'hello.sh'
}

workflow {
  hello('Hello!').view()
}
```


By default, Nextflow looks for the template script in the `templates` directory located alongside the Nextflow script in which the process is defined. An absolute path can be used to specify a different location. However, this practice is discouraged because it hinders pipeline portability.

An example template script is provided below:

```
#!/bin/bash
echo "process started at `date`"
echo $STR
echo "process completed"
```

Variables prefixed with the dollar character (`$`) are interpreted as Nextflow variables when the template script is executed by Nextflow and Bash variables when executed directly. For example, the above script can be executed from the command line by providing each input as an environment variable:

```
STR='Hello!' bash templates/hello.sh
```

The following caveats should be considered:

- Template scripts are recommended only for Bash scripts. Languages that do not prefix variables with `$` (e.g. Python and R) can't be executed directly as a template script.
- Variables escaped with `\$` will be interpreted as Bash variables when executed by Nextflow, but will not be interpreted as variables when executed from the command line. This practice should be avoided to ensure that the template script behaves consistently.
- Template variables are evaluated even if they are commented out in the template script. If a template variable is missing, it will cause the pipeline to fail regardless of where it occurs in the template.

tip

Template scripts are generally discouraged due to the caveats described above. The best practice for using a custom script is to embed it in the process definition at first and move it to a separate file with its own command line interface once the code matures.

Deprecated in version 25.04

Use the `script` section instead.

The `shell` section is a string expression that defines the script that is executed by the process. It is an alternative to the `Script` definition with one important difference: it uses the exclamation mark `!` character, instead of the usual dollar `$` character, to denote Nextflow variables.

This way, it is possible to use both Nextflow and Bash variables in the same script without having to escape the latter, which makes process scripts easier to read and maintain. For example:

```

process hello {
  input:
  val str

  shell:
  '''
  echo "User $USER says !{str}"
  '''
}

workflow {
  channel.of('Hello', 'Hola', 'Bonjour') | hello
}

```

In the above example, `$USER` is treated as a Bash variable, while `!{str}` is treated as a Nextflow variable.

note

- Shell script definitions require the use of single-quote `'` delimited strings. When using double-quote `"` delimited strings, dollar variables are interpreted as Nextflow variables as usual. See [String interpolation](#) for more information.
- Variables prefixed with `!` must always be enclosed in curly brackets, i.e. `!{str}` is a valid variable whereas `!str` is ignored.
- Shell scripts support the use of the [Template](#) mechanism. The same rules are applied to the variables defined in the template script.

The `exec` section executes the given code without launching a job.

For example:

```

process hello {
  input:
  val name

  exec:
  println "Hello Mr. $name"
}

workflow {
  channel.of('a', 'b', 'c') | hello
}

```

will display:

```

Hello Mr. b
Hello Mr. a
Hello Mr. c

```

A native process is very similar to a [function](#). However, it provides additional capabilities such as parallelism, caching, and progress logging.

You can define a command stub, which replaces the actual process command when the `-stub-run` or `-stub` command-line option is enabled:

```

process salmon_index {
  input:
  path transcriptome

  output:
  path 'index'

  script:
  """
  salmon index --threads $task.cpus -t $transcriptome -i index
  """

  stub:
  """
  mkdir index
  touch index/seq.bin
  touch index/info.json
  touch index/refseq.bin
  """
}

```

The `stub` section can be defined before or after the `script` section. When the pipeline is executed with the `-stub-run` option and a process's `stub` is not defined, the `script` section is executed.

This feature makes it easier to quickly prototype the workflow logic without using the real commands. The developer can use it to provide a dummy script that mimics the execution of the real one in a quicker manner. In other words, it is a way to perform a dry-run.

The `input` section defines the input of a process, similar to function arguments. A process may have at most one input section, which must contain at least one input declaration.

The input section follows the syntax shown below:

```

input:
  <input qualifier> <input name>

```

An input declaration consists of a qualifier and a name. The input qualifier defines the type of data to be received. This information is used by Nextflow to apply the semantic rules associated with each qualifier, and handle it properly depending on the target execution platform (grid, cloud, etc).

When a process is invoked in a workflow, it must be provided a channel or dataflow value for each input in the process input section, similar to calling a function with specific arguments. The examples provided in the following sections demonstrate how a process is invoked.

The following input qualifiers are available:

- `val` : Access the input value by name in the process script.
- `path` : Handle the input value as a path, staging the file properly in the execution context.
- `env` : Use the input value to set an environment variable in the process script.
- `stdin` : Forward the input value to the process `stdin` special file.
- `tuple` : Handle a group of input values having any of the above qualifiers.
- `each` : Execute the process for each element in the input collection.

See [process reference](#) for the full list of input methods and options.

The `val` qualifier accepts any data type. It can be accessed in the process script by using the specified input name, as shown in the following example:

```

process echo {
  input:
  val x

  script:
  """
  echo "process job $x"
  """
}

workflow {
  def num = channel.of(1,2,3)
  echo(num)
}

```

In the above example, the process is executed three times: once for each value emitted by the `num` channel. The resulting output is similar to the one shown below:

```

process job 3
process job 1
process job 2

```

note

Processes do not necessarily process items in the order that they are received. In the above example, the value `3` was processed before the others.

note

When the process declares exactly one input, the pipe `|` operator can be used to provide inputs to the process, instead of passing it as a parameter. Both methods have identical semantics:

```

process echo {
  input:
  val x

  script:
  """
  echo "process job $x"
  """
}

workflow {
  channel.of(1,2,3) | echo
}

```

The `path` qualifier allows you to provide input files to the process execution context. Nextflow will stage the files into the process execution directory, and they can be accessed in the script by using the specified input name. For example:

```

process blast {
  input:
  path query_file

  script:
  """
  blastp -query ${query_file} -db nr
  """
}

workflow {
  def proteins = channel.fromPath( '/some/path/*.fa' )
  blast(proteins)
}

```

In the above example, all the files ending with the suffix `.fa` are sent over the channel `proteins`. These files are received by the process, which executes a BLAST query on each of them.

It's worth noting that in the above example, the name of the file in the file-system is not used. You can access the file without even knowing its name, because you can reference it in the process script by the input name.

There may be cases where your task needs to use a file whose name is fixed, i.e. it does not have to change along with the actual provided file. In this case, you can specify a fixed name with the `name` attribute in the input file parameter definition, as shown in the following example:

```

input:
path query_file, name: 'query.fa'

```

or, using a shorter syntax:

```

input:
path 'query.fa'

```

The previous example can be re-written as shown below:

```

process blast {
  input:
  path 'query.fa'

  script:
  """
  blastp -query query.fa -db nr
  """
}

workflow {
  def proteins = channel.fromPath( '/some/path/*.fa' )
  blast(proteins)
}

```

In this example, each file received by the process is staged with the name `query.fa` in a different execution context (i.e. the folder where a task is executed).

tip

This feature allows you to execute the process command multiple times without worrying about the file names changing. In other words, Nextflow helps you write pipeline tasks that are self-contained and decoupled from the execution environment. As a best practice, you should avoid referencing files in your process script other than those defined in your input section.

Channel factories like `channel.fromPath` produce file objects, but a `path` input can also accept a string literal path. The string value should be an absolute path, i.e. it must be prefixed with a `/` character or a supported URI protocol (`file://`, `http://`, `s3://`, etc), and it cannot contain special characters (`\n`, etc).

```
process cat {
  input:
  path x

  script:
  """
  cat $x
  """
}

workflow {
  cat('/some/data/file.txt')
}
```

note

Process `path` inputs have nearly the same interface as described in [Path](#), with one difference which is relevant when files are staged into a subdirectory. Given the following input:

```
path x, name: 'my-dir/file.txt'
```

In this case, `x.name` returns the file name with the parent directory (e.g. `my-dir/file.txt`), whereas normally it would return the file name (e.g. `file.txt`). You can use `x.fileName.name` to get the file name.

A `path` input can also accept a collection of files instead of a single value. In this case, the input variable will be a list.

When the input has a fixed file name and a collection of files is received by the process, the file name will be appended with a numerical suffix representing its ordinal position in the list. For example:

```

process blast {
  input:
  path 'seq'

  script:
  """
  echo seq*
  """
}

workflow {
  def fasta = channel.fromPath( "/some/path/*.fa" ).buffer(size: 3)
  blast(fasta)
}

```

will output:

```

seq1 seq2 seq3
seq1 seq2 seq3
...

```

The target input file name may contain the `*` and `?` wildcards, which can be used to control the name of staged files. The following table shows how the wildcards are replaced depending on the cardinality of the received input collection.

Arity	Name pattern	Staged file names
any	<code>*</code>	named as the source file
one	<code>file*.ext</code>	<code>file.ext</code>
one	<code>file?.ext</code>	<code>file1.ext</code>
one	<code>file???.ext</code>	<code>file01.ext</code>
many	<code>file*.ext</code>	<code>file1.ext</code> , <code>file2.ext</code> , <code>file3.ext</code> , ..
many	<code>file?.ext</code>	<code>file1.ext</code> , <code>file2.ext</code> , <code>file3.ext</code> , ..
many	<code>file???.ext</code>	<code>file01.ext</code> , <code>file02.ext</code> , <code>file03.ext</code> , ..
many	<code>dir/*</code>	named as the source file, created in <code>dir</code> subdirectory
many	<code>dir??/*</code>	named as the source file, created in a progressively indexed subdirectory e.g. <code>dir01/</code> , <code>dir02/</code> , etc.
many	<code>dir*/*</code>	(as above)

The following example shows how a wildcard can be used in the input file definition:

```

process blast {
  input:
    path 'seq?.fa'

  script:
    """
    cat seq1.fa seq2.fa seq3.fa
    """
}

workflow {
  def fasta = channel.fromPath( "/some/path/*.fa" ).buffer(size: 3)
  blast(fasta)
}

```

note

Rewriting input file names according to a named pattern is an extra feature and not at all required. The normal file input syntax introduced in the [Input files \(path \)](#) section is valid for collections of multiple files as well. To handle multiple input files while preserving the original file names, use a variable identifier or the `*` wildcard.

Added in version 23.10

The `arity` option can be used to enforce the expected number of files, either as a number or a range.

For example:

```

input:
path('one.txt', arity: '1')           // exactly one file is expected
path('pair_*.txt', arity: '2')        // exactly two files are expected
path('many_*.txt', arity: '1..*')     // one or more files are expected

```

When a task is executed, Nextflow will check whether the received files for each path input match the declared arity, and fail if they do not. When the arity is `'1'`, the corresponding input variable will be a single file; otherwise, it will be a list of files.

When the input file name is specified by using the `name` option or a string literal, you can also use other input values as variables in the file name string. For example:

```

process grep {
  input:
    val x
    path "${x}.fa"

  script:
    """
    cat ${x}.fa | grep '>'
    """
}

```

In the above example, the input file name is determined by the current value of the `x` input value.

This approach allows input files to be staged in the task directory with a name that is coherent with the current execution context.

tip

In most cases, you won't need to use dynamic file names, because each task is executed in its own directory, and input files are automatically staged into this directory by Nextflow. This behavior guarantees that input files with the same name won't overwrite each other. The above example is useful specifically when there are potential file name conflicts within a single task.

The `env` qualifier allows you to define an environment variable in the process execution context based on the input value. For example:

```
process echo_env {
  input:
  env 'HELLO'

  script:
  '''
  echo "$HELLO world!"
  '''
}

workflow {
  channel.of('hello', 'hola', 'bonjour', 'ciao') | echo_env
}
```

```
hello world!
ciao world!
bonjour world!
hola world!
```

The `stdin` qualifier allows you to forward the input value to the [standard input](#) of the process script. For example:

```
process cat {
  input:
  stdin

  script:
  """
  cat -
  """
}

workflow {
  channel.of('hello', 'hola', 'bonjour', 'ciao')
    | map { v -> v + '\n' }
    | cat
}
```

will output:

```
hola
bonjour
ciao
hello
```

The `tuple` qualifier groups multiple values into a single input definition. Each element in the tuple is associated with a corresponding element in the `tuple` definition. For example:

```

process cat {
  input:
    tuple val(id), path('input.txt')

  script:
    """
    echo "Processing $id"
    cat input.txt > copy
    """
}

workflow {
  channel.of( [1, 'alpha.txt'], [2, 'beta.txt'], [3, 'delta.txt'] ) | cat
}

```

In the above example, the `tuple` input consists of the value `x` and the file `input.txt`.

A `tuple` definition may contain any of the following qualifiers, as previously described: `val`, `env`, `path` and `stdin`. Files specified with the `path` qualifier are treated exactly the same as standalone `path` inputs.

The `each` qualifier allows you to repeat the execution of a process for each item in a collection, each time a new value is received. For example:

```

process align {
  input:
    path seq
    each mode

  script:
    """
    t_coffee -in $seq -mode $mode > result
    """
}

workflow {
  sequences = channel.fromPath('*.fa')
  methods = ['regular', 'espresso', 'psicoffee']

  align(sequences, methods)
}

```

In the above example, each time a file of sequences is emitted from the `sequences` channel, the process executes three tasks, each running a T-coffee alignment with a different value for the `mode` parameter. This behavior is useful when you need to repeat the same task over a given set of parameters.

Input repeaters can be applied to files as well. For example:

```

process align {
  input:
  path seq
  each mode
  each path(lib)

  script:
  """
  t_coffee -in $seq -mode $mode -lib $lib > result
  """
}

workflow {
  sequences = channel.fromPath('*.fa')
  methods = ['regular', 'espresso']
  libraries = [ file('PQ001.lib'), file('PQ002.lib'), file('PQ003.lib') ]

  align(sequences, methods, libraries)
}

```

In the above example, each sequence input file emitted by the `sequences` channel triggers six alignment tasks, three with the `regular` method against each library file, and three with the `espresso` method.

note

When multiple repeaters are defined, the process is executed for each combination of them.

note

Input repeaters do not support tuples. Use the `combine` operator to combine the repeated input with the other inputs to produce all of the desired input combinations.

A process can declare multiple inputs, which allows it to accept inputs from multiple dataflow sources.

warning

Do not supply more than one channel when calling a process with multiple inputs. Invoking a process with multiple channels can lead to [non-deterministic behavior](#). All additional inputs should be dataflow values.

When a process is defined with multiple inputs, it waits for a value from each input and launches a new task with the combined values. When one of the inputs is a channel, the process repeats until all values in the channel are consumed. If the channel is empty, the process will not launch any tasks.

For example:

```

process echo {
  input:
  val x
  val y

  script:
  """
  echo $x and $y
  """
}

workflow {
  x = channel.value(1)
  y = channel.of('a', 'b', 'c')
  echo(x, y)
}

```

The above example executes the `echo` process three times. The dataflow value `x` is reused for each value in `y`. It outputs:

```

1 and a
1 and b
1 and c

```

note

In general, multiple input channels should be used to process combinations of different inputs, using the `each` qualifier or value channels. Having multiple queue channels as inputs is equivalent to using the `merge` operator, which is not recommended as it may lead to [non-deterministic process inputs](#).

See also: [Singleton outputs](#).

The `output` section defines the outputs of a process, similar to a function return. A process may have at most one output section, which must contain at least one output declaration.

The output section follows the syntax shown below:

```

output:
  <output qualifier> <output name> [, <option>: <option value>]

```

An output declaration consists of a qualifier and a name. Some optional attributes can also be specified.

When a process is invoked, each process output is returned as a channel (except under [certain conditions](#)). The examples provided in the following sections demonstrate how to access the outputs of a process.

The following output qualifiers are available:

- `val` : Emit the variable with the specified name.
- `path` : Emit a file produced by the process with the specified name.
- `env` : Emit the variable defined in the process environment with the specified name.
- `stdout` : Emit the `stdout` of the executed process.
- `tuple` : Emit multiple values.
- `eval` : Emit the result of a script or command evaluated in the task execution context.

Refer to the [process reference](#) for the full list of available output methods and options.

The `val` qualifier allows you to output any Nextflow variable defined in the process. A common use case is to output a variable that was defined in the `input` section, as shown in the following example:

```
process echo {
  input:
  each x

  output:
  val x

  script:
  """
  echo $x > file
  """
}

workflow {
  methods = ['prot', 'dna', 'rna']

  received = echo(methods)
  received.view { method -> "Received: $method" }
}
```

The output value can be a value literal, an input variable, any other Nextflow variable in the process scope, or a value expression. For example:

```
process cat {
  input:
  path infile

  output:
  val x
  val 'BB11'
  val "${infile.baseName}.out"

  script:
  x = infile.name
  """
  cat $x > file
  """
}

workflow {
  ch_dummy = channel.fromPath('*').first()
  (ch_var, ch_str, ch_exp) = cat(ch_dummy)

  ch_var.view { var -> "ch_var: $var" }
  ch_str.view { str -> "ch_str: $str" }
  ch_exp.view { exp -> "ch_exp: $exp" }
}
```

The `path` qualifier allows you to output one or more files produced by the process. For example:

```
process random_number {
  output:
  path 'result.txt'

  script:
  """
  echo $RANDOM > result.txt
  """
}

workflow {
  numbers = random_number()
  numbers.view { file -> "Received: ${file.text}" }
}
```

In the above example, the `random_number` process creates a file named `result.txt` which contains a random number. Since a `path` output with the same name is declared, that file is emitted by the corresponding process output. A downstream process or operator with a compatible input will be able to receive it.

Refer to the [process reference](#) for the list of available options for `path` outputs.

When an output file name contains a `*` or `?` wildcard character, it is interpreted as a [glob](#) path matcher. This allows you to capture multiple files into a list and emit the list as a single value. For example:

```
process split_letters {
  output:
  path 'chunk_*'

  script:
  """
  printf 'Hola' | split -b 1 - chunk_
  """
}

workflow {
  split_letters
  | flatten
  | view { chunk -> "File: ${chunk.name} => ${chunk.text}" }
}
```

It prints:

```
File: chunk_aa => H
File: chunk_ab => o
File: chunk_ac => l
File: chunk_ad => a
```

By default, all the files matching the specified glob pattern are emitted as a single list. However, as the above example demonstrates, the `flatten` operator can be used to transform the list of files into a channel that emits each file individually.

Some caveats on glob pattern behavior:

- Input files are not included (unless `includeInputs` is `true`)
- Directories are included, unless the `**` pattern is used to recurse through directories

warning

Although the input files matching an output glob pattern are not included in the resulting output, these files may still be transferred from the task scratch directory to the original task work directory. Therefore, to avoid unnecessary file copies, avoid using loose wildcards when defining output files, e.g. `path '*'`. Instead, use a prefix or a suffix to restrict the set of matching files to only the expected ones, e.g. `path 'prefix_*.sorted.bam'`.

Read more about glob syntax at the following link [What is a glob?](#)

Added in version 23.10

The `arity` option can be used to enforce the expected number of files, either as a number or a range.

For example:

```
output:
path('one.txt', arity: '1')           // exactly one file is expected
path('pair_*.txt', arity: '2')        // exactly two files are expected
path('many_*.txt', arity: '1..*')     // one or more files are expected
```

When a task completes, Nextflow will check whether the produced files for each path output match the declared arity, and fail if they do not. When the arity is `'1'`, the corresponding output will be a single file; otherwise, it will be a list of files.

When an output file name needs to be expressed dynamically, it is possible to define it using a dynamic string which references variables in the `input` section or in the script global context. For example:

```
process align {
  input:
  val species
  path seq

  output:
  path "${species}.aln"

  script:
  """
  t_coffee -in $seq > ${species}.aln
  """
}
```

In the above example, each process execution produces an alignment file whose name depends on the actual value of the `species` input.

tip

The management of output files in Nextflow is often misunderstood.

With other tools it is generally necessary to organize the output files into some kind of directory structure or to guarantee a unique file name scheme, so that result files don't overwrite each other and so they can be referenced unequivocally by downstream tasks.

With Nextflow, in most cases, you don't need to manage the naming of output files, because each task is executed in its own unique directory, so files produced by different tasks can't overwrite each other. Also, metadata can be associated with outputs by using the [tuple output](#) qualifier, instead of including them in the output file name.

One example in which you'd need to manage the naming of output files is when you use the `publishDir` directive to have output files also in a specific path of your choice. If two tasks have the same filename for their output and you want them to be in the same path specified by `publishDir`, the last task to finish will overwrite the output of the task that finished before. You can dynamically change that by adding the `saveAs` option to your `publishDir` directive.

To sum up, the use of output files with static names over dynamic ones is preferable whenever possible, because it will result in simpler and more portable code.

The `env` qualifier allows you to output a variable defined in the process execution environment:

```

process seq {
  output:
  env 'RESULT'

  script:
  """
  RESULT=$(seq 5)
  """
}

workflow {
  seq | view
}

```

The `stdout` qualifier allows you to output the `stdout` of the executed process:

```

process hello {
  output:
  stdout

  script:
  """
  echo "Hello world!"
  """
}

workflow {
  hello | view { message -> "I say... $message" }
}

```

Added in version 24.04

The `eval` qualifier allows you to capture the standard output of an arbitrary command evaluated the task shell interpreter context:

```

process hello {
  output:
  eval('echo Hello world!')

  script:
  """
  true
  """
}

workflow {
  hello | view
}

```

Only one-line Bash commands are supported. You can use a semi-colon `;` to specify multiple Bash commands on a single line, and many interpreters can execute arbitrary code on the command line, e.g. `python -c 'print("Hello world!")'`.

If the command fails, the task will also fail. In Bash, you can append `|| true` to a command to suppress any command failure.

The `tuple` qualifier outputs multiple values in a single output as a [tuple](#). For example:


```

process blast {
  input:
    tuple val(species), path(query)

  output:
    tuple val(species), path('result')

  script:
    """
    blast -db nr -query $query > result
    """
}

workflow {
  ch_species = channel.of('human', 'cow', 'horse')
  ch_query = channel.fromPath('*.fa')

  blast(ch_species.combine(ch_query))
}

```

In the above example, a `blast` task is executed for each pair of `species` and `query` that are received. Each task produces a new tuple containing the value for `species` and the file `result`.

A `tuple` definition may contain any of the following qualifiers, as previously described: `val`, `path`, `env` and `stdout`. Files specified with the `path` qualifier are treated exactly the same as standalone `path` inputs.

note

While parentheses for input and output qualifiers are generally optional, they are required when specifying elements in an input/output tuple.

Here's an example with a single path output (parentheses optional):

```

process hello {
  output:
    path 'result.txt', hidden: true

  script:
    """
    echo 'another new line' >> result.txt
    """
}

```

And here's an example with a tuple output (parentheses required):

```

process hello {
  output:
    tuple path('last_result.txt'), path('result.txt', hidden: true)

  script:
    """
    echo 'another new line' >> result.txt
    echo 'another new line' > last_result.txt
    """
}

```

The `emit` option is used to define the name of a process output, which can be used to access the output by name in the calling workflow. For example:

```

process hello_bye {
  output:
  path 'hello.txt', emit: hello
  stdout emit: bye

  script:
  """
  echo "hello" > hello.txt
  echo "bye"
  """
}

workflow {
  hello_bye()
  hello_bye.out.hello.view()
}

```

See [Calling processes and workflows](#) for more details.

Normally, if a specified output is not produced by the task, the task will fail. Setting `optional: true` will cause the task to not fail, and instead emit nothing to the corresponding output channel.

```

output:
path("output.txt"), optional: true

```

In this example, the process is normally expected to produce an `output.txt` file, but in this case, if the file is missing, the task will not fail. The output channel will only contain values for those tasks that produced `output.txt`.

note

While this option can be used with any process output, it cannot be applied to individual elements of a [tuple](#) output. The entire tuple must be optional or not optional.

When a process is only supplied with dataflow values, regular values, or no inputs, it returns outputs as dataflow values. For example:

```

process echo {
  input:
  val greeting

  output:
  val greeting

  exec:
  true
}

process greet {
  input:
  val greeting
  val name

  output:
  val "$greeting, $name!"

  exec:
  true
}

workflow {
  names = channel.of( 'World', 'Mundo', 'Welt' )
  greeting = echo('Hello')
  result = greet(greeting, names)
  result.view()
}

```

In the above example, the `echo` process is invoked with a regular value that is wrapped in a dataflow value. As a result, `echo` returns a dataflow value and `greet` is executed three times.

If the call to `echo` was changed to `echo( channel.of('hello') )`, the process would instead return a channel, and `greet` would be executed only once.

See also: [Multiple inputs](#).

note

As a best practice, conditional logic should be implemented in the calling workflow (e.g. using an `if` statement or `filter` operator) instead of the process definition.

The `when` section allows you to define a condition that must be satisfied in order to execute the process. The condition can be any expression that returns a boolean value.

It can be useful to enable/disable the process execution depending on the state of various inputs and parameters. For example:

```

process blast_search {
  input:
  path proteins
  val dbtype

  when:
  proteins.name =~ /^BB11.* / && dbtype == 'nr'

  script:
  """
  blastp -query $proteins -db nr
  """
}

```

Directives are optional settings that affect the execution of the current process.

By default, directives are evaluated when the process is defined. However, if the value is a dynamic string or closure, it will be evaluated separately for each task, which allows task-specific variables like `task` and `val` inputs to be used.

Some directives are only supported by specific executors. See [Executors](#) for more information about each executor.

Refer to the [process reference](#) for the full list of process directives. If you are new to Nextflow, here are some commonly-used operators to learn first:

General:

- [errorStrategy](#): strategy for handling task failures
- [executor](#): the [executor](#) to execute tasks
- [tag](#): a semantic name used to differentiate between task executions of the same process

Resource requirements:

- [cpus](#): the number of CPUs to request for each task
- [memory](#): the amount of memory to request for each task
- [time](#): the amount of walltime to request for each task

Software dependencies:

- [conda](#): list of conda packages to provision for tasks
- [container](#): container image to use for tasks

The `task` object also contains the values of all process directives for the given task, which allows you to access these settings at runtime. For examples:

```
process hello {
  script:
  """
  some_tool --cpus $task.cpus --mem $task.memory
  """
}
```

In the above snippet, `task.cpus` and `task.memory` hold the values for the [cpus directive](#) and [memory directive](#) directives, respectively, which were resolved for this task based on the process configuration.

A directive can be assigned dynamically, during the process execution, so that its actual value can be evaluated based on the process inputs.

To be defined dynamically, the directive's value needs to be expressed using a [closure](#). For example:

```
process hello {
  executor 'sge'
  queue { entries > 100 ? 'long' : 'short' }

  input:
  tuple val(entries), path('data.txt')

  script:
  """
  your_command --here
  """
}
```

In the above example, the `queue` directive is evaluated dynamically, depending on the input value `entries`. When it is larger than 100, jobs will be submitted to the `long` queue, otherwise the `short` queue will be used.

All directives can be assigned a dynamic value except the following:

- `executor`
- `label`
- `maxForks`
- `secret`

Added in version 25.10

Dynamic directives can be specified without a closure when using the `strict parser`:

```
queue (entries > 100 ? 'long' : 'short')
```

Directives will be evaluated dynamically if they reference task inputs. Process configuration options must still be specified with a closure to be dynamic.

It's a very common scenario that different instances of the same process may have very different needs in terms of computing resources. In such situations requesting, for example, an amount of memory too low will cause some tasks to fail. Instead, using a higher limit that fits all the tasks in your execution could significantly decrease the execution priority of your jobs.

`Dynamic directives` can be used to adjust the requested task resources when a task fails and is re-executed. For example:

```
process hello {
  memory { 2.GB * task.attempt }
  time { 1.hour * task.attempt }

  errorStrategy { task.exitStatus in 137..140 ? 'retry' : 'terminate' }
  maxRetries 3

  script:
  """
  your_command --here
  """
}
```

In the above example the `memory` and execution `time` limits are defined dynamically. The first time the process is executed the `task.attempt` is set to 1, thus it will request 2 GB of memory and 1 hour of wall time.

If the task execution fails with an exit status between 137 and 140, the task is re-executed; otherwise, the run is terminated immediately. The re-executed task will have `task.attempt` set to 2, and will request 4 GB of memory and 2 hours of wall time.

The `maxRetries` directive sets the maximum number of times the same task can be re-executed.

tip

Directives with named arguments, such as `accelerator` and `disk`, must use a more verbose syntax when they are dynamic. For example:

```
// static request
disk 375.GB, type: 'local-ssd'

// dynamic request
disk { [request: 375.GB * task.attempt, type: 'local-ssd'] }
```

Task resources can also be defined in terms of task inputs. For example:

```
process hello {
  memory { 8.GB + 1.GB * Math.ceil(input_file.size() / 1024 ** 3) }

  input:
  path input_file

  script:
  """
  your_command --here
  """
}
```

In this example, each task requests 8 GB of memory, plus the size of the input file rounded up to the next GB. This way, each task requests only as much memory as it needs based on the size of the inputs. The specific function that you use should be tuned for each process.

Added in version 24.10

Task resource requests can be updated relative to the [trace file](#) metrics of the previous task attempt. The metrics can be accessed through the `task.previousTrace` variable. For example:

```
process hello {
  memory { task.attempt > 1 ? task.previousTrace.memory * 2 : (1.GB) }
  errorStrategy { task.exitStatus in 137..140 ? 'retry' : 'terminate' }
  maxRetries 3

  script:
  """
  your_command --here
  """
}
```

In the above example, the `memory` is set according to previous trace record metrics. In the first attempt, when no trace metrics are available, it is set to 1 GB. In each subsequent attempt, the requested memory is doubled.

note

Many fields from the previous task attempts are accessible. See [Trace file](#) for a list of available fields.

There are cases in which the required execution resources may be temporarily unavailable e.g. network congestion. In these cases immediately re-executing the task will likely result in the identical error. A retry with an exponential backoff delay can better recover these error conditions:

```
process hello {
  errorStrategy { sleep(Math.pow(2, task.attempt) * 200 as long); return 'retry' }
  maxRetries 5

  script:
  """
  your_command --here
  """
}
```

Processes (typed)

Added in version 25.10

warning

Typed processes are a preview feature. The syntax and behavior may change in future releases.

Typed processes use a new syntax for inputs and outputs that supports static typing.

```
nextflow.enable.types = true

process hello {
  input:
  message: String

  output:
  file('hello.txt')

  script:
  """
  echo '${message}' > hello.txt
  """
}
```

To use this feature:

1. Enable the [strict parser](#) by setting the `NXF_SYNTAX_PARSER` environment variable to `v2`:

```
export NXF_SYNTAX_PARSER=v2
```

2. Set `nextflow.enable.types = true` in every script that uses typed processes.

See [Process \(typed\)](#) for the complete syntax reference and [Migrating to static typing](#) to migrate existing code to static typing.

The `input:` section declares process inputs. In typed processes, each input declaration consists of a name and type:

```
process fastqc {
  input:
  meta: Map
  fastq: Path
  extra_args: String

  script:
  """
  echo 'meta: ${meta}'
  echo 'fastq: ${fastq}'
  echo 'extra_args: ${extra_args}'
  """
}
```

All [standard types](#) except for the dataflow types (`Channel` and `Value`) can be used as type annotations in processes.

Nextflow automatically stages `Path` inputs and `Path` collections (such as `Set<Path>`) into the task directory.

By default, tasks fail if any input receives a `null` value. To allow `null` values, add `?` to the type annotation:

```
process cat_opt {
  input:
  input: Path?

  stage:
  stageAs input, 'input.txt'

  output:
  stdout()

  script:
  """
  [[ -f input.txt ]] && cat input.txt || echo 'empty input'
  """
}
```

Record inputs can be declared using a record type:

```
process fastqc {
  input:
  sample: Sample

  script:
  """
  echo 'id: ${sample.id}'
  echo 'fastq: ${sample.fastq}'
  """
}

record Sample {
  id: String
  fastq: Path
}
```

In this example, the record input is staged as `sample`, and `sample.fastq` is staged as an input file since it is declared with type `Path` in the `Sample` record type. Each field in the record type is staged into the task the same way as an individual input.

When the process is invoked, the incoming record should contain the specified fields, or else the run will fail. If the incoming record has additional fields not declared by the process input, they are ignored.

Record inputs can also be declared as a destructured input:

```
process fastqc {
  input:
  record(
    id: String,
    fastq: Path
  )

  script:
  """
  echo 'id: ${id}'
  echo 'fastq: ${fastq}'
  """
}
```

This pattern mirrors the standard `record()` function used to construct records. In this example, `fastq` is staged as an input file since the `fastq` field is declared with type `Path`.

tip

Record inputs are a useful way to select a subset of fields from a larger record. This way, the process stages only what it needs, keeping related data together in your workflow logic.

Tuple inputs can be declared as a destructured input:

```
process fastqc {
  input:
  tuple(id: String, fastq: Path)

  script:
  """
  echo 'id: ${id}'
  echo 'fastq: ${fastq}'
  """
}
```

This pattern mirrors the standard `tuple()` function used to construct tuples. Each tuple component is staged into the task the same way as an individual input.

The `stage:` section defines custom staging behavior using stage directives. It should be specified after the `input:` section. These directives serve the same purpose as input qualifiers such as `env` and `stdin` in the legacy syntax.

The `env` directive declares an environment variable in terms of task inputs:

```
process echo_env {
  input:
  hello: String

  stage:
  env 'HELLO', hello

  script:
  """
  echo "$HELLO world!"
  """
}
```

The `stdin` directive defines the standard input of the task script:

```
process cat {
  input:
  message: String

  stage:
  stdin message

  script:
  """
  cat -
  """
}
```

The `stageAs` directive stages an input file (or files) under a custom file pattern:

```

process blast {
  input:
    fasta: Path

  stage:
    stageAs fasta, 'query.fa'

  script:
    """
    blastp -query query.fa -db nr
    """
}

```

The file pattern can also reference task inputs:

```

process grep {
  input:
    id: String
    fasta: Path

  stage:
    stageAs fasta, "${id}.fa"

  script:
    """
    cat ${id}.fa | grep '>'
    """
}

```

See [Inputs and outputs \(typed\)](#) for available stage directives.

The `output:` section declares the outputs of a typed process. Each output declaration consists of a name, an optional type, and an output value:

```

process echo {
  input:
    message: String

  output:
    out_env: String = env('MESSAGE')
    out_file: Path = file('message.txt')
    out_std: String = stdout()

  script:
    """
    export MESSAGE='${message}'

    echo \${MESSAGE} > message.txt

    cat message.txt
    """
}

```

When there is only one output, the name can be omitted:

```
process echo {
  input:
    message: String

  output:
    stdout()

  script:
    """
    echo '${message}'
    """
}
```

See [Inputs and outputs \(typed\)](#) for available output functions.

You can use the `file()` and `files()` functions in the `output:` section to get a single file or collection of files from the task directory.

By default, the `file()` function fails if the specified file is not present in the task directory. You can specify `optional: true` to allow missing files. The `file()` function returns `null` for missing files. For example:

```
process foo {
  output:
    file('output.txt', optional: true)

  script:
    """
    exit 0
    """
}
```

Whereas legacy process outputs could only be structured using specific qualifiers like `val` and `tuple`, typed process outputs are regular values.

The `record()` standard library function can be used to create a record:

```
process fastqc {
  input:
    record(
      id: String,
      fastq: Path
    )

  output:
    record(
      id: id,
      fastqc: file('fastqc_logs')
    )

  script:
    // ...
}
```

The `tuple()` standard library function can be used to create a tuple:

```

process fastqc {
  input:
    tuple(id: String, fastq: Path)

  output:
    tuple(id, file('fastqc_logs'))

  script:
    // ...
}

```

The `topic:` section emits values to [topic channels](#). A topic emission consists of an output value and a topic name:

```

process cat {
  input:
    message: Path

  output:
    stdout()

  topic:
    tuple('bash', eval('bash --version')) >> 'versions'
    tuple('cat', eval('cat --version')) >> 'versions'

  script:
    """
    cat ${message}
    """
}

```

Topic emissions can use the same [output functions](#) as the `output:` section.

The `script:` and `exec:` sections behave the same way as [legacy processes](#).

The `stub:` section behaves the same way as [legacy processes](#).

Directives behave the same way as [legacy processes](#).

Workflows

In Nextflow, a **workflow** is a specialized function for composing [processes](#) and dataflow logic:

- An [entry workflow](#) is the entrypoint of a pipeline. It can take [parameters](#) as inputs using the `params` block, and it can publish [outputs](#) using the `output` block.
- A [named workflow](#) is a workflow that can be called by other workflows. It can define its own inputs and outputs, which are called takes and emits.
- Both entry workflows and named workflows can contain [dataflow logic](#) such as calling processes, workflows, and channel operators.

A script can define up to one entry workflow, which does not have a name and serves as the entrypoint of the script:

```
workflow {
  channel.of('Bonjour', 'Ciao', 'Hello', 'Hola')
    .map { v -> "$v world!" }
    .view()
}
```

Parameters can be declared by assigning a `params` property to a default value:

```
params.input = '/some/data/file'
params.save_intermeds = false

workflow {
  if( params.input )
    analyze(params.input, params.save_intermeds)
  else
    analyze(fake_input(), params.save_intermeds)
}
```

The default value can be overridden by the command line, params file, or config file. Parameters from multiple sources are resolved in the order described in [Pipeline parameters](#).

Added in version 25.10

Workflow outputs are available as a preview in Nextflow [24.04](#), [24.10](#), and [25.04](#).

note

Workflow outputs are intended to replace the [publishDir](#) directive. See [Migrating to workflow outputs](#) for guidance on migrating from `publishDir` to workflow outputs.

A script can define an output block to declare the top-level workflow outputs. Each output should be assigned in the `publish` section of the entry workflow. Any channel in the workflow can be assigned to an output, including process and subworkflow outputs.

Example:

```

process fetch {
  // ...

  output:
  path 'sample.txt'

  // ...
}

workflow {
  main:
  ch_samples = fetch(params.input)

  publish:
  samples = ch_samples
}

output {
  samples {
    path '.'
  }
}

```

In the above example, the output of process `fetch` is assigned to the `samples` workflow output. How this output is published to a directory structure is described in the next section.

Each workflow output can define how files are published from the work directory to a designated output directory.

Output directory

You can set the top-level output directory for a run using the `-output-dir` command-line option or the `outputDir` config option:

```
nextflow run main.nf -output-dir 'my-results'
```

```
// nextflow.config
outputDir = 'my-results'
```

The default output directory is `results` in the launch directory.

Publish path

By default, Nextflow publishes all output files to the output directory. Each workflow output can define where to publish files within the output directory using the `path` directive:

```

workflow {
  main:
    ch_step1 = step1()
    ch_step2 = step2(ch_step1)

    publish:
    step1 = ch_step1
    step2 = ch_step2
}

output {
  step1 {
    path 'step1'
  }
  step2 {
    path 'step2'
  }
}

```

The following directory structure is created:

```

results/
├─ step1/
│   └─ ...
├─ step2/
│   └─ ...

```

Nextflow publishes all files received by an output into the specified directory. Nextflow recursively scans lists, maps, and tuples for nested files:

```

workflow {
  main:
    ch_samples = channel.of(
      tuple( [id: 'SAMP1'], [ file('1.txt'), file('2.txt') ] )
    )

    publish:
    samples = ch_samples // 1.txt and 2.txt are published
}

```

note

Files that do not originate from the work directory are not published.

Dynamic publish path

The `path` directive can also be a closure which defines a custom publish path for each channel value:


```

workflow {
  main:
    ch_samples = channel.of(
      [id: 'SAMP1', fastq_1: file('1.fastq'), fastq_2: file('2.fastq')]
    )

    publish:
      samples = ch_samples
}

output {
  samples {
    path { sample -> "fastq/${sample.id}/" }
  }
}

```

The above example publishes each channel value to a different subdirectory. In this case, each pair of FASTQ files is published into a subdirectory based on the sample ID.

Alternatively, you can define a different path for each individual file using the `>>` operator:

```

output {
  samples {
    path { sample ->
      sample.fastq_1 >> "fastq/${sample.id}/"
      sample.fastq_2 >> "fastq/${sample.id}/"
    }
  }
}

```

Each `>>` specifies a source file and publish target. The source file should be a file or collection of files, and the publish target should be a directory or file name. If the publish target ends with a slash, Nextflow treats it as the directory in which to publish source files.

When using this syntax, only files captured with the `>>` operator are saved to the output directory.

Conditional publishing

Outputs can be conditionally published using pipeline parameters:

```

output {
  samples {
    path { sample ->
      sample.fastqc >> "fastqc/"
      sample.bam >> (params.save_bams ? "align/" : null)
    }
  }
}

```

In the above example, the BAM files specified by `sample.bam` are published only when `params.save_bams` is `true`.

Index files are structured metadata files that catalog published outputs and their associated metadata. An index file preserves the structure of channel values, including metadata, which is more robust than encoding this information into file paths. The index file can be a CSV (`.csv`), JSON (`.json`), or YAML (`.yaml`, `.yml`) file. The channel values should be files, lists, maps, or tuples.

Each output can create an index file of its published values:

```

workflow {
  main:
    ch_samples = channel.of(
      [id: 1, name: 'sample 1', fastq_1: '1a.fastq', fastq_2: '1b.fastq'],
      [id: 2, name: 'sample 2', fastq_1: '2a.fastq', fastq_2: '2b.fastq'],
      [id: 3, name: 'sample 3', fastq_1: '3a.fastq', fastq_2: null]
    )

    publish:
      samples = ch_samples
}

output {
  samples {
    path 'fastq'
    index {
      path 'samples.csv'
    }
  }
}

```

The above example writes the following CSV file to `results/samples.csv`:

```

"1","sample 1","results/fastq/1a.fastq","results/fastq/1b.fastq"
"2","sample 2","results/fastq/2a.fastq","results/fastq/2b.fastq"
"3","sample 3","results/fastq/3a.fastq",""

```

You can customize the index file with additional directives, for example:

```

index {
  path 'samples.csv'
  header true
  sep '|'
}

```

This example produces the following index file:

```

"id"|"name"|"fastq_1"|"fastq_2"
"1"|"sample 1"|"results/fastq/1a.fastq"|"results/fastq/1b.fastq"
"2"|"sample 2"|"results/fastq/2a.fastq"|"results/fastq/2b.fastq"
"3"|"sample 3"|"results/fastq/3a.fastq"|"

```

note

Files that do not originate from the work directory are not published, but are included in the index file.

See [Output directives](#) for the list of available index directives.

You can apply labels to each workflow output using the `label` directive:

```

output {
  multiqc_report {
    label 'qc'
    label 'summary'
  }
}

```

Labels can be used to find and filter output files across workflow runs with data lineage. See [Data lineage](#) for details on how to query output files by label.

The following directives are available for each output in the output block:

Create an index file containing a record of each published value.

The following directives are available in an index definition:

When `true`, the keys of the first record are used as the column names (default: `false`). Can also be a list of column names. Only used for CSV files.

The name of the index file relative to the base output directory (required). Can be a CSV, JSON, or YAML file.

The character used to separate values (default: `,`). Only used for CSV files.

Attach a label to every file published by this output. Can be specified multiple times to attach multiple labels.

Labels are stored in the `labels` field of `FileOutput` records in the [lineage store](#).

Specify the publish path relative to the output directory (default: `''`). Can be a path, a closure that defines a custom directory for each published value, or a closure that publishes individual files using the `>>` operator.

Additionally, the following options from the [workflow](#) config scope can be specified as directives:

- `contentType`
- `enabled`
- `ignoreErrors`
- `mode`
- `overwrite`
- `storageClass`
- `tags`

For example:

```
output {
  samples {
    mode 'copy'
  }
}
```

A named workflow is a workflow that can be called by other workflows:

```
workflow my_workflow {
  ch_hello = hello()
  bye( ch_hello.collect() )
}

workflow {
  my_workflow()
}
```

The above example defines a workflow named `my_workflow` which is called by the entry workflow. Both `hello` and `bye` could be any other process or workflow.

The `take:` section declares the inputs of a named workflow:

```

workflow my_workflow {
  take:
    data1
    data2

  main:
    ch_hello = hello(data1, data2)
    bye(hello.out)
}

```

Inputs can be specified like arguments when calling the workflow:

```

workflow {
  my_workflow( channel.of('/some/data') )
}

```

The `emit:` section declares the outputs of a named workflow:

```

workflow my_workflow {
  main:
    ch_hello = hello(data)
    ch_bye = bye(ch_hello)

  emit:
    hello = ch_hello
    bye = ch_bye
}

workflow {
  result = my_workflow()
  result.hello.view()
  result.bye.view()
}

```

The name should be omitted when only one output is declared:

```

workflow my_workflow {
  take:
    data

  main:
    ch_bye = bye(hello(data))

  emit:
    ch_bye
}

workflow {
  result = my_workflow()
  result.view()
}

```

Workflows consist of dataflow logic, in which processes are connected to each other through dataflow channels and dataflow values.

A dataflow channel (or simply channel) is an asynchronous sequence of values.

The values in a channel cannot be accessed directly, but only through an operator or process. For example:

```

channel.of(1, 2, 3).view { v -> "channel emits ${v}" }

```

```
channel emits 1
channel emits 2
channel emits 3
```

A dataflow value is an asynchronous value.

Dataflow values can be created using the `channel.value` factory, and they are created by processes (under [certain conditions](#)).

A dataflow value cannot be accessed directly, but only through an operator or process. For example:

```
channel.value(1).view { v -> "dataflow value is ${v}" }
```

```
dataflow value is 1
```

A channel can be created by factories in the `channel` namespace. For example, the `channel.fromPath()` factory creates a channel from a file name or glob pattern, similar to the `files()` function:

```
channel.fromPath('input/*.txt').view()
```

See [Channel factories](#) for the full list of channel factories.

Channel operators, or operators for short, are functions that consume and produce channels. Because channels are asynchronous, operators are necessary to manipulate the values in a channel. Operators are particularly useful for implementing glue logic between processes.

Commonly used operators include:

- [collect](#): collect the channel values into a collection
- [combine](#): emit the combinations of two channels
- [filter](#): emit only the channel values that satisfy a condition
- [flatMap](#): emit multiple values for each channel value with a closure
- [grouptuple](#): group the channel values based on a grouping key
- [join](#): join the values from two channels based on a matching key
- [map](#): transform each channel value with a mapping function
- [mix](#): emit the values from multiple channels
- [view](#): print each channel value to standard output

See [Operators](#) for the full set of operators. See [Value<V>](#) for the set of available methods for dataflow values.

Processes and workflows are called like functions, passing their inputs as arguments:

```

process hello {
  output:
  path 'hello.txt', emit: txt

  script:
  """
  your_command > hello.txt
  """
}

process bye {
  input:
  path 'hello.txt'

  output:
  path 'bye.txt', emit: txt

  script:
  """
  another_command hello.txt > bye.txt
  """
}

workflow hello_bye {
  take:
  data

  main:
  hello()
  bye(data)
}

workflow {
  data = channel.fromPath('/some/path/*.txt')
  hello_bye(data)
}

```

Processes and workflows can only be called by workflows. A given process or workflow can only be called once in a given workflow. To use a process or workflow multiple times in the same workflow, [include](#) it from another script with multiple aliases:

```

include { hello_bye as hello_bye1 } from './modules/hello_bye'
include { hello_bye as hello_bye2 } from './modules/hello_bye'

workflow {
  data1 = channel.fromPath('data1/*.txt')
  data2 = channel.fromPath('data2/*.txt')
  hello_bye1(data1)
  hello_bye2(data2)
}

```

The "return value" of a process or workflow call is the process outputs or workflow emits, respectively. The return value can be assigned to a variable or passed into another call:

```

workflow hello_bye {
  take:
  data

  main:
  bye_out = bye(hello(data))

  emit:
  bye_out
}

workflow {
  data = channel.fromPath('/some/path/*.txt')
  bye_out = hello_bye(data)
}

```

Named outputs can be accessed as properties of the return value:

```

workflow hello_bye {
  take:
  data

  main:
  hello_out = hello(data)
  bye_out = bye(hello_out.txt)

  emit:
  hello = hello_out.txt
  bye = bye_out.txt
}

workflow {
  data = channel.fromPath('/some/path/*.txt')
  flow_out = hello_bye(data)
  bye_out = flow_out.bye
}

```

As a convenience, process and workflow outputs can also be accessed without first assigning to a variable, by using the `.out` property of the process or workflow name:

```

workflow hello_bye {
  take:
  data

  main:
  hello(data)
  bye(hello.out)

  emit:
  hello = hello.out
  bye = bye.out
}

workflow {
  data = channel.fromPath('/some/path/*.txt')
  hello_bye(data)
  hello_bye.out.bye.view()
}

```

note

Process named outputs are defined using the `emit` option on a process output. See [naming process outputs](#) for more information.

Workflows can be composed in the same way:

```
workflow flow1 {
  take:
  data

  emit:
  tack(tick(data))
}

workflow flow2 {
  take:
  data

  emit:
  tock(tick(data))
}

workflow {
  data = channel.fromPath('/some/path/*.txt')
  flow2(flow1(data))
}
```

The same process can be called in different workflows without using an alias, like `tick` in the above example, which is used in both `flow1` and `flow2`. The workflow call stack determines the fully qualified process name, which is used to distinguish the different process calls, i.e. `flow1:tick` and `flow2:tick` in the above example.

tip

The fully qualified process name can be used as a [process selector](#) in a Nextflow configuration file, and it takes priority over the simple process name.

Deprecated in version 26.04

These operators are not supported when [static typing](#) is enabled. Use standard method calls and assignments instead.

The following operators have a special meaning when used with process and workflow calls in a workflow:

- The `|` pipe operator can be used to chain processes, operators, and workflows.
- The `&` and operator can be used to call multiple processes in parallel with the same channel(s).

For example:


```

process greet {
  input:
  val data

  output:
  val result

  exec:
  result = "$data world"
}

process to_upper {
  input:
  val data

  output:
  val result

  exec:
  result = data.toUpperCase()
}

workflow {
  channel.of('Hello')
  | map { v -> v.reverse() }
  | (greet & to_upper)
  | mix
  | view
}

```

In the above snippet, the initial channel is piped to the `map` operator, which reverses the string value. Then, the result is passed to the processes `greet` and `to_upper`, which are executed in parallel. Each process outputs a channel, and the two channels are combined using the `mix` operator. Finally, the result is printed using the `view` operator.

The same code can also be written as:

```

workflow {
  ch = channel.of('Hello').map { v -> v.reverse() }
  ch_greet = greet(ch)
  ch_upper = to_upper(ch)
  ch_greet.mix(ch_upper).view()
}

```

Added in version 22.04

note

This is a preview feature and requires the `nextflow.preview.recursion` feature flag to be enabled. The syntax and behavior may change in future releases.

Processes can be invoked recursively using the `recurse` method.

```

nextflow.preview.recursion = true

params.start = 10

workflow {
    count_down
        .recurse(params.start)
        .until { v -> v == 0 }
        .view { v -> "${v}..." }
}

process count_down {
    input:
    val v

    output:
    val v

    exec:
    sleep(1000)
    v = v - 1
}

```

```

9...
8...
7...
6...
5...
4...
3...
2...
1...
0...

```

In the above example, the `count_down` process is first invoked with the value `params.start`. On each subsequent iteration, the process is invoked again using the output from the previous iteration. The recursion continues until the specified condition is satisfied, as defined by the `until` method, which terminates the recursion.

The recursive output can also be limited using the `times` method:

```

count_down
    .recurse(params.start)
    .times(3)
    .view { v -> "${v}..." }

```

Workflows can also be invoked recursively:

```

nextflow.preview.recursion = true

params.input = "recurse-workflow.in"

workflow {
    clock
        .recurse(file(params.input))
        .until { file -> file.size() > 64 }
        .view { file -> file.text }
}

workflow clock {
    take:
    logfile

    emit:
    tock(tick(logfile))
}

process tick {
    input:
    path 'input.txt'

    output:
    path 'result.txt'

    script:
    """
    cat input.txt > result.txt
    echo "Task ${task.index} : tick" >> result.txt
    """
}

process tock {
    input:
    path 'input.txt'

    output:
    path 'result.txt'

    script:
    """
    cat input.txt > result.txt
    echo "Task ${task.index} : tock" >> result.txt
    """
}

```

```

hello
Task 1 : tick
Task 1 : tock

```

```

hello
Task 1 : tick
Task 1 : tock
Task 2 : tick
Task 2 : tock

```

```

hello
Task 1 : tick
Task 1 : tock
Task 2 : tick
Task 2 : tock
Task 3 : tick
Task 3 : tock

```

Limitations

- A recursive process or workflow must have matching inputs and outputs, such that the outputs for each iteration can be supplied as the inputs for the next iteration.

- Recursive workflows cannot use reduction operators such as `collect`, `reduce`, and `toList`, because these operators cause the recursion to hang indefinitely after the initial iteration.

Workflows (typed)

Typed workflows use a new syntax for inputs and outputs that supports static typing.

To use this feature:

1. Enable the [strict parser](#) by setting the `NXF_SYNTAX_PARSER` environment variable to `v2`:

```
export NXF_SYNTAX_PARSER=v2
```

2. Set `nextflow.enable.types = true` in every script that uses typed workflows. The `params` block and `output` block can be used without this feature flag.

See [Workflow \(typed\)](#) for the complete syntax reference and [Migrating to static typing](#) to migrate existing code to static typing.

Added in version 25.10

A script can declare parameters using the `params` block:

```
params {  
    // Path to input data.  
    input: Path  
  
    // Whether to save intermediate files.  
    save_intermeds: Boolean  
}  
  
workflow {  
    analyze(params.input, params.save_intermeds)  
}
```

All [standard types](#) except for the dataflow types (`Channel` and `Value`) can be used for parameters.

Typed parameters should only be referenced in the entry workflow or `output` block. Parameters can be passed to workflows and processes as explicit inputs.

The default value can be overridden by the command line, params file, or config file. Parameters from multiple sources are resolved in the order described in [Pipeline parameters](#). Parameters specified on the command line are converted to the appropriate type based on the corresponding type annotation.

A parameter that doesn't specify a default value is a required parameter. If a required parameter is not given a value at runtime, the run will fail.

Added in version 26.04

Boolean parameters that don't specify a default value will default to `false`.

Added in version 25.10

Workflow outputs can use type annotations:

```

nextflow.enable.types = true

params {
    input: String
}

process fastqc {
    input:
    reads: Path

    output:
    file('fastqc')

    // ...
}

process summary {
    input:
    logs: Path

    output:
    file('report.html')

    // ...
}

workflow {
    main:
    ch_reads = channel.fromPath(params.input)
    ch_fastqc = fastqc(ch_reads)
    summary_report = summary(ch_fastqc.collect())

    publish:
    fastqc = ch_fastqc
    summary_report = summary_report
}

output {
    fastqc: Channel<Path> {
        path '.'
    }

    summary_report: Path {
        path '.'
    }
}

```

In the above example, the workflow declares two outputs:

- `fastqc`: a channel of FastQC results (`Channel<Path>`)
- `summary_report`: a dataflow value containing a summary report (`Value<Path>` or `Path`)

Type annotations are useful for documenting the structure of each workflow output, and they can be used by the language server to validate the type of each published output.

Outputs that receive a dataflow value can be declared as `Value<V>` or `V` for short. In this example, the `summary_report` is declared with type `Path` as a shorthand for `Value<Path>`.

Added in version 26.04

warning

Typed workflows are a preview feature. The syntax and behavior may change in future releases.

Typed workflows can use type annotations in the `take:` and `emit:` sections:

```
nextflow.enable.types = true

workflow hello_bye {
  take:
  samples: Channel<Path>

  main:
  ch_hello = hello(samples)
  val_bye = bye(ch_hello.collect())

  emit:
  result: Value<Path> = val_bye
}
```

In the above example, `hello_bye` takes a channel of files (`Channel<Path>`) and emits a dataflow value with a single file (`Value<Path>`). See [Types](#) for the list of available types.

The following syntax patterns are no longer supported in typed workflows:

- Using `channel` to access channel factories (use `channel` instead)
- Using implicit closure parameters (declare parameters explicitly instead)
- Using `set` or `tap` to assign channels (use standard assignments instead)
- Composing dataflow logic with `|` and `&` (use standard method calls instead)
- Accessing process and workflow outputs via `.out` (use standard assignments instead)

See [Preparing for static typing](#) for more information.

The operator library has been updated to provide first-class support for static typing and records. All operators can be used in both typed workflows and legacy workflows. However, only a [core subset](#) of operators are recommended for use with static typing.

See [Using operators with static typing](#) for best practice guidelines when migrating existing code.

Notifications

This page documents how to handle workflow events and send notifications.

Due to the asynchronous nature of Nextflow, the termination of a script does not correspond to the termination of the running workflow. Thus some information, only available on execution completion, needs to be accessed by using an asynchronous handler.

Nextflow invokes the `onComplete` event handler when the workflow execution is completed. It allows one to access the workflow termination status and other useful information. For example:

```
workflow.onComplete {
    println "Pipeline completed at: $workflow.complete"
    println "Execution status: ${ workflow.success ? 'OK' : 'failed' }"
}
```

Added in version 25.10

Entry workflows can define an `onComplete` section instead of using `workflow.onComplete`:

```
workflow {
    main:
    // ...

    onComplete:
    println "Pipeline completed at: $workflow.complete"
    println "Execution status: ${ workflow.success ? 'OK' : 'failed' }"
}
```

Nextflow invokes the `onError` event handler when a runtime or process error caused the pipeline execution to stop. For example:

```
workflow.onError {
    println "Error: Pipeline execution stopped with the following message: ${workflow.errorMessage}"
}
```

note

Both the `onError` and `onComplete` handlers are invoked when an error condition is encountered. The `onError` handler is called as soon as the error is raised, while `onComplete` is called just before the pipeline execution is about to terminate. When using the `finish` [errorStrategy](#), there may be a significant gap between the two, depending on the time required to complete any pending job.

Added in version 25.10

Entry workflows can define an `onError` section instead of using `workflow.onError`:


```

workflow {
  main:
    // ...

  onError:
    println "Error: Pipeline execution stopped with the following message: ${workflow.errorMessage}"
}

```

The built-in function `sendMail` allows you to send a mail message from a workflow script.

The mail attributes are specified as named parameters or an equivalent map. For example:

```

sendMail(
  to: 'you@gmail.com',
  subject: 'Catch up',
  body: 'Hi, how are you!',
  attach: '/some/path/attachment/file.txt'
)

```

which is equivalent to:

```

mail = [
  to: 'you@gmail.com',
  subject: 'Catch up',
  body: 'Hi, how are you!',
  attach: '/some/path/attachment/file.txt'
]

sendMail(mail)

```

The following parameters can be specified:

Multiple email addresses can be specified by separating them with a comma.

The mail target recipients.

Multiple email addresses can be specified by separating them with a comma.

The mail CC recipients.

Multiple email addresses can be specified by separating them with a comma.

The mail BCC recipients.

Multiple email addresses can be specified by separating them with a comma.

The mail sender address.

The mail subject.

The mail content charset (default: UTF-8).

The mail plain text content.

The mail body content. It can be either plain text or HTML content.

The mail body mime type. If not specified it's automatically detected.

Single file or a list of files to be included as mail attachments.

Another version of `sendMail` allows a more idiomatic syntax:

```

sendMail {
  to 'you@gmail.com'
  from 'me@gmail.com'
  attach '/some/path/attachment/file.txt'
  attach '/other/path/image.png'
  subject 'Catch up'

  ...
  Hi there,
  Look! Multi-lines
  mail content!
  ...
}

```

The same attributes listed in the table in the previous section are allowed.

tip

A string expression at the end is implicitly interpreted as the mail body content, therefore the `body` parameter can be omitted as shown above.

tip

To send an email that includes text and HTML content, use both the `text` and `body` attributes. The first is used for the plain text content, while the second is used for the rich HTML content.

When using the curly brackets syntax, the `attach` parameter can be repeated two or more times to include multiple attachments in the mail message.

Moreover for each attachment it's possible to specify any of the following options:

Defines the `Content-ID` header field for the attachment.

Defines the `Content-Disposition` header field for the attachment.

Defines the `filename` parameter of the `Content-Disposition` header field.

Defines the `Content-Description` header field for the attachment.

For example:

```

sendMail {
  to 'you@dot.com'
  attach '/some/file.txt', fileName: 'manuscript.txt'
  attach '/other/image.png', disposition: 'inline'
  subject 'Sending documents'
  ...
  the mail body
  ...
}

```

If no mail server configuration is provided, Nextflow tries to send the email using the external mail command eventually provided by the underlying system (e.g. `sendmail` or `mail`).

If your system does not provide access to any of the above you can configure a SMTP server in the `nextflow.config` file. For example:

```
mail {
  smtp.host = 'your.smtp-server.com'
  smtp.port = 475
  smtp.user = 'my-user'
}
```

note

Some versions of Java (e.g. Java 11 Corretto) do not default to TLS v1.2, and as a result may have issues with 3rd party integrations that enforce TLS v1.2 (e.g. Azure Active Directory OIDC). This problem can be addressed by setting the following config option:

```
mail {
  smtp.ssl.protocols = 'TLSv1.2'
}
```

See the [mail scope](#) section to learn more the mail server configuration options.

Added in version 23.10

Nextflow supports the [AWS Simple Email Service](#) API as an alternative provider to send emails in place of an SMTP server.

To enable this feature, set the following environment variable in the launch environment:

```
export NXF_ENABLE_AWS_SES=true
```

Make also sure to add the following AWS IAM permission to the AWS user (or role) used to launch the pipeline execution:

```
ses:SendRawEmail
```

The following snippet shows how to configure Nextflow to send emails through SES:

```
mail {
  smtp.host = 'email-smtp.us-east-1.amazonaws.com'
  smtp.port = 587
  smtp.user = '<Your AWS SES access key>'
  smtp.password = '<Your AWS SES secret key>'
  smtp.auth = true
  smtp.starttls.enable = true
  smtp.starttls.required = true
}
```

You can use the `sendMail` function with a [workflow completion handler](#) to notify a workflow completion. For example:

```

workflow.onComplete {
    def msg = """\r
    Pipeline execution summary
    -----
    Completed at: ${workflow.complete}
    Duration    : ${workflow.duration}
    Success     : ${workflow.success}
    workDir     : ${workflow.workDir}
    exit status : ${workflow.exitStatus}
    """
    .stripIndent()

    sendMail(to: 'you@gmail.com', subject: 'My pipeline execution', body: msg)
}

```

This is useful to send a custom notification message. Note however that Nextflow includes a built-in notification mechanism which is the most convenient way to notify the completion of a workflow execution in most cases. Read the following section to learn about it.

Nextflow includes built-in workflow notification features that automatically sends a notification message when a workflow execution terminates.

To enable simply specify the `-N` option when launching the pipeline execution. For example:

```
nextflow run <pipeline name> -N <recipient address>
```

It will send a notification mail when the execution completes.

warning

By default the notification message is sent with the `sendmail` system tool, which is assumed to be available in the environment where Nextflow is running. Make sure it's properly installed and configured. Alternatively, you can provide the SMTP server configuration settings to use the Nextflow built-in mail support, which doesn't require any external system tool.

See the [Mail configuration](#) section to learn about the available mail delivery options and configuration settings.

See [Completion handler](#) to learn more about the workflow notification configuration details.

Secrets

Nextflow has built-in support for pipeline secrets, allowing users to safely provide sensitive information to a pipeline run.

This feature allows you to decouple the use of secrets in your pipelines from the pipeline code and configuration files. Secrets are managed by Nextflow and stored separately into a local store only accessible to the secrets owner.

When a pipeline is launched, Nextflow injects the secrets into the run without leaking them into temporary execution files. Secrets are provided to tasks as environment variables.

Secrets can be used with the local executor and grid executors (e.g., Slurm or Grid Engine). Secrets can be used with the AWS Batch executor when launched from [Seqera Platform](#).

The Nextflow [secrets](#) sub-command can be used to manage secrets:

```
# create a new secret
nextflow secrets set F00 "Hello world"

# list all secrets
nextflow secrets list

# get the value of secret F00
nextflow secrets get F00

# delete the secret F00
nextflow secrets delete F00
```

Secrets can be used in configuration files using the built-in `secrets` variable. For example:

```
aws {
  accessKey = secrets.MY_ACCESS_KEY
  secretKey = secrets.MY_SECRET_KEY
}
```

The above snippet accesses the secrets `MY_ACCESS_KEY` and `MY_SECRET_KEY` and assigns them to the corresponding AWS config settings.

warning

Secrets should not be assigned to pipeline parameters, as they can be leaked by the pipeline.

Added in version 25.10

Nextflow supports the use of secrets provided by plugins (e.g., AWS secrets) in configuration. However, due to the way that plugins are loaded, there are specific considerations when using config secrets:

- **Initial config load:** Nextflow first loads the configuration without secrets enabled. Any reference to a secret will return the empty string `''`.

- **Plugin resolution:** Plugins are resolved after the initial configuration load. This is because the configuration can specify additional plugins.
- **Config reloading:** If secrets are accessed during configuration and the initial load succeeds, Nextflow will reload the configuration with secrets enabled.

As a result, config secrets must be used in a way that does not cause the config resolution to fail when secrets are not present.

For example:

```
includeConfig secrets.MY_SECRET
? "https://example.com/extra.config?secret=${secrets.MY_SECRET}"
: '/dev/null'
```

The above snippet includes a secured config only if the secret is present. Otherwise, it includes `/dev/null`, which is equivalent to including an empty file. The reference to `secrets.MY_SECRET` in the condition causes the config to be reloaded with secrets enabled, including secrets from plugins such as AWS secrets.

Added in version 24.04

Secrets can be accessed in the pipeline script using the built-in `secrets` variable. For example:

```
workflow {
  println "The secret is: ${secrets.MY_SECRET}"
}
```

warning

The above example is only meant to demonstrate how to access a secret, not how to use it. In practice, sensitive information should not be printed to the console or output files.

Secrets can be accessed by processes using the `secret` directive. For example:

```
process my_task {
  secret 'MY_ACCESS_KEY'
  secret 'MY_SECRET_KEY'

  script:
  """
  your_command --access \${MY_ACCESS_KEY} --secret \${MY_SECRET_KEY}
  """
}
```

In the above example, the secrets `MY_ACCESS_KEY` and `MY_SECRET_KEY` are injected into the process script as environment variables.

warning

Secrets are made available as environment variables in the process script. To prevent evaluation in the Nextflow script context, escape variable names with a backslash (e.g., `\${MY_ACCESS_KEY}`) as shown above.

Sharing pipelines

Nextflow seamlessly integrates with popular Git providers, including [BitBucket](#), [GitHub](#), and [GitLab](#) for managing Nextflow pipelines as version-controlled Git repositories. This feature allows you to easily use other people's Nextflow pipelines and publish your own pipelines.

note

Nextflow is not meant to completely replace the [Git](#) tool. You may still need `git` to create new repositories or commit changes, etc.

You can configure your credentials for various Git providers in the Git configuration file, located at `$HOME/.nextflow/scm`. See [Git](#) for more information.

Nextflow can work with repositories stored in a local or shared file system. The repository must be created as a [bare repository](#).

For example, given a bare repository at `/shared/projects/hello.git`, Nextflow is able to run it using the following syntax:

```
nextflow run file:/shared/projects/hello.git
```

See [Git documentation](#) for more details about how to create and manage bare repositories.

In order to publish your Nextflow pipeline to GitHub (or any other supported platform) and allow other people to use it, you only need to create a GitHub repository containing all your project script and data files. If you don't know how to do it, follow this simple tutorial that explains how to [create a GitHub repository](#).

Nextflow only requires that the main script in your pipeline project is called `main.nf`. A different name can be used by specifying the `manifest.mainScript` attribute in the `nextflow.config` file that must be included in your project. For example:

```
manifest.mainScript = 'my_very_long_script_name.nf'
```

To learn more about this and other project metadata information, that can be defined in the Nextflow configuration file, read the [Manifest](#) section on the Nextflow configuration page.

Once you have uploaded your pipeline project to GitHub other people can execute it simply using the project name or the repository URL.

For example, if your GitHub account name is `acme` and you have uploaded a project into a repository named `hello` the repository URL will be `http://github.com/acme/hello` and people will be able to download and run it by using either the command:

```
nextflow run acme/hello
```

or

```
nextflow run http://github.com/acme/hello
```

See [Command line](#) to learn how to use the Nextflow command line to run pipelines and manage pipeline projects.

Computational pipelines are rarely composed by a single script. In real world applications they depend on many other components, including other scripts and tools, databases, and specialized environments which provide compute and storage.

These external dependencies are the primary challenge when sharing software, because the users need to recreate the environment around a tool in order to use it. This setup process is often painful and error prone, which severely hinders the ability to reproduce computational results on a system other than the one on which it was originally developed.

Nextflow tackles this problem by integrating with existing tools for reproducible software, namely Git for source code and Docker for containers. These tools allow you to keep all the dependencies of your pipeline project in one place and track changes over time with version control.

By making your pipeline project self-contained, meaning all of its dependencies are fully defined in the project itself, you gain two major advantages:

- **Portability:** the pipeline can be run in virtually any environment with a Java VM and a container runtime
- **Reproducibility:** any results produced by the pipeline can be easily reproduced, even across different environments

One way to account for dependencies is to break them down into three categories: code, data, and environment. Here we will describe how to include each of these dependencies in your Nextflow pipeline:

Aside from pipeline scripts, you may have additional scripts and tools used by individual tasks.

Many standard tools can be accessed as Docker containers or Conda/Spack packages. In this case, you only need to specify the container image URL (e.g. from [DockerHub](#)) or package name with the `container` or `conda` directive, respectively.

Make sure to enable the desired method in your Nextflow configuration:

```
// containers
docker.enabled = true

// conda packages
conda.enabled = true
```

This way, when you launch your pipeline, Nextflow will automatically download the necessary dependencies to run your tasks based on this configuration.

Read the [Containers](#) page to learn more about how to use containers with Nextflow, and the [Conda environments](#) page for Conda packages.

tip

For maximal reproducibility, make sure to define a specific version for each tool. Otherwise, your pipeline might use different versions across subsequent runs, which can introduce subtle differences to your results.

As for custom scripts, you can include executable scripts in the `bin` directory of your pipeline repository. When configured correctly, these scripts can be executed like a regular command from any process script (i.e. without modifying the `PATH` environment variable or using an absolute path), and changing the script will cause the task to be re-executed on a resumed run (i.e. just like changing the process script itself).

To configure a custom script:

1. Save the script in the `bin` directory (relative to the pipeline repository root).
2. Specify a portable shebang (see note below for details).
3. Make the script executable. For example: `chmod a+x bin/my_script.py`

tip

To maximize the portability of your bundled script, use `env` to dynamically resolve the location of the interpreter instead of hard-coding it in the shebang line.

For example, shebang definitions `#!/usr/bin/python` and `#!/usr/local/bin/python` both hard-code specific paths to the Python interpreter. Instead, the following approach is more portable:

```
#!/usr/bin/env python
```

The `lib` directory can be used to add utility code or external libraries without cluttering the pipeline scripts. The `lib` directory in the Nextflow project root is added to the classpath by default. Classes defined in the `lib` directory will be available in pipeline scripts. Functions defined outside of classes will not be available in pipeline scripts.

For example, `lib/DNASequence.groovy` defines the `DNASequence` class:

```
// lib/DNASequence.groovy
class DNASequence {
    String sequence

    // Constructor
    DNASequence(String sequence) {
        this.sequence = sequence.toUpperCase() // Ensure sequence is in uppercase for consistency
    }

    // Method to calculate melting temperature using the Wallace rule
    double getMeltingTemperature() {
        int g_count = sequence.count('G')
        int c_count = sequence.count('C')
        int a_count = sequence.count('A')
        int t_count = sequence.count('T')

        // Wallace rule calculation
        double tm = 4 * (g_count + c_count) + 2 * (a_count + t_count)
        return tm
    }

    String toString() {
        return "DNA[$sequence]"
    }
}
```

The `DNASequence` class is available in the execution context:

```
// main.nf
workflow {
    channel.of('ACGTTGCAATGCCGTA', 'GCGTACGGTACGTTAC')
        .map { seq -> new DNASequence(seq) }
        .view { dna ->
            "Found sequence '$dna' with melting temperature ${dna.getMeltingTemperature()}°C"
        }
}
```

It prints:

```
Found sequence 'DNA[ACGTTGCAATGCCGTA]' with melting temperature 48.0°C
Found sequence 'DNA[GCGTACGGTACGTTAC]' with melting temperature 50.0°C
```

note

Package declarations in the `lib` directory are ignored. The package of a class is determined by the directory structure within the `lib` directory.

For example, if the above example were defined in `lib/Utils/DNASequence.groovy`, the class would need to be referenced in pipeline scripts as `Utils.DNASequence`.

In general, input data should be provided by external sources using parameters which can be controlled by the user. This way, a pipeline can be easily reused to process different datasets which are appropriate for the pipeline.

Parameters can be declared with default values in the main script or in the configuration file:

```
params.my_input = 'default input file'
params.my_output = 'default output path'
params.my_flag = false
// ...
```

When launching a pipeline, parameter values can be provided on the command line or in a params file (using the `-params-file` option). Options prefixed with a double dash (`--`) are interpreted as parameters:

```
nextflow run <your pipeline> --my_input /path/to/input/file --my_output /other/path --my_flag true
```

When a pipeline requires some small data that rarely changes, it may be easier to include the data in the pipeline repository. You can reference this data from the pipeline script in a portable manner (i.e. without relying on an absolute path) by using the `projectDir` implicit variable, which refers to the local copy of the pipeline repository.

The following example references the file `dataset/sequences.fa` in the pipeline repository:

```
sequences = file("$projectDir/dataset/sequences.fa")
sequences.splitFasta {
    println it
}
```

The "environment" refers to any other aspects of the environment in which your pipeline is executed, such as environment variables and resource managers like SLURM.

Any environment variable that may be required by the tools in your pipeline can be defined under the `env` scope in your Nextflow configuration. For example:

```
env {
    DELTA = 'hello'
    GAMMA = 'world'
}
```

Similarly, if you use an HPC scheduler like SLURM or a cloud batch service like AWS Batch to execute tasks in a distributed manner, you can use a configuration profile to define the settings for a given environment.

See [Configuration](#) for more information about Nextflow configuration and [Executors](#) for more information about executors.

VS Code integration

The [Nextflow VS Code extension](#) provides language support for Nextflow pipelines in [VS Code](#).

This page describes the [Nextflow language server](#) used by the extension. See the extension README in [GitHub](#) or the [Visual Studio Marketplace](#) for details on additional extension features.

The VS Code extension highlights Nextflow scripts and config files for better readability.

The extension highlights source code in red for errors and yellow for warnings.

To view all diagnostics for the workspace, open the **Problems** tab. Here, you can search for diagnostics by diagnostic message, filename, and so on.

note

The language server parses scripts and config files according to the [Nextflow language specification](#), which is more strict than the Nextflow CLI. See [Preparing for strict syntax](#) for more information.

Added in version 25.10

The extension can perform static type checking. Enable it using the **Nextflow > Type Checking** extension setting. See [Migrating to static types](#) for more information about migrating to static types.

When you hover over certain source code elements, such as variable names and function calls, the extension provides a tooltip with related information, such as the definition and/or documentation for the element.

If a [Javadoc](#) comment is defined above a workflow, process, function, or parameter in a `params` block, the extension will include the contents of the comment in hover hints. The following is an example Javadoc comment:

```
/**
 * Say hello to someone.
 *
 * @param name
 */
def sayHello(name) {
    println "Hello, ${name}!"
}
```

The **Outline** section in the Explorer panel lists top-level definitions when you view a script. Include declarations in scripts and config files act as links, and ctrl-clicking them opens the corresponding script or config file.

To view the definition of a symbol (e.g., a workflow, process, function, or variable), right-click the symbol and select **Go to Definition**. Ctrl-click the symbol to view the definition. Ctrl-click the definition to show all references.

The extension suggests auto-completions for variable names, function names, config settings, and other symbols as you type. The extension also provides several snippets for common script declarations, such as processes and workflows.

The extension can format your scripts and config files based on a standard set of formatting rules. Rules can be customized using the **Nextflow > Formatting** extension settings.

Use the **Format Document** command in the command palette to format the current file.

The extension can rename all references of a symbol (e.g., a workflow, process, function, or variable) throughout the code.

To rename a symbol, right-click the symbol, select **Rename Symbol**, and enter a new name.

If a `nextflow_schema.json` file exists in the same directory as a script with an entry workflow, the extension uses the schema to provide validation, hover hints, and code completion for params in the entry workflow.

The extension can generate a workflow DAG that includes the workflow inputs, outputs, and any processes or workflows that are called by the selected workflow. The workflow DAG is displayed in a new panel to the side.

The DAG preview is similar to the [workflow diagram](#) that can be generated at runtime using the `-with-dag` option. However, it is different in several ways:

- The DAG preview does not require executing Nextflow code.
- The DAG preview shows only a single workflow, whereas the workflow diagram shows the entire pipeline.
- The DAG preview shows all conditional processes, whereas the workflow diagram shows only the processes that were called in a specific run.

To preview the DAG of a workflow, select the **Preview DAG** CodeLens above the workflow definition.

The extension can automatically migrate scripts to static types. See [Migrating to static types](#) for details.

To migrate a script, open the Command Palette, search for **Convert script to static types**, and select it.

To migrate an entire pipeline, use the **Convert pipeline to static types** command.

In the event of a language server error, you can use the **Nextflow: Restart language server** command in the command palette to restart the language server.

Report issues at [nextflow-io/vscode-language-nextflow](#) or [nextflow-io/language-server](#). When reporting, include a minimal code snippet that reproduces the issue and any error logs from the server. To view logs, open the **Output** tab and select **Nextflow Language Server** from the dropdown. Enable **Nextflow > Debug** in the extension settings to show additional log messages while debugging.

Git filesystem changes

The language server does not detect certain filesystem changes, such as changing the current Git branch. Restart the language server from the command palette to sync it with your workspace.

Plugin definitions

The language server does not recognize configuration options from third-party plugins and will report "Unrecognized config option" warnings for them.

Added in version 25.10

The language server can recognize plugin definitions, including configuration options and functions, for plugins that use the [Nextflow Gradle plugin](#) and require Nextflow ≥ 25.10 . Custom factories and operators are not currently recognized.

The `lib` directory

The language server provides limited support for Groovy scripts in the `lib` directory. Errors in Groovy scripts are not reported as diagnostics, and changing a Groovy script does not automatically re-compile the Nextflow scripts that reference it. Edit the Nextflow script or close and re-open it to refresh the diagnostics.

The Nextflow language server implements the [Language Server Protocol \(LSP\)](#) for Nextflow scripts and config files. It is distributed as a standalone Java application and can be integrated with any editor that functions as an LSP client.

Currently, only the VS Code integration is officially supported, but community contributions for other editors are welcome. Visit the [GitHub issues](#) page for the latest updates on community-led integrations.

Using plugins

Nextflow core plugins require no additional configuration. When a pipeline uses a core plugin, Nextflow automatically downloads and uses the latest compatible plugin version. In contrast, third-party plugins must be explicitly declared. When a pipeline uses one or more third-party plugins, Nextflow must be configured to download and use the plugin.

A plugin identifier consists of the plugin name and version, separated by an @ symbol:

```
nf-hello@0.5.0
```

The plugin version is optional. If it is not specified, Nextflow will download the latest version of the plugin that meets the minimum Nextflow version requirements specified by the plugin.

note

Plugin versions are required for [offline usage](#).

Added in version 25.04

The plugin version can be prefixed with ~ to pin the major and minor versions and allow the latest patch release to be used. For example, `nf-amazon@~2.9.0` will resolve to the latest version matching `2.9.x`. When working offline, Nextflow will resolve version ranges against the local plugin cache defined by `NXF_PLUGINS_DIR`.

tip

It is recommended to pin the major and minor version of each plugin to minimize the risk of breaking changes while allowing patch updates to be used automatically.

Plugins can be configured via Nextflow configuration files or at runtime.

To configure a plugin via configuration files, use the `plugins` block. For example:

```
plugins {
  id 'nf-hello@0.5.0'
  id 'nf-amazon@2.9.0'
}
```

To configure plugins at runtime, use the `-plugins` option. For example:

```
nextflow run main.nf -plugins nf-hello@0.5.0,nf-amazon@2.9.0
```

note

Plugin declarations in Nextflow configuration files are ignored when specifying plugins via the `-plugins` option.

When Nextflow downloads plugins, it caches them in the directory specified by `NXF_PLUGINS_DIR` (`$HOME/.nextflow/plugins` by default).

When running Nextflow in an offline environment, any required plugins must be downloaded and moved into the offline environment prior to any runs.

To use Nextflow plugins in an offline environment:

1. Install a self-contained version of Nextflow in an environment with an internet connection. See [Standalone distribution](#) for more information.
2. Run `nextflow plugin install <plugin-name>@<version>` for each required plugin to download it. Alternatively, run the pipeline once, which will automatically download all plugins required by the pipeline.
3. Copy the `nextflow` binary and `$HOME/.nextflow` directory to the offline environment.
4. Specify each plugin and its version in Nextflow configuration files or at runtime. See [Configuration](#) for more information.

warning

Nextflow will attempt to download newer versions of plugins if their versions are not set. See [Identifiers](#) for more information.

Developing plugins

This page describes how to develop plugins for Nextflow.

The [Nextflow plugin template](#) is a scaffold for plugin development. It uses [Gradle](#), a build automation tool optimized for Java and Groovy projects, as well as the [Nextflow Gradle plugin](#).

You can use the `nextflow plugin create` sub-command to create plugins using the plugin template. See [Creating a plugin](#) for more information.

The plugin template includes the source directories, build configuration files, and metadata required for development, testing, and publishing. Depending on the developer's preference, plugins can be written in Java or Groovy.

For example, a plugin created from the plugin template with the name `nf-hello` and organization `nextflow` will have the following structure:

```
nf-hello
├── .github/workflows/build.yml
├── COPYING
├── Makefile
├── README.md
├── build.gradle
├── gradle
│   ├── wrapper
│   │   ├── gradle-wrapper.jar
│   │   └── gradle-wrapper.properties
│   └── gradlew
├── settings.gradle
├── src
│   ├── main
│   │   ├── groovy
│   │   │   ├── nextflow
│   │   │   │   └── hello
│   │   │   │       ├── HelloExtension.groovy
│   │   │   │       ├── HelloFactory.groovy
│   │   │   │       ├── HelloObserver.groovy
│   │   │   │       └── HelloPlugin.groovy
│   │   └── test
│   │       ├── groovy
│   │       │   ├── nextflow
│   │       │   │   └── hello
│   │       │   │       └── HelloObserverTest.groovy
│   └── validation
│       ├── main.nf
│       └── nextflow.config
```

This structure contains the following key files and folders:

- `.github/workflows/build.yml`: GitHub Action which implements continuous integration for the plugin.
- `build.gradle`: The Gradle build script.
- `COPYING`: The project license, detailing the terms under which the code can be used and distributed.
- `gradle/wrapper/`: Helper files for the Gradle Wrapper.

- `gradlew`: The Gradle Wrapper script, which allows you to use Gradle without installing it into your environment.
- `Makefile`: Defines common tasks for building, testing, and publishing the plugin with Make.
- `README.md`: The project README, which provides an overview of the project, including its purpose, features, and instructions for usage and development.
- `settings.gradle`: The Gradle project configuration, which specifies project-specific settings such as the project name and included modules.
- `src/main/groovy/<organization>/`: The main source directory, which contains the plugin source code and resources.
- `src/test/groovy/<organization>/`: The test source directory, which contains the plugin unit tests.
- `validation`: A small Nextflow pipeline which serves as an end-to-end test for the plugin.

The plugin template also implements the following example features:

- A custom trace observer that prints a message when the workflow starts and when the workflow completes (see `HelloObserver`).
- A custom function called `sayHello` (see `HelloExtension`).

The [Nextflow Gradle plugin](#) simplifies the development of Nextflow plugins. It provides default configuration required for Nextflow integration, as well as custom Gradle tasks for building, testing, and publishing plugins.

It is versioned and published to the [Gradle Plugin Portal](#), and can be declared and managed like any other dependency in the `build.gradle` file:

```
plugins {
    id 'io.nextflow.nextflow-plugin' version '1.0.0-beta.6'
}
```

note

You can develop Nextflow plugins without the Gradle plugin. However, this approach is only suggested if you are an advanced developer and your project is incompatible with the Gradle plugin.

The plugin template includes a Makefile which wraps the most important Gradle tasks provided by the Nextflow Gradle plugin.

These tasks can be executed with [Make](#). For example:

```
make assemble
```

The following `make` commands are available:

Compiles the Nextflow plugin code and assembles it into a zip archive.

Installs the plugin into the local Nextflow plugins directory.

Publishes the plugin. See [Publish a plugin](#) for more information.

Runs plugin unit tests. See [Test a plugin](#) for more information.

Nextflow's plugin system exposes various extension points. This section gives examples of typical extension points and how to use them.

Plugins can define custom CLI commands that are executable with the `nextflow plugin` command.

To implement a plugin-specific command, implement the `PluginExecAware` interface in your plugin entry point (the class that extends `BasePlugin`). Alternatively, implement the `PluginAbstractExec` trait, which provides an abstract implementation with some boilerplate code. This trait requires you to implement the `getCommands()` and `exec()` methods. For example:

```
import nextflow.cli.PluginAbstractExec
import nextflow.plugin.BasePlugin

class MyPlugin extends BasePlugin implements PluginAbstractExec {
    @Override
    List<String> getCommands() {
        [ 'hello' ]
    }

    @Override
    int exec(String cmd, List<String> args) {
        if( cmd == 'hello' ) {
            println "Hello! You gave me these arguments: ${args.join(' ')}"
            return 0
        }
        else {
            System.err.println "Invalid command: ${cmd}"
            return 1
        }
    }
}
```

The command can be run using the `nextflow plugin` command:

```
nextflow plugin my-plugin:hello --alpha --beta
```

See the [plugin](#) for usage information.

Plugins can access the resolved Nextflow configuration through the session object using `session.config.navigate()`. Several extension points provide the session object for this reason. This method allows you to query any configuration option safely. If the option isn't defined, it will return null.

A common practice is to use a custom config scope to define any configuration for your plugin. For example:

```
import nextflow.Session
import nextflow.trace.TraceObserver

class MyObserver implements TraceObserver {

    @Override
    void onFlowCreate(Session session) {
        final message = session.config.navigate('myplugin.createMessage')
        println message
    }
}
```

This option can then be set in your configuration file:

```
// dot syntax
myplugin.createMessage = "I'm alive!"

// block syntax
myplugin {
    createMessage = "I'm alive!"
}
```

Added in version 25.04

Changed in version 25.10

The `nextflow.config.schema` package was renamed to `nextflow.config.spec`.

Plugins can declare their configuration options by implementing the `ConfigScope` interface and declaring each config option as a field with the `@ConfigOption` annotation. For example:

```
import nextflow.config.spec.ConfigOption
import nextflow.config.spec.ConfigScope
import nextflow.config.spec.ScopeName
import nextflow.script.dsl.Description

@ScopeName('myplugin')
@Description('')
    The `myplugin` scope allows you to configure the `nf-myplugin` plugin.
    '')
class MyPluginConfig implements ConfigScope {

    @ConfigOption
    @Description('')
        Message to print to standard output when a run is initialized.
        '')
    final String createMessage

    // no-arg constructor is required to enable validation of config options
    MyPluginConfig() {
    }

    MyPluginConfig(Map opts) {
        this.createMessage = opts.createMessage
    }
}
```

This approach is not required to support plugin config options. However, it allows Nextflow to recognize plugin definitions when validating the configuration. See [Configuration scopes](#) for more information.

Plugins can define custom executors that can be used with the `executor` process directive.

To implement an executor, create a class in your plugin that extends the `Executor` class and implements the `ExtensionPoint` interface. Add the `@ServiceName` annotation to your class with the name of your executor. For example:

```
import nextflow.executor.Executor
import nextflow.util.ServiceName
import org.pf4j.ExtensionPoint

@ServiceName('my-executor')
class MyExecutor extends Executor implements ExtensionPoint {

    // ...

}
```

You can then use this executor in your pipeline:

```
process hello {
    executor 'my-executor'

    // ...

}
```

tip

See the source code of Nextflow's built-in executors for examples of how to implement various executor components.

Plugins can define custom filesystems that Nextflow can use to interact with external storage systems using a single interface. For more information about accessing remote files, see [Remote files](#).

To implement a custom filesystem, create a class in your plugin that extends [FileSystemProvider](#). Implement the `getScheme()` method to define the URI scheme for your filesystem. For example:

```
import java.nio.file.spi.FileSystemProvider

class MyFileSystemProvider extends FileSystemProvider {

    @Override
    String getScheme() {
        return 'myfs'
    }

    // ...

}
```

You can then use this filesystem in your pipeline:

```
input = file('myfs://<path-to-file>')
```

See [Developing a Custom File System Provider](#) for more information and the `nf-amazon` plugin (`S3FileSystemProvider`) for an example of a custom filesystem.

tip

Custom filesystems are an advanced plugin extension. Before creating a new filesystem, check that your use case cannot already be supported by an existing filesystem such as HTTP or S3.

Plugins can define custom functions that can be included in Nextflow pipelines.

To implement a custom function, create a plugin class that extends the `PluginExtensionPoint` class and implement your function with the `Function` annotation. For example:

```
import nextflow.Session
import nextflow.plugin.extension.Function
import nextflow.plugin.extension.PluginExtensionPoint

class MyExtension extends PluginExtensionPoint {

    @Override
    void init(Session session) {}

    @Function
    String reverseString(String origin) {
        origin.reverse()
    }
}
```

You can then add this function to your pipeline:

```
include { reverseString } from 'plugin/my-plugin'

channel.of( reverseString('hi') )
```

Alternatively, you can use an alias:

```
include { reverseString as anotherReverseMethod } from 'plugin/my-plugin'
```

Plugins can define custom channel factories and operators that can then be included in pipelines.

To implement a custom channel factory or operator, create a class in your plugin that extends the `PluginExtensionPoint` class and implement your function with the `Factory` or `Operator` annotation. For example:

```
import groovyx.gpars.dataflow.DataflowReadChannel
import groovyx.gpars.dataflow.DataflowWriteChannel
import nextflow.Session
import nextflow.plugin.extension.Factory
import nextflow.plugin.extension.Operator
import nextflow.plugin.extension.PluginExtensionPoint

class MyExtension extends PluginExtensionPoint {

    @Override
    void init(Session session) {}

    @Factory
    DataflowWriteChannel fromQuery(Map opts, String query) {
        // ...
    }

    @Operator
    DataflowWriteChannel sqlInsert(DataflowReadChannel source, Map opts) {
        // ...
    }
}
```

You can then use the custom channel factories or operators in your pipeline:

```
include { sqlInsert; fromQuery as fromTable } from 'plugin/nf-sqlldb'

def sql = 'select * from F00'
channel
    .fromTable(sql, db: 'test', emitColumns: true)
    .sqlInsert(into: 'BAR', columns: 'id', db: 'test')
```

note

The above snippet is based on the [nf-sqlldb](#) plugin. The `fromQuery` factory is included under the alias `fromTable`.

tip

Before creating a custom operator, consider whether the operator can be defined as a [function](#) that can be composed with existing operators such as `map` or `subscribe`. Functions are easier to implement and can be used anywhere in your pipeline, not just channel logic.

Plugins that implement a custom executor will likely need to access [process directives](#) that affect the task execution. When an executor receives a task, the process directives can be accessed through that task's configuration. Custom executors should try to support all process directives that have executor-specific behavior and are relevant to the executor.

Nextflow does not provide the ability to define custom process directives in a plugin. Instead, use the `ext` directive to provide custom process settings to your executor. Use specific names that are not likely to conflict with other plugins or existing pipelines.

For example, a custom executor can use existing process directives and a custom setting through the `ext` directive:

```
class MyExecutor extends Executor {

    @Override
    TaskHandler createTaskHandler(TaskRun task) {
        final cpus = task.config.cpus
        final memory = task.config.memory
        final myOption = task.config.ext.myOption

        println "This task is configured with cpus=${cpus}, memory=${memory}, myOption=${myOption}"

        // ...
    }

    // ...
}
```

Changed in version 25.04

The `TraceObserver` interface is now deprecated. Use [TraceObserverV2](#) and [TraceObserverFactoryV2](#) instead.

A trace observer is an entity that can listen and react to workflow events, such as when a workflow starts, a task is completed, or a file is published. Several components in Nextflow, such as the execution report and DAG visualization, are implemented as trace observers.

Plugins can define custom trace observers that react to workflow events with custom behavior. To implement a trace observer, create a class that implements the `TraceObserver` trait and another class that implements the `TraceObserverFactory` interface. Implement any of the hooks defined in `TraceObserver` and implement the `create()` method in your observer factory. For example:

```
import java.nio.file.Path

import nextflow.processor.TaskHandler
import nextflow.trace.TraceObserver
import nextflow.trace.TraceRecord

class MyObserver implements TraceObserver {

    @Override
    void onFlowBegin() {
        println "Okay, let's begin!"
    }

    @Override
    void onProcessComplete(TaskHandler handler, TraceRecord trace) {
        println "I completed a task! It's name is '${handler.task.name}'"
    }

    @Override
    void onProcessCached(TaskHandler handler, TraceRecord trace) {
        println "I found a task in the cache! It's name is '${handler.task.name}'"
    }

    @Override
    void onFilePublish(Path destination, Path source) {
        println "I published a file! It's located at ${path.toUriString()}"
    }

    @Override
    void onFlowError(TaskHandler handler, TraceRecord trace) {
        println "Uh oh, something went wrong..."
    }

    @Override
    void onFlowComplete() {
        println 'All done!'
    }
}
```

You can then use your trace observer by simply enabling the plugin in your pipeline. In the above example, the observer must also be enabled with a config option:

```
myplugin.enabled = true
```

See the [TraceObserver source code](#) for descriptions of the available workflow events.

The following environment variables are available to develop and test plugins:

The plugin execution mode. Either `prod` for production or `dev` for development.

The path where the plugin archives are loaded and stored (default: `$NXF_HOME/plugins` in production and `./plugins` in development).

Whether to use the default plugins when no plugins are specified in the Nextflow configuration (default: `true`).

Comma-separated list of development plugin root directories.

Added in version 23.04

Comma-separated list of URIs for additional plugin registries or meta files, which will be used in addition to the default registry.

The URI should refer to a plugin repository JSON file or a specific plugin JSON meta file. In the latter case, it should match the pattern `https://host.name/some/path/<PLUGIN_NAME>-X.Y.Z-meta.json`. For example:

```
# custom plugin repository at https://github.com/my-org/plugins
export NXF_PLUGINS_TEST_REPOSITORY="https://raw.githubusercontent.com/my-org/plugins/main/plugins.json"

# custom plugin release
export NXF_PLUGINS_TEST_REPOSITORY="https://github.com/nextflow-io/nf-hello/releases/download/0.3.0/nf-hello-0.3.0-meta.json"

nextflow run main.nf -plugins nf-hello
```

This variable is useful for testing a plugin release before publishing it to the main registry.

Nextflow plugin registry

The [Nextflow plugin registry](#) is a central repository for Nextflow plugins. It hosts an index of plugin metadata that supports plugin discovery, accessibility, and version tracking.

Nextflow 25.10 and later can use the plugin registry as a drop-in replacement for the [legacy plugin index](#) hosted on GitHub. See [Migrating to the Nextflow plugin registry](#) for more information about migrating to the Nextflow plugin registry.

See [Publishing a plugin](#) for instructions on how to publish plugins to the registry, including the [README.md requirement](#).

Ownership of a plugin is required to publish plugins to the Nextflow plugin registry.

To claim ownership of a plugin:

1. Open the [Nextflow plugin registry](#) in a browser.
2. Log in to [Seqera](#) with your GitHub or Google account, or by providing an email address.

note

If you are logging in for the first time, Seqera will send an authentication link to your email address to complete the login process.

3. Go to the **My plugins** page and select **Claim a plugin**.
4. Enter your unique plugin name or select the plugin you wish to claim in the **Plugin name** field.
5. Enter your organization name in the **Provider** field.

note

Your organization must match the provider specified when publishing your plugin.

6. Select **Submit Request**.

The plugin will show as **PENDING REVIEW** under **Pending Ownership Requests** until an admin approves the claim. Admin approval is required only once.

An API access token is required to publish plugins to the Nextflow plugin registry.

To create an API access token:

1. Open the [Nextflow plugin registry](#) in a browser.
2. Log in to [Seqera](#) with your GitHub or Google account, or by providing an email address.

note

If you are logging in for the first time, Seqera will send an authentication link to your email address to complete the login process.

3. Go to the **Access tokens** page.
 4. Under **Create New Access Token**, enter a descriptive name for the **Token name** and select the token duration from the **Expiry** drop down.
 5. Select **Generate token**.
 6. Copy and past token somewhere safe, you won't be able to see it again.
- Once you have your token, see [Publishing a plugin](#) for instructions on how to use it.

Syntax

This page provides a comprehensive description of the Nextflow language.

A line comment starts with `//` and includes the rest of the line.

```
println 'Hello world!' // line comment
```

A block comment starts with `/*` and includes all subsequent characters up to the first `*/`.

```
/*  
 * block comment  
 */  
println 'Hello again!'
```

A Nextflow script may contain the following top-level declarations:

- Shebang
- Feature flags
- Include declarations
- Params block
- Parameter declarations (legacy)
- Workflow definitions
- Process definitions
- Function definitions
- Enum types
- Record types
- Output block

Script declarations are in turn composed of statements and expressions.

If there are no script declarations, a script may contain one or more [statements](#), in which case the entire script is treated as an entry workflow. For example:

```
println 'Hello world!'
```

Is equivalent to:

```
workflow {  
    println 'Hello world!'  
}
```

warning

Statements and script declarations can not be mixed at the same level.

The first line of a script can be a [shebang](#):

```
#!/usr/bin/env nextflow
```

A feature flag declaration is an assignment. The target should be a valid [feature flag](#) and the source should be a literal (i.e. number, string, boolean):

```
nextflow.preview.recursion = true
```

An include declaration consists of an include source and one or more include clauses:

```
include { hallo as sayHello } from './some/module'
```

The include source should be a string literal. Each include clause should specify a name, and may also specify an alias. In the above example, `hallo` is included under the alias `sayHello`.

Include clauses can be separated by semi-colons or newlines:

```
// semi-colons
include { hallo ; bye as goodbye } from './some/module'

// newlines
include {
    hallo
    bye as goodbye
} from './some/module'
```

Include clauses can also be specified as separate includes:

```
include { hallo } from './some/module'
include { bye as goodbye } from './some/module'
```

The params block consists of one or more parameter declarations. A parameter declaration consists of a name, type, and an optional default value:

```
params {
    input: Path
    save_intermeds: Boolean = false
}
```

Only one params block may be defined in a script.

A legacy parameter declaration is an assignment. The target should be a pipeline parameter and the source should be an expression:

```
params.message = 'Hello world!'
```

Parameters supplied via command line options, params files, and config files take precedence over parameter definitions in a script.

A workflow can be a named workflow or an entry workflow.

A named workflow consists of a name and a body, and may consist of a take, main, and emit section:

```

workflow greet {
  take:
  greetings

  main:
  messages = greetings.map { v -> "$v world!" }

  emit:
  messages
}

```

- The take, emit, and publish sections are optional. The `main:` section label can be omitted if they are not specified.
- The take section consists of one or more parameters.
- The main section consists of one or more [statements](#).
- The emit section consists of one or more emit statements. An emit statement can be a [variable name](#), an [assignment](#), or an [expression statement](#). If an emit statement is an expression statement, it must be the only emit.

An entry workflow has no name and may consist of a main, publish, onComplete, and onError section:

```

workflow {
  main:
  greetings = channel.of('Bonjour', 'Ciao', 'Hello', 'Hola')
  messages = greetings.map { v -> "$v world!" }
  greetings.view { v -> "$v world!" }

  publish:
  messages = messages

  onComplete:
  log.info 'Workflow completed successfully!'

  onError:
  log.error 'Workflow failed.'
}

```

- Only one entry workflow may be defined in a script.
- The `main:` section label can be omitted if the other sections are not specified.
- The publish section consists of one or more publish statements. A publish statement is an [assignment](#), where the assignment target is the name of a workflow output.
- The `onComplete` and `onError` sections consist of one or more [statements](#).

A typed workflow is a workflow that uses static typing for inputs and outputs:

```

nextflow.enable.types = true

workflow greet {
  take:
  greetings: Channel<String>

  main:
  messages = greetings.map { v -> "$v world!" }

  emit:
  messages: Channel<String>
}

```

Typed workflows have the following new features:

- Each workflow input in the `take:` section has a name and a type.
- Each named workflow output in the `emit:` section may specify a type.

See [Workflows \(typed\)](#) for more information on the semantics of typed workflows.

A process consists of a name and a body. The process body consists of one or more [statements](#). A minimal process definition must return a string:

```
process hello {  
  """  
  echo 'Hello world!'  
  """  
}
```

A process may define additional sections for directives, inputs, outputs, script, exec, and stub:

```
process greet {  
  // directives  
  errorStrategy 'retry'  
  tag { "${greeting}/${name}" }  
  
  input:  
  val greeting  
  val name  
  
  output:  
  stdout  
  
  script: // or exec:  
  """  
  echo '${greeting}, ${name}!'  
  """  
  
  stub:  
  """  
  # do nothing  
  """  
}
```

- A process must define a script or exec section (see below). All other sections are optional. Directives do not have an explicit section label, but must be defined first.
- The `script:` section label can be omitted only when there are no other sections in the body.
- Sections must be defined in the order shown above, with the exception of the output section, which can also be specified after the script and stub.

Each section may contain one or more statements. For directives, inputs, and outputs, these statements must be [function calls](#). See [Process reference](#) for the set of available input qualifiers, output qualifiers, and directives.

The script section can be substituted with an exec section:

```

process greet {
  input:
  val greeting
  val name

  exec:
  message = "${greeting}, ${name}!"

  output:
  val message
}

```

The script and stub sections must return a string in the same manner as a [function](#).

See [Processes](#) for more information on the semantics of each process section.

A typed process is a process that uses static types for inputs and/or outputs:

```

process greet {
  input:
  greeting: String
  name: String

  stage:
  env 'NAME', name

  output:
  stdout()

  topic:
  eval('bash --version') >> 'versions'

  script:
  """
  echo "${greeting}, \${NAME}!"
  """
}

```

Typed processes may specify the following sections:

Consists of one or more process inputs. Each input has a name and type.

Consists of one or more stage directives. See [Inputs and outputs \(typed\)](#) for the set of available stage directives.

Consists of one or more output statements. An output statement can be a [variable name](#), an [assignment](#), or an [expression statement](#). An output statement must be the only output if it is an expression statement. See [Inputs and outputs \(typed\)](#) for the set of available output functions.

Consists of one or more topic statements. A topic statement is a right-shift expression with an output value on the left side and a string on the right side.

note

Typed processes use the same behavior as legacy processes for all other sections.

See [Process \(typed\)](#) for more information on the semantics of typed processes.

A function consists of a name, parameter list, and a body:


```
def greet(greeting, name) {
  println "${greeting}, ${name}!"
}
```

The function body consists of one or more [statements](#).

The [return statement](#) can be used to explicitly return from a function:

```
// return with no value
def greet(greeting, name) {
  if( !greeting || !name )
    return
  println "${greeting}, ${name}!"
}

// return a value
def fib(x) {
  if( x <= 1 )
    return x
  fib(x - 1) + fib(x - 2)
}
```

An enum type declaration consists of a name and a body. The body consists of a comma-separated list of identifiers:

```
enum Day {
  MONDAY,
  TUESDAY,
  WEDNESDAY,
  THURSDAY,
  FRIDAY,
  SATURDAY,
  SUNDAY
}
```

A record type declaration consists of a name and a body. The body consists of one or more fields, where each field has a name and a type:

```
record FastqPair {
  id: String
  fastq_1: Path
  fastq_2: Path
}
```

The output block consists of one or more output declarations. Each output declaration consists of a name and one or more output directives for defining the output:

```
output {
  samples {
    path 'fastq'
    index {
      path 'index.csv'
    }
  }
}
```

Only one output block may be defined in a script. See [Workflow outputs](#) for the set of available output directives.

Statements can be separated by either a newline or a semi-colon:

```
// newline
println 'Hello!'
println 'Hello again!'

// semi-colon
println 'Hello!' ; println 'Hello again!'
```

Variables can be declared with the `def` keyword:

```
def x = 42
```

Multiple variables can be declared in a single statement if the initializer is a tuple with the same number of elements as declared variables:

```
def (x, y) = tuple(1, 2)
```

An assignment statement consists of a target expression and a source expression separated by an equals sign:

```
v = 42
list[0] = 'first'
map.key = 'value'
```

The target expression must be a [variable](#), [index](#), or [property](#) expression. The source expression can be any expression.

Multiple variables can be assigned in a single statement when the source expression is a tuple with the same number of elements as assigned variables:

```
(x, y) = tuple(1, 2)
```

A compound assignment combines a binary operator with the equals sign. The following compound assignment operators are available: `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`, `?=`. For example, `x += 1` is equivalent to `x = x + 1`:

```
x += 1
```

Any [expression](#) can be a statement.

In general, the only expressions that can have any effect as expression statements are function calls that have side effects (e.g. `println`) or an implicit return statement in a [function](#) or [closure](#).

An assert statement consists of the `assert` keyword followed by a boolean expression, with an optional error message separated by a colon:

```
assert 2 + 2 == 4 : 'The math broke!'
```

An if/else statement consists of an if branch and an optional else branch. Each branch consists of a boolean expression in parentheses, followed by either a single statement or a block statement (one or more statements in curly braces). For example:

```
def x = Math.random()
if( x < 0.5 ) {
  println 'You lost.'
}
else {
  println 'You won!'
}
```

If/else statements can be chained any number of times by making the else branch another if/else statement:

```
def grade = 89
if( grade >= 90 )
  println 'You get an A!'
else if( grade >= 80 )
  println 'You get a B!'
else if( grade >= 70 )
  println 'You get a C!'
else if( grade >= 60 )
  println 'You get a D!'
else
  println 'You failed.'
```

A return statement consists of the `return` keyword with an optional expression:

```
def add(a, b) {
  return a + b
}

def sayHello(name) {
  if( !name )
    return
  println "Hello, ${name}!"
}
```

A throw statement consists of the `throw` keyword followed by an expression that returns an error type:

```
throw new Exception('something failed!')
```

A try/catch statement consists of a try block followed by any number of catch clauses:

```
def text = null
try {
  text = file('hello.txt').text
}
catch( IOException e ) {
  log.warn "Could not load hello.txt"
}
```

An expression represents a value. A literal value is an expression whose value is known at compile-time, such as a number, string, or boolean. All other expressions must be evaluated at run-time.

Every expression has a type, which may be resolved at compile-time or run-time.

A variable expression is a reference to a variable or other named value:

```
def x = 42

x
// -> 42
```

A number literal can be an integer or floating-point number, and can be positive or negative. Integers can be specified in binary with `0b`, octal with `0o`, or hexadecimal with `0x`. Floating-point numbers can use scientific notation with the `e` or `E` prefix. Underscores can be used as thousands separators to make long numbers more readable.

```
// integer
42
-1
0b1001 // -> 9
031    // -> 25
0xabcd // -> 43981

// real
3.14
-0.1
1.59e7 // -> 15_900_000
1.59e-7 // -> 0.000000159
```

A boolean literal can be `true` or `false`:

```
assert true != false
assert !true == false
assert true == !false
```

The null literal is specified as `null`. It can be used to represent an "empty" value:

```
def x = null
x = 42
```

A string literal consists of arbitrary text enclosed by single or double quotes:

```
println "I said 'hello'"
println 'I said "hello" again!'
```

A triple-quoted string can span multiple lines:

```
println '''
  Hello,
  How are you today?
'''

println """
  We don't have to escape quotes anymore!
  Even "double" quotes!
"""
```

A slashy string is enclosed by slashes instead of quotes:

```
/no escape!/
```

note

A slashy string cannot be empty because it would become a line comment.

In single- and double-quoted strings (including their triple-quoted forms), the backslash character (`\`) escapes characters that would otherwise have special meaning, or inserts special characters:

Escape	Character
<code>\b</code>	Backspace
<code>\t</code>	Tab (horizontal)
<code>\n</code>	Newline (line feed)
<code>\f</code>	Form feed
<code>\r</code>	Carriage return
<code>\s</code>	Space
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\\</code>	Backslash
<code>\\$</code>	Dollar sign (prevents interpolation)
<code>\ newline</code>	Line continuation (the newline is removed)
<code>\u HHHH</code>	Unicode character with the given hexadecimal code point (e.g. <code>é</code>)
<code>\ NNN</code>	Character with the given octal value (e.g. <code>\101</code>)

A backslash followed by any other character is a syntax error. To include a literal backslash, escape it as `\\`.

A slashy string does not interpret any of the above escapes, which makes it well-suited for regular expressions. The only exception is `\/`, which escapes a forward slash so that it does not terminate the string.

Double-quoted strings can be interpolated using the `${}` placeholder with an expression:

```
def names = ['Thing 1', 'Thing 2']
println "Hello, ${names.join(' and ')}!"
// -> Hello, Thing 1 and Thing 2!
```

If the expression is a name or simple property expression (one or more identifiers separated by dots), the curly braces can be omitted:

```
def name = [first: '<FIRST_NAME>', last: '<LAST_NAME>']
println "Hello, $name.first $name.last!"
// -> Hello, <FIRST_NAME> <LAST_NAME>!
```

Multi-line double-quoted strings can also be interpolated:

```
"""
blastp \
-in $input \
-out $output \
-db $blast_db \
-html
"""
```

A list literal consists of a comma-separated list of zero or more expressions, enclosed in square brackets:

```
[1, 2, 3]
```

A map literal consists of a comma-separated list of one or more map entries, enclosed in square brackets. Each map entry consists of a key expression and value expression separated by a colon:

```
[alpha: 1, beta: 2, gamma: 3]
```

An empty map is specified with a single colon to distinguish it from an empty list:

```
[:]
```

Both the key and value can be any expression. Identifier keys are treated as string literals (i.e. the quotes can be omitted). A variable can be used as a key by enclosing it in parentheses:

```
def x = 'alpha'
[(x): 1]
// -> ['alpha': 1]
```

A closure, also known as an anonymous function, consists of a parameter list followed by zero or more statements, enclosed in curly braces:

```
{ a, b -> a + b }
```

The above closure takes two arguments and returns their sum.

The closure body is identical to that of a [function](#). Statements should be separated by newlines or semi-colons:

```
{ v ->
  println 'Hello!'
  println "We're in a closure!"
  println 'Goodbye...'
  v * v
}
```

See [standard library](#) and [operator](#) for more examples of how closures are used in practice.

An index expression consists of a left expression and a right expression, with the right expression enclosed in square brackets:

```
myList[0]
```

A property expression consists of an object expression and a property, separated by a dot, spread dot, or safe dot:

```
myFile.text      // dot
myFiles*.text    // spread dot:  myFiles.collect { myFile -> myFile.text }
myFile?.text     // safe dot:   myFile != null ? myFile.text : null
```

The property must be an identifier or string literal.

A function call consists of a name and argument list:

```
printf('Hello %s!\n', 'World')
```

A method call consists of an object expression and a function call separated by a dot, spread dot, or safe dot:

```
myFile.getText()    // dot
myFiles*.getText()  // spread dot:  myFiles.collect { myFile -> myFile.getText() }
myFile?.getText()   // safe dot:    myFile != null ? myFile.getText() : null
```

The argument list may contain any number of positional arguments and named arguments:

```
file('hello.txt', checkIfExists: true)
```

The argument name must be an identifier or string literal.

The parentheses can be omitted when the function call is also an [expression statement](#) and there is at least one argument:

```
// positional args
printf 'Hello %s!\n', 'World'

// positional and named args
file 'hello.txt', checkIfExists: true

// named args
resourceLabels group: 'my-group', user: 'me'
```

If the last argument is a closure, it can be specified outside of the parentheses:

```
// closure arg with additional args
[1, 2, 3].inject('result:') { acc, v -> acc + ' ' + v }

// single closure arg
[1, 2, 3].each() { v -> println v }

// single closure arg without parentheses
[1, 2, 3].each { v -> println v }
```

A constructor call consists of the `new` keyword followed by a type name and an argument list enclosed in parentheses:

```
new java.util.Date()
```

If the type is implicitly available in the script, the fully-qualified type name can be elided to the simple type name:

```
new Date()
```

See [Standard library](#) for the set of types that are available in Nextflow scripts.

A unary expression consists of a unary operator followed by an expression:

```
!(2 + 2 == 4)
```

The following unary operators are available:

- `~`: bitwise NOT
- `!`: logical NOT
- `+`: unary plus
- `-`: unary minus

A binary expression consists of a left expression and a right expression separated by a binary operator:

```
2 + 2
```

The following binary operators are available:

- `**`: power (i.e. exponentiation)
- `*`: multiplication
- `/`: division
- `%`: remainder (i.e. modulo)
- `+`: addition
- `-`: subtraction
- `<<`: left shift
- `>>`: right shift
- `>>>`: unsigned right shift
- `..`: inclusive range
- `..<`: right-exclusive range
- `as`: type cast
- `instanceof`: type relation
- `!instanceof`: negated type relation
- `<`: less than
- `>`: greater than
- `<=`: less than or equals
- `>=`: greater than or equals
- `in`: membership
- `!in`: negated membership
- `==`: equals
- `!=`: negated equals
- `<=>`: spaceship (i.e. three-way comparison)
- `==~`: regex find
- `==~`: regex match
- `&`: bitwise AND
- `^`: bitwise XOR (exclusive or)
- `|`: bitwise OR
- `&&`: logical AND
- `||`: logical OR
- `?:` elvis (i.e. short ternary)

A ternary expression consists of a test expression, a true expression, and a false expression, separated by a question mark and a colon:

```
println x % 2 == 0 ? 'x is even!' : 'x is odd!'
```

Any expression can be enclosed in parentheses:


```
1 + 2 * 3
// -> 1 + 6 -> 7

(1 + 2) * 3
// -> 3 * 3 -> 9
```

Compound expressions are evaluated in the following order:

- parentheses
- property expressions
- function calls
- index expressions
- `~`, `!`
- `**`
- `+`, `-` (unary)
- `*`, `/`, `%`
- `+`, `-` (binary)
- `<<`, `>>>`, `>>`, `..`, `..<`
- `as`
- `instanceof`, `!instanceof`
- `<`, `>`, `<=`, `>=`, `in`, `!in`
- `==`, `!=`, `<=>`
- `==~`, `==~`
- `&`
- `^`
- `|`
- `&&`
- `||`
- `?:` (ternary)
- `?:` (elvis)

The following legacy features were excluded from this page because they are deprecated:

- The `addParams` and `params` clauses of include declarations.
- The `when:` section of a process definition.
- The `shell:` section of a process definition.

See [strict syntax](#) for more information.

Semantics

The semantics of the Nextflow language are the meaning and behavior of each language construct. This page is a companion to the [Syntax](#) reference, which describes the grammar of the language (how constructs are written). See the [Scripts](#) page for a practical introduction to the language.

Every expression in Nextflow produces a value, and every value has a [type](#) that determines the operations it supports. This section describes the kinds of values in the language and how they behave. See the [Types](#) reference for the full set of operations available on each type, and [Syntax](#) for the grammar of each literal.

A boolean is either `true` or `false`. Booleans are produced by the [relational](#) and [logical](#) operators and consumed by constructs that test a condition, such as [if/else](#) and [assertions](#).

Any value can be coerced to a boolean based on its truthiness. A value that coerces to `true` is truthy, and a value that coerces to `false` is falsy.

The truthiness of a value depends on its type:

Type	Falsy values	Truthy values
Boolean	<code>false</code>	<code>true</code>
Number	zero (<code>0</code> , <code>0.0</code>)	any non-zero number
String	empty string (<code>''</code>)	any non-empty string
Collection	empty collection (<code>[]</code>)	any non-empty collection
Map	empty map (<code>[:]</code>)	any non-empty map
Everything else	<code>null</code>	any non-null value

For example:

```
assert true
assert 42
assert 'hello'
assert [1, 2, 3]
assert [key: 'value']

assert !false
assert !0
assert !''
assert ![ ]
assert ![ :]
assert !null
```

A number is either an integer or a floating-point number.

Numbers support the standard [arithmetic operators](#). Arithmetic between two integers produces an integer, with one exception: division always produces a floating-point result. Use the `intdiv` method for integer division.

```
assert 2 + 3 == 5
assert 7 / 2 == 3.5      // division produces a float
assert 7.intdiv(2) == 3  // integer division
```

When an operation mixes an integer and a floating-point operand, the integer is coerced to floating-point.

See [Integer](#) and [Float](#) for the available operations.

A string is a sequence of characters, written with single or double quotes (see [String](#) for the grammar). Strings are immutable, and can be concatenated with the `+` operator:

```
def greeting = 'Hello, ' + 'world!'
```

A double-quoted string supports interpolation -- embedded expressions are evaluated and their results inserted into the string. A single-quoted string is a literal and is never interpolated:

```
def name = 'world'
println "Hello, ${name}!" // -> Hello, world!
println 'Hello, ${name}!' // -> Hello, ${name}!
```

An interpolated expression is written as `${expression}`. The curly braces can be omitted for a simple variable or property reference:

```
def sample = [id: 'A', count: 42]
println "Sample $sample.id has $sample.count reads"
// -> Sample A has 42 reads
```

Each interpolated expression is evaluated when the string is created, and its result is converted to a string using the value's `toString()` method. Interpolation is eager -- the expression is evaluated immediately, not deferred:

```
def x = 1
def s = "x is ${x}"
x = 2
println s // -> x is 1
```

Single-quoted strings are not interpolated:

```
println 'Hello, ${name}!'
// -> Hello, ${name}!
```

See [String](#) for the available operations.

The special value `null` represents the absence of a value. Accessing a property or calling a method on `null` raises a null reference error. Avoid `null` where possible. The [safe navigation operator](#) can be used to access a property that may be `null`.

A collection is a group of values. Nextflow provides three kinds of collection, which differ in whether they preserve the order of their elements and whether they allow duplicates:

Collection	Ordered	Duplicates
List	yes	yes
Set	no	no
Bag	no	yes

A list is the most commonly used collection. It is written with square brackets, and its elements are accessed by index, starting at zero. Lists are mutable -- elements can be added, removed, or replaced:

```
def numbers = [1, 2, 3]
assert numbers[0] == 1
```

A set is an unordered collection that cannot contain duplicates. It is typically created from a list with the `toSet()` method:

```
def unique = [1, 2, 2, 3].toSet()
assert unique.size() == 3
```

A bag is an unordered collection that allows duplicates. It is used where the order of elements is not significant.

All collections can be iterated with [higher-order functions](#) and tested with the [membership operators](#). The `+` operator returns a new collection containing the elements of both operands, rather than modifying either one:

```
assert [1, 2] + [3, 4] == [1, 2, 3, 4]
```

See [List<E>](#), [Set<E>](#), and [Bag<E>](#) for the available operations.

A map is a collection of key-value pairs, written with square brackets:

```
def scores = [alice: 100, bob: 92]
```

Values are accessed by key, using either the index operator or a property expression:

```
assert scores['alice'] == 100
assert scores.alice == 100
```

The `+` operator returns a new map containing the entries of both operands, with the right operand taking precedence on conflicting keys:

```
assert [a: 1, b: 2] + [b: 3] == [a: 1, b: 3]
```

See [Map<K,V>](#) for the available operations.

Added in version 26.04

A record holds a set of named fields, where each field can have its own type. Records are created with the `record()` function:

```
def person = record(name: 'Alice', age: 42)
```

Fields are accessed by name:

```
assert person.name == 'Alice'
```

Records are immutable -- once a record is created, it cannot be modified. Combine two records using the `+` operator to produce a new record:

```
def older = person + record(age: 43)
// -> record(name: 'Alice', age: 43)
```

See [Record](#) for the available operations.

A tuple is a fixed sequence of values, which may have different types. Tuples are created with the `tuple()` function:

```
def pair = tuple('sample_1', file('sample_1.fastq'))
```

Tuple elements are accessed by index:

```
assert pair[0] == 'sample_1'
```

Tuples are immutable -- once a tuple is created, its elements cannot be modified.

A tuple can be destructured into multiple variables, both in [assignments](#) and in [closure](#) parameters.

See [Tuple](#) for the available operations.

A closure is an anonymous function that is itself a value. A closure consists of a parameter list and a body, enclosed in curly braces:

```
def square = { v -> v * v }
assert square.call(9) == 81
```

Closures have the same parameter and return semantics as [function](#).

A closure can access variables from the enclosing scope, and can declare local variables that exist only for the duration of each invocation:

```
def factor = 2
assert [1, 2, 3].collect { v -> factor * v } == [2, 4, 6]
```

When a closure receives a single [tuple](#) argument, the parameter list can destructure the tuple. The number of parameters must match the number of tuple elements:

```
def samples = [
  tuple('sample_1', 100),
  tuple('sample_2', 200),
]

samples.each { id, count ->
  println "${id}: ${count}"
}
```

Within a nested closure, `return` exits the nearest enclosing closure, not the outer function.

The primary use of a closure is as an argument to a higher-order function -- a function that takes a closure and applies it, such as the [iteration](#) methods of a collection or the [operators](#) of a channel.

An operator is a symbol that performs an operation on one or two operands. The full set of operators and their grammar is listed in the [Syntax](#) reference. This section describes their semantics, grouped by category.

note

The operators described here are language operators, which apply to ordinary values. They are different from channel operators, which are functions for manipulating [channels](#). See [Operators](#) for channel operators.

Arithmetic operators perform numeric computation:

Operator	Operation	Example
<code>+</code>	addition	<code>2 + 3 -> 5</code>
<code>-</code>	subtraction	<code>5 - 2 -> 3</code>
<code>*</code>	multiplication	<code>2 * 3 -> 6</code>
<code>/</code>	division	<code>7 / 2 -> 3.5</code>
<code>%</code>	remainder (modulo)	<code>7 % 3 -> 1</code>
<code>**</code>	power (exponentiation)	<code>2 ** 8 -> 256</code>

Division of two integers produces a floating-point result (`7 / 2` is `3.5`, not `3`). To perform integer division, use the `intdiv` method: `7.intdiv(2)` is `3`.

Some arithmetic operators are overloaded for non-numeric types. In particular, `+` concatenates strings, lists, and maps:

```
assert 'foo' + 'bar' == 'foobar'
assert [1, 2] + [3, 4] == [1, 2, 3, 4]
assert [a: 1] + [b: 2] == [a: 1, b: 2]
```

Relational operators compare two values and produce a boolean:

Operator	Operation
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code>	less than
<code>></code>	greater than
<code><=</code>	less than or equal to
<code>>=</code>	greater than or equal to
<code><=></code>	three-way comparison (spaceship)

In Nextflow, `==` tests value equality, not object identity -- two distinct objects with the same contents are equal:

```
assert [1, 2, 3] == [1, 2, 3]
assert 'hello' == 'hello'
```

Ordering operators (`<`, `>`, `<=`, `>=`) compare values according to their natural order: numbers numerically and strings alphabetically. The spaceship operator `<=>` returns `-1`, `0`, or `1` depending on whether the left operand is less than, equal to, or greater than the right operand, and is useful for defining custom sort comparators:

```
def items = ['banana', 'apple', 'cherry']
items.toSorted { a, b -> a <=> b }
// -> ['apple', 'banana', 'cherry']
```

Logical operators combine boolean values. Their operands are coerced based on [truthiness](#):

Operator	Operation
<code>&&</code>	logical AND
<code> </code>	logical OR
<code>!</code>	logical NOT

The `&&` and `||` operators short-circuit: the right operand is evaluated only if necessary. For `&&`, the right operand is evaluated only if the left is `truthy`. For `||`, it is evaluated only if the left is `falsy`. This allows the right operand to safely depend on the left:

```
if( reads != null && reads.size() > 0 )
    // reads.size() is only evaluated if reads is not null
```

Bitwise operators operate on the individual bits of integer values:

Operator	Operation
<code>&</code>	bitwise AND
<code> </code>	bitwise OR
<code>^</code>	bitwise XOR
<code>~</code>	bitwise NOT
<code><<</code>	left shift
<code>>></code>	right shift
<code>>>></code>	unsigned right shift

```
assert (0b1100 & 0b1010) == 0b1000 // -> 8
assert (0b1100 | 0b1010) == 0b1110 // -> 14
assert (1 << 4) == 16
```

note

The `>>` operator is also used in [typed processes](#) to emit values to a topic, and in [workflow outputs](#) to publish files.

Conditional operators select a value based on a condition.

The ternary operator `cond ? a : b` evaluates `cond` (using [truthiness](#)) and returns `a` if it is truthy, otherwise `b`:

```
def label = sample.single_end ? 'single' : 'paired'
```

The Elvis operator `a ?: b` is a shorthand that returns `a` if it is truthy, otherwise `b`. It is commonly used to supply a default value:

```
def threads = params.threads ?: 1
```

In both cases, only the selected branch is evaluated.

Navigation operators access properties and methods of a value.

The dot operator `a.b` accesses the property `b` (or calls the method `b()`) of `a`:

```
myFile.name  
myFile.getName()
```

The safe dot operator `a?.b` returns `null` if `a` is `null`, instead of raising a null reference error. This avoids the need for explicit null checks:

```
def name = sample?.reads?.name  
// null if sample or sample.reads is null
```

The spread dot operator `a*.b` applies the property access or method call to each element of a collection, returning a list of the results:

```
def files = [file('a.txt'), file('b.txt')]  
files*.name  
// -> ['a.txt', 'b.txt']  
// equivalent to: files.collect { f -> f.name }
```

Range operators create a [list](#) of consecutive values. A range can be formed from any ordered type, such as integers or characters:

Operator	Operation
<code>..</code>	inclusive range
<code>..<code><</code></code>	right-exclusive range

```
assert (1..5) == [1, 2, 3, 4, 5]  
assert (1..<5) == [1, 2, 3, 4]  
assert ('a'..'f') == ['a', 'b', 'c', 'd', 'e', 'f']
```


Membership operators test whether a [collection](#) contains a value:

Operator	Operation
<code>in</code>	membership
<code>!in</code>	negated membership

```
assert 2 in [1, 2, 3]
assert 4 !in [1, 2, 3]
```

For a [map](#), the `in` operator tests for the presence of a key.

Type operators convert or test the type of a value:

Operator	Operation
<code>as</code>	type cast
<code>instanceof</code>	type test
<code>!instanceof</code>	negated type test

The `as` operator casts a value to a different type (see [Types](#)). The `instanceof` operator tests whether a value is an instance of a given type:

```
assert ('42' as Integer) == 42
assert 'foo' instanceof String
assert 42 !instanceof String
```

Regex operators match a string against a regular expression:

Operator	Operation
<code>==~</code>	regex find
<code>===~</code>	regex match

The `==~` operator is truthy if the pattern occurs anywhere in the string, whereas `===~` is true only if the entire string matches the pattern:

```
assert 'hello world' ==~ /world/           // find: pattern occurs in the string
assert !('hello world' ===~ /world/)       // match: pattern must match the whole string
assert '2.7.3' ===~ /\d+\.\d+\.\d+/
```

When an expression contains multiple operators, precedence determines the order in which they are applied. Operators with higher precedence are applied before those with lower precedence. For example, `*` has higher precedence than `+`, so `1 + 2 * 3` is 7, not 9.

The full precedence order, from highest to lowest, is listed in the [Syntax](#) reference. The most important rules to remember are:

- Property access, method calls, and index expressions bind most tightly.
- Unary operators (`!`, `~`, unary `-`) bind more tightly than binary arithmetic.
- Arithmetic binds more tightly than comparison, which binds more tightly than the logical operators.

- The conditional operators (`?:`) have the lowest precedence.

When in doubt, use parentheses to make the intended grouping explicit:

```
(1 + 2) * 3    // -> 9
1 + 2 * 3     // -> 7
```

A statement performs an action, such as declaring a variable, branching on a condition, or raising an error. Statements are executed in order, top to bottom.

A variable is declared with the `def` keyword and an initial value:

```
def x = 42
```

Multiple variables can be declared in a single statement by destructuring a [tuple](#). The number of variables must match the number of elements:

```
def (id, reads) = tuple('sample_1', file('sample_1.fastq'))
// id      -> 'sample_1'
// reads   -> file('sample_1.fastq')
```

Each variable has a scope, which is the region of code in which it can be used:

- Variables declared in a function, along with the function's parameters, exist for the duration of each function call. The same applies to closures.
- Workflow inputs exist for the entire workflow body. Variables declared in the `main:` section exist for the `main`, `emit`, and `publish` sections. Named outputs are not variable declarations and therefore have no scope.
- Process input variables exist for the entire process body. Variables declared in the process `script`, `exec`, and `stub` sections exist only within their respective section, with one exception -- variables declared without the `def` keyword also exist in the `output` section.
- Variables declared in an `if` or `else` branch exist only within that branch:

```
if( true )
  def x = 'hello'
println x           // error: `x` is undefined

// solution: declare `x` outside of the if branch
def x
if( true )
  x = 'hello'
println x
```

A variable cannot be declared with the same name as another variable in the same scope or an enclosing scope:

```
def clash(x) {
  def x           // error: `x` is already declared
  if( true )
    def x         // error: `x` is already declared
}
```

An assignment updates the value of a target, which can be a variable, an index expression, or a property expression:

```
x = 42
list[0] = 'first'
map.key = 'value'
```

Multiple targets can be assigned in a single statement by destructuring a [tuple](#), in the same way as a [declaration](#) but without `def`:

```
(id, reads) = tuple('sample_1', file('sample_1.fastq'))
```

A compound assignment combines a binary operation with the assignment. For example, `x += y` is equivalent to `x = x + y` -- the target is read, combined with the source, and the result assigned back:

```
def x = 1
x += 2
assert x == 3
```

Because the underlying operators are [overloaded](#), compound assignment also works for strings and collections:

```
def items = [1, 2]
items += [3]
assert items == [1, 2, 3]
```

The following compound assignment operators are available, each corresponding to a [binary operator](#):

`+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`.

The elvis assignment `?=` is a special case: `x ?= y` is equivalent to `x = x ?: y`, assigning `y` only if `x` is currently [falsy](#). It is useful for setting a default value:

```
def x = null
x ?= 42
assert x == 42
```

An [if/else statement](#) conditionally executes one of two branches based on a boolean condition. The condition does not need to be a boolean value -- it is coerced to a boolean using [truthiness](#):

```
def files = file('*.txt')
if( files ) {
  println "Found ${files.size()} files"
}
else {
  println 'No files found'
}
```

An if/else statement is a statement, not an expression -- it does not produce a value. To select between two values based on a condition, use a [conditional expression](#) instead:

```
def message = files ? "Found ${files.size()} files" : 'No files found'
```

Nextflow does not support imperative loop statements such as `for` and `while`. Instead, iteration is expressed declaratively using higher-order functions -- methods that take a [closure](#) and apply it to each element of a collection.

For example, instead of a `for` loop:

```
// Groovy (not supported in Nextflow)
def result = []
for( x in [1, 2, 3] ) {
    result.add(x * x)
}
```

Use the `collect` method to transform each element:

```
def result = [1, 2, 3].collect { x -> x * x }
// -> [1, 4, 9]
```

Higher-order functions make the intent of the iteration explicit. Common operations include:

- `each`: perform a side effect for each element
- `collect`: transform each element into a new value (i.e. `map`)
- `find` / `findAll`: select the first / all elements that satisfy a condition (i.e. `filter`)
- `inject`: combine all elements into a single value (i.e. `reduce` or `fold`)
- `any` / `every`: test whether any / all elements satisfy a condition

```
def numbers = [1, 2, 3, 4, 5]

numbers.each { v -> println v }           // print each number
numbers.collect { v -> v * 2 }           // -> [2, 4, 6, 8, 10]
numbers.findAll { v -> v % 2 == 0 }      // -> [2, 4]
numbers.inject(0) { acc, v -> acc + v }  // -> 15
numbers.every { v -> v > 0 }             // -> true
```

A higher-order function returns a value rather than mutating external state, which makes it easier to reason about and avoids common errors. For example, `inject` accumulates a result without an external accumulator variable:

```
// instead of mutating an external variable
def total = 0
[1, 2, 3].each { v -> total += v }

// accumulate the result directly
def total = [1, 2, 3].inject(0) { acc, v -> acc + v }
```

See [Iterable<E>](#) for the full set of higher-order functions for collections such as lists and sets. To process the values in a [channel](#), use [operators](#) such as `map`, `filter`, and `reduce` instead.

An [assert statement](#) evaluates a boolean condition and raises an error if the condition is false. Like an if condition, the asserted expression is coerced to a boolean using [truthiness](#):

```
assert params.input != null : 'The --input parameter is required'
```

If the condition is false, an error is raised. The optional message after the colon is included in the error. If no message is given, Nextflow reports the values of the sub-expressions in the condition:

```
assert 2 + 2 == 5
// Assertion failed:
// assert 2 + 2 == 5
//      |   |
//      4  false
```

Assertions are intended for verifying invariants -- conditions that should always hold if the code is correct. They are commonly used in tests and for sanity checks. To validate user inputs or report expected error conditions, use the `error` function instead, which produces a cleaner message without the assertion trace.

When an error occurs at runtime, an exception is raised. An exception propagates up through the call stack until it is caught by an enclosing `try/catch statement`, or else it reaches the Nextflow runtime and terminates the pipeline.

The recommended way to raise an error in pipeline code is the `error` function, which stops the pipeline with a clear error message:

```
if( params.aligner !in ['bowtie2', 'bwamem'] )
    error "Unsupported aligner: ${params.aligner}"
```

The lower-level `throw` statement can be used to raise a specific exception type, but `error` should be preferred for reporting pipeline errors, as it produces a cleaner message and does not require constructing an exception:

```
// preferred
error 'Something failed!'

// equivalent, but more verbose
throw new Exception('Something failed!')
```

A `try/catch` statement is appropriate when an error is expected and can be handled in pipeline logic -- for example, falling back to a default value when an optional file cannot be read:

```
def config
try {
    config = file('custom.config').text
}
catch( e: IOException ) {
    log.warn 'Could not read custom.config, using defaults'
    config = ''
}
```

A catch clause matches an exception if the raised exception is an instance of the declared type. If no catch clause matches, the exception propagates to the next enclosing `try/catch` statement, or terminates the pipeline. Use `try/catch` only for errors you intend to recover from -- otherwise, let the error propagate so the failure is visible.

note

The `error` function and `try/catch` handle errors in pipeline code (functions, closures, the workflow body). They are distinct from two other mechanisms that operate at different levels:

- The `errorStrategy` directive determines what happens when a process task fails (e.g. a command exits with a non-zero status): the task can be retried, ignored, or terminate the pipeline. This is the appropriate mechanism for handling failures of the tools that a process executes.
- The `onError` workflow handler runs when the pipeline as a whole fails, allowing for cleanup or notification.

A **function** is a named, reusable block of code that takes zero or more parameters and may return a value:

```
def sayHello(name: String) {  
  println "Hello, ${name}!"  
}  
  
def add(a: Integer, b: Integer) -> Integer {  
  a + b  
}
```

Arguments are passed by reference -- the function receives a reference to the same object, not a copy. Mutating a mutable argument (such as a list or map) inside a function affects the caller's object:

```
def addItem(list, item) {  
  list.add(item)  
}  
  
def items = [1, 2]  
addItem(items, 3)  
println items // -> [1, 2, 3]
```

To avoid this, create a copy before mutating, or use operations that return a new value (e.g. `list + [item]` instead of `list.add(item)`).

Reassigning a parameter, on the other hand, does not affect the caller. The parameter is a separate reference to the same object, so assigning a new value to it changes only the function's local reference:

```
def replace(list) {  
  list = [4, 5, 6] // reassigns the local reference only  
}  
  
def items = [1, 2, 3]  
replace(items)  
println items // -> [1, 2, 3]
```

A function parameter can declare a default value, which is used when the caller does not supply an argument for that parameter:

```
def greet(name, greeting = 'Hello') {  
  "${greeting}, ${name}!"  
}  
  
greet('world') // -> Hello, world!  
greet('world', 'Hi') // -> Hi, world!
```

Parameters with default values must come after parameters without defaults. A default value can be any expression, and is evaluated each time the function is called without that argument.

A **function call** may use named arguments -- arguments specified by name rather than position. Named arguments are collected into a map and passed as the first positional argument to the function:

```
file('hello.txt', checkIfExists: true)
```

is equivalent to:

```
file([checkIfExists: true], 'hello.txt')
```

Named arguments are commonly used by built-in functions and process directives to specify optional settings:

```
// process directive with named options
accelerator 4, type: 'nvidia-tesla-v100'

// channel factory with named options
channel.fromPath('*.txt', checkIfExists: true)
```

Because named arguments are collected into a single map, their order does not matter, and they can be freely mixed with positional arguments. A function that accepts named arguments must declare a `Map` as its first parameter.

A function returns the value of its last evaluated expression, or the value of an explicit `return` statement:

```
def add(a, b) {
  a + b      // implicit return
}

def addExplicit(a, b) {
  return a + b
}
```

If a function has multiple return statements (including implicit returns), they must either all return a value or all return nothing. When a value is returned, every conditional branch must return a value:

```
def isEven1(n) {
  if( n % 2 == 1 )
    return      // error: a return value is required here
  return true
}

def isEven2(n) {
  if( n % 2 == 0 )
    return true
  // error: a return value is required here
}
```

If the last statement of a function is not a return or expression statement, it is equivalent to a bare `return` with no value.

A process defines a task that runs in an isolated environment. A process is called like a function from a workflow, but instead of running immediately, each call adds a node to the [dataflow graph](#). Nextflow creates a separate task for each input (or combination of inputs) and executes these tasks asynchronously as their inputs become available, potentially in parallel and on distributed compute.

See [Processes](#) for a full description of how to define and use processes.

A Nextflow workflow describes a dataflow -- a graph of [processes](#) and [operators](#) connected by channels. See [Processes](#) and [Workflows](#) for a full description of how to define and compose them.

A workflow body is not executed step by step like an ordinary function. Instead, it is evaluated once, up front, to construct the dataflow graph. The statements in a workflow body declare how processes, operators, and channels are wired together. The actual computation happens afterward, asynchronously, as data flows through the graph.

In this sense, a workflow definition behaves like a compile-time macro: it runs once to build the dataflow graph, and the operations it describes (process tasks, operator logic) execute later as data becomes available.

Consider:

```
workflow {
  ch_input = channel.of(1, 2, 3)
  ch_result = ch_input.map { v -> expensive(v) }
  ch_result.view()
}
```

When the workflow body runs:

1. `channel.of(1, 2, 3)` creates a channel.
2. `.map { ... }` connects a `map` operator to that channel and returns a new channel. The closure is not called yet -- it is registered to run later, once per value.
3. `.view()` connects a `view` operator.

Only after the entire graph is constructed does Nextflow begin executing it: the channel emits its values, the `map` closure runs for each value, and `view` prints each result.

This has several consequences:

- **Channel contents cannot be accessed directly.** Because the workflow body runs before any data flows, you cannot inspect the values in a channel from within the workflow body. You must use an operator or process to act on each value:

```
// does NOT work -- the channel has no values yet when this runs
def first = ch_input[0]

// correct -- the closure runs later, for each value
ch_input.map { v -> transform(v) }
```

- **Operator closures run asynchronously.** The closure passed to an operator such as `map` or `filter` is invoked later, once per channel value, and possibly out of order. It should not depend on mutable state shared with other closures.
- **The order of statements describes connections, not execution order.** Two independent branches of a workflow are constructed in the order written, but their tasks may execute concurrently.

This is the fundamental difference between the workflow definition (evaluated synchronously to build the graph) and the workflow execution (runs the graph asynchronously). See [Dataflow](#) for more information.

A script is a single Nextflow source file. It consists of [script declarations](#) -- such as workflows, processes, functions, and types -- and may use declarations from other scripts.

When a script is executed, its entry workflow -- the workflow with no name -- serves as the entrypoint. A script that contains only [statements](#), with no declarations, is treated as the body of an implicit entry workflow. To be executable, a script must either be a code snippet or define an entry workflow.

A script can use the definitions of another script through an [include declaration](#):

```
include { hallo as sayHello } from './some/module'
```


The include source should refer to either a local path (e.g. `./module.nf`), a registry module (e.g. `nf-core/fastq`), or a plugin (e.g. `plugin/nf-hello`).

The following definitions can be included from a script:

- Functions
- Processes
- Named workflows
- New in 26.04: Record types
- New in 26.04: Enum types

The entry workflow of an included script is ignored. This allows the same script to be executed directly or included by another script.

Every value in Nextflow has a type, which determines the set of operations that can be performed on it. The available types are described in the [Types](#) reference.

Nextflow supports two styles of typing:

- **Dynamic typing** (the default): variables and parameters do not declare a type, and the type of each value is determined at runtime. This is the traditional behavior.
- **Static typing** (opt-in): variables, parameters, process inputs/outputs, and workflow inputs/outputs can declare a type, which is checked at compile time. Static typing is enabled with the `nextflow.enable.types` feature flag.

A type annotation is written after the name, separated by a colon:

```
def count: Integer = 0
def name: String = 'sample'
def reads: List<Path> = []
```

A type annotation can be suffixed with `?` to indicate that the value may be `null`:

```
def x: String? = null
```

Some types are generic -- they are parameterized by one or more type arguments that specify the types of their contents. A type argument is written in angle brackets after the type name:

- `List<E>` is a list whose elements have type `E`
- `Channel<E>` is a channel whose values have type `E`
- `Map<K,V>` is a map whose keys have type `K` and values have type `V`

For example:

```
// a list of strings
def sequences: List<String> = ['ATCG', 'GCTA']

// a map from string keys to integer values
def counts: Map<String,Integer> = [sample1: 1000, sample2: 1500]

// a channel of file paths
def ch_bams: Channel<Path> = channel.fromPath('*.bam')
```

Generic type arguments allow the type checker to verify that the contents of a collection or channel are used consistently -- for example, that a list of `Path` is not mistakenly treated as a list of `String`.

A record type is a user-defined type that specifies a set of named fields, each with its own type (see [Syntax](#) for the declaration grammar):

```
record Sample {
  id: String
  fastq_1: Path
  fastq_2: Path?
}
```

A field can be marked as optional by appending `?` to its type.

A [record](#) created with the `record()` function has the generic type `Record` and makes no guarantee about which fields it provides. A record type, by contrast, specifies a minimum set of required fields, and is used to make a stronger guarantee in a particular context, such as a workflow, process, or function input.

Record types are structural (or duck-typed): a record satisfies a record type if it provides at least the fields declared by that type. The record may have additional fields, and it does not matter how the record was created. A record is validated against a record type when it is supplied as an argument to a workflow, process, or function that declares a record type input, or cast to a record type with the [as operator](#):

```
def hello(sample: Sample) {
  println "Hello sample ${sample.id}!"
}

workflow {
  // error: missing `fastq_1` field required by Sample
  hello(record(id: '1', fastq_2: file('1_2.fastq')))

  // ok: all required fields are present (extra fields are allowed)
  hello(record(id: '1', fastq_1: file('1_1.fastq'), single_end: true))
}
```

Because matching is structural, two record types with the same fields and field types are interchangeable, even if they have different names.

An enum type is a user-defined type with a fixed set of named values (see [Syntax](#) for the declaration grammar):

```
enum Day {
  MONDAY,
  TUESDAY,
  WEDNESDAY,
  THURSDAY,
  FRIDAY,
  SATURDAY,
  SUNDAY
}
```

Each value is accessed as a property of the enum type:

```
def day = Day.MONDAY
```

An enum type is useful for representing a value that can take only one of a fixed set of options, in place of a free-form string.

A value can be cast to a different type explicitly with the `as` operator. The value must be convertible to the target type, or an error is raised:

```
def x = '1h' as Duration
def n = '42' as Integer
```

A value is also cast implicitly in the following situations:

- **Numeric coercion** -- when an [arithmetic operation](#) mixes an integer and a floating-point operand, the integer is coerced to a floating-point number.
- **Boolean coercion** -- in a condition, such as [if/else](#), [assertions](#), or the [logical operators](#), any value is coerced to a boolean according to its [truthiness](#).
- **String coercion** -- when a value is interpolated into a [string](#), it is converted to a string with its `toString()` method.

Standard library

This section describes the Nextflow standard library, which consists of built-in namespaces and types.

Namespaces

[Reference for namespaces in the Nextflow standard library.](#)

Types

[Reference for the standard types in the Nextflow standard library.](#)

Groovy and Java classes

[Reference for Groovy and Java classes available to Nextflow at runtime.](#)

Source: <https://docs.seqera.io/nextflow/reference/stdlib>

Process reference

This page lists the task properties, input/output methods, and directives available in [process](#) definitions.

The following task properties are defined in the process body:

The current task attempt.

Available only in `script:` and `shell:` blocks

The exit code returned by the task script.

The exit code is only available after the task has been executed (e.g., the [errorStrategy](#) directive).

Available only in `exec:` blocks

The task hash.

The process-level task index.

Available only in `exec:` blocks

The task name.

Added in version 24.10

The exception reported by the previous task attempt.

Since the exception is available after a failed task attempt, it can only be accessed when retrying a failed task execution, i.e., when `task.attempt` is greater than 1.

Added in version 24.10

The trace record associated with the previous task attempt.

Since the trace record is available after a failed task attempt, it can only be accessed when retrying a failed task execution, i.e., when `task.attempt` is greater than 1. See [Trace file](#) for a list of available fields.

note

The trace fields `%cpu` and `%mem` can be accessed as `pcpu` and `pmem`, respectively.

The name of the process that spawned the task.

Available only in `exec:` blocks

The unique directory path for the task.

note

Directive values for a task can be accessed via `task.<directive>`. See [Task directive values](#) for more information.

Added in version 25.10

note

Typed processes require the `nextflow.enable.types` feature flag to be enabled in every script that uses them.

The following directives can be used in the `stage:` section of a typed process:

Declares an environment variable with the specified name and value in the task environment.

Stages a file into the task directory under the given alias.

Stages a collection of files into the task directory under the given alias.

Stages the given value as the standard input (i.e., `stdin`) to the task script.

The following functions are available in the `output:` and `topic:` sections of a typed process:

Returns the value of an environment variable from the task environment.

Returns the standard output of the specified command, which is executed in the task environment after the task script completes.

Returns a file from the task environment that matches the specified pattern.

Available options:

When `true`, target files are returned in place of any matching symlink (default: `true`).

When `true`, the file name is interpreted as a glob pattern (default: `true`).

When `true`, hidden files are included in the matching output files (default: `false`).

When `true` and the file name is a glob pattern, any input files matching the pattern are also included in the output (default: `false`).

Maximum number of directory levels to visit (default: no limit).

When `true`, the task will not fail if the given file is missing (default: `false`).

Type of paths returned, either `file`, `dir` or `any` (default: `any`, or `file` if the given file name contains a double star (`**`)).

Returns files from the task environment that match the given pattern.

Supports the same options as `file()` (except for `optional`).

Returns the standard output of the task script.

Declare a variable input. The received value can be any type, and it will be made available to the process body (i.e. `script`, `shell`, `exec`) as a variable given by `identifier`.

Deprecated in version 19.10

Use `path` instead.

Declare a file input. The received value can be any type, and it will be staged into the task directory. If the received value is not a file or collection of files, it is implicitly converted to a string and written to a file.

The argument can be an identifier or string. If an identifier, the received value will be made available to the process body as a variable. If a string, the received value will be staged into the task directory under the given alias.

Declare a file input. The received value should be a file or collection of files and will be staged into the task directory.

tip

See [Multiple input files](#) for more information about accepting collections of files.

The argument can be an identifier or string. If an identifier, the received value will be made available to the process body as a variable. If a string, the received value will be staged into the task directory under the given alias.

Available options:

Added in version 23.10

Specify the number of expected files. Can be a number, e.g. `'1'`, or a range, e.g. `'1..*'`. If a task receives an invalid number of files for this `path` input, it will fail.

Specify how the file should be named in the task work directory. Can be a name or a pattern.

Alias of `name`.

Declare an environment variable input. The received value should be a string, and it will be exported to the task environment as an environment variable given by `name`.

Declare a `stdin` input. The received value should be a string, and it will be provided as the standard input (i.e. `stdin`) to the task script. It should be declared only once for a process.

Declare a tuple input. Each argument should be an input declaration such as `val`, `path`, `env`, or `stdin`.

The received value should be a tuple with the same number of elements as the `tuple` declaration, and each received element should be compatible with the corresponding `tuple` argument. Each tuple element is treated the same way as if it were a standalone input.

Declare a variable output. The argument can be any value, and it can reference any output variables defined in the process body (i.e. variables declared without the `def` keyword).

Deprecated in version 19.10

Use `path` instead.

Declare a file output. It receives the output files from the task environment that match the given pattern.

Multiple patterns can be specified using the colon separator (`:`). The union of all files matched by each pattern will be collected.

Declare a file output. It receives the output files from the task environment that match the given pattern.

Available options:

Added in version 23.10

Specify the number of expected files. Can be a number or a range. If a task produces an invalid number of files for this `path` output, it will fail.

If the arity is `1`, a single file will be emitted. Otherwise, a list will always be emitted, even if only one file is produced.

warning

If the arity is not specified, a single file or list will be emitted based on whether a single file or multiple files are produced at runtime, resulting potentially in an output channel with a mixture of files and file collections.

When `true`, target files are returned in place of any matching symlink (default: `true`).

When `true`, the specified name is interpreted as a glob pattern (default: `true`).

When `true`, hidden files are included in the matching output files (default: `false`).

When `true` and the output path is a glob pattern, any input files matching the pattern are also included in the output (default: `false`).

Maximum number of directory levels to visit (default: no limit).

Type of paths returned, either `file`, `dir` or `any` (default: `any`, or `file` if the specified file name pattern contains a double star (`**`)).

Declare an environment variable output. It receives the value of the environment variable (given by `name`) from the task environment.

Changed in version 24.04

Prior to this version, if the environment variable contained multiple lines of output, the output would be compressed to a single line by converting newlines to spaces.

Declare a `stdout` output. It receives the standard output of the task script.

Added in version 24.04

Declare an `eval` output. It receives the standard output of the given command, which is executed in the task environment after the task script.

If the command fails, the task will also fail.

Declare a tuple output. Each argument should be an output declaration such as `val`, `path`, `env`, `stdin`, or `eval`. Each tuple element is treated the same way as if it were a standalone output.

The following options are available for all process outputs:

Defines the name of the output channel.

When `true`, the task will not fail if the specified output is missing (default: `false`).

Added in version 25.04

Send the output to a [topic channel](#) with the given name.

The `accelerator` directive defines the number of hardware accelerators (e.g. GPUs) required by each task execution. For example:

```
process hello {  
  accelerator 4, type: 'nvidia-tesla-k80'  
  
  script:  
  """  
  your_gpu_enabled --command --line  
  """  
}
```

The above example requests 4 GPUs of type `nvidia-tesla-k80` for each task.

note

This directive is only used by certain executors. Refer to the [Executors](#) page to see which executors support this directive.

note

Additional options may be required to fully enable the use of accelerators. When using containers with GPUs, you must pass the GPU drivers through to the container. For Docker, this requires the option `--gpus all` in the `docker run` command. For Apptainer/Singularity, this requires the option `--nv`. The specific implementation details depend on the accelerator and container type being used.

The following options are available:

The number of requested accelerators.

Specifying this directive with a number (e.g., `accelerator 4`) is equivalent to the `request` option (e.g., `accelerator request: 4`).

The accelerator type.

The meaning of this option depends on the target execution platform. See the platform-specific documentation for more information about the available accelerators:

- [Google Cloud](#)
- [Kubernetes](#)

This option is not supported for AWS Batch. You can control the accelerator type indirectly through the allowed instance types in your Compute Environment. See the [AWS Batch FAQs](#) for more information.

The `afterScript` directive executes a custom (Bash) snippet immediately after the main process has run. This may be useful to clean up your staging area.

When combined with the `container` directive, the `afterScript` is executed outside the specified container. In other words, the `afterScript` is always executed in the host environment.

The `arch` directive defines the CPU architecture to build the software in use by the process' task. For example:

```
process blast {
  spack 'blast-plus@2.13.0'
  arch 'linux/x86_64', target: 'cascadelake'

  script:
  """
  blastp -query input_sequence -num_threads ${task.cpus}
  """
}
```

The example above declares that the CPU generic architecture is `linux/x86_64` (X86 64 bit), and more specifically that the microarchitecture is `cascadelake` (a specific generation of Intel CPUs).

This directive is currently used by the following Nextflow functionalities:

- by the `spack` directive, to build microarchitecture-optimized applications;
- by the [Wave](#) service, to build containers for one of the generic families of CPU architectures (see below);
- by the `spack` strategy within [Wave](#), to optimize the container builds for specific CPU microarchitectures.

Allowed values for the `arch` directive are as follows, grouped by equivalent family (choices available for the sake of compatibility):

- X86 64 bit: `linux/x86_64`, `x86_64`, `linux/amd64`, `amd64`
- ARM 64 bit: `linux/aarch64`, `aarch64`, `linux/arm64`, `arm64`, `linux/arm64/v8`
- ARM 64 bit, older generation: `linux/arm64/v7`

Examples of values for the architecture `target` option are `cascadelake`, `icelake`, `zen2` and `zen3`. See the [Spack documentation](#) for the full and up-to-date list of meaningful targets.

Added in version 24.04

The `array` directive submits tasks as job arrays for executors that support it.

A job array is a collection of jobs with the same resource requirements and the same script (parameterized by an index). Job arrays incur significantly less scheduling overhead compared to individual jobs, and as a result they are preferred by HPC schedulers where possible.

The directive should be specified with a given array size, along with an executor that supports job arrays. For example:

```
process hello {  
  executor 'slurm'  
  array 100  
  
  script:  
  """  
  your_command --here  
  """  
}
```

Nextflow currently supports job arrays for the following executors:

- [AWS Batch](#)
- [Google Cloud Batch](#)
- [LSF](#)
- [PBS](#)
- [PBS Pro](#)
- [SGE](#)
- [SLURM](#)

A process using job arrays collects tasks and submits each batch as a job array when it is ready. Any "leftover" tasks are submitted as a partial job array.

Once a job array is submitted, each "child" task is executed as an independent job. Any tasks that fail (and can be retried) are retried without interfering with the tasks that succeeded. Retried tasks are submitted individually rather than through a job array, in order to allow for the use of [dynamic resources](#).

The following directives must be uniform across all tasks in a process that uses job arrays, because these directives are specified once for the entire job array:

- [accelerator](#)
- [clusterOptions](#)
- [cpus](#)
- [disk](#)
- [machineType](#)
- [memory](#)
- [queue](#)
- [resourceLabels](#)
- [resourceLimits](#)
- [time](#)

For cloud-based executors like AWS Batch, or when using Fusion with any executor, the following additional directives must be uniform:

- [container](#)
- [containerOptions](#)

When using Wave, the following additional directives must be uniform:

- [conda](#)

The `beforeScript` directive executes a custom (Bash) snippet before the main process script is run. This may be useful to initialize the underlying cluster environment or for other custom initialization.

For example:

```
process hello {
  beforeScript 'source /cluster/bin/setup'

  script:
  """
  echo 'hello'
  """
}
```

When the process is containerized (using the [container](#) directive), the `beforeScript` is executed in the container only if the executor is container-native (e.g. cloud batch executors, Kubernetes). Otherwise, the `beforeScript` is executed outside the container.

The `cache` directive controls whether and how task executions are cached.

By default, cached task executions are re-used when the pipeline is launched with the [resume](#) option. The `cache` directive can be used to disable caching for a specific process:

```
process hello {
  cache false

  // ...
}
```

See [Cache and resume](#) for more information.

The following options are available:

Disable caching.

Enable caching. Input file metadata (name, size, last updated timestamp) are included in the cache keys.

Enable caching. Input file content is included in the cache keys.

Enable caching. Minimal input file metadata (name and size only) are included in the cache keys.

This strategy provides a workaround for incorrect caching invalidation observed on shared file systems due to inconsistent file timestamps.

The `clusterOptions` directive specifies additional submission options for grid executors. You can use it to specify options for your cluster that are not supported directly by other process directives.

The cluster options can be a string:

```
process hello {
  clusterOptions '-x 1 -y 2'

  // ...
}
```

Changed in version 24.04

Prior to this version, grid executors that require each option to be on a separate line in the job script would attempt to split multiple options using a variety of different conventions. Multiple options can now be specified more clearly using a string list as shown below.

The cluster options can also be a string list:

```
process hello {
  clusterOptions '-x 1', '-y 2', '--flag'

  // ...
}
```

Grid executors that require one option per line will write each option to a separate line, while grid executors that allow multiple options per line will write all options to a single line, the same as with a string. This form is useful to control how the options are split across lines when it is required by the scheduler.

note

This directive is only used by grid executors. Refer to the [Executors](#) page to see which executors support this directive.

warning

While you can use the `clusterOptions` directive to specify options that are supported as process directives (`queue`, `memory`, `time`, etc), you should not use both at the same time, as it will cause undefined behavior. Most HPC schedulers will either fail or simply ignore one or the other.

The `conda` directive defines the set of [Conda](#) packages required by each task. For example:

```
process hello {
  conda 'bwa=0.7.15'

  script:
  """
  your_command --here
  """
}
```

Nextflow automatically creates an environment for each unique set of Conda packages.

The name of the desired channel for a specific package can be specified using the standard Conda notation, e.g. `bioconda::bwa=0.7.15`. Multiple packages can be specified separating them with a blank space, e.g. `bwa=0.7.15 fastqc=0.11.5`.

The `conda` directive can also accept a Conda environment file path or the path of an existing Conda environment. See [Conda](#) for more information.

The `container` directive defines the container required by each task. For example:

```
process hello_docker {
  container 'busybox:latest'

  script:
  """
  your_command --here
  """
}
```

The corresponding container runtime (e.g. Docker, Singularity) should be running on the compute nodes where tasks are executed. See [Containers](#) for the container runtimes supported by Nextflow.

note

This directive is ignored by [native processes](#) (i.e. `exec` processes).

The `containerOptions` directive specifies additional container options for the underlying container runtime (e.g. Docker, Singularity). For example:

```
process hello_docker {
  container 'busybox:latest'
  containerOptions '--volume /data/db:/db'

  output:
  path 'output.txt'

  script:
  """
  your_command --data /db > output.txt
  """
}
```

The above example provides a custom volume mount for a specific process.

warning

This directive is not supported by the [Kubernetes](#) executor.

The `cpus` directive defines the number of CPUs required by each task execution. For example:

```
process blast {
  cpus 8

  script:
  """
  blastp -query input_sequence -num_threads ${task.cpus}
  """
}
```

This directive is required for tasks that execute multi-process or multi-threaded commands/tools and it is meant to reserve enough CPUs when a pipeline task is executed through a cluster resource manager.

See also: [disk](#), [memory](#), [time](#), [queue](#), [dynamic task resources](#)

The `debug` directive prints the standard output of each task to the pipeline standard output.

For example:

```
process hello {
  debug true

  script:
  """
  echo Hello
  """
}
```

Prints:

```
Hello
```

Removing the `debug` directive or setting it to `false` in the above example will cause `Hello` to not be printed.

The `disk` directive defines the amount of disk storage required by each task execution. For example:

```
process hello {
  disk 2.GB

  script:
  """
  your_command --here
  """
}
```

The following suffixes can be used to specify disk values:

- `B`: Bytes
- `KB`: Kilobytes
- `MB`: Megabytes
- `GB`: Gigabytes
- `TB`: Terabytes

See [MemoryUnit](#) for more information.

note

The `disk` directive is only used by certain executors. Refer to the [Executors](#) page to see which executors support this directive.

See also: [cpus](#), [memory](#), [time](#), [queue](#), [dynamic task resources](#)

The `errorStrategy` directive defines how to handle task failures.

A task failure occurs when the executed script returns a non-zero exit code. By default, the pipeline run is aborted.

The following error strategies are available:

When a task fails, terminate the pipeline immediately and report an error. Pending and running jobs are killed.

When a task fails, wait for submitted and running tasks to finish and then terminate the pipeline, reporting an error.

When a task fails, ignore it and continue the pipeline execution. If the `workflow.failOnIgnore` config option is set to `true`, the pipeline will report an error (i.e. return a non-zero exit code) upon completion. Otherwise, the pipeline will complete successfully.

See the [workflow](#) namespace for more information.

When a task fails, retry it.

When setting the `errorStrategy` directive to `ignore` the process doesn't stop on an error condition, it just reports a message notifying you of the error event.

For example:

```
process hello {  
  errorStrategy 'ignore'  
  
  // ...  
}
```

In this case, the workflow will complete successfully and return an exit status of 0. However, if you set `workflow.failOnIgnore = true` in your Nextflow configuration, the workflow will return a non-zero exit status and report the failed tasks as an error.

The `retry` error strategy retries failed tasks. For example:

```
process hello {  
  errorStrategy 'retry'  
  
  // ...  
}
```

The number of times a failing process is re-executed is defined by the [maxRetries](#) and [maxErrors](#) directives.

tip

More complex strategies depending on the task exit status or other parametric values can be defined using a dynamic `errorStrategy`. See [Dynamic directives](#) for details.

See also: [maxErrors](#), [maxRetries](#), [dynamic task resources](#)

The `executor` directive defines the underlying system where tasks are executed. For example:

```
process hello {  
  executor 'slurm'  
  
  // ...  
}
```

Commonly used executors include:

- `awsbatch`: [AWS Batch](#)
- `azurebatch`: [Azure Batch](#)
- `google-batch`: [Google Cloud Batch](#)
- `k8s`: [Kubernetes](#) cluster
- `local`: The local machine where the pipeline is launched

- `lsf`: [Platform LSF](#) job scheduler
- `slurm`: [SLURM](#) workload manager

Each executor supports additional configuration options under the `executor` config scope. See [Executors](#) for more information.

The `ext` is a generic directive for user-defined properties. For example:

```
process star {
  container "biocontainers/star:${task.ext.version}"

  input:
    path genome
    tuple val(sampleId), path(reads)

  script:
    """
    STAR --genomeDir $genome --readFilesIn $reads ${task.ext.args ? : ''}
    """
}
```

In the above example, the process container version is controlled by `ext.version`, and the script supports additional command line arguments through `ext.args`.

The `ext` directive can be set in the process definition:

```
process hello {
  ext version: '2.5.3', args: '--alpha --beta'

  // ...
}
```

Or in the Nextflow configuration:

```
process.ext.version = '2.5.3'
process.ext.args = '--alpha --beta'
```

Added in version 23.04

The `fair` directive, when enabled, guarantees that process outputs will be emitted in the order in which they were received. For example:

```
process hello {
  fair true

  input:
    val x

  output:
    tuple val(task.index), val(x)

  script:
    """
    sleep \${(RANDOM % 3)}
    """
}

workflow {
  channel.of('A','B','C','D') | hello | view
}
```

The above example produces:

```
[1, A]
[2, B]
[3, C]
[4, D]
```

The `hints` directive specifies executor-specific hints as key-value pairs. Each executor uses the hints it recognizes and ignores the rest. Hint values can be any raw value (i.e., numbers, strings, booleans, lists, and maps).

Unprefixed keys are available to **every** executor. Any executor that recognizes the key consumes it. Prefixing a key with an executor name (e.g., `awsbatch/...`) restricts the hint to that executor only. For example:

```
process hello {
  hints consumableResources: ['my-license': 1]

  script:
  """
  your_command --here
  """
}
```

To restrict a hint to a single executor, prefix the key with the executor name:

```
hints 'awsbatch/consumableResources': ['my-license': 1]
```

When the same hint is provided both unprefixed and with a matching executor prefix, the prefixed form takes precedence for that executor.

Calling `hints` multiple times in a process definition accumulates entries, with later calls overwriting entries for the same key. Setting `hints` via configuration (e.g., in `nextflow.config`) replaces the entire map.

See [Executors](#) to see which hints are recognized by each executor.

The `label` directive attaches a custom label to the process. For example:

```
process hello {
  label 'big_mem'

  script:
  """
  your_command --here
  """
}
```

A label may contain alphanumeric characters or `_`. It must start and end with an alphabetic character.

The same label can be applied to multiple processes. Multiple labels can be applied to the same process by using the `label` directive multiple times.

Process labels are used to apply shared process configuration via `withLabel` selectors. They are not recorded in execution logs, trace reports, or lineage metadata. See [Process selectors](#) for more information.

note

To tag individual task executions for logging and debugging, use [tag](#). To tag cloud computing resources for cost tracking, use [resourceLabels](#). To attach metadata labels to output files for lineage tracking, use the `label` [output directive](#) in the `output` block.

The `machineType` can be used to specify a predefined Google Compute Platform [machine type](#) when running using the [Google Batch](#), or when using auto-pools with [Azure Batch](#).

For example:

```
process hello {
  machineType 'n1-highmem-8'

  script:
  """
  your_command --here
  """
}
```

See also: [cpus](#), [memory](#)

The `maxErrors` directive defines the maximum number of task failures allowed for a process when using the `retry` error strategy. For example:

```
process hello {
  errorStrategy 'retry'
  maxErrors 5

  script:
  """
  echo 'do this as that .. '
  """
}
```

In the above example, the run will fail if the `hello` process accrues more than 5 failures across all of its task executions.

By default, there is no limit. However, the run can still fail if an individual task exceeds the number of retries allowed by the `maxRetries` directive.

See also: [errorStrategy](#), [maxRetries](#)

The `maxForks` directive defines the maximum number of concurrent task executions for a process. For example:

```
process hello {
  maxForks 1

  script:
  """
  your_command --here
  """
}
```

The above example forces the `hello` process to execute tasks sequentially.

By default, there is no limit. However, the number of concurrent tasks can still be limited globally by the number of CPUs (for local tasks) and the `executor.queueSize` config option.

The `maxRetries` directive defines the maximum number of times a task can be retried when using the `retry` error strategy. For example:

```
process hello {
  errorStrategy 'retry'
  maxRetries 3

  script:
  """
  echo 'do this as that .. '
  """
}
```

In the above example, the run will fail if any task executed by `hello` fails more than three times.

By default, only one retry per task is allowed. However, the run can still fail if the total number of failures for the process exceeds the number allowed by the `maxErrors` directive.

See also: [errorStrategy](#), [maxErrors](#)

The `maxSubmitAwait` directive defines how long a task can remain in submission queue without being executed. Tasks that exceed this duration in the queue will fail.

It can be used with the `retry` error strategy to re-submit tasks to a different queue or with different resource requirements. For example:

```
process hello {
  errorStrategy 'retry'
  maxSubmitAwait 10.m
  maxRetries 3
  queue "${task.submitAttempt==1 ? 'spot-compute' : 'on-demand-compute'}"

  script:
  """
  your_command --here
  """
}
```

In the above example, each task is submitted to the `spot-compute` queue on the first attempt (`task.submitAttempt==1`). If a task remains in the queue for more than 10 minutes, it fails and is re-submitted to the `on-demand-compute` queue.

The `memory` directive defines how much memory is required by each task execution. For example:

```
process hello {
  memory 2.GB

  script:
  """
  your_command --here
  """
}
```

The following suffixes can be used to specify memory values:

- `B`: Bytes
- `KB`: Kilobytes
- `MB`: Megabytes

- GB : Gigabytes
- TB : Terabytes

See [MemoryUnit](#) for more information.

See also: [cpus](#), [disk](#), [time](#), [queue](#), [dynamic task resources](#)

The `module` directive defines the set of [Environment Modules](#) required by each task, if supported by your compute environment. For example:

```
process blast {
  module 'ncbi-blast/2.2.27'

  script:
  """
  blastp -query <etc..>
  """
}
```

Multiple modules can be specified using the `:` separator:

```
process blast {
  module 'ncbi-blast/2.2.27:t_coffee/10.0:clustalw/2.1'

  script:
  """
  blastp -query <etc..>
  """
}
```

The `penv` directive defines the parallel environment to use when submitting tasks to the [SGE](#) resource manager. For example:

```
process blast {
  cpus 4
  penv 'smp'
  executor 'sge'

  script:
  """
  blastp -query input_sequence -num_threads ${task.cpus}
  """
}
```

Refer to your cluster documentation or your system administrator to determine whether this feature is supported in your environment.

The `pod` directive defines pod-specific settings, such as environment variables, secrets, and config maps, when using the [Kubernetes](#) executor.

For example:

```
process echo {
  pod env: 'MESSAGE', value: 'hello world'

  script:
  """
  echo $MESSAGE
  """
}
```

The above snippet defines an environment variable named `MESSAGE` whose value is `'hello world'`.

Pod settings can be specified in Nextflow configuration:

```
// single setting
process.pod = [env: 'MESSAGE', value: 'hello world']

// multiple settings
process.pod = [
  [env: 'MESSAGE', value: 'hello world'],
  [secret: 'my-secret/key1', mountPath: '/etc/file.txt']
]
```

The following options are available:

Specifies the pod [affinity](#) with the given configuration.

Can be specified multiple times

Defines a pod [annotation](#) with the given name and value.

Specifies whether to [automount service account token](#) into the pod (default: `true`).

Can be specified multiple times

Mounts a [ConfigMap](#) with name and optional key to the given path. If the key is omitted, the path is interpreted as a directory and all entries in the `ConfigMap` are exposed in that path.

Can be specified multiple times

Mounts a [CSI ephemeral volume](#) with the given configuration to the given path.

Can be specified multiple times

Mounts an [emptyDir](#) with the given configuration to the given path.

Can be specified multiple times

Defines an environment variable whose value is defined by the given [ConfigMap](#) and key.

Can be specified multiple times

Defines an environment variable whose value is defined by the given [field path](#) value.

For example, the following pod option:

```
process.pod = [env: 'MY_NODE_NAME', fieldPath: 'spec.nodeName']
```

Maps to the following pod spec:

```
env:
- name: MY_NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
```

Can be specified multiple times

Defines an environment variable whose value is defined by the given [Secret](#) and key.

Can be specified multiple times

Defines an environment variable with the given name and value.

Added in version 23.10

Can be specified multiple times

Allows creating [hostPath](#) volume and access it with the specified `mountPath` in the pod.

Specifies the [image pull policy](#) used by the pod to pull the container image.

Specifies the [image pull secret](#) used to access a private container image registry.

Can be specified multiple times

Defines a pod [label](#) with the given name and value.

Specifies the [node selector](#) with the given configuration.

The configuration can be a map or a string:

```
// map
process.pod = [nodeSelector: [disktype: 'ssd', cpu: 'intel']]

// string
process.pod = [nodeSelector: 'disktype=ssd,cpu=intel']
```

Specifies the [priority class name](#) for pods.

Specifies whether the pod should run as a privileged container (default: `false`).

Specifies the user ID with which to run the container. Shortcut for the `securityContext` option.

Specifies the [runtime class](#).

Specifies which [scheduler](#) is used to schedule the container.

Can be specified multiple times

Mounts a [Secret](#) with name and optional key to the given path. If the key is omitted, the path is interpreted as a directory and all entries in the `Secret` are exposed in that path.

Specifies the pod [security context](#) with the given configuration.

Can be specified multiple times

Specifies the pod [toleration](#) with the given configuration.

The configuration should be a map corresponding to a single toleration rule. For example, the following pod options:

```
process.pod = [
  [toleration: [key: 'key1', operator: 'Equal', value: 'value1', effect: 'NoSchedule']],
  [toleration: [key: 'key1', operator: 'Exists', effect: 'NoSchedule']],
]
```

Maps to the following pod spec:

```

tolerations:
- key: "key1"
  operator: "Equal"
  value: "value1"
  effect: "NoSchedule"
- key: "key1"
  operator: "Exists"
  effect: "NoSchedule"

```

Added in version 24.04

Specifies the [TTL mechanism](#) for finished jobs in seconds. Applies to both successful and failed jobs.

Can be specified multiple times

Mounts a [Persistent volume claim](#) with the given name to the given path.

The `subPath` option can be used to mount a sub-directory of the volume instead of its root.

The `readOnly` option can be used to mount the volume as read-only (default: `false`)

note

[Workflow outputs](#) can be used instead of `publishDir`. See [Migrating to workflow outputs](#) to learn how to migrate existing code.

The `publishDir` directive publishes matching process output files to a target directory. For example:

```

process hello {
  publishDir '/data/chunks'

  output:
  path 'chunk_*'

  script:
  """
  printf 'Hola' | split -b 1 - chunk_
  """
}

```

The above example publishes the `chunk_*` output files into the `/data/chunks` directory.

Only files that match the declaration in the `output` block are published, not all the outputs of the process.

The `publishDir` directive can be specified more than once in order to publish output files to different target directories based on different rules.

By default, files are published via symbolic link from the task directory to the target directory. Use the `mode` option to control this behavior:


```
process hello {
  publishDir '/data/chunks', mode: 'copy', overwrite: false

  output:
  path 'chunk_*'

  script:
  """
  printf 'Hola' | split -b 1 - chunk_
  """
}
```

warning

Output files are published asynchronously after the task execution, so they may not be immediately available in the publish directory during the pipeline run. Downstream processes should access output files through the declared process outputs, not the publish directory.

Available options:

Experimental: currently only supported for S3.

Allow specifying the media content type of the published file a.k.a. [MIME type](#). If set to `true`, the content type is inferred from the file extension (default: `false`).

Enable or disable the publish rule depending on the boolean value specified (default: `true`).

Changed in version 24.04

The default value was changed from `false` to `true`

When `true` abort the execution if some file can't be published to the specified target directory or bucket for any cause (default: `true`)

The file publishing method. Can be one of the following values:

- `'copy'`: Copies the output files into the publish directory.
- `'copyNoFollow'`: Copies the output files into the publish directory without following symlinks ie. copies the links themselves.
- `'link'`: Creates a hard link in the publish directory for each output file.
- `'move'`: Moves the output files into the publish directory. **Note:** this is only supposed to be used for a terminal process i.e. a process whose output is not consumed by any other downstream process.
- `'relink'`: Creates a relative symbolic link in the publish directory for each output file.
- `'symlink'`: Creates an absolute symbolic link in the publish directory for each output file (default).

When `true` any existing file in the target directory will be overridden (default: `true` during normal pipeline execution and `false` when pipeline execution is `resumed`).

Specifies the directory where files need to be published. **Note:** the syntax `publishDir '/some/dir'` is a shortcut for `publishDir path: '/some/dir'`.

Specifies a [glob](#) file pattern that selects which files to publish from the overall set of output files.

A closure which, given the name of the file being published, returns the actual file name or a full path where the file is required to be stored. This can be used to rename or change the destination directory of the published files dynamically by using a custom strategy. Return the value `null` from the closure to not publish a file. This is useful when the process has multiple output files, but you want to publish only some of them.

Added in version 23.04

Experimental: currently only supported for S3.

Allow specifying the storage class to be used for the published file.

Experimental: currently only supported for S3.

Allow the association of arbitrary tags with the published file e.g. `tags: [MESSAGE: 'Hello world']`.

The `queue` directive defines the queue to which tasks should be submitted, for executors that support queues. For example:

```
process hello {
  queue 'long'
  executor 'slurm'

  script:
  """
  your_command --here
  """
}
```

Some executors can accept multiple queue names as a comma-separated string:

```
queue 'short,long,cn-el6'
```

However, this is not generally supported by cloud executors such as AWS Batch, Azure Batch, and Google Batch.

See [Executors](#) to see which executors support this directive.

The `resourceLabels` directive attaches custom name-value pairs to task executions, for executors that support it. For example:

```
process hello {
  resourceLabels region: 'some-region', user: 'some-username'

  script:
  """
  your_command --here
  """
}
```

Resource labels are attached to underlying resources such as cloud VMs, and are intended for operational purposes such as cost tracking. They are not recorded in lineage metadata.

When `resourceLabels` is specified multiple times in the config, only the last setting is used. Additionally, when `resourceLabels` is specified both in the config and the process definition, only the process definition is used.

As a best practice, define all resource labels in a single config setting:

```
process {
  resourceLabels = [ region: 'some-region', user: 'some-username' ]
}
```

Use process selectors (`withName:` or `withLabel:`) to override resource labels for a specific process.

Resource labels are currently supported by the following executors:

- [AWS Batch](#)
- [Azure Batch](#)
- [Google Cloud Batch](#)
- [Kubernetes](#)
- [Seqera executor](#)

note

The limits and the syntax of the corresponding executor should be taken into consideration when using resource labels.

Added in version 23.10

Resource labels in Azure are added to auto-pools, rather than jobs, in order to facilitate cost analysis. A new pool will be created for each new set of resource labels. Therefore, it is recommended to also set `azure.batch.deletePoolsOnCompletion = true` when using process-specific resource labels.

See also: [label](#) (for shared process configuration), [tag](#) (for per-task identification)

Added in version 24.04

The `resourceLimits` directive defines environment-specific limits for task resource requests.

Resource limits can be specified in a process:

```
process hello {
  resourceLimits cpus: 24, memory: 768.GB, time: 72.h

  script:
  """
  your_command --here
  """
}
```

Or in the Nextflow configuration:

```
process.resourceLimits = [
  cpus: 24,
  memory: 768.GB,
  time: 72.h
]
```

Resource limits can be defined for the following directives:

- `cpus`
- `disk`
- `memory`
- `time`

When a task resource request exceeds the corresponding limit, the task resources are automatically reduced to comply with these limits before the job is submitted.

Resource limits are a useful way to prevent tasks with `dynamic resources` from requesting more resources than can be provided by an executor (e.g. a task requests 32 cores but the largest node in the cluster has 24).

The `scratch` directive executes each task in a temporary directory that is local to the compute node.

This is useful when executing tasks on an executor with a shared filesystem, because it decreases the network overhead of reading and writing files. Only the files declared as process outputs are copied to the pipeline work directory.

For example:

```
process hello {
  scratch true

  output:
  path 'data_out'

  script:
  """
  your_command --here
  """
}
```

It can also be specified in the Nextflow configuration:

```
process.scratch = true
```

By default, the `scratch` directive uses the `$TMPDIR` environment variable in the underlying node as the base scratch directory. If `$TMPDIR` is not defined, then it creates a scratch directory using the `mktemp` command.

Each task creates a subdirectory within the base scratch directory and automatically deletes it upon completion.

note

Cloud-based executors enable `scratch` by default since the pipeline work directory resides in object storage.

The following values are supported:

Do not use a scratch directory.

Create a scratch directory in the directory defined by the `$TMPDIR` environment variable, or `$(mktemp /tmp)` if `$TMPDIR` is not set.

Create a scratch directory in the directory defined by the given environment variable, or `$(mktemp /tmp)` if that variable is not set. The value must use single quotes, otherwise the environment variable will be evaluated in the pipeline script context.

Create a scratch directory in the specified directory.

Create a scratch directory in the RAM disk `/dev/shm/`.

The `secret` directive allows a process to access secrets.

For example:

```
process hello_secret {
  secret 'MY_ACCESS_KEY'
  secret 'MY_SECRET_KEY'

  script:
  """
  your_command --access \${MY_ACCESS_KEY} --secret \${MY_SECRET_KEY}
  """
}
```

Each secret is provided to the task as an environment variable.

See [Secrets](#) for more information.

note

Secrets can only be used with the local or grid executors (e.g., Slurm or Grid Engine). Secrets can be used with AWS Batch and Google Batch when launched from Sequera Platform.

The `shell` directive defines a custom shell command for process scripts. By default, script blocks are executed with `/bin/bash -ue`.

```
process hello {
  shell '/bin/bash', '-euo', 'pipefail'

  script:
  """
  your_command --here
  """
}
```

It can also be specified in the Nextflow configuration:

```
process.shell = ['/bin/bash', '-euo', 'pipefail']
```

The `spack` directive defines the set of [Spack](#) packages required by each task. For example:

```
process hello {
  spack 'bwa@0.7.15'

  script:
  """
  your_command --here
  """
}
```

Nextflow automatically creates a Spack environment for each unique set of packages.

Multiple packages can be specified separating them with a blank space, e.g. `bwa@0.7.15 fastqc@0.11.5`.

The `spack` directive also accepts a Spack environment file path or the path of an existing Spack environment. See [Spack](#) for more information.

The `stageInMode` directive defines how input files are staged into the task work directory.

The following modes are supported:

Input files are staged in the task work directory by creating a copy.

Input files are staged in the task work directory by creating a hard link for each of them.

Input files are staged in the task work directory by creating a symbolic link with a relative path for each of them.

Input files are staged in the task work directory by creating a symbolic link with an absolute path for each of them (default).

The `stageOutMode` directive defines how output files are staged out from the scratch directory to the task work directory.

The following modes are supported:

Output files are copied from the scratch directory to the work directory.

Added in version 23.04

Output files are copied from the scratch directory to the work directory by using the `fc` utility (note: it must be available in the task environment).

Output files are moved from the scratch directory to the work directory.

Added in version 23.04

Output files are copied from the scratch directory to the work directory by using the `rclone` utility (note: it must be available in the task environment).

Output files are copied from the scratch directory to the work directory by using the `rsync` utility.

See also: [scratch](#)

The `storeDir` directive stores task outputs in a permanent store directory instead of the work directory.

On subsequent runs, each task is executed only if the declared output files do not exist in the store directory. When the files are present, the task is skipped and these files are used as the task outputs.

The following example shows how to use the `storeDir` directive to create a directory containing a BLAST database for each species specified by an input parameter:

```

process make_blast_db {
  storeDir '/db/genomes'

  input:
  path species

  output:
  path "${dbName}.*"

  script:
  dbName = species.baseName
  """
  makeblastdb -dbtype nucl -in ${species} -out ${dbName}
  """
}

```

Caveats:

- The `env`, `eval`, and `stdout` output qualifiers cannot be used with `storeDir` because they rely on helper files in the task directory. Use `path` outputs instead.
- If a process uses `storeDir` and all of its outputs are optional, the process will always be skipped, even if the store directory is empty. Avoid this issue by specifying at least one required file output.
- The `storeDir` directive is not a replacement for publishing outputs. Use the [publishDir](#) directive or [workflow outputs](#) instead.

The `tag` directive defines a custom identifier for each task execution. For example:

```

process hello {
  tag "$code"

  input:
  val code

  script:
  """
  echo $code
  """
}

workflow {
  ch_codes = channel.of('alpha', 'gamma', 'omega')
  hello(ch_codes)
}

```

The above example logs each task with its corresponding tag:

```

[6e/28919b] Submitted process > hello (alpha)
[d2/1c6175] Submitted process > hello (gamma)
[1c/3ef220] Submitted process > hello (omega)

```

Tags are a useful way to track related tasks in a pipeline run. Tasks can be identified by tag in the [execution log](#) and the [trace report](#).

note

The name of a task in both [reports](#) and [lineage](#) is defined as `<process> (<tag>)`.

note

The `tag` directive is not related to the `label` directive. Process labels are only used for shared process configuration, not for tracking.

The `time` directive defines the maximum runtime for each task. For example:

```
process hello {  
  time 1.h  
  
  script:  
  """  
  your_command --here  
  """  
}
```

The following suffixes can be used to specify duration values:

- `ms`: milliseconds
- `s`: seconds
- `m`: minutes
- `h`: hours
- `d`: days

See [Duration](#) for more information.

note

This directive is only used by certain executors. Refer to the [Executors](#) page to see which executors support this directive.

See also: [cpus](#), [disk](#), [memory](#), [queue](#), [dynamic task resources](#)

Channel factories

This page describes the channel factories that are available in the `channel` namespace.

`empty()` -> `Channel`

The `channel.empty` factory creates a channel that emits nothing:

```
channel.empty().view() // prints nothing
```

note

As a best practice, use a samplesheet instead of matching file pairs directly with glob patterns. See [Samplesheets](#) for more information.

The `channel.fromFilePairs` factory creates a channel that emits file pairs matching a [glob](#) pattern:

```
ch = channel.fromFilePairs('/my/data/SRR*_{1,2}.fastq')
ch.view()
```

Each file pair is emitted as a 2-tuple containing the grouping key (from the `*` wildcard) and the list of files (sorted lexicographically):

```
[SRR493366, [/my/data/SRR493366_1.fastq, /my/data/SRR493366_2.fastq]]
[SRR493367, [/my/data/SRR493367_1.fastq, /my/data/SRR493367_2.fastq]]
[SRR493368, [/my/data/SRR493368_1.fastq, /my/data/SRR493368_2.fastq]]
[SRR493369, [/my/data/SRR493369_1.fastq, /my/data/SRR493369_2.fastq]]
[SRR493370, [/my/data/SRR493370_1.fastq, /my/data/SRR493370_2.fastq]]
[SRR493371, [/my/data/SRR493371_1.fastq, /my/data/SRR493371_2.fastq]]
```

The glob pattern must contain at least one `*` wildcard character.

Available options:

`checkIfExists` : When `true`, throws an error if the file path does not exist in the file system (default: `false`).

`flat` : When `true`, tuples are emitted with the matching files flattened instead of as a nested list (default: `false`).

`followLinks` : When `true`, follows symbolic links when traversing a directory tree, otherwise treats them as files (default: `true`).

`hidden` : When `true`, matches hidden files when using a glob pattern (default: `false`).

`maxDepth` : Maximum number of directory levels to visit with the `**` wildcard (default: no limit).

`size` : The number of expected files for each file pair (default: `2`). Set to `-1` to allow any size.

`type` : Whether to return only files (`'file'`), only directories (`'dir'`), or both (`'any'`) when using a glob pattern. By default, only files are returned (`'file'`).

Added in version 25.04

warning

Experimental: may change in a future release.

```
fromLineage( [opts] ) -> Channel<Path>
```

The `channel.fromLineage` factory creates a channel that emits files from the `lineage` store that match the given key-value params:

```
ch = channel.fromLineage(  
  workflowRun: 'lid://0d1d1622ced3e4edc690bec768919b45',  
  label: ['alpha', 'beta']  
)  
ch.view()
```

The above snippet emits files published by the given workflow run that are labeled as `alpha` and `beta`.

Available options:

`label` : List of labels associated with the desired files.

`taskRun` : LID of the task run that produced the desired files.

`workflowRun` : LID of the workflow run that produced the desired files.

```
fromList( values: Iterable<E> ) -> Channel<E>
```

The `channel.fromList` factory creates a channel that emits each element in a collection:

```
ch = channel.fromList( ['a', 'b', 'c', 'd'] )  
ch.view { v -> "value: $v" }
```

Prints:

```
value: a  
value: b  
value: c  
value: d
```

See also: [channel.of](#)

```
fromPath( pattern: String, [opts] ) -> Channel<Path>
```

The `channel.fromPath` factory creates a channel that emits file paths matching a name or `glob` pattern:

```
// match single file
channel.fromPath('data/some/bigfile.txt')

// match `txt` files in `data/bag`
channel.fromPath('data/big/*.txt')

// match `fa` files in `data` and its subdirectories
channel.fromPath('data/**/*.fa')

// match `fa` files with same suffix in any subdirectory of `data`
channel.fromPath('data/**/*.fa')

// match file pair (`file_1.fq` and `file_2.fq`)
channel.fromPath('data/file_{1,2}.fq')
```

By default, glob patterns do not match hidden files (i.e. files with names that start with `.`). Use a glob pattern that explicitly starts with `.` or set `hidden: true` to match hidden files:

```
// match hidden files in `data`
channel.fromPath('data/.*)
channel.fromPath('data/*', hidden: true)

// match hidden files in `data` with `fa` extension
channel.fromPath('data/*.fa')
```

By default, glob patterns only match regular files, not directories. Use the `type` option to control whether to match files, directories, or both:

```
// match only directories
channel.fromPath('data/*', type: 'dir')

// match files and directories
channel.fromPath('data/*', type: 'any')
```

Available options:

`checkIfExists` : When `true`, throws an error if the file path does not exist in the file system (default: `false`).

`followLinks` : When `true`, follows symbolic links when traversing a directory tree, otherwise treats them as files (default: `true`).

`glob` : When `true`, interprets the characters `*`, `?`, `[]`, and `{}` as glob wildcards, otherwise treats them as normal characters (default: `true`).

`hidden` : When `true`, matches hidden files when using a glob pattern (default: `false`).

`maxDepth` : Maximum number of directory levels to visit with the `**` wildcard (default: no limit).

`relative` : When `true`, returns file paths as relative to the top-most common directory (default: `false`).

`type` : Whether to return only files (`'file'`), only directories (`'dir'`), or both (`'any'`) when using a glob pattern. By default, only files are returned (`'file'`).

Deprecated in version 26.04

Use the [Entrez Direct](#) command-line tool to query the SRA database.

The `channel.fromSRA` factory queries the [NCBI SRA](#) database and returns a channel emitting the FASTQ files matching the specified criteria i.e project or accession number(s). For example:

```
channel.fromSRA('SRP043510').view()
```

It returns:

```
[SRR1448794, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR144/004/SRR1448794/SRR1448794.fastq.gz]
[SRR1448795, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR144/005/SRR1448795/SRR1448795.fastq.gz]
[SRR1448792, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR144/002/SRR1448792/SRR1448792.fastq.gz]
[SRR1448793, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR144/003/SRR1448793/SRR1448793.fastq.gz]
[SRR1910483, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR191/003/SRR1910483/SRR1910483.fastq.gz]
[SRR1910482, ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR191/002/SRR1910482/SRR1910482.fastq.gz]
(remaining omitted)
```

Multiple accession IDs can be specified as a list:

```
ids = ['ERR908507', 'ERR908506', 'ERR908505']
channel.fromSRA(ids).view()
```

```
[ERR908507, [ftp://ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908507/ERR908507_1.fastq.gz, ftp://
ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908507/ERR908507_2.fastq.gz]]
[ERR908506, [ftp://ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908506/ERR908506_1.fastq.gz, ftp://
ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908506/ERR908506_2.fastq.gz]]
[ERR908505, [ftp://ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908505/ERR908505_1.fastq.gz, ftp://
ftp.sra.ebi.ac.uk/vol1/fastq/ERR908/ERR908505/ERR908505_2.fastq.gz]]
```

Each read pair is implicitly managed and returned as a list of files.

This method uses the NCBI [ESearch](#) API behind the scenes, therefore it allows the use of any query term supported by this API.

To access the ESearch API, you must provide your [NCBI API keys](#) through one of the following ways:

- The `apiKey` option:

```
channel.fromSRA(ids, apiKey='0123456789abcdef')
```

- The `NCBI_API_KEY` variable in your environment:

```
export NCBI_API_KEY=0123456789abcdef
```

Available options:

`apiKey` : NCBI user API key.

`cache` : Enable/disable the caching API requests (default: `true`).

`max` : Maximum number of entries that can be retried (default: unlimited) .

`protocol` : Allow choosing the protocol for the resulting remote URLs. Available choices: `ftp`, `http`, `https` (default: `ftp`).

`retryPolicy` : Set a retry policy in case the SRA request fails with a retrievable error.

: Available properties:

- `delay`: Delay between attempts (default: 500ms)
- `jitter`: Jitter value (default: 0.25)
- `maxAttempts`: Max attempts (default: 3)
- `maxDelay`: Max delay (default: 30s)

: For example:

```
channel.fromSRA(ids, retryPolicy: [delay: '250ms', maxAttempts: 5])
```

`interval( interval: String ) -> Channel<Integer>`

The `channel.interval` factory emits an incrementing index (starting from zero) at a periodic interval. For example:

```
channel.interval('1s').view()
```

The above snippet will emit 0, 1, 2, and so on, every second, forever. You can use an operator such as [take](#) or [until](#) to close the channel based on a stopping condition.

`of( values... ) -> Channel`

The `channel.of` factory allows you to create a channel that emits each argument:

```
ch = channel.of( 1, 3, 5, 7 ).view()
```

Prints:

```
1
3
5
7
```

Ranges of values are expanded accordingly:

```
channel.of(1..23, 'X', 'Y').view()
```

Prints:

```
1
2
3
4
:
23
X
Y
```

See also: [channel.fromList](#)

Added in version 25.04

note

This feature was previewed in versions 24.04 and 24.10 with the `nextflow.preview.topic` feature flag.

`topic( name: String ) -> Channel`

A topic channel is a channel that can receive values from many sources implicitly based on a matching topic name.

A typed process can emit values to a topic using the `topic:` section:

```
nextflow.enable.types = true

process hello {
    topic:
    file('hello.txt') >> 'my-topic'

    // ...
}

process bye {
    topic:
    file('bye.txt') >> 'my-topic'

    // ...
}
```

A legacy process can assign outputs in the `output:` section to a topic using the `topic` option:

```
process hello {
    output:
    path('hello.txt'), topic: 'my-topic'

    // ...
}
```

The `channel.topic` factory returns the topic channel for the given name:

```
channel.topic('my-topic').view()
```

The above example emits all values sent to the `my-topic` topic from processes such as `hello` and `bye`.

This approach is a convenient way to collect related items from many different sources without explicitly connecting them (e.g. using the `mix` operator).

warning

Any process that consumes a topic channel (directly or indirectly) should not send any outputs to that topic, or else the pipeline will hang forever.

See also: [Topics](#) for process outputs

`value( value: V ) -> Value<V>`

The `channel.value` factory creates a dataflow value bound to the given argument:

```
channel.value( 'Hello there' )  
channel.value( [1,2,3,4,5] )
```

The first line creates a dataflow value bound to the string 'Hello there'. The second line creates a dataflow value bound to the list [1,2,3,4,5], which is treated as a single value in dataflow logic.

watchPath(pattern: String, events: String = 'create') -> Channel<Path>

The `channel.watchPath` factory creates a channel that watches a [glob](#) pattern and emits matching files as they appear.

For example:

```
ch = channel.watchPath('/path/*.fa')  
ch.view { fa -> "Fasta file: $fa" }
```

The second argument specifies which filesystem events to watch as a comma-separated string:

```
ch = channel.watchPath('/path/*.fa', 'create,modify')  
ch.view { fa -> "File created or modified: $fa" }
```

By default, only new files are watched. The following events are supported:

- `create`: A new file is created
- `modify`: A file is modified
- `delete`: A file is deleted

warning

The `channel.watchPath` factory waits endlessly for matching files, which means that it will cause your pipeline to run forever. Consider using the `take` or `until` operator to apply a stopping condition (e.g. receiving 10 files, receiving a file named `DONE`).

note

The `channel.watchPath` factory only works with local and shared filesystems. It does not support object storage such as S3.

See also: [channel.fromPath](#)

Operators (typed)

Added in version 26.04

This page describes the core operators that are recommended for use with static typing.

Channel<E> collect() -> Value<Bag<E>>

The `collect` operator collects all values from a source channel into a collection and emits it as a dataflow value:

```
channel.of( 1, 2, 3, 4 )
    .collect()
    .view()
```

```
[1, 2, 3, 4]
```

Channel<L> combine(other: Channel<R>) -> Channel<Tuple>

Channel<L> combine(other: Value<R>) -> Channel<Tuple>

The `combine` operator emits every pairwise combination of a source channel with another channel or dataflow value:

```
numbers = channel.of(1, 2, 3)
words = channel.of('hello', 'ciao')

numbers
    .combine(words)
    .view()
```

```
[1, hello]
[2, hello]
[3, hello]
[1, ciao]
[2, ciao]
[3, ciao]
```

Tuples in both the left- and right-hand sources are flattened in the combined tuple. For example, `tuple(1, 2)` and `tuple('red', 'blue')` are combined as `tuple(1, 2, 'red', 'blue')`.

Channel<Record> combine([opts]) -> Channel<Record>

When the `combine` operator is called on a channel of records with named arguments, the named arguments are appended to each record in the source channel. Each named argument can be a value or dataflow value.

Channel<E> filter(condition: (E) -> Boolean) -> Channel<E>

The `filter` operator emits the values from a source channel that satisfy a condition, discarding all other values:


```
channel.of( 1, 2, 3, 4, 5 )
    .filter { v -> v % 2 == 1 }
    .view()
```

```
1
3
5
```

Channel<E> flatMap(transform: (E) -> Iterable<R>) -> Channel<R>

The `flatMap` operator applies a mapping function to each value from a source channel. The mapping function should return a collection, and each element in the collection is emitted separately.

For example:

```
channel.of( 1, 2, 3 )
    .flatMap { n -> [ n, n*2, n*3 ] }
    .view()
```

```
1
2
3
2
4
6
3
6
9
```

Channel<Tuple<K, V>> groupBy() -> Channel<Tuple<K, Bag<V>>>

Channel<Tuple<K, Integer, V>> groupBy() -> Channel<Tuple<K, Bag<V>>>

The `groupBy` operator collects values from a source channel into groups based on a grouping key. A tuple is emitted for each group, containing the grouping key and collection of values.

The source channel should supply either 2-tuples of the form (`<key>`, `<value>`) or 3-tuples of the form (`<key>`, `<size>`, `<value>`).

If the source tuples do not specify a size, `groupBy` will not emit any groups until all inputs have been received:

```
channel.of( tuple(1, 'A'), tuple(1, 'B'), tuple(2, 'C'), tuple(3, 'B'), tuple(1, 'C'), tuple(2, 'A'),
    tuple(3, 'D') )
    .groupBy()
    .map { key, values -> tuple(key, values.toSorted()) }
    .view()
```

```
[1, [A, B, C]]
[2, [A, C]]
[3, [B, D]]
```

If the source tuples do specify a size, then `groupBy` will emit each group as soon as it is ready:

```
channel.of(
  tuple('chr1', ['/path/to/region1_chr1.vcf', '/path/to/region2_chr1.vcf']),
  tuple('chr2', ['/path/to/region1_chr2.vcf', '/path/to/region2_chr2.vcf', '/path/to/region3_chr2.vcf']),
)
.flatMap { chr, vcfs ->
  vcfs.collect { vcf ->
    tuple(chr, vcfs.size(), vcf)    // preserve group size
  }
}
.view { v -> "scattered: ${v}" }
.groupBy()
.map { key, values -> tuple(key, values.toSorted()) }
.view { v -> "gathered: ${v}" }
```

```
scattered: [chr1, 2, /path/to/region1_chr1.vcf]
scattered: [chr1, 2, /path/to/region2_chr1.vcf]
scattered: [chr2, 3, /path/to/region1_chr2.vcf]
scattered: [chr2, 3, /path/to/region2_chr2.vcf]
scattered: [chr2, 3, /path/to/region3_chr2.vcf]
gathered: [chr1, [/path/to/region1_chr1.vcf, /path/to/region2_chr1.vcf]]
gathered: [chr2, [/path/to/region1_chr2.vcf, /path/to/region2_chr2.vcf, /path/to/region3_chr2.vcf]]
```

note

When specifying the group size, make sure that the number of inputs for a given group matches the specified size for that group. Otherwise, the run will fail.

Channel<Record> join(other: Channel<Record>, [opts]) -> Channel<Record>

The `join` operator emits the relational join of two channels of records, using a matching key given by the `by` option:

```
left = channel.of( record(id: 'X', a: 1), record(id: 'Y', a: 2), record(id: 'Z', a: 3), record(id: 'P', a: 7) )
right = channel.of( record(id: 'Z', b: 6), record(id: 'Y', b: 5), record(id: 'X', b: 4) )

left.join(right, by: 'id').view()
```

```
[id:Z, a:3, b:6]
[id:Y, a:2, b:5]
[id:X, a:1, b:4]
```

Duplicate matching keys are handled by emitting each matching combination (like a relational join):

```
left = channel.of( record(id: 'X', a: 1), record(id: 'X', a: 3) )
right = channel.of( record(id: 'X', b: 2), record(id: 'X', b: 4) )

left.join(right, by: 'id').view()
```

```
[id:X, a:1, b:2]
[id:X, a:1, b:4]
[id:X, a:3, b:2]
[id:X, a:3, b:4]
```

By default, unmatched values are discarded. The `remainder` option can be used to emit them at the end:

```

left = channel.of( record(id: 'X', a: 1), record(id: 'Y', a: 2), record(id: 'Z', a: 3), record(id: 'P', a: 7) )
right = channel.of( record(id: 'Z', b: 6), record(id: 'Y', b: 5), record(id: 'X', b: 4), record(id: 'Q', b: 8) )

left.join(right, by: 'id', remainder: true).view()

```

```

[id:Y, a:2, b:5]
[id:Z, a:3, b:6]
[id:X, a:1, b:4]
[id:P, a:7]
[id:Q, b:8]

```

Available options:

by: `String` : This option is required. : The record field to use as the matching key.

remainder: `Boolean` : When `true`, unmatched values are emitted at the end, otherwise they are discarded (default: `false`).

Channel<E> map(transform: (E) -> R) -> Channel<R>

The `map` operator applies a mapping function to each value from a source channel:

```

channel.of( 1, 2, 3, 4, 5 )
  .map { v -> v * v }
  .view()

```

```

1
4
9
16
25

```

Channel<E> mix(other: Channel<E>) -> Channel<E>

Channel<E> mix(other: Value<E>) -> Channel<E>

The `mix` operator emits the values from a channel and another channel or dataflow value into a single output channel:

```

c1 = channel.of( '1', '2', '3' )
c2 = channel.of( 'a', 'b' )
v3 = channel.value( 'z' )

c1.mix(c2).mix(v3).view()

```

```

1
2
3
a
b
z

```

The values in the output channel may be emitted in any order, for example:

```
z
1
a
2
b
3
```

```
Channel<E> reduce( seed: R, accumulator: (R, E) -> R ) -> Value<R>
```

```
Channel<E> reduce( accumulator: (E, E) -> E ) -> Value<E>
```

The `reduce` operator applies an accumulator function sequentially to each value in a source channel, and emits the final accumulated value.

The accumulator function takes two parameters -- the accumulated value and the i-th emitted value -- and it should return the accumulated result, which is passed to the next invocation with the i+1-th value. This process is repeated for each value in the source channel.

For example:

```
channel.of( 1, 2, 3, 4, 5 )
  .reduce { a, b ->
    println "a: $a b: $b"
    a + b
  }
  .view { result -> "result = $result" }
```

```
a: 1 b: 2
a: 3 b: 3
a: 6 b: 4
a: 10 b: 5
result = 15
```

By default, the first value is used as the initial accumulated value (the seed). You can optionally specify a different initial value as shown below:

```
channel.of( 1, 2, 3, 4, 5 )
  .reduce( 'result:' ) { acc, v ->
    println acc
    acc + ' ' + v
  }
  .view { result -> "final $result" }
```

```
result:
result: 1
result: 1 2
result: 1 2 3
result: 1 2 3 4
final result: 1 2 3 4 5
```

```
Channel<E> subscribe( action: (E) -> () )
```

```
Channel<E> subscribe( [opts] )
```

The `subscribe` operator invokes a custom function for each value from a source channel:

```
source = channel.of( 'alpha', 'beta', 'delta' )

source.subscribe { str ->
    println "Got: ${str}; len: ${str.length()}"
}
```

```
Got: alpha; len: 5
Got: beta; len: 4
Got: delta; len: 5
```

The `subscribe` operator supports multiple types of event handlers:

```
channel.of( 1, 2, 3 ).subscribe(
    onNext: { v -> println v },
    onComplete: { println 'Done' }
)
```

```
1
2
3
Done
```

note

Unlike most operators, `subscribe` does not return anything. It should only be used for side effects, such as printing to the console, writing to a file, or making HTTP requests.

Available options:

`onNext: (E) -> ()` : Closure that is invoked when an value is emitted. Equivalent to providing a single closure argument.

`onComplete: () -> ()` : Closure that is invoked after the last value is emitted by the channel.

`onError: (T) -> ()` : Closure that is invoked when an exception is raised while handling the `onNext` event. It will not make further calls to `onNext` or `onComplete`. The `onError` method takes as its parameter the `Throwable` that caused the error.

`Channel<E> unique( transform: (E) -> ? ) -> Channel<E>` **`Channel<E> unique() -> Channel<E>`**

The `unique` operator emits the unique values from a source channel:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 2, 3 )
    .unique()
    .view()
```

```
1
2
3
```

An optional closure can be used to transform each value before it is evaluated for uniqueness:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 4, 6 )
    .unique { v -> v % 2 }
    .view()
```

```
1
2
```

Channel<E> until(condition: (E) -> Boolean) -> Channel<E>

The `until` operator emits each value from a source channel until a stopping condition is satisfied:

```
channel.of( 3, 2, 1, 5, 1, 5 )
    .until { v -> v == 5 }
    .view()
```

```
3
2
1
```

Channel<E> view(transform: (E) -> String, [opts]) -> Channel<E>

Channel<E> view([opts]) -> Channel<E>

The `view` operator prints each value from a source channel to standard output:

```
channel.of(1, 2, 3).view()
```

```
1
2
3
```

An optional closure can be used to transform each value before it is printed:

```
channel.of(1, 2, 3)
    .map { v -> [v, v*v] }
    .view { num, sqr -> "The square of $num is $sqr" }
```

```
The square of 1 is 1
The square of 2 is 4
The square of 3 is 9
```

The `tag` option can be used to print the channel only when the `-dump-channels` command-line option is specified with the given tag:

```
channel.of( 1, 2, 3 )
    .map { v -> v + 1 }
    .view(tag: 'plus1')

channel.of( 1, 2, 3 )
    .map { v -> v ** 2 }
    .view(tag: 'exp2')
```

You can run this script with `-dump-channels plus1` or `-dump-channels exp2` to print either channel, or `-dump-channels plus1,exp2` to print both.

The `view` operator also emits every value that it receives, allowing it to be chained with other operators.

Available options:

`newLine: Boolean` : Print each value to a separate line (default: `true`).

`tag: String` : Print the channel values only when `-dump-channels` is specified on the command line with the given tag.

Operators (legacy)

Added in version 26.04

See [Operators \(typed\)](#) for the new typed operator reference. See [Using operators with static typing](#) for the migration guide.

Returns: multiple channels

The `branch` operator forwards each item from a source channel to one of multiple output channels, based on a selection criteria.

The selection criteria is a [closure](#) that defines, for each output channel, a unique label followed by a boolean expression. When an item is received, it is routed to the first output channel whose expression evaluates to `true`. For example:

```
channel.of(1, 2, 3, 40, 50)
  .branch { v ->
    small: v < 10
    large: v > 10
  }
  .set { result }

result.small.view { v -> "$v is small" }
result.large.view { v -> "$v is large" }
```

```
1 is small
2 is small
3 is small
40 is large
50 is large
```

note

The above output may be printed in any order since the two `view` operations are executed asynchronously.

A fallback condition can be specified using `true` as the last branch condition:

```
channel.of(1, 2, 3, 40, 50)
  .branch { v ->
    small: v < 10
    large: v < 50
    other: true
  }
  .set { result }

result.small.view { v -> "$v is small" }
result.large.view { v -> "$v is large" }
result.other.view { v -> "$v is other" }
```



```
1 is small
2 is small
3 is small
40 is large
50 is other
```

The value emitted to each branch can be customized with an expression statement (or statements) after the branch condition:

```
channel.of(1, 2, 3, 40, 50)
  .branch { v ->
    alpha: v < 10
      return v + 2

    beta: v < 50
      return v - 2

    other: true
      return 0
  }
  .set { result }

result.alpha.view { v -> "$v is alpha" }
result.beta.view { v -> "$v is beta" }
result.other.view { v -> "$v is other" }
```

```
3 is alpha
4 is alpha
5 is alpha
38 is beta
0 is other
```

tip

When the `return` keyword is omitted, the value of the last expression statement is implicitly returned.

The `branchCriteria()` method can be used to create a branch criteria as a variable that can be passed as an argument to any number of `branch` operations, as shown below:

```
def criteria = branchCriteria { v ->
  small: v < 10
  large: v > 10
}

channel.of(1, 2, 30).branch(criteria).set { ch1 }
channel.of(10, 20, 3).branch(criteria).set { ch2 }

ch1.small.view { v -> "$v is small" }
ch1.large.view { v -> "$v is large" }
ch2.small.view { v -> "$v is small" }
ch2.large.view { v -> "$v is large" }
```

```
1 is small
2 is small
3 is small
20 is large
30 is large
```

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: queue channel

The `buffer` operator collects items from a source channel into subsets and emits each subset separately.

This operator has multiple variants:

Emits each subset when `closingCondition` is satisfied. The closing condition can be a literal value, a [regular expression](#), a type qualifier (i.e. Java class), or a boolean predicate. For example:

```
channel.of( 1, 2, 3, 1, 2, 3 )
        .buffer { v -> v == 2 }
        .view()
```

```
[1, 2]
[3, 1, 2]
```

Creates a new subset when `openingCondition` is satisfied and emits the subset when `closingCondition` is satisfied. The opening and closing conditions can each be a literal value, a [regular expression](#), a type qualifier (i.e. Java class), or a boolean predicate. For example:

```
// emits bundles starting with `2` and ending with `4`
channel.of( 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2 )
        .buffer( 2, 4 )
        .view()
```

```
[2, 3, 4]
[2, 3, 4]
```

Emits a new subset for every `n` items. Remaining items are discarded. For example:

```
channel.of( 1, 2, 3, 1, 2, 3, 1 )
        .buffer( size: 2 )
        .view()
```

```
[1, 2]
[3, 1]
[2, 3]
```

The `remainder` option can be used to emit any remaining items as a partial subset:

```
channel.of( 1, 2, 3, 1, 2, 3, 1 )
        .buffer( size: 2, remainder: true )
        .view()
```

```
[1, 2]
[3, 1]
[2, 3]
[1]
```

Emits a new subset for every `n` items, skipping `m` items before collecting each subset. For example:

```
channel.of( 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2 )
    .buffer( size: 3, skip: 2 )
    .view()
```

```
[3, 4, 5]
[3, 4, 5]
```

The `remainder` option can be used to emit any remaining items as a partial subset.

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: channel

The `collate` operator collects items from a source channel into groups of `N` items.

This operator has multiple variants:

Collects items into groups of `size` items:

```
channel.of(1, 2, 3, 1, 2, 3, 1)
    .collate( 3 )
    .view()
```

```
[1, 2, 3]
[1, 2, 3]
[1]
```

By default, any remaining items are emitted as a partial group. You can specify `false` as the second parameter to discard them instead:

```
channel.of(1, 2, 3, 1, 2, 3, 1)
    .collate( 3, false )
    .view()
```

```
[1, 2, 3]
[1, 2, 3]
```

note

This version of `collate` is equivalent to `buffer( size: n, remainder: true | false )`.

Collects items into groups of `size` items using a sliding window that moves by `step` items at a time:

```
channel.of(1, 2, 3, 4)
  .collate( 3, 1 )
  .view()
```

```
[1, 2, 3]
[2, 3, 4]
[3, 4]
[4]
```

You can specify `false` as the third parameter to discard any remaining items.

Returns: dataflow value

The `collect` operator collects all items from a source channel into a list and emits it as a single item:

```
channel.of( 1, 2, 3, 4 )
  .collect()
  .view()
```

```
[1, 2, 3, 4]
```

An optional `closure` can be used to transform each item before it is collected:

```
channel.of( 'hello', 'ciao', 'bonjour' )
  .collect { v -> v.length() }
  .view()
```

```
[5, 4, 7]
```

Available options:

When `true`, nested list structures are flattened and their items are collected individually (default: `true`).

When `true`, the collected items are sorted by their natural ordering (default: `false`). Can also be a `closure` or a `Comparator` which defines how items are compared during sorting.

Returns: channel

The `collectFile` operator collects the items from a source channel and saves them to one or more files, emitting the collected file(s).

This operator has multiple variants:

Collects the items and saves them to a single file specified by the `name` option:

```
channel.of('alpha', 'beta', 'gamma')
  .collectFile(name: 'sample.txt', newLine: true)
  .subscribe { file ->
    println "Entries are saved to file: $file"
    println "File content is: ${file.text}"
  }
```

Collects the items into groups and saves each group to a file, using a grouping criteria. The grouping criteria is a `closure` that maps each item to a pair, where the first element is the file name for the group and the second element is the content to be appended to that file. For example:

```
channel.of('Hola', 'Ciao', 'Hello', 'Bonjour', 'Halo')
  .collectFile { item ->
    [ "${item[0]}.txt", item + '\n' ]
  }
  .subscribe { file ->
    println "File '${file.name}' contains:"
    println file.text
  }
```

```
File 'B.txt' contains:
Bonjour
```

```
File 'C.txt' contains:
Ciao
```

```
File 'H.txt' contains:
Halo
Hola
Hello
```

When the items from the source channel are files, the grouping criteria can be omitted. In this case, the items will be grouped by their source filename.

The following example shows how to use a closure to collect and sort all sequences in a FASTA file from shortest to longest:

```
channel.fromPath('/data/sequences.fa')
  .splitFasta( record: [id: true, sequence: true] )
  .collectFile( name: 'result.fa', sort: { v -> v.size() } ) {
    v -> v.sequence
  }
  .view { fa -> fa.text }
```

warning

The `collectFile` operator needs to store files in a temporary directory that is automatically deleted on workflow completion. For performance reasons, this directory is located in the machine's local storage, and it should have as much free space as the data that is being collected. The `tempDir` option can be used to specify a different temporary directory.

Available options:

Controls the caching ability of the `collectFile` operator when using the resume feature. It follows the same semantic of the `cache` directive (default: `true`).

Prepend the resulting file with the header fetched in the first collected file. The header size (i.e. lines) can be specified by using the `skip` option (default: `0`), to determine how many lines to remove from all collected files except for the first (where no lines will be removed).

Name of the file where all received values are stored.

Appends a `newline` character automatically after each entry (default: `false`).

A value or a map of values used to initialize the files content.

Skip the first `n` lines e.g. `skip: 1` (default: `0`).

Defines sorting criteria of content in resulting file(s). Can be one of the following values:

- `false`: Disable content sorting. Entries are appended as they are produced.
- `true`: Order the content by the entry's natural ordering, i.e. numerical for number, lexicographic for string, etc. See the [Java documentation](#) for more information.
- `'index'`: Order the content by the incremental index number assigned to each entry while they are collected.
- `'hash'`: (default) Order the content by the hash number associated to each entry
- `'deep'`: Similar to the previous, but the hash number is created on actual entry content, e.g. when the entry is a file, the hash is created on the actual file content.
- A custom sorting criteria can be specified with a [closure](#) or a [Comparator](#) object.

The file content is sorted in such a way that it does not depend on the order in which entries were added to it, which guarantees that it is consistent (i.e. does not change) across different executions with the same data.

Folder where the resulting file(s) are stored.

Folder where temporary files, used by the collecting process, are stored.

Returns: channel

The `combine` operator produces the combinations (i.e. cross product, "Cartesian" product) of two source channels, or a channel and a list (as the right operand), emitting each combination separately.

For example:

```
numbers = channel.of(1, 2, 3)
words = channel.of('hello', 'ciao')

numbers
  .combine(words)
  .view()
```

```
[1, hello]
[2, hello]
[3, hello]
[1, ciao]
[2, ciao]
[3, ciao]
```

The `by` option can be used to combine items that share a matching key. The value should be the zero-based index of the tuple, or a list of indices. For example:

```
source = channel.of( [1, 'alpha'], [2, 'beta'] )
target = channel.of( [1, 'x'], [1, 'y'], [1, 'z'], [2, 'p'], [2, 'q'], [2, 't'] )

source.combine(target, by: 0).view()
```

```
[1, alpha, x]
[1, alpha, y]
[1, alpha, z]
[2, beta, p]
[2, beta, q]
[2, beta, t]
```

note

The `combine` operator is similar to `cross` and `join`, making them easy to confuse. Their differences can be summarized as follows:

- `combine` and `cross` both produce an outer product or cross product, whereas `join` produces an inner product.
- `combine` filters pairs with a matching key only if the `by` option is used, whereas `cross` always filters pairs with a matching key.
- `combine` with the `by` option merges and flattens each pair, whereas `cross` does not. Compare the examples for `combine` and `cross` to see this difference.

Returns: channel

The `concat` operator emits the items from two or more source channels into a single output channel. Each source channel is emitted in the order in which it was specified.

In other words, given N channels, the items from the i+1-th channel are emitted only after all of the items from the i-th channel have been emitted.

For example:

```
a = channel.of( 'a', 'b', 'c' )
b = channel.of( 1, 2, 3 )
c = channel.of( 'p', 'q' )

c.concat( b, a ).view()
```

```
p
q
1
2
3
a
b
c
```

tip

As a best practice, use `mix` instead of `concat`. The `mix` operator does not wait for each source channel to emit all values before processing the next one.

Returns: dataflow value

The `count` operator computes the total number of items in a source channel and emits it:

```
channel.of(9, 1, 7, 5)
  .count()
  .view()
```

```
4
```

An optional filter can be provided to select which items to count. The selection criteria can be a literal value, a [regular expression](#), a type qualifier (i.e. Java class), or a boolean predicate. For example:

```
channel.of(4, 1, 7, 1, 1)
  .count(1)
  .view()
```

3

```
channel.of('a', 'c', 'c', 'q', 'b')
  .count( ~/c/ )
  .view()
```

2

```
channel.of('a', 'c', 'c', 'q', 'b')
  .count { v -> v <= 'c' }
  .view()
```

4

Returns: dataflow value

Counts the total number of records in a channel of FASTA files, equivalent to `splitFasta | count`. See [splitFasta](#) for the list of available options.

Returns: dataflow value

Counts the total number of records in a channel of FASTQ files, equivalent to `splitFastq | count`. See [splitFastq](#) for the list of available options.

Returns: dataflow value

Counts the total number of records in a channel of JSON files, equivalent to `splitJson | count`. See [splitJson](#) for the list of available options.

Returns: dataflow value

Counts the total number of lines in a channel of text files, equivalent to `splitText | count`. See [splitLines](#) for the list of available options.

Returns: channel

The `cross` operator emits every pairwise combination of two channels for which the pair has a matching key.

By default, the key is defined as the first entry in a list or map, or the value itself for any other data type. For example:

```
source = channel.of( [1, 'alpha'], [2, 'beta'] )
target = channel.of( [1, 'x'], [1, 'y'], [1, 'z'], [2, 'p'], [2, 'q'], [2, 't'] )

source.cross(target).view()
```



```
[[1, alpha], [1, x]]
[[1, alpha], [1, y]]
[[1, alpha], [1, z]]
[[2, beta], [2, p]]
[[2, beta], [2, q]]
[[2, beta], [2, t]]
```

An optional [closure](#) can be used to define the matching key for each item:

```
source = channel.of( [1, 'alpha'], [2, 'beta'] )
target = channel.of( [1, 'a'], [1, 'b'], [2, 'a'], [2, 'b'] )

source .cross(target) { v -> v[1][0] } .view()
```

```
[[1, alpha], [1, a]]
[[1, alpha], [2, a]]
[[2, beta], [1, b]]
[[2, beta], [2, b]]
```

There are two important caveats when using the `cross` operator:

1. The operator is not commutative, i.e. `a.cross(b)` is not the same as `b.cross(a)`
2. Each source channel should not emit any items with duplicate keys, i.e. each item should have a unique key.

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: channel

The `distinct` operator forwards a source channel with consecutively repeated items removed, such that each emitted item is different from the preceding one:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 2, 3 )
    .distinct()
    .view()
```

```
1
2
3
1
2
3
```

An optional [closure](#) can be used to transform each value before it is evaluated for distinct-ness:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 4, 6 )
    .distinct { v -> v % 2 }
    .view()
```

```
1
2
3
2
```

Returns: channel

The `dump` operator prints each item in a source channel when the pipeline is executed with the `-dump-channels` command-line option, otherwise it does nothing. It is a useful way to inspect and debug channels quickly without having to modify the pipeline script.

The `tag` option can be used to select which channels to dump:

```
channel.of( 1, 2, 3 )
  .map { v -> v + 1 }
  .dump(tag: 'plus1')

channel.of( 1, 2, 3 )
  .map { v -> v ** 2 }
  .dump(tag: 'exp2')
```

Then, you can run your pipeline with `-dump-channels plus1` or `-dump-channels exp2` to dump the content of either channel. Multiple tag names can be specified as a comma-separated list.

Available options:

When `true`, format the output as pretty-printed JSON (default: `false`).

Associate the channel with a tag that can be specified with the `-dump-channels` option to select which channels to dump.

Returns: channel

The `filter` operator emits the items from a source channel that satisfy a condition, discarding all other items. The filter condition can be a literal value, a [regular expression](#), a type qualifier (i.e. Java class), or a boolean predicate.

The following example filters a channel with a regular expression that only matches strings beginning with `a`:

```
channel.of( 'a', 'b', 'aa', 'bc', 3, 4.5 )
  .filter( ~/^a./ )
  .view()
```

```
a
aa
```

The following example filters a channel with the `Number` type qualifier so that only numbers are emitted:

```
channel.of( 'a', 'b', 'aa', 'bc', 3, 4.5 )
  .filter( Number )
  .view()
```

```
3
4.5
```

The following example filters a channel using a boolean predicate, which is a [closure](#) that returns a boolean value. In this case, the predicate is used to select only odd numbers:

```
channel.of( 1, 2, 3, 4, 5 )
  .filter { v -> v % 2 == 1 }
  .view()
```

```
1
3
5
```

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: dataflow value

The `first` operator emits the first item in a source channel, or the first item that matches a condition. The condition can be a [regular expression](#), a type qualifier (i.e. Java class), or a boolean predicate. For example:

```
// no condition is specified, emits the very first item: 1
channel.of( 1, 2, 3 )
  .first()
  .view()

// emits the first item matching the regular expression: 'aa'
channel.of( 'a', 'aa', 'aaa' )
  .first( ~/aa.*/ )
  .view()

// emits the first String value: 'a'
channel.of( 1, 2, 'a', 'b', 3 )
  .first( String )
  .view()

// emits the first item for which the predicate evaluates to true: 4
channel.of( 1, 2, 3, 4, 5 )
  .first { v -> v > 3 }
  .view()
```

Returns: channel

The `flatMap` operator applies a mapping function to each item from a source channel.

When the mapping function returns a list, each element in the list is emitted separately:

```
channel.of( 1, 2, 3 )
  .flatMap { n -> [ n, n*2, n*3 ] }
  .view()
```

```
1
2
3
2
4
6
3
6
9
```

When the mapping function returns a map, each key-value pair in the map is emitted separately:

```
channel.of( 1, 2, 3 )
  .flatMap { n -> [ number: n, square: n*n, cube: n*n*n ] }
  .view { entry -> "${entry.key}: ${entry.value}" }
```

```
number: 1
square: 1
cube: 1
number: 2
square: 4
cube: 8
number: 3
square: 9
cube: 27
```

Returns: channel

The `flatten` operator flattens each item from a source channel that is a list or other collection, such that each element in each collection is emitted separately:

```
channel.of( [1, [2, 3]], 4, [5, [6]] )
  .flatten()
  .view()
```

```
1
2
3
4
5
6
```

As shown in the above example, deeply nested collections are also flattened.

tip

As a best practice, use `flatMap` instead of `flatten`. The `flatMap` operator only flattens one level and has a well-defined return type.

Returns: queue channel

The `groupTuple` operator collects lists (i.e. tuples) from a source channel into groups based on a grouping key. A new tuple is emitted for each distinct key.

To be more precise, the operator transforms a sequence of tuples like (K, V1, V2, ..) into a sequence of tuples like (K, list(V), list(W), ..).

For example:

```
channel.of( [1, 'A'], [1, 'B'], [2, 'C'], [3, 'B'], [1, 'C'], [2, 'A'], [3, 'D'] )
  .groupTuple()
  .view()
```

```
[1, [A, B, C]]
[2, [C, A]]
[3, [B, D]]
```

By default, the first element of each tuple is used as the grouping key. The `by` option can be used to specify a different index, or list of indices. For example, to group by the second element of each tuple:

```
channel.of( [1, 'A'], [1, 'B'], [2, 'C'], [3, 'B'], [1, 'C'], [2, 'A'], [3, 'D'] )
  .groupTuple(by: 1)
  .view()
```

```
[[1, 2], A]
[[1, 3], B]
[[2, 1], C]
[[3], D]
```

By default, if you don't specify a size, the `groupTuple` operator will not emit any groups until all inputs have been received. If possible, you should always try to specify the number of expected elements in each group using the `size` option, so that each group can be emitted as soon as it's ready. In cases where the size of each group varies based on the grouping key, you can use the built-in `groupKey()` function, which allows you to define a different expected size for each group:

```
channel.of(
  ['chr1', ['/path/to/region1_chr1.vcf', '/path/to/region2_chr1.vcf']],
  ['chr2', ['/path/to/region1_chr2.vcf', '/path/to/region2_chr2.vcf', '/path/to/region3_chr2.vcf']],
)
.flatMap { chr, vcfs ->
  vcfs.collect { vcf ->
    tuple(groupKey(chr, vcfs.size()), vcf) // preserve group size with key
  }
}
.view { v -> "scattered: ${v}" }
.groupTuple()
.map { key, vcfs -> tuple(key.getGroupTarget(), vcfs) } // unwrap group key
.view { v -> "gathered: ${v}" }
```

```
scattered: [chr1, /path/to/region1_chr1.vcf]
scattered: [chr1, /path/to/region2_chr1.vcf]
scattered: [chr2, /path/to/region1_chr2.vcf]
scattered: [chr2, /path/to/region2_chr2.vcf]
scattered: [chr2, /path/to/region3_chr2.vcf]
gathered: [chr1, [/path/to/region1_chr1.vcf, /path/to/region2_chr1.vcf]]
gathered: [chr2, [/path/to/region1_chr2.vcf, /path/to/region2_chr2.vcf, /path/to/region3_chr2.vcf]]
```

Available options:

The zero-based index of the element to use as the grouping key. Can also be a list of indices, e.g. `by: [0,2]` (default: `[0]`).

When `true`, incomplete tuples (i.e. groups with less than `size` items) are emitted as partial groups, otherwise they are discarded (default: `false`). This option can only be used with `size`.

The required number of items for each group. When a group reaches the required size, it is emitted.

Defines the sorting criteria for the grouped items. Can be one of the following values:

warning

The `sort` option is discouraged because it can lead to inconsistent sorting when there are multiple groups. Perform sorting separately (e.g., in a subsequent `map` operation) to ensure correct results.

Returns: channel

The `ifEmpty` operator emits a source channel, or a default value if the source channel is empty (doesn't emit any value):

```
channel.of(1, 2, 3).ifEmpty('Hello').view()
```

```
1
2
3
```

```
channel.empty().ifEmpty('Hello').view()
```

```
Hello
```

The default value can also be a [closure](#), in which case the closure is evaluated and the result is emitted when the source channel is empty.

See also: [empty](#) channel factory

Returns: channel

The `join` operator emits the inner product of two source channels using a matching key.

To be more precise, the operator transforms a sequence of tuples like (K, V1, V2, ...) and (K, W1, W2, ...) into a sequence of tuples like (K, V1, V2, ..., W1, W2, ...).

For example:

```
left  = channel.of( ['X', 1], ['Y', 2], ['Z', 3], ['P', 7] )
right = channel.of( ['Z', 6], ['Y', 5], ['X', 4] )

left.join(right).view()
```

```
[Z, 3, 6]
[Y, 2, 5]
[X, 1, 4]
```

By default, the first element of each item is used as the key. The `by` option can be used to specify a different index, or list of indices.

By default, unmatched items are discarded. The `remainder` option can be used to emit them at the end:

```
left  = channel.of( ['X', 1], ['Y', 2], ['Z', 3], ['P', 7] )
right = channel.of( ['Z', 6], ['Y', 5], ['X', 4] )

left.join(right, remainder: true).view()
```

```
[Y, 2, 5]
[Z, 3, 6]
[X, 1, 4]
[P, 7, null]
```

note

The `join` operator is similar to an SQL inner join, or an SQL outer join when `remainder` is `true`. The only difference is that `join` does not support duplicate keys, whereas an SQL join produces the cross-product of duplicate keys. The `combine` operator with the `by` option is equivalent to an SQL join.

Available options:

The zero-based index of each item to use as the matching key. Can also be a list of indices, e.g. `by: [0, 2]` (default: `[0]`).

When `true`, an error is reported when the operator receives multiple items from the same channel with the same key (default: `false`). Value is set to `true` if `strict mode` is enabled.

When `true`, an error is reported when the operator receives an item from one channel for which there no matching item from the other channel (default: `false`). Value is set to `true` if `strict mode` is enabled. This option cannot be used with `remainder`.

When `true`, unmatched items are emitted at the end, otherwise they are discarded (default: `false`).

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: dataflow value

The `last` operator emits the last item from a source channel:

```
channel.of( 1, 2, 3, 4, 5, 6 )
  .last()
  .view()
```

```
6
```

Returns: channel

The `map` operator applies a mapping function to each item from a source channel:

```
channel.of( 1, 2, 3, 4, 5 )
  .map { v -> v * v }
  .view()
```

```
1
4
9
16
25
```

note

`null` values are not emitted by `map`.

Returns: dataflow value

The `max` operator emits the item with the greatest value from a source channel:

```
channel.of( 8, 6, 2, 5 )
    .max()
    .view { v -> "Max value is $v" }
```

```
Max value is 8
```

An optional [closure](#) can be used to control how the items are compared. The closure can be a mapping function, which transforms each item before it is compared, or a comparator function, which defines how to compare two items more generally.

The following examples show how to find the longest string in a channel:

```
// mapping function
channel.of( "hello", "hi", "hey" )
    .max { v -> v.length() }
    .view()
```

```
hello
```

```
// comparator function
channel.of( "hello", "hi", "hey" )
    .max { a, b -> a.length() <=> b.length() }
    .view()
```

```
hello
```

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Use [combine](#) or [join](#) instead to combine multiple channels in a deterministic way, such as a matching key.

Returns: channel

The `merge` operator joins the items from two or more channels into a new channel:


```
odds  = channel.of(1, 3, 5, 7, 9)
evens = channel.of(2, 4, 6)

odds
  .merge( evens )
  .view()
```

```
[1, 2]
[3, 4]
[5, 6]
```

An optional [closure](#) can be used to control how two items are merged:

```
odds  = channel.of(1, 3, 5, 7, 9)
evens = channel.of(2, 4, 6)

odds
  .merge( evens ) { a, b -> tuple(b*b, a) }
  .view()
```

```
[16, 3]
[36, 5]
[4, 1]
```

The `merge` operator may return a channel or value depending on the inputs:

- If the first argument is a channel, the `merge` operator returns a channel merging as many values as are available for all inputs. Dataflow values are re-used for each merged value.
- If the first argument is a dataflow value, the `merge` operator returns a dataflow value, merging the first value from each input, regardless of whether there are channel inputs with additional values.

Returns: dataflow value

The `min` operator emits the item with the lowest value from a source channel:

```
channel.of( 8, 6, 2, 5 )
  .min()
  .view { v -> "Min value is $v" }
```

```
Min value is 2
```

An optional [closure](#) can be used to control how the items are compared. The closure can be a mapping function, which transforms each item before it is compared, or a comparator function, which defines how to compare two items more generally.

The following examples show how to find the shortest string in a channel:

```
// mapping function
channel.of( "hello", "hi", "hey" )
  .min { v -> v.length() }
  .view()
```

```
hi
```

```
// comparator function
channel.of( "hello", "hi", "hey" )
  .min { a, b -> a.length() <=> b.length() }
  .view()
```

```
hi
```

Returns: channel

The `mix` operator emits the items from two or more source channels into a single output channel:

```
c1 = channel.of( '1', '2', '3' )
c2 = channel.of( 'a', 'b' )
v3 = channel.value( 'z' )

c1.mix(c2).mix(v3).view()
```

```
1
2
3
a
b
z
```

The items in the mixed output channel may appear in any order, regardless of which source channel they came from. Thus, the previous example could also output the following:

```
z
1
a
2
b
3
```

Returns: multiple channels

The `multiMap` operator applies a set of mapping functions to a source channel, producing a separate output channel for each mapping function.

The multi-map criteria is a [closure](#) that defines, for each output channel, a label followed by a mapping expression.

For example:

```
channel.of( 1, 2, 3, 4 )
  .multiMap { v ->
    inc: v + 1
    squared: v * v
  }
  .set { result }

result.inc.view { v -> "inc $v" }
result.squared.view { v -> "squared $v" }
```

```
inc 2
inc 3
inc 4
inc 5
squared 1
squared 4
squared 9
squared 16
```

Multiple labels can share the same mapping expression using the following shorthand:

```
channel.of( 1, 2, 3 )
  .multiMap { v -> alpha: beta: v }
  .set { result }

result.alpha.view { v -> "alpha $v" }
result.beta.view { v -> "beta $v" }
```

```
alpha 1
alpha 2
alpha 3
beta 1
beta 2
beta 3
```

The above example creates two channels as before, but now they both receive the same items.

You can use the `multiMapCriteria()` method to create a multi-map criteria as a variable that can be passed as an argument to any number of `multiMap` operations, as shown below:

```
def criteria = multiMapCriteria { v ->
  small: [v, v < 10]
  large: [v, v > 10]
}

channel.of(1, 2, 30).multiMap(criteria).set { ch1 }
channel.of(10, 20, 1).multiMap(criteria).set { ch2 }

ch1.small.view { v, is_small -> "ch1: $v is small: $is_small" }
ch1.large.view { v, is_large -> "ch1: $v is large: $is_large" }
ch2.small.view { v, is_small -> "ch2: $v is small: $is_small" }
ch2.large.view { v, is_large -> "ch2: $v is large: $is_large" }
```

note

If you use `multiMap` to split a tuple or map into multiple channels, it is recommended that you retain a matching key (e.g. sample ID) with each new channel, so that you can re-combine these channels later on if needed. In general, you should not expect to be able to merge channels correctly without a matching key, due to the concurrent nature of Nextflow pipelines.

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: channel

The `randomSample` operator emits a randomly-selected subset of items from a source channel:

```
channel.of( 1..100 )
  .randomSample( 10 )
  .view()
```

The above snippet will print 10 randomly-selected numbers between 1 and 100 (without replacement).

An optional second parameter can be used to set the initial seed for the random number generator, which ensures that the `randomSample` operator produces the same pseudo-random sequence across runs:

```
channel.of( 1..100 )
  .randomSample( 10, 234 )
  .view()
```

The above example will print 10 randomly-selected numbers between 1 and 100 (without replacement). Each subsequent script execution will produce the same sequence.

Returns: dataflow value

The `reduce` operator applies an accumulator function sequentially to each item in a source channel, and emits the final accumulated value. The accumulator function takes two parameters -- the accumulated value and the i-th emitted item -- and it should return the accumulated result, which is passed to the next invocation with the i+1-th item. This process is repeated for each item in the source channel.

For example:

```
channel.of( 1, 2, 3, 4, 5 )
  .reduce { a, b ->
    println "a: $a b: $b"
    a + b
  }
  .view { result -> "result = $result" }
```

```
a: 1 b: 2
a: 3 b: 3
a: 6 b: 4
a: 10 b: 5
result = 15
```

By default, the first item is used as the initial accumulated value. You can optionally specify a different initial value as shown below:

```
channel.of( 1, 2, 3, 4, 5 )
  .reduce( 'result:' ) { acc, v ->
    println acc
    acc + ' ' + v
  }
  .view { result -> "final $result" }
```

```
result:
result: 1
result: 1 2
result: 1 2 3
result: 1 2 3 4
final result: 1 2 3 4 5
```

Returns: nothing

The `set` operator assigns a source channel to a variable, whose name is specified as a closure parameter:

```
channel.of(10, 20, 30).set { my_channel }
```

Using `set` is semantically equivalent to assigning a variable:

```
my_channel = channel.of(10, 20, 30)
```

tip

As a best practice, use a standard assignment (`=`) instead of `set`. Standard assignments enable more effective [type checking](#).

Returns: channel

The `splitCsv` operator parses and splits [CSV](#) files or text from a source channel into records.

For example:

```
channel.of( '10,20,30\n70,80,90' )  
  .splitCsv()  
  .view { row -> "${row[0]} - ${row[1]} - ${row[2]}" }
```

```
10 - 20 - 30  
70 - 80 - 90
```

The above example shows hows CSV text is parsed and split into individual rows, where each row is simply a list of columns.

When the CSV begins with a header line defining the column names, and the `header` option is `true`, each row is returned as a map instead:

```
channel.of( 'alpha,beta,gamma\n10,20,30\n70,80,90' )  
  .splitCsv( header: true )  
  .view { row -> "${row.alpha} - ${row.beta} - ${row.gamma}" }
```

```
10 - 20 - 30  
70 - 80 - 90
```

The `header` option can also just be a list of columns:

```
channel.of( 'alpha,beta,gamma\n10,20,30\n70,80,90' )  
  .splitCsv( header: ['col1', 'col2', 'col3'], skip: 1 )  
  .view { row -> "${row.col1} - ${row.col2} - ${row.col3}" }
```

```
10 - 20 - 30  
70 - 80 - 90
```

Available options:

When specified, group rows into chunks with the given size (default: none).

Parse the content with the specified charset, e.g. UTF-8. See the list of [standard charsets](#) for available options.

When `true`, decompress the content using the GZIP format before processing it (default: `false`). Files with the `.gz` extension are decompressed automatically.

When `true`, the first line is used as the columns names (default: `false`). Can also be a list of columns names.

Limits the number of records to retrieve for each source item (default: no limit).

The character used to quote values (default: `'` or `"`).

The character used to separate values (default: `,`).

Number of lines to ignore from the beginning when parsing the CSV text (default: `0`).

When `true`, remove leading and trailing blanks from values (default: `false`).

Returns: channel

The `splitFasta` operator splits [FASTA](#) files or text from a source channel into individual sequences.

The `by` option can be used to group sequences into chunks of a given size. The following example shows how to read a FASTA file and split it into chunks of 10 sequences each:

```
channel.fromPath('misc/sample.fa')
  .splitFasta( by: 10 )
  .view()
```

warning

Chunks are stored in memory by default. When splitting large files, specify `file: true` to save the chunks into files in order to avoid running out of memory. See the list of options below for details.

The `record` option can be used to split FASTA content into records instead of text chunks. Each record is a map that allows you to access the FASTA sequence data with ease. For example:

```
channel.fromPath('misc/sample.fa')
  .splitFasta( record: [id: true, seqString: true] )
  .filter { record -> record.id =~ /^ENST0.* / }
  .view { record -> record.seqString }
```

The above example loads the `misc/sample.fa` file, splits it into records containing the `id` and `seqString` fields (i.e. the sequence id and the sequence data), filters records by their ID, and finally prints the sequence string of each record.

Available options:

Defines the number of sequences in each chunk (default: `1`).

Parse the content with the specified charset, e.g. UTF-8. See the list of [standard charsets](#) for available options.

When `true`, resulting file chunks are GZIP compressed (default: `false`). The `.gz` suffix is automatically added to chunk file names.

When `true`, decompress the content using the GZIP format before processing it (default: `false`). Files with the `.gz` extension are decompressed automatically.

The index of the element to split when the source items are lists or tuples (default: first file object or first element).

When `true`, saves each split to a file. Use a string instead of `true` value to create split files with a specific name (split index number is automatically added). Finally, set this attribute to an existing directory, in order to save the split files into the specified directory.

Limits the number of sequences to retrieve for each source item (default: no limit).

Parse each entry in the FASTA file into a record. The following fields are available:

- `id`: The FASTA sequence identifier, i.e. the word following the `>` symbol up to the first blank or newline character
- `header`: The first line in a FASTA sequence without the `>` character
- `desc`: The text in the FASTA header following the ID value
- `text`: The complete FASTA sequence including the header
- `seqString`: The sequence as a single-line string, i.e. containing no newline characters
- `sequence`: The sequence as a multi-line string, i.e. always ending with a newline character
- `width`: Define the length of a single line when the `sequence` field is used, after which the sequence continues on a new line.

Defines the size of the expected chunks as a memory unit, e.g. `1.MB`.

See also: [countFasta](#)

Returns: channel

The `splitFastq` operator splits [FASTQ](#) files or text from a source channel into individual sequences.

The `by` option can be used to group sequences into chunks of a given size. The following example shows how to read a FASTQ file and split it into chunks of 10 sequences each:

```
Channel
channel.fromPath('misc/sample.fastq')
  .splitFastq( by: 10 )
  .view()
```

warning

Chunks are stored in memory by default. When splitting large files, specify `file: true` to save the chunks into files in order to avoid running out of memory. See the list of options below for details.

The `record` option can be used to split FASTQ content into records instead of text chunks. Each record is a map that allows you to access the FASTQ sequence data with ease. For example:

```
channel.fromPath('misc/sample.fastq')
  .splitFastq( record: true )
  .view { record -> record.readHeader }
```

The `pe` option can be used to split paired-end FASTQ files. The source channel must emit tuples containing the file pairs. For example:

```
channel.fromFilePairs('/my/data/SRR*_{1,2}.fastq', flat: true)
  .splitFastq(by: 100_000, pe: true, file: true)
  .view()
```

note

`channel.fromFilePairs()` requires the `flat: true` option in order to emit the file pairs as separate elements in the produced tuples.

note

This operator assumes that the order of the paired-end reads correspond with each other and that both files contain the same number of reads.

Available options:

Defines the number of sequences in each chunk (default: `1`).

Parse the content with the specified charset, e.g. UTF-8. See the list of [standard charsets](#) for available options.

When `true`, resulting file chunks are GZIP compressed (default: `false`). The `.gz` suffix is automatically added to chunk file names.

When `true`, decompress the content using the GZIP format before processing it (default: `false`). Files with the `.gz` extension are decompressed automatically.

The index of the element to split when the source items are lists or tuples (default: first file object or first element).

When `true`, saves each split to a file. Use a string instead of `true` value to create split files with a specific name (split index number is automatically added). Finally, set this attribute to an existing directory, in order to save the split files into the specified directory.

Limits the number of sequences to retrieve for each source item (default: no limit).

When `true`, splits paired-end read files. Items emitted by the source channel must be tuples with the file pairs.

Parse each entry in the FASTQ file into a record. The following fields are available:

- `readHeader`: Sequence header (without the `@` prefix)
- `readString`: The raw sequence content
- `qualityHeader`: Base quality header (it may be empty)
- `qualityString`: Quality values for the sequence

See also: [countFastq](#)

Returns: channel

The `splitJson` operator splits [JSON](#) files or text from a source channel into individual records.

If the source item is a JSON array, each element of the array will be emitted:

```
// Example with a JSON array
channel.of('[1, null, ["A", {}], true]')
  .splitJson()
  .view { v -> "Item: ${v}" }
```

```
Item: 1
Item: null
Item: [A, [{}]]
Item: true
```

If the source item is a JSON object, each key-value pair will be emitted as a map with the properties `key` and `value`:

```
// Example with a JSON object
channel.of('{ "A": 1, "B": [1, 2, 3], "C": { "D": null } }')
  .splitJson()
  .view { v -> "Item: ${v}" }
```

```
Item: [key:A, value:1]
Item: [key:B, value:[1, 2, 3]]
Item: [key:C, value:[D:null]]
```

The `path` option can be used to query a section of the JSON document to parse and split:

```
channel.of('{ "A": 1, "B": [2, 3, { "C": { "D": null, "E": 4, "F": 5 } } ] }')
  .splitJson(path: 'B[2].C')
  .view { v -> "Item: ${v}" }
```

```
Item: [key:D, value:null]
Item: [key:E, value:4]
Item: [key:F, value:5]
```

Available options:

Limits the number of records to retrieve for each source item (default: no limit).

Defines a query for a section of each source item to parse and split. The expression should be a path similar to [JSONPath](#). The empty string is the document root (default). An integer in brackets is a zero-based index in a JSON array. A string preceded by a dot `.` is a key in a JSON object.

See also: [countJson](#)

Returns: channel

The `splitText` operator splits multi-line text content from a source channel into chunks of N lines:

```
Channel
  .fromPath('/some/path/*.txt')
  .splitText()
  .view()
```

The above example loads a collection of text files, splits the content of each file into individual lines, and prints each line.

The `by` option can be used to emit chunks of N lines:

```
channel.fromPath('/some/path/*.txt')
  .splitText( by: 10 )
  .subscribe { chunk ->
    print chunk
    print "--- end of the chunk ---\n"
  }
```

An optional [closure](#) can be used to transform each text chunk produced by the operator. The following example shows how to split text files into chunks of 10 lines and transform them to uppercase letters:

```
channel.fromPath('/some/path/*.txt')
  .splitText( by: 10 ) { v -> v.toUpperCase() }
  .view()
```

note

Text chunks returned by the `splitText` operator are always terminated by a `\n` newline character.

Available options:

Defines the number of lines in each `chunk` (default: `1`).

Parse the content with the specified charset, e.g. `UTF-8`. See the list of [standard charsets](#) for available options.

When `true`, resulting file chunks are GZIP compressed (default: `false`). The `.gz` suffix is automatically added to chunk file names.

When `true`, decompresses the content using the GZIP format before processing it (default: `false`). Files with the `.gz` extension are decompressed automatically.

The index of the element to split when the source items are lists or tuples (default: first file object or first element).

When `true`, saves each split to a file. Use a string instead of `true` value to create split files with a specific name (split index number is automatically added). Finally, set this attribute to an existing directory, in order to save the split files into the specified directory.

Parses the first line as header and prepends it to each emitted chunk (default: `false`).

Limits the number of lines to retrieve for each source item (default: no limit).

See also: [countLines](#)

Returns: nothing

The `subscribe` operator invokes a custom function for each item from a source channel:

```
source = channel.of( 'alpha', 'beta', 'delta' )

source.subscribe { str ->
  println "Got: ${str}; len: ${str.length()}"
}
```

```
Got: alpha; len: 5
Got: beta; len: 4
Got: delta; len: 5
```

The `subscribe` operator supports multiple types of event handlers:

```
channel.of( 1, 2, 3 ).subscribe(
  onNext: { v -> println v },
  onComplete: { println 'Done' }
)
```

```
1
2
3
Done
```

note

Unlike most operators, `subscribe` is a terminal operator and does not emit any values. It should only be used for side effects, such as printing to the console, writing to a file, or making HTTP requests.

Available options:

Closure that is invoked when an item is emitted. Equivalent to providing a closure as the first argument.

Closure that is invoked after the last item is emitted by the channel.

Closure that is invoked when an exception is raised while handling the `onNext` event. It will not make further calls to `onNext` or `onComplete`. The `onError` method takes as its parameter the `Throwable` that caused the error.

Returns: dataflow value

The `sum` operator emits the sum of all items in a source channel:

```
channel.of( 8, 6, 2, 5 )
  .sum()
  .view { result -> "The sum is $result" }
```

```
The sum is 21
```

An optional `closure` can be used to transform each item before it is added to the sum:

```
channel.of( 4, 1, 7, 5 )
  .sum { v -> v * v }
  .view { result -> "Square: $result" }
```

```
Square: 91
```

warning

This operator depends on the ordering of values in the source channel. It can lead to [non-deterministic behavior](#) if used improperly.

Returns: channel

The `take` operator takes the first N items from a source channel:

```
channel.of( 1..10 )  
  .take( 3 )  
  .view()
```

```
1  
2  
3
```

tip

Specifying a size of `-1` causes the operator to take all values.

Returns: channel

The `tap` operator assigns a source channel to a variable, and emits the source channel. It is a useful way to extract intermediate output channels from a chain of operators. For example:

```
channel.of( 'a', 'b', 'c' )  
  .tap { log1 }  
  .map { v -> v * 2 }  
  .tap { log2 }  
  .map { v -> v.toUpperCase() }  
  .view { result -> "Result: $result" }  
  
log1.view { v -> "Log 1: $v" }  
log2.view { v -> "Log 2: $v" }
```

```
Result: AA  
Result: BB  
Result: CC  
Log 1: a  
Log 1: b  
Log 1: c  
Log 2: aa  
Log 2: bb  
Log 2: cc
```

tip

As a best practice, use a standard assignment (`=`) instead of `tap`. Standard assignments enable more effective [type checking](#).

Returns: dataflow value

The `toList` operator collects all the items from a source channel into a list and emits the list as a single item:

```
channel.of( 1, 2, 3, 4 )
    .toList()
    .view()
```

```
[1, 2, 3, 4]
```

note

There are two main differences between `toList` and `collect`:

- When there is no input, `toList` emits an empty list whereas `collect` emits nothing.
- By default, `collect` flattens list items by one level.

In other words, `toList` is equivalent to:

```
collect(flat: false).ifEmpty([])
```

Returns: dataflow value

The `toSortedList` operator collects all the items from a source channel into a sorted list and emits the list as a single item:

```
channel.of( 3, 2, 1, 4 )
    .toSortedList()
    .view()
```

```
[1, 2, 3, 4]
```

An optional `closure` can be used to control how items are compared when sorting. For example, to sort tuples by their second element in descending order:

```
channel.of( ['homer', 5], ['bart', 2], ['lisa', 10], ['marge', 3], ['maggie', 7] )
    .toSortedList { a, b -> b[1] <=> a[1] }
    .view()
```

```
[[lisa, 10], [maggie, 7], [homer, 5], [marge, 3], [bart, 2]]
```

note

`toSortedList` is equivalent to:

```
collect(flat: false, sort: true).ifEmpty([])
```

Returns: channel

The `transpose` operator "transposes" each tuple from a source channel by flattening any nested list in each tuple, emitting each nested item separately.

To be more precise, the operator transforms a sequence of tuples like (K, list(V1), list(V2), ..) into a sequence of tuples like (K, V1, V2, ..).

For example:

```
channel.of(
    [1, ['A', 'B', 'C']],
    [2, ['C', 'A']],
    [3, ['B', 'D']]
)
.transpose()
.view()
```

```
[1, A]
[1, B]
[1, C]
[2, C]
[2, A]
[3, B]
[3, D]
```

If each source item has more than two elements, these will be flattened by the first element in the item, and a new item will be emitted only when it is complete:

```
channel.of(
    [1, [1], ['A']],
    [2, [1, 2], ['B', 'C']],
    [3, [1, 2, 3], ['D', 'E']]
)
.transpose()
.view()
```

```
[1, 1, A]
[2, 1, B]
[2, 2, C]
[3, 1, D]
[3, 2, E]
```

The `remainder` option can be used to emit any incomplete items:

```
channel.of(
    [1, [1], ['A']],
    [2, [1, 2], ['B', 'C']],
    [3, [1, 2, 3], ['D', 'E']]
)
.transpose(remainder: true)
.view()
```

```
[1, 1, A]
[2, 1, B]
[2, 2, C]
[3, 1, D]
[3, 2, E]
[3, 3, null]
```

Available options:

The zero-based index of the element to be transposed. Can also be a list of indices, e.g. `by: [0,2]`. By default, every list element is transposed.

When `true`, incomplete tuples are emitted with `null` values for missing elements, otherwise they are discarded (default: `false`).

tip

As a best practice, use `flatMap` instead of `transpose`, since `flatMap` has a well-defined return type.

Returns: channel

The `unique` operator emits the unique items from a source channel:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 2, 3 )
  .unique()
  .view()
```

```
1
2
3
```

An optional `closure` can be used to transform each item before it is evaluated for uniqueness:

```
channel.of( 1, 1, 2, 2, 2, 3, 1, 1, 2, 4, 6 )
  .unique { v -> v % 2 }
  .view()
```

```
1
2
```

note

The difference between `unique` and `distinct` is that `unique` removes all duplicate values, whereas `distinct` removes only consecutive duplicate values.

Returns: channel

The `until` operator emits each item from a source channel until a stopping condition is satisfied:

```
channel.of( 3, 2, 1, 5, 1, 5 )
  .until { v -> v == 5 }
  .view()
```

```
3
2
1
```

Returns: channel

The `view` operator prints each item from a source channel to standard output:

```
channel.of(1, 2, 3).view()
```

```
1
2
3
```

An optional [closure](#) can be used to transform each item before it is printed:

```
channel.of(1, 2, 3)
  .map { v -> [v, v*v] }
  .view { num, sqr -> "The square of $num is $sqr" }
```

```
The square of 1 is 1
The square of 2 is 4
The square of 3 is 9
```

The `view` operator also emits every item that it receives, allowing it to be chained with other operators.

Available options:

Print each item to a separate line (default: `true`).

Added in version 25.04

Associate the channel with a tag. For example:

```
channel.of( 1, 2, 3 )
  .map { v -> v + 1 }
  .view(tag: 'plus1')

channel.of( 1, 2, 3 )
  .map { v -> v ** 2 }
  .view(tag: 'exp2')
```


Configuration options

This page lists all of the available settings in the [Nextflow configuration](#).

The following settings are available:

The remote work directory used by hybrid workflows. Equivalent to the `-bucket-dir` option of the `run` command.

Delete all files associated with a run in the work directory when the run completes successfully (default `false`).

warning

This option will prevent the use of the resume feature on subsequent executions of that pipeline run.

warning

This option is not supported for remote work directories, such as Amazon S3, Google Cloud Storage, and Azure Blob Storage.

Added in version 24.10

The pipeline output directory. Equivalent to the `-output-dir` option of the `run` command.

Enable the use of previously cached task executions. Equivalent to the `-resume` option of the `run` command.

The pipeline work directory. Equivalent to the `-work-dir` option of the `run` command.

The `appleContainer` scope controls how [Apple container](#) is used by Nextflow.

The following settings are available:

Execute tasks with Apple container (default: `false`).

Specify additional options supported by the `container` CLI (i.e., `container [OPTIONS] run`).

Comma separated list of environment variable names to be included in the container environment.

Set the signal used to stop a running container (default: `true`, which sends `SIGTERM` via `container stop`). Set to a signal name (e.g. `'SIGKILL'`) to use `container kill -s`, or `false` to disable.

The registry from where container images are pulled. It should NOT include the protocol prefix (i.e., `http://`).

Clean up the container after the execution (default: `true`).

Specify extra command line options supported by the `container run` command (e.g., `--rosetta`, `--ssh`).

Mounts a path of your choice as the `/tmp` directory in the container.

Allocate a pseudo-tty (default: `false`).

When `false`, mount input directories as read-only (default: `true`).

The `apptainer` scope controls how [Apptainer](#) containers are executed by Nextflow.

The following settings are available:

Automatically mount host paths in the executed container (default `true`). It requires the `user bind control` feature to be enabled in your Apptainer installation.

The directory where remote Apptainer images are stored. When using a computing cluster it must be a shared folder accessible to all compute nodes.

Execute tasks with Apptainer containers (default `false`).

Specify additional options supported by the Apptainer engine i.e. `apptainer [OPTIONS]`.

Comma separated list of environment variable names to be included in the container environment.

Directory where remote Apptainer images are retrieved. When using a computing cluster it must be a shared folder accessible to all compute nodes.

Pull the Apptainer image with http protocol (default `false`).

Added in version 24.04

When enabled, OCI (and Docker) container images are pulled and converted to the SIF format by the Apptainer run command, instead of Nextflow (default `false`).

note

Leave `ociAutoPull` disabled if you are willing to build a Singularity/Apptainer native image with Wave (see the [Build Singularity native images](#) section).

The amount of time the Apptainer pull can last, exceeding which the process is terminated (default `20 min`).

The registry from where Docker images are pulled. It should be only used to specify a private registry server. It should NOT include the protocol prefix i.e. `http://`.

Specify extra command line options supported by `apptainer exec`.

The `aws` scope controls the interactions with AWS, including AWS Batch and S3.

The following settings are available:

AWS account access key.

AWS profile from `~/.aws/credentials`.

AWS region (e.g. `us-east-1`).

AWS account secret key.

The path where the AWS command line tool is installed in the host AMI.

Delay between download attempts from S3 (default 10 sec).

The AWS Batch [Execution Role](#) ARN that needs to be used to execute the Batch Job. It is mandatory when using AWS Fargate.

Added in version 26.04

When `true`, add the `--force-glacier-transfer` flag to AWS CLI S3 download commands (default `false`).

This option is needed when staging directories that have been restored from [S3 Glacier](#). It does not restore objects from Glacier.

The AWS Batch Job Role ARN that needs to be used to execute the Batch Job.

The name of the logs group used by Batch Jobs (default `/aws/batch/job`).

Max parallel upload/download transfer operations per job (default 4).

Changed in version 24.10

The default value was changed from 5 to 0.

Max number of execution attempts of a job interrupted by a EC2 Spot reclaim event (default 0)

Max number of downloads attempts from S3 (default 1).

The compute platform type used by AWS Batch. Can be either `ec2` or `fargate`. Set to `fargate` to use [AWS Fargate](#).

The [retry mode](#) used to handle rate-limiting by AWS APIs. Can be one of `standard`, `legacy`, `adaptive`, or `built-in` (default `standard`).

The scheduling priority for all tasks when using [fair-share scheduling](#) (default 0).

The share identifier for all tasks when using [fair-share scheduling](#).

Added in version 25.04

When `true`, jobs that cannot be scheduled due to lack of resources or misconfiguration are terminated and handled as task failures (default `false`).

List of container mounts. Mounts can be specified as simple e.g. `/some/path` or canonical format e.g. `/host/path:/mount/path[:ro|rw]`.

Allow the access of public S3 buckets without providing AWS credentials (default `false`). Any service that does not accept unsigned requests will return a service access error.

The amount of time to wait (in milliseconds) when initially establishing a connection before timing out (default 10000).

The AWS S3 API entry point e.g. `https://s3-us-west-1.amazonaws.com`. The endpoint must include the protocol prefix e.g. `https://`.

The maximum number of open HTTP connections used by the S3 client (default `50`).

The maximum size for the heap memory buffer used by concurrent downloads. It must be at least 10 times the `minimumPartSize` (default: `400 MB`).

The maximum number of retry attempts for failed retryable requests (default `-1`).

Added in version 25.10

The minimum part size used for multipart S3 transfers (default: `8 MB`).

Added in version 25.10

The object size threshold used for multipart S3 transfers (default: same as `aws.client.minimumPartSize`).

Deprecated in version 25.10

This option is no longer supported.

The protocol to use when connecting to AWS. Can be `http` or `https` (default `'https'`).

The proxy host to connect through.

The password to use when connecting through a proxy.

The port to use when connecting through a proxy.

Added in version 25.10

The protocol scheme to use when connecting through a proxy. Can be `http` or `https` (default `'http'`).

The user name to use when connecting through a proxy.

Added in version 25.10

Use [Requester Pays](#) for S3 buckets (default `false`).

Specify predefined bucket permissions, also known as [canned ACL](#). Can be one of `Private`, `PublicRead`, `PublicReadWrite`, `AuthenticatedRead`, `LogDeliveryWrite`, `BucketOwnerRead`, `BucketOwnerFullControl`, or `AwsExecRead`.

Use the path-based access model to access objects in S3-compatible storage systems (default `false`).

Deprecated in version 25.10

This option is no longer supported.

The name of the signature algorithm to use for signing requests made by the client.

Deprecated in version 25.10

This option is no longer supported.

The Size hint (in bytes) for the low level TCP send buffer (default 0).

Deprecated in version 25.10

This option is no longer supported.

The Size hint (in bytes) for the low level TCP receive buffer (default 0).

Changed in version 25.10

The default socket timeout changed from 50000 to 30000.

The amount of time to wait (in milliseconds) for data to be transferred over an established, open connection before the connection is timed out (default: 30000).

The S3 storage class applied to stored objects, one of [STANDARD , STANDARD_IA , ONEZONE_IA , INTELLIGENT_TIERING] (default STANDARD).

The S3 server side encryption to be used when saving objects on S3. Can be AES256 or aws:kms (default none).

The AWS KMS key Id to be used to encrypt files stored in the target S3 bucket.

Added in version 25.10

The target network throughput (in Gbps) used for S3 uploads and downloads (default 10).

Deprecated in version 25.10

This option is no longer supported.

The HTTP user agent header passed with all HTTP requests.

Deprecated in version 25.10

This option is no longer supported.

The size of a single part in a multipart upload (default 100 MB).

Deprecated in version 25.10

This option is no longer supported.

The maximum number of upload attempts after which a multipart upload returns an error (default 5).

Deprecated in version 25.10

This option is no longer supported.

The maximum number of threads used for multipart upload (default 10).

Deprecated in version 25.10

This option is no longer supported.

The time to wait after a failed upload attempt to retry the part upload (default 500ms).

The S3 storage class applied to stored objects. Can be `STANDARD` , `STANDARD_IA` , `ONEZONE_IA` , or `INTELLIGENT_TIERING` (default `STANDARD`).

The `azure` scope allows you to configure the interactions with Azure, including Azure Batch and Azure Blob Storage.

The following settings are available:

The service principal client ID. Defaults to environment variable `AZURE_CLIENT_ID` .

The service principal client secret. Defaults to environment variable `AZURE_CLIENT_SECRET` .

The Azure tenant ID. Defaults to environment variable `AZURE_TENANT_ID` .

The blob `access tier` used by `azcopy` to upload files to Azure Blob Storage. Valid options are `None` , `Hot` , or `Cool` (default `None`).

The block size (in MB) used by `azcopy` to transfer files between Azure Blob Storage and compute nodes (default 4).

The batch service account key. Defaults to environment variable `AZURE_BATCH_ACCOUNT_KEY` .

The batch service account name. Defaults to environment variable `AZURE_BATCH_ACCOUNT_NAME` .

Enable the automatic creation of batch pools specified in the Nextflow configuration file (default `false`).

Enable the automatic creation of batch pools depending on the pipeline resources demand (default `true`).

The mode in which the `azcopy` tool is installed by Nextflow (default `'node'`).

Available options:

The `azcopy` tool is installed once during the pool creation.

The `azcopy` tool is installed for each task execution.

The `azcopy` tool is not installed.

Changed in version 23.10

Default value was changed from `true` to `false`.

Delete all jobs when the workflow completes (default `false`).

Delete all compute node pools when the workflow completes (default `false`).

Added in version 23.10

Delete each task when it completes (default `true`). Although this setting is enabled by default, failed tasks will not be deleted unless it is explicitly enabled. This way, the default behavior is that successful tasks are deleted while failed tasks are preserved for debugging purposes.

The batch service endpoint e.g. `https://nfbatch1.westeurope.batch.azure.com`.

Added in version 25.04

The maximum elapsed time that jobs may run, measured from the time they are created (default `30d`). If jobs do not complete within this time limit, the Batch service terminates them and any tasks still running.

The name of the batch service region, e.g. `westeurope` or `eastus2`. Not needed when the endpoint is specified.

Added in version 25.10

The client ID for an Azure [managed identity](#) that is available on all Azure Batch node pools. This identity is used by Fusion to authenticate to Azure storage. If set to `'auto'`, Fusion will use the first available managed identity.

Enable autoscaling feature for the pool identified with `<name>`.

The internal root mount point when mounting File Shares. Must be `/mnt/resource/batch/tasks/fsmounts` for CentOS nodes or `/mnt/batch/tasks/fsmounts` for Ubuntu nodes (default `CentOS`).

Enable the use of low-priority VMs (default `false`).

warning

As of September 30, 2025, Low Priority VMs will no longer be supported in Azure Batch accounts that use Batch Managed mode for pool allocation. You may continue to use this setting to configure Spot VMs in Batch accounts configured with User Subscription mode.

The max number of virtual machines when using auto scaling.

The mount options for mounting the file shares (default `-o vers=3.0,dir_mode=0777,file_mode=0777,sec=ntlmssp`).

The offer type of the virtual machine type used by the pool identified with `<name>` (default `centos-container`).

Enable the task to run with elevated access. Ignored if `runAs` is set (default `false`).

The publisher of virtual machine type used by the pool identified with `<name>` (default `microsoft-azure-batch`).

The username under which the task is run. The user must already exist on each node of the pool.

The [scale formula](#) for the pool identified with `<name>`.

The interval at which to automatically adjust the Pool size according to the autoscale formula. Must be at least 5 minutes and at most 168 hours (default `10 mins`).

The scheduling policy for the pool identified with `<name>`. Can be either `spread` or `pack` (default `spread`).

The ID of the Compute Node agent SKU which the pool identified with `<name>` supports (default `batch.node.centos.8`).

Added in version 25.04

Enable the `startTask` to run with elevated access (default `false`).

Added in version 25.04

The `startTask` that is executed as the node joins the Azure Batch node pool.

Added in version 23.04

The subnet ID of a virtual network in which to create the pool.

The number of virtual machines provisioned by the pool identified with `<name>`.

The virtual machine type used by the pool identified with `<name>`.

Added in version 23.10

When the workflow completes, set all jobs to terminate on task completion (default `true`).

The client ID for an Azure [managed identity](#). Defaults to environment variable `AZURE_MANAGED_IDENTITY_USER`.

When `true`, use the system-assigned [managed identity](#) to authenticate Azure resources. Defaults to environment variable `AZURE_MANAGED_IDENTITY_SYSTEM`.

The password to connect to a private container registry.

The container registry from which to pull the Docker images (default `docker.io`).

The username to connect to a private container registry.

Delay when retrying failed API requests (default `250ms`).

Jitter value when retrying failed API requests (default `0.25`).

Max attempts when retrying failed API requests (default `10`).

Max delay when retrying failed API requests (default 90s).

The blob storage account key. Defaults to environment variable `AZURE_STORAGE_ACCOUNT_KEY`.

The blob storage account name. Defaults to environment variable `AZURE_STORAGE_ACCOUNT_NAME`.

The file share mount options.

The file share mount path.

The blob storage shared access signature (SAS) token, which can be provided instead of an account key. Defaults to environment variable `AZURE_STORAGE_SAS_TOKEN`.

The duration of the SAS token generated by Nextflow when the `sasToken` option is not specified (default 48h).

The `charliecloud` scope controls how [Charliecloud](#) containers are executed by Nextflow.

The following settings are available:

The directory where remote Charliecloud images are stored. When using a computing cluster it must be a shared folder accessible to all compute nodes.

Execute tasks with Charliecloud containers (default `false`).

Comma separated list of environment variable names to be included in the container environment.

The amount of time the Charliecloud pull can last, exceeding which the process is terminated (default 20 min).

The registry from where images are pulled. It should be only used to specify a private registry server. It should NOT include the protocol prefix i.e. `http://`.

Specify extra command line options supported by the `ch-run` command.

Mounts a path of your choice as the `/tmp` directory in the container. Use the special value `'auto'` to create a temporary directory each time a container is created.

When `false`, mount input directories as read-only (default `true`).

Run containers from storage in writeable mode using overlayfs (default `true`). This option requires unprivileged `overlayfs` (Linux kernel ≥ 5.11). For full support, tempfs with xattrs in the user namespace (Linux kernel ≥ 6.6) is required. See [charliecloud documentation](#) for details.

The `conda` scope controls the creation of Conda environments by the Conda package manager.

The following settings are available:

The path where Conda environments are stored. It should be accessible from all compute nodes when using a shared file system.

Changed in version 26.04

The default was changed to `'conda-forge,bioconda'`.

The list of Conda channels that can be used to resolve Conda packages (default `'conda-forge,bioconda'`). Channel priority decreases from left to right.

Extra command line options for the `conda create` command. See the [Conda documentation](#) for more information.

The amount of time to wait for the Conda environment to be created before failing (default `20 min`).

Execute tasks with Conda environments (default `false`).

Use [Mamba](#) instead of `conda` to create Conda environments (default `false`).

Use [Micromamba](#) instead of `conda` to create Conda environments (default `false`).

The `dag` scope controls the generation of the [workflow diagram](#).

The following settings are available:

Added in version 23.10

Supported by the HTML and Mermaid renderers.

Controls the maximum depth at which to render sub-workflows (default `no limit`).

Added in version 23.10

Supported by the Graphviz, DOT, HTML and Mermaid renderers.

Controls the direction of the DAG, can be `'LR'` (left-to-right) or `'TB'` (top-to-bottom) (default `'TB'`).

When `true` enables the generation of the DAG file (default `false`).

Graph file name (default `'dag-<timestamp>.html'`).

The output format is inferred from the file extension. The following formats are supported:

Graphviz [DOT](#) file.

Graph Exchange XML Format ([GEXF](#)).

HTML file with Mermaid diagram.

Mermaid diagram.

Requires [Graphviz](#).

Graphviz PDF file.

Requires [Graphviz](#)

Graphviz PNG file.

Requires [Graphviz](#)

Graphviz SVG file.

When `true` overwrites any existing DAG file with the same name (default `false`).

Added in version 23.10

Only supported by the HTML and Mermaid renderers.

When `false`, channel names are omitted, operators are collapsed, and empty workflow inputs are removed (default `false`).

The `docker` scope controls how [Docker](#) containers are executed by Nextflow.

The following settings are available:

Enable Docker execution (default `false`).

Specify additional options supported by the Docker engine i.e. `docker [OPTIONS]`.

Comma separated list of environment variable names to be included in the container environment.

Fix ownership of files created by the Docker container (default `false`).

Use command line options removed since Docker 1.10.0 (default `false`).

Add the specified flags to the volume mounts e.g. `'ro,Z'`.

The registry from where Docker images are pulled. It should be only used to specify a private registry server. It should NOT include the protocol prefix i.e. `http://`.

Added in version 25.10

When `true`, forces the override of the registry name in fully qualified container image names with the registry specified by `docker.registry` (default `false`). This setting allows you to redirect container image pulls from their original registry to a different registry, such as a private mirror or proxy.

Clean up the container after the execution (default `true`). See the [Docker documentation](#) for details.

Specify extra command line options supported by the `docker run` command. See the [Docker documentation](#) for details.

Executes Docker run command as `sudo` (default `false`).

Mounts a path of your choice as the `/tmp` directory in the container. Use the special value `'auto'` to create a temporary directory each time a container is created.

Allocates a pseudo-tty (default `false`).

When `false`, mount input directories as read-only (default `true`).

The `env` scope allows the definition one or more variables that will be exported into the environment where workflow tasks are executed.

Simply prefix your variable names with the `env` scope or surround them by curly brackets, as shown below:

```
env.ALPHA = 'some value'
env.BETA = "$HOME/some/path"

env {
  DELTA = 'one more'
  GAMMA = "/my/path:$PATH"
}
```

note

In the above example, variables like `$HOME` and `$PATH` are evaluated when the workflow is launched. If you want these variables to be evaluated during task execution, escape them with `\$`. This difference is important for variables like `$PATH`, which may be different in the workflow environment versus the task environment.

warning

The `env` scope provides environment variables to tasks, not Nextflow itself. Nextflow environment variables such as `NXF_VER` should be set in the environment in which Nextflow is launched.

The `executor` scope controls various executor behaviors.

The following settings are available:

Added in version 24.04

Used only by the [SLURM](#), [LSF](#), [PBS/Torque](#) and [PBS Pro](#) executors.

The project or organization account that should be charged for running the pipeline jobs.

Used only by the local executor.

The maximum number of CPUs made available by the underlying system.

Determines how often to log the executor status (default `5 min`).

Used only by grid executors.

Determines how long to wait for the `.exitcode` file to be created after the task has completed, before returning an error status (default `270 sec`).

Used only by grid executors and Google Batch.

Determines the name of jobs submitted to the underlying cluster executor:

```
executor.jobName = { "$task.name - $task.hash" }
```

The job name should satisfy the validation constraints of the underlying scheduler.

Determines the number of jobs that can be killed in a single command execution (default `100`).

Used only by the local executor.

The maximum amount of memory made available by the underlying system.

The name of the executor to be used (default `local`).

Added in version 23.10

Used only by the [SLURM](#) executor.

When `true`, memory allocations for SLURM jobs are specified as `--mem-per-cpu <task.memory / task.cpus>` instead of `--mem <task.memory>`.

Added in version 26.04

Used only by the [SLURM](#) executor.

Requires SLURM 24.05 or later.

When `true`, job status queries use `squeue --only-job-state` without partition (`-p`) or user (`-u`) filters. This can reduce the load on the SLURM controller, especially if your SLURM administrator has enabled `SchedulerParameters=enable_job_state_cache` in your [SLURM configuration](#). See [--only-job-state](#) for more information (default `false`).

Used only by the [LSF](#) executor.

Enables the per-job memory limit mode for LSF jobs.

Used only by the [LSF](#) executor.

Enables the per-task memory reserve mode for LSF jobs.

Determines how often to check for process termination. Default varies for each executor.

Added in version 23.04

Determines how job status is retrieved. When `false` only the queue associated with the job execution is queried. When `true` the job status is queried globally i.e. irrespective of the submission queue (default `false`).

The number of tasks the executor will handle in a parallel manner. A queue size of zero corresponds to no limit. Default varies for each executor.

Used only by grid executors.

Determines how often to fetch the queue status from the scheduler (default `1 min`).

Determines the max rate of job submission per time unit, for example `'10sec'` (10 jobs per second) or `'50/2min'` (50 jobs every 2 minutes) (default `unlimited`).

Used only by grid executors.

Delay when retrying failed job submissions (default `500ms`).

Used only by grid executors.

Jitter value when retrying failed job submissions (default `0.25`).

Used only by grid executors.

Max attempts when retrying failed job submissions (default `3`).

Used only by grid executors.

Max delay when retrying failed job submissions (default `30s`).

Changed in version 25.10

This option was renamed from `executor.submit.retry.reason` to `executor.retry.reason`.

Used only by grid executors.

Regex pattern that when verified causes a failed submit operation to be re-tried (default `Socket timed out`).

Some executor settings have different default values depending on the executor.

Executor	queueSize	pollInterval
AWS Batch	1000	10s
Azure Batch	1000	10s
Google Batch	1000	10s
Grid Executors	100	5s
Kubernetes	100	5s
Local	N/A	100ms

Executor config settings can be applied to specific executors by prefixing the executor name with the symbol `$` and using it as special scope. For example:

```
// block syntax
executor {
  $sge {
    queueSize = 100
    pollInterval = '30sec'
  }

  $local {
    cpus = 8
    memory = '32 GB'
  }
}

// dot syntax
executor.$sge.queueSize = 100
executor.$sge.pollInterval = '30sec'
executor.$local.cpus = 8
executor.$local.memory = '32 GB'
```

The `fusion` scope provides advanced configuration for the use of the [Fusion file system](#).

The following settings are available:

Added in version 24.04

The maximum size of the local cache used by the Fusion client.

The URL of the container layer that provides the Fusion client. Supports `http(s)://...` and `file:/...` (absolute, no authority) schemes. The latter is intended for local development against a custom manifest. For example, `file:/path/to/manifest.json`.

Enable the Fusion file system (default `false`).

Export access credentials required by the underlying object storage as environment variables (e.g., `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` for AWS S3) to task execution environments (default `false`).

note

This configuration does not mount or provide access to credential files. For example, AWS credentials like `~/.aws/credentials`, `~/.aws/config`, and SSO cache files are not mounted. AWS SSO users must export credentials to environment variables:

```
eval "$(aws configure export-credentials --format env)"
```

warning

This option leaks credentials in the task launcher script. It should only be used for testing and development purposes.

The log level of the Fusion client.

The output location of the Fusion log.

Added in version 23.10

Enable privileged containers for Fusion (default `true`). Non-privileged use is supported only on Kubernetes with the [k8s-fuse-plugin](#) or a similar FUSE device plugin.

Added in version 25.04

Currently only supported for AWS Batch

Enable Fusion snapshotting (preview, default `false`). This feature allows Fusion to automatically restore a job when it is interrupted by a spot reclamation.

Currently only supported for S3.

The pattern that determines how tags are applied to files created via the Fusion client (default `[.command.*|.exitcode|.fusion.*](nextflow.io/metadata=true),[*](nextflow.io/temporary=true)`). Set to `false` to disable tags.

The `google` scope allows you to configure the interactions with Google Cloud, including Google Cloud Batch and Google Cloud Storage.

The following settings are available:

Use the given Google Cloud project ID as the billing project for storage access (default `false`). Required when accessing data from [requester pays](#) buckets.

Added in version 23.10

The HTTP connection timeout for Cloud Storage API requests (default `'60s'`).

Added in version 23.10

The HTTP read timeout for Cloud Storage API requests (default `'60s'`).

The Google Cloud location where jobs are executed (default `us-central1`).

The Google Cloud project ID to use for pipeline execution.

The set of [allowed locations](#) for VMs to be provisioned (default `no restriction`).

Added in version 24.10

The list of exit codes that should be automatically retried by Google Batch when `google.batch.maxSpotAttempts` is greater than 0 (default `[50001]`). See [Google Batch documentation](#) for the complete list of retryable exit codes.

Added in version 24.10

The image URI of the virtual machine boot disk, e.g. `batch-debian` (default `none`). See [Google documentation](#) for details.

The size of the virtual machine boot disk, e.g. `50.GB` (default `none`).

The [minimum CPU Platform](#), e.g. `'Intel Skylake'` (default `none`).

Added in version 25.04

List of custom mount options for `gcsfuse` (default `['-o rw', '-implicit-dirs']`).

Enables Ops Agent installation on Google Batch instances for enhanced monitoring and logging (default: `false`). See the [Google Batch documentation](#) for details.

note

The Ops Agent requires a compatible boot disk image. For Google Batch, use [Batch-debian images](#) (e.g., `batch-debian`) with `google.batch.bootDiskImage` . The default Container-Optimized OS (`batch-cos`) is not compatible with the Ops Agent.

Added in version 26.04

The Google Cloud Storage path where job logs should be stored, e.g. `gs://my-logs-bucket/logs` . When specified, Google Batch will write job logs to this location instead of [Cloud Logging](#) . The bucket must be accessible and writable by the service account.

Added in version 24.10

The default value was changed from `5` to `0` .

Max number of execution attempts of a job interrupted by a Compute Engine Spot reclaim event (default 0). See also `google.batch.autoRetryExitCodes`

The URL of an existing network resource to which the VM will be attached.

You can specify the network as a full or partial URL. For example, the following are all valid URLs:

- `https://www.googleapis.com/compute/v1/projects/{project}/global/networks/{network}`
- `projects/{project}/global/networks/{network}`
- `global/networks/{network}`

The [network tags](#) to be applied to the instances created by Google Batch jobs (e.g., `['allow-ssh', 'allow-http']`). Network tags are ignored when using instance templates.

The Google service account email to use for the pipeline execution. If not specified, the default Compute Engine service account for the project will be used.

The service account must have the IAM roles listed in the [Google Cloud Batch documentation for Nextflow](#). Without these roles, Cloud Batch jobs stay in the `SCHEDULED` state and never run.

This service account will only be used for tasks submitted by Nextflow, not for Nextflow itself. See [Credentials](#) for more information on Google Cloud credentials.

Enable the use of spot virtual machines (default `false`).

The URL of an existing subnetwork resource in the network to which the VM will be attached.

You can specify the subnetwork as a full or partial URL. For example, the following are all valid URLs:

- `https://www.googleapis.com/compute/v1/projects/{project}/regions/{region}/subnetworks/{subnetwork}`
- `projects/{project}/regions/{region}/subnetworks/{subnetwork}`
- `regions/{region}/subnetworks/{subnetwork}`

Do not provision public IP addresses for VMs, such that they only have an internal IP address (default `false`). When this option is enabled, jobs can only load Docker images from Google Container Registry, and cannot use external services other than Google APIs.

Added in version 24.10

Max attempts when retrying failed API requests to Cloud Storage (default 10).

Added in version 24.10

Max delay when retrying failed API requests to Cloud Storage (default '90s').

Added in version 24.10

Delay multiplier when retrying failed API requests to Cloud Storage (default 2.0).

The `k8s` scope controls the deployment and execution of workflow applications in a Kubernetes cluster.

The following settings are available:

Automatically mount host paths into the task pods (default `false`). Only intended for development purposes when using a single node.

When `true`, successful pods are automatically deleted (default `true`).

Map of options for the Kubernetes HTTP client.

If this option is specified, it will be used instead of `.kube/config`.

Available options:

- `server`
- `token`
- `tokenFile`
- `verifySsl`
- `sslCert`
- `sslCertFile`
- `clientCert`
- `clientCertFile`
- `clientKey`
- `clientKeyFile`

Added in version 26.04

The interval after which the Kubernetes client configuration is refreshed (default: `50m`).

This setting is useful when the Kubernetes authentication token has a limited lifespan and needs to be periodically refreshed. The client configuration will be automatically reloaded after the specified interval, allowing Nextflow to obtain fresh credentials from the Kubernetes configuration.

Whether to use Kubernetes `Pod` or `Job` resource type to carry out Nextflow tasks (default `Pod`).

The Kubernetes [configuration context](#) to use.

Added in version 24.04

When `true`, set both the pod CPUs `request` and `limit` to the value specified by the `cpus` directive, otherwise set only the `request` (default `false`). This setting is useful when a K8s cluster requires a CPU limit to be defined through a [LimitRange](#).

Include the hostname of each task in the execution trace (default `false`).

Added in version 24.04

The FUSE device plugin to be used when enabling Fusion in unprivileged mode (default `['nextflow.io/fuse'1]`).

The Kubernetes HTTP client request connection timeout e.g. `'60s'`.

The Kubernetes HTTP client request connection read timeout e.g. `'60s'`.

The strategy for pulling container images. Can be `IfNotPresent`, `Always`, `Never`.

The path where the workflow is launched and the user data is stored (default `<volume-claim-mount-path>/<user-name>`). Must be a path in a shared K8s persistent volume.

The Kubernetes namespace to use (default `default`).

Additional pod configuration options such as environment variables, config maps, secrets, etc. Allows the same settings as the `pod` process directive. When using the `kuberun` command, this setting also applies to the submitter pod.

The path where Nextflow projects are downloaded (default `<volume-claim-mount-path>/projects`). Must be a path in a shared K8s persistent volume.

The user ID to be used to run the containers. Shortcut for the `securityContext` option.

The `security context` to use for all pods.

The Kubernetes `service account name` to use.

The name of the persistent volume claim where the shared work directory is stored.

The mount path for the persistent volume claim (default `/workspace`).

The path in the persistent volume to be mounted (default `/`).

The path of the shared work directory (default `<user-dir>/work`). Must be a path in a shared K8s persistent volume.

Save the pod spec for each task to `.command.yaml` in the task directory (default `false`).

Delay when retrying failed API requests (default `250ms`).

Jitter value when retrying failed API requests (default `0.25`).

Max attempts when retrying failed API requests (default `4`).

Max delay when retrying failed API requests (default `90s`).

The `lineage` scope controls the generation of `lineage` metadata.

The following settings are available:

Enable generation of lineage metadata (default `false`).

The location of the lineage metadata store (default `./.lineage`).

The `mail` scope controls the mail server used to send email notifications.

The following settings are available:

Enable Java Mail logging for debugging purposes (default `false`).

Default email sender address.

Host name of the mail server.

User password to connect to the mail server.

Port number of the mail server.

User name to connect to the mail server.

Host name of an HTTP web proxy server that will be used for connections to the mail server.

Port number for the HTTP web proxy server.

Any SMTP configuration property supported by the [Java Mail API](#), which Nextflow uses to send emails. See the table of available properties [here](#).

The `manifest` scope allows you to define some metadata that is useful when publishing or running your pipeline.

The following settings are available:

Deprecated in version 24.10

Use `manifest.contributors` instead.

Project author name (use a comma to separate multiple names).

Added in version 24.10

List of project contributors. Should be a list of maps.

The following fields are supported in the contributor map:

The contributor name.

The contributor affiliated organization.

The contributor email address.

The contributor GitHub URL.

List of contribution types, each element can be one of `'author'`, `'maintainer'`, or `'contributor'`.

The contributor [ORCID](#) URL.

Deprecated in version 26.04

Nextflow can automatically detect the default branch when using a remote pipeline.

Git repository default branch (default `master`).

Free text describing the workflow project.

Project documentation URL.

Project related publication DOI identifier.

Controls whether git sub-modules should be cloned with the main repository.

Can be either a boolean value, a list of submodule names, or a comma-separated string of submodule names.

Project home page URL.

Project related icon location (Relative path or URL).

Project license.

Project main script (default `main.nf`).

Project short name.

Minimum required Nextflow version.

This setting may be useful to ensure that a specific version is used:

```
manifest.nextflowVersion = '1.2.3'           // exact match
manifest.nextflowVersion = '1.2+'           // 1.2 or later (excluding 2 and later)
manifest.nextflowVersion = '>=1.2'          // 1.2 or later
manifest.nextflowVersion = '>=1.2, <=1.5'    // any version in the 1.2 .. 1.5 range
manifest.nextflowVersion = '!=1.2'          // with ! prefix, stop execution if current version does not match
required version.
```

See [VersionNumber](#) for details.

Project organization.

Pull submodules recursively from the Git repository.

Project version number.

Changed in version 24.10

The `nextflow.publish.retryPolicy` settings were moved to `workflow.output.retryPolicy`.

Changed in version 25.10

The `workflow.output.retryPolicy` settings were moved to `nextflow.retryPolicy`.

Delay used for retryable operations (default `350ms`).

Jitter value used for retryable operations (default `0.25`).

Max attempts used for retryable operations (default `5`).

Max delay used for retryable operations (default `90s`).

The `notification` scope controls the automatic sending of an email notification on workflow completion.

The following settings are available:

Map of variables that can be used in the template file.

Send an email notification when the workflow execution completes (default `false`).

Sender address for the email notification.

Path of a template file containing the contents of the email notification.

Recipient address for the email notification. Multiple addresses can be specified as a comma-separated list.

The `podman` scope controls how [Podman](#) containers are executed by Nextflow.

The following settings are available:

Execute tasks with Podman containers (default `false`).

Specify additional options supported by the Podman engine i.e. `podman [OPTIONS]`.

Comma separated list of environment variable names to be included in the container environment.

Add the specified flags to the volume mounts e.g. `'ro,Z'`.

The registry from where container images are pulled. It should be only used to specify a private registry server. It should NOT include the protocol prefix i.e. `http://`.

Clean-up the container after the execution (default `true`).

Specify extra command line options supported by the `podman run` command.

Mounts a path of your choice as the `/tmp` directory in the container. Use the special value `'auto'` to create a temporary directory each time a container is created.

The `report` scope controls the generation of the [Execution report](#).

The following settings are available:

Create the execution report on workflow completion (default `false`).

The path of the created execution report file (default `'report-<timestamp>.html'`).

Overwrite any existing report file with the same name (default `false`).

The `sarus` scope controls how [Sarus](#) containers are executed by Nextflow.

The following settings are available:

Execute tasks with Sarus containers (default `false`).

Comma-separated list of environment variable names to be included in the container environment.

Specify extra command line options supported by the `sarus run` command. See the [Sarus user guide](#) for details.

Allocates a pseudo-tty (default `false`).

Added in version 26.04

warning

Preview feature: may change in a future release.

The `sequera` scope allows you to configure the interactions with Seqera services.

The `sequera.executor` scope configures the Seqera scheduler service for the [Seqera executor](#).

The following settings are available:

The following settings are available:

When `true`, automatically adds workflow metadata labels to the session with the `nextflow.io/` prefix (default: `false`). The following labels are added: `projectName`, `userName`, `runName`, `sessionId`, `resume`, `revision`, `commitId`, `repository`, `manifestName`, `runtimeVersion`. A `seqla.io/runId` label is also added, computed as a SipHash of the session ID and run name.

The Seqla Platform compute environment ID. When specified, the scheduler resolves the compute environment directly by this ID instead of inferring a suitable compute environment. Used as a fallback when the workflow launch does not include a compute environment reference.

The Seqla scheduler service endpoint URL (required).

The EC2 key pair name for SSH access to instances.

The resource prediction model used to estimate task resource requirements from historical execution metrics. Supported values: `'qr/v1'`, `'qr/v2'` (quantile regression). When not set, no resource estimation is applied.

The compute backend provider type (e.g. `'aws'`, `'local'`). When specified, used together with `region` to select the matching compute environment.

The cloud region for task execution.

When `true`, enables on-demand interactive shell access (e.g. SSH) to the run's task containers for the VM and local backends (default: `false`). A running task can then be reached with `sched task ssh <task-id>` (or a plain `ssh <task-id>@<scheduler>`), and the connection survives task completion.

The execution strategy within the chosen `provider`. Narrows compute-environment selection when a provider offers multiple strategies (e.g. AWS supports `'ecs'` and `'vm'`). When omitted, the provider's canonical default strategy is used (AWS → `'ecs'`).

Custom environment variables to apply to all tasks submitted by the Seqla executor. These are merged with the Fusion environment variables, with Fusion variables taking precedence. For example: `taskEnvironment = [MY_VAR: 'value']`.

The ECS capacity provider mode. Can be `'managed'` (default) for ECS Managed Instances, or `'asg'` for an Auto Scaling Group-backed capacity provider.

The disk allocation strategy. Can be `'task'` (default) for per-task EBS volumes, or `'node'` for per-node instance storage. When using `'node'` allocation, EBS-specific options (`diskType`, `diskIops`, `diskThroughputMiBps`, `diskEncrypted`) are not applicable.

Enable KMS encryption for the EBS volume (default: `false`). Only applicable when `diskAllocation` is `'task'`.

The IOPS for io1/io2/gp3 volumes. Required for io1/io2 volume types. Only applicable when `diskAllocation` is `'task'`.

The container path where the disk is mounted (default: `'/tmp'`). Applicable to all disk allocation strategies.

The disk size for session-level storage (e.g. `100.GB`).

The throughput in MiB/s for gp3 volumes (125-1000). Default: `325` (Fusion recommended). Only applicable when `diskAllocation` is `'task'`.

The EBS volume type for task scratch disk. Supported types: `'ebs/gp3'` (default), `'ebs/gp2'`, `'ebs/io1'`, `'ebs/io2'`, `'ebs/st1'`, `'ebs/sc1'`. Only applicable when `diskAllocation` is `'task'`.

List of acceptable EC2 instance families. For example, `['m5', 'c5', 'r5']`.

The maximum number of spot retry attempts before falling back to on-demand. Only used when provisioning is `'spot'` or `'spotFirst'`.

The instance provisioning mode. Can be `'spot'`, `'ondemand'`, or `'spotFirst'`.

The initial delay when a failing HTTP request is retried (default: `'450ms'`).

The maximum delay when a failing HTTP request is retried (default: `'90s'`).

The maximum number of retry attempts (default: `10`).

The jitter factor for randomizing retry delays (default: `0.25`).

The multiplier for exponential backoff (default: `2.0`).

The `shifter` scope controls how [Shifter](#) containers are executed by Nextflow.

The following settings are available:

Execute tasks with Shifter containers (default `false`).

Comma-separated list of environment variable names to be included in the container environment.

The `singularity` scope controls how [Singularity](#) containers are executed by Nextflow.

The following settings are available:

Changed in version 23.10

Default value was changed from `false` to `true`.

Automatically mount host paths in the executed container (default `true`). It requires the `user bind control` feature to be enabled in your Singularity installation.

The directory where remote Singularity images are stored. When using a compute cluster, it must be a shared folder accessible to all compute nodes.

Execute tasks with Singularity containers (default `false`).

Specify additional options supported by the Singularity engine i.e. `singularity [OPTIONS]`.

Comma separated list of environment variable names to be included in the container environment.

Directory where remote Singularity images are retrieved. When using a computing cluster it must be a shared folder accessible to all compute nodes.

Pull the Singularity image with http protocol (default `false`).

Added in version 24.04

Requires Singularity 3.11 or later

When enabled, OCI (and Docker) container images are pull and converted to a SIF image file format implicitly by the Singularity run command, instead of Nextflow (default `false`).

note

Leave `ociAutoPull` disabled if willing to build a Singularity native image with Wave (see [Build Singularity native images](#)).

Added in version 24.04

Requires Singularity 4 or later

Enable OCI-mode, that allows running native OCI compliant container image with Singularity using `crun` or `runc` as low-level runtime (default `false`).

See `--oci` flag in the [Singularity documentation](#) for more details and requirements (default `false`).

note

Leave `ociMode` disabled if you are willing to build a Singularity native image with Wave (see [Build Singularity native images](#)).

The amount of time the Singularity pull can last, after which the process is terminated (default `20 min`).

The registry from where Docker images are pulled. It should be only used to specify a private registry server. It should NOT include the protocol prefix i.e. `http://`.

Specify extra command line options supported by `singularity exec`.

The `spack` scope controls the creation of a Spack environment by the Spack package manager.

The following settings are available:

The path where Spack environments are stored. It should be accessible from all compute nodes when using a shared file system.

Enable checksum verification of source tarballs (default `true`).

Only disable when requesting a package version not yet encoded in the corresponding Spack recipe.

The amount of time to wait for the Spack environment to be created before failing (default `60 min`).

Execute tasks with Spack environments (default `false`).

The maximum number of parallel package builds (default the number of available CPUs).

The `timeline` scope controls the generation of the [Execution timeline](#).

The following settings are available:

Create the execution timeline on workflow completion (default `false`).

Timeline file name (default `'timeline-<timestamp>.html'`).

Overwrite any existing timeline file with the same name (default `false`).

The `tower` scope controls the settings for [Seqera Platform](#) (formerly Tower Cloud).

The following settings are available:

The unique access token for your Seqera Platform account.

Your `accessToken` can be obtained from your Seqera Platform instance in the [Tokens page](#).

The compute environment ID in your Seqera Platform account used to launch pipelines (default: the primary compute environment in the selected workspace).

Enable workflow monitoring with Seqera Platform (default `false`).

The endpoint of your Seqera Platform instance (default `https://api.cloud.seqera.io`).

Added in version 26.04

The HTTP connection timeout for Seqera Platform API requests (default: `'60s'`).

Added in version 26.04

The HTTP read timeout for Seqera Platform API requests (default: `'60s'`).

The workspace ID in Seqera Platform in which to save the run (default the launching user's personal workspace).

The workspace ID can also be specified using the environment variable `TOWER_WORKSPACE_ID` (config file has priority over the environment variable).

The `trace` scope controls the generation of the [Trace file](#).

The following settings are available:

Create the trace file on workflow completion (default `false`).

Comma-separated list of fields to include in the trace file.

Available fields:

Task ID.

Task hash code.

Task ID given by the underlying execution system e.g. POSIX process PID when executed locally, job ID when executed by a grid engine, etc.

Nextflow process name.

User provided identifier associated with this task.

Task name.

Task status. Options `NEW`, `SUBMITTED`, `RUNNING`, `COMPLETED`, `FAILED`, and `ABORTED`.

POSIX process exit status.

Environment module used to run the task.

Docker image name used to execute the task.

The CPUs number request for the task execution.

The time request for the task execution

The disk space request for the task execution.

The memory request for the task execution.

Attempt at which the task completed.

Timestamp when the task has been submitted.

Timestamp when the task execution has started.

Timestamp when task execution has completed.

Time elapsed to complete since the submission.

Task execution time i.e. delta between completion and start timestamp.

The queue that the executor attempted to run the process on.

Percentage of CPU used by the process.

Percentage of memory used by the process.

Real memory (resident set) size of the process. Equivalent to `ps -o rss` .

Virtual memory size of the process. Equivalent to `ps -o vsize` .

Peak of real memory. Data is read from field `VmHWM` in `/proc/$pid/status` file.

Peak of virtual memory. Data is read from field `VmPeak` in `/proc/$pid/status` file.

Number of bytes the process read, using any read-like system call from files, pipes, tty, etc. Data is read from `/proc/$pid/io` .

Number of bytes the process wrote, using any write-like system call. Data is read from `/proc/$pid/io` .

Number of read-like system call invocations that the process performed. Data is read from `/proc/$pid/io` .

Number of write-like system call invocations that the process performed. Data is read from `/proc/$pid/io` .

Number of bytes the process directly read from disk. Data is read from `/proc/$pid/io` .

Number of bytes the process originally dirtied in the page-cache (assuming they will go to disk later). Data is read from `/proc/$pid/io` .

Number of voluntary context switches. Data is read from field `voluntary_ctxt_switches` in `/proc/$pid/status` file.

Number of involuntary context switches. Data is read from field `nonvoluntary_ctxt_switches` in `/proc/$pid/status` file.

The variables defined in task execution environment.

The directory path where the task was executed.

The task command script.

The value of the process `scratch` directive.

The action applied on error for task failure.

The host on which the task was executed. Supported only for the Kubernetes executor yet. Activate with `k8s.fetchNodeName = true` in the Nextflow config file.

The name of the CPU model used to execute the task. This data is read from `/proc/cpuinfo`.

Trace file name (default `'trace-<timestamp>.txt'`).

Overwrite any existing trace file with the same name (default `false`).

Report trace metrics as raw numbers where applicable, i.e. report duration values in milliseconds and memory values in bytes (default `false`).

Character used to separate values in each row (default `\t`).

The `wave` scope provides advanced configuration for the use of [Wave containers](#).

The following settings are available:

Enable the use of Wave containers (default `false`).

The Wave service endpoint (default `https://wave.seqera.io`).

Added in version 23.10

Enable Wave container freezing (default `false`). Wave will provision a non-ephemeral container image that will be pushed to a container repository of your choice. The target registry must be specified using the `wave.build.repository` setting. It is also recommended to specify a custom cache repository using `wave.build.cacheRepository`.

note

The container registry authentication must be managed by the underlying infrastructure.

Added in version 24.10

Enable Wave container mirroring (default `false`). Wave will mirror (i.e., copy) the containers in your pipeline to a container registry of your choice, so that pipeline tasks can pull the containers from this registry instead of the original one.

The mirrored containers will have the same name, digest, and metadata.

The target registry must be specified using the `wave.build.repository` setting.

This option is only compatible with `wave.strategy = 'container'`. It cannot be used with `wave.freeze`.

note

The container registry authentication must be managed by the underlying infrastructure.

The strategy to be used when resolving multiple Wave container requirements (default `'container,dockerfile,conda'`).

The container repository used to cache image layers built by the Wave service.

The corresponding credentials must be provided in your Seqera Platform account.

Added in version 25.10

The compression algorithm that should be used when building the container. Allowed values are `gzip` , `estargz` and `zstd` (default `gzip`).

Added in version 25.10

Level of compression used when building a container depending the chosen algorithm `gzip` , `estargz` (0-9) and `zstd` (0-22).

Added in version 25.10

Forcefully apply compression option to all layers, including already existing layers (default `false`).

Added in version 24.04

The base image for the final stage in multi-stage Conda container builds (default `ubuntu:24.04`). This option only applies when using `wave.build.template` set to `conda/micromamba:v2` or `conda/pixi:v1` .

One or more Conda packages to be always added in the resulting container (default `conda-forge::procping`).

One or more commands to be added to the Dockerfile used to build a Conda based image.

The Mamba container image is used to build Conda based container. This is expected to be [micromamba-docker](#) image.

The container repository where images built by Wave are uploaded.

The corresponding credentials must be provided in your Seqera Platform account.

Added in version 26.04

The build template to use for container builds (default `conda/micromamba:v1`). Supported values:

- `conda/micromamba:v1` Standard Micromamba 1.x single-stage build. Default when unspecified.
- `conda/micromamba:v2` Micromamba 2.x with multi-stage builds.
- `conda/pixi:v1` Pixi package manager with multi-stage builds for optimized image sizes.
- `cran/installr:v1` R/CRAN packages using `installr`.

Multi-stage templates produce smaller images by excluding build tools from the final image.

The connection timeout for the Wave HTTP client (default `30s`).

Added in version 25.04

The maximum request rate for the Wave HTTP client (default 1/sec).

The initial delay when a failing HTTP request is retried (default 450ms).

The jitter factor used to randomly vary retry delays (default 0.25).

The max number of attempts a failing HTTP request is retried (default 5).

The max delay when a failing HTTP request is retried (default 90s).

Added in version 24.10

Comma-separated list of allowed vulnerability levels when scanning containers for security vulnerabilities in `required` mode.

Allowed values are `low`, `medium`, `high`, `critical`.

This option requires `wave.scan.mode = 'required'`.

Added in version 24.10

Enable Wave container security scanning. Wave will scan the containers in your pipeline for security vulnerabilities.

The following options can be specified:

No security scanning is performed.

The containers used by your pipeline are scanned for security vulnerabilities. The task execution is carried out regardless of the security scan result.

The containers used by your pipeline are scanned for security vulnerabilities. The task is only executed if the corresponding container is free of vulnerabilities.

Added in version 24.10

The `workflow` scope provides workflow execution options.

The following settings are available:

Added in version 24.10

When `true`, the pipeline will exit with a non-zero exit code if any failed tasks are ignored using the `ignore error strategy` (default `false`).

Specify a closure that will be invoked at the end of a workflow run (including failed runs). See [Workflow handlers](#) for more information.

Specify a closure that will be invoked if a workflow run is terminated. See [Workflow handlers](#) for more information.

Currently only supported for S3.

Specify the media type, also known as [MIME type](#), of published files (default `false`). Can be a string (e.g. `'text/html'`), or `true` to infer the content type from the file extension.

Currently only supported for local and shared filesystems.

Copy file attributes (such as the last modified timestamp) to the published file (default `false`).

Enable or disable publishing (default `true`).

When `true`, the workflow will not fail if a file can't be published for some reason (default `false`).

The file publishing method (default `'symlink'`).

Available options:

Copy each file into the output directory.

Copy each file into the output directory without following symlinks, i.e. only the link is copied.

Create a hard link in the output directory for each file.

Move each file into the output directory.

Should only be used for files which are not used by downstream processes in the workflow.

Create a relative symbolic link in the output directory for each file.

Create an absolute symbolic link in the output directory for each output file.

When `true` any existing file in the specified folder will be overwritten (default `'standard'`).

Available options:

Never overwrite existing files.

Always overwrite existing files.

Overwrite existing files when the file content is different.

Overwrite existing files when the file size is different.

Overwrite existing files when the file size or last modified timestamp is different.

Currently only supported for S3.

Specify the storage class for published files.

Currently only supported for S3.

Specify arbitrary tags for published files.

For example:

```
workflow.output.tags = [F00'hello', BAR'world']
```

Environment variables

The following environment variables control the configuration of the Nextflow runtime and the underlying Java virtual machine.

Defines the path location of the Java binary command used to launch Nextflow.

Defines the path location of the Java VM installation used to run Nextflow.

Added in version 26.04

When `true`, enables agent output mode. In this mode, Nextflow replaces the interactive ANSI log with minimal, structured output optimized for AI agents and non-interactive environments. The output uses tagged lines such as `[PIPELINE]`, `[PROCESS]`, `[WARN]`, `[ERROR]`, and `[SUCCESS]` / `[FAILED]` written to standard output (default: `false`).

Enables/disables ANSI console output (default `true` when ANSI terminal is detected).

Enables/disables ANSI completion summary: `true|false` (default: print summary if execution last more than 1 minute).

Defines the directory where downloaded pipeline repositories are stored (default: `$NXF_HOME/assets`)

Added in version 24.04

Defines the base cache directory when using the default cache store (default: `"$launchDir/.nextflow"`).

Directory where remote Charliecloud images are stored. When using a computing cluster it must be a shared folder accessible from all compute nodes.

Added in version 23.10

Defines the base cache path when using the cloud cache store.

Defines the default cloud driver to be used if not specified in the config file or as command line option, either `aws` or `google`.

Directory where Conda environments are stored. When using a computing cluster it must be a shared folder accessible from all compute nodes.

Enable the use of Conda recipes defined by using the `conda` directive. (default: `false`).

Deprecated in version 22.10

When `true`, override the container entrypoint with `/bin/bash` (default: `false`).

Added in version 25.10

Defines the format for date and time representations in notifications and reports. Supports custom formats (e.g., `yyyy-MM-dd HH:mm:ss`) or `iso` for ISO 8601 format with timezone (default: `dd-MMM-yyyy HH:mm:ss`).

Deprecated in version 25.04

Defines the DSL version that should be used in not specified otherwise in the script of config file (default: `2`)

Added in version 23.10

Nextflow automatically checks for a newer version of itself unless this option is enabled (default: `false`).

Disables the cancellation of child jobs on workflow execution termination.

Added in version 23.10

Disables the automatic type detection of command line parameters.

Added in version 23.10

Disables the requirement for Wave service when enabling the Fusion file system.

Added in version 23.10

Enable to use of AWS SES native API for sending emails in place of legacy SMTP settings (default: `false`)

Added in version 23.10

When enabled the job script will execute Linux `sync` command on job completion. This may be useful to synchronize the job state over shared file systems (default: `false`)

Enable Nextflow secrets features (default: `true`)

Deprecated in version 26.04

Enable Nextflow strict execution mode (default: `false`)

Added in version 23.10

Enable the use of virtual threads in the Nextflow runtime (default: `false`)

Defines the default process executor, e.g. `sge`

Added in version 23.10

The file storage path against which relative file paths are resolved.

For example, with `NXF_FILE_ROOT=/some/root/path`, the use of `file('hello')` will be resolved to the absolute path `/some/root/path/hello`. A remote root path can be specified using the usual protocol prefix, e.g. `NXF_FILE_ROOT=s3://my-bucket/data`. Files defined using an absolute path are not affected by this setting.

Added in version 26.04

When set to `true`, collect task resource metrics (CPU, memory, I/O) from the Fusion trace file (`.fusion/trace.json`) produced in the task work directory, replacing the metrics collected by the default bash command-trace wrapper. Requires Fusion to be enabled. GPU metrics from Fusion are always collected regardless of this setting (default: `false`).

Nextflow home directory (default: `$HOME/.nextflow`).

Defines the path location of the Java VM installation used to run Nextflow. This variable overrides the `JAVA_HOME` variable if defined.

Allows the setting Java VM options. This is similar to `NXF_OPTS` however it's only applied the JVM running Nextflow and not to any java pre-launching commands.

The filename of the Nextflow log (default: `.nextflow.log`).

When `true` prevents Nextflow from automatically downloading and updating remote project repositories (default: `false`).

Changed in version 23.10

This option also disables the automatic version check (see `NXF_DISABLE_CHECK_LATEST`).

Changed in version 24.04

This option also prevents plugins from being downloaded. Plugin versions must be specified in offline mode, or else Nextflow will fail.

Provides extra options for the Java and Nextflow runtime. It must be a blank separated list of `-Dkey[=value]` properties.

Default `organization` prefix when looking for a hosted repository (default: `nextflow-io`).

Defines the path location of the pipeline parameters file.

Name of the file where the process PID is saved when Nextflow is launched in background.

Added in version 25.04

Comma separated list of plugin IDs that can be used in a workflow executions e.g. `NXF_PLUGINS_ALLOWED=nf-amazon,nf-seqera,nf-wave`. Use empty string to disallow all plugins.

Whether to use the default plugins when no plugins are specified in the Nextflow configuration (default: `true`).

The path where the plugin archives are loaded and stored (default: `$NXF_HOME/plugins`).

Added in version 25.10

Specifies the URL of the plugin registry used to download and resolve plugins. This allows using custom or private plugin registries instead of the default public registry.

Added in version 23.04

Defines a custom plugin registry or plugin release URL for testing plugins outside of the main registry.

Added in version 24.04

Defines the default behavior of `publishDir.failOnError` setting. See [publishDir](#) directive for more information.

Added in version 25.10

Delay used for HTTP retryable operations (default: `350ms`).

Added in version 25.10

Jitter value used for HTTP retryable operations (default: `0.25`).

Added in version 25.10

Max number of attempts used for HTTP retryable operations (default: `5`).

Added in version 25.10

Max delay used for HTTP retryable operations (default: `90s`).

Added in version 25.10

Delay multiplier used for HTTP retryable operations (default: `2.0`).

Defines the path location of the SCM config file .

Directory where remote Singularity images are stored. When using a computing cluster it must be a shared folder accessible from all compute nodes.

Directory where remote Singularity images are retrieved. It should be a directory accessible to all compute nodes.

Directory where Spack environments are stored. When using a computing cluster it must be a shared folder accessible from all compute nodes.

Added in version 23.04

Enable the use of Spack recipes defined by using the [spack](#) directive. (default: `false`).

Added in version 25.04

The strict parser is disabled by default.

Changed in version 26.04

The strict parser is enabled by default.

Set to `'v2'` to use the [strict syntax parser](#) for Nextflow code. Set to `'v1'` to fall back to the legacy parser.

Directory where temporary files are stored

Enable trace level logging for the specified packages. Equivalent to the `-trace` command-line option.

Defines which version of Nextflow to use.

Directory where working files are stored (usually your scratch directory)

Added in version 23.10

Enables writing large staging scripts to a separate `.command.stage` file. The value defines the minimum size of the `.command.run` staging script for it to be written to the separate file (default when enabled: `'1 MB'`).

This setting is useful for executors that impose a size limit on job scripts.

The access token for authenticating with Seqera Platform. Can also be configured using the `tower.accessToken` config option.

The Seqera Platform API endpoint (default: `https://api.cloud.seqera.io`). Can also be configured using the `tower.endpoint` config option.

Added in version 25.10

The Auth0 domain for authenticating with Seqera Platform when connecting to a custom endpoint. When specified, this value takes precedence over the built-in mappings for known Seqera endpoints. Must be used in conjunction with `TOWER_AUTH_CLIENT_ID`.

Added in version 25.10

The Auth0 client ID for authenticating with a custom Seqera Platform endpoint. Must be used in conjunction with `TOWER_AUTH_DOMAIN`.

The refresh token for maintaining authentication with Seqera Platform. Can also be configured using the `tower.refreshToken` config option.

Specifies the Seqera Platform compute environment ID.

When specified, the scheduler resolves the compute environment directly by this ID instead of inferring a suitable compute environment. Can also be configured using the `tower.computeEnvId` config option.

The Seqera Platform workspace ID. Can also be configured using the `tower.workspaceId` config option.

Defines the FTP proxy server. Proxy authentication is supported by providing the credentials in the proxy URL, e.g. `ftp://user:password@proxy-host.com:port`.

Defines the HTTP proxy server.

Defines the HTTPS proxy server.

Disables ANSI color codes in Nextflow log output. When this variable is set, Nextflow prints plain text logs following the [NO_COLOR standard](#).

If both `NO_COLOR` and `NXF_ANSI_LOG` are set, `NXF_ANSI_LOG` takes precedence.

Defines one or more host names that should not use the proxy server. Separate multiple names using a comma character.

Forces the terminal width of ANSI-formatted log output. Overrides automatic terminal width detection and uses the specified width for line wrapping when set to an integer value.

Namespaces

This page lists all of the available namespaces in the Nextflow standard library.

The global namespace contains globally available constants and functions.

Alias of `workflow.projectDir`.

Alias of `workflow.launchDir`.

Directory where a module script is located (equivalent to `projectDir` if used in the main script).

Map of workflow parameters specified in the config file or as command line options.

Alias of `workflow.projectDir`.

Added in version 24.04

Map of pipeline secrets. See [Secrets](#) for more information.

Alias of `workflow.workDir`.

Create a branch criteria to use with the [branch](#) operator.

Added in version 25.04

Get the value of the environment variable with the specified name in the Nextflow launch environment.

Throw a script runtime error with an optional error message.

Deprecated in version 22.10

Use `error()` instead

Stop the pipeline execution and return an exit code and optional error message.

Get a file from a file name or glob pattern. Returns a collection of files if the glob pattern yields zero or multiple files.

Available options:

When `true`, throws an exception if the specified path does not exist in the file system (default: `false`).

When `true`, follows symbolic links when traversing a directory tree, otherwise treats them as files (default: `true`).

When `true`, interprets characters `*`, `?`, `[]` and `{}` as glob wildcards, otherwise handles them as normal characters (default: `true`).

When `true`, includes hidden files in the resulting paths (default: `false`).

Maximum number of directory levels to visit (default: no limit).

Type of paths returned, can be `'file'`, `'dir'` or `'any'` (default: `'file'`).

See also: [channel.fromPath](#).

Get a collection of files from a file name or glob pattern. Supports the same options as `file()`.

Create a grouping key to use with the [groupTuple](#) operator.

Create a multi-map criteria to use with the [multiMap](#) operator.

Print a value to standard output.

Print a formatted string with the given values to standard output.

Print a value to standard output with a newline.

Send an email. See [Notifications](#) for more information.

Sleep for the given number of milliseconds.

Create a record from the given named arguments.

Create a tuple from the given arguments.

The `channel` namespace contains the built-in channel factories. See [Channel factories](#) for details.

The `log` namespace contains functions for logging messages to the console.

Log an error message to the console.

This function does not terminate the pipeline -- use the global `error()` function instead.

Log an info message to the console.

Log a warning message to the console.

The `nextflow` namespace contains information about the current Nextflow runtime.

Nextflow runtime build number.

Nextflow runtime compile timestamp.

Nextflow runtime version number. See [VersionNumber](#) for more information.

The `workflow` namespace contains information about the current workflow run.

Command line as entered by the user to launch the workflow execution.

Git commit ID of the executed workflow repository.

When providing a Git tag, branch name, or commit hash using the `-r` CLI option, the associated `workflow.commitId` is also populated.

Available only in the `workflow.onComplete` handler

Timestamp of workflow when execution is completed.

Configuration files used for the workflow execution.

Docker image used to run workflow tasks, or a map of process names to process containers when multiple images are used.

Returns the name of the container engine (e.g. docker or singularity) or null if no container engine is enabled.

Available only in the `workflow.onComplete` handler

Time elapsed to complete workflow execution.

Available only in the `workflow.onComplete` and `workflow.onError` handlers

Error message of the task that caused the workflow execution to fail.

Available only in the `workflow.onComplete` and `workflow.onError` handlers

Detailed error of the task that caused the workflow execution to fail.

Available only in the `workflow.onComplete` and `workflow.onError` handlers

Exit status of the task that caused the workflow execution to fail.

Added in version 24.10

Whether the `workflow.failOnIgnore` config option was enabled.

See also: [errorStrategy](#)

Namespace containing information about the current Fusion runtime.

The following properties are available:

Whether Fusion is enabled.

The Fusion version being used.

User system home directory.

Directory where the workflow was launched.

Namespace corresponding to the `manifest` config scope.

Added in version 24.10

Workflow output directory.

Added in version 24.04

Whether the current workflow run is a preview run.

Comma-separated list of active configuration profiles.

Directory where the workflow project is located.

Project repository Git remote URL.

Returns `true` whenever the current instance is resumed from a previous execution.

Git branch/tag of the executed workflow repository.

When providing a Git tag or branch name using the `-r` CLI option, the `workflow.revision` is also populated.

Mnemonic name assigned to this execution instance.

Project main script file path.

Project main script unique hash ID.

Project main script file name.

Unique identifier (UUID) associated to current execution.

Timestamp of workflow at execution start.

Returns `true` whenever the current instance is a stub-run execution .

Available only in the `workflow.onComplete` and `workflow.onError` handlers

Reports if the execution completed successfully.

User system account name.

Namespace containing Wave runtime information.

The following properties are available:

Whether Wave is enabled.

The directory where task temporary files are stored.

Define an action to take when the workflow completes (whether successful or not).

Define an action to take if the workflow is terminated due to a runtime error or task failure.

Types

This page describes the standard types in the Nextflow standard library.

Implements the [stdlib-types-iterable](#) trait.

A bag is an unordered collection of values of type `E`.

The following operations are supported for bags:

Concatenates two bags.

Given a value and a bag, returns `true` if the bag contains the value (or not).

note

Lists in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

A boolean can be `true` or `false`.

The following operations are supported for booleans:

- `&&`: logical AND
- `||`: logical OR
- `!`: logical NOT

note

Booleans in Nextflow can be backed by any of the following Java types: `boolean`, `Boolean`.

A channel (also known as a dataflow channel or queue channel) is an asynchronous sequence of values of type `E`. It is used to facilitate dataflow logic in a workflow.

See [Dataflow](#) for an overview of dataflow types. See [Operators \(typed\)](#) for the set of operators that are recommended for use with static typing.

A Duration represents a duration of time with millisecond precision.

A Duration can be created by adding a unit suffix to an integer (e.g. `1.h`), or by explicitly casting to `Duration`:

```
// integer with suffix
oneDay = 24.h

// integer value (milliseconds)
oneSecond = 1000 as Duration

// simple string value
oneHour = '1h' as Duration

// complex string value
complexDuration = '1day 6hours 3minutes 30seconds' as Duration
```

The following suffixes are available:

Unit	Description
ms, milli, millis	Milliseconds
s, sec, second, seconds	Seconds
m, min, minute, minutes	Minutes
h, hour, hours	Hours
d, day, days	Days

Durations can be compared like numbers, and they support basic arithmetic operations:

```
a = 1.h
b = 2.h

assert a < b
assert a + a == b
assert b - a == a
assert a * 2 == b
assert b / 2 == a
```

The following methods are available for a Duration:

Get the duration value in days (rounded down).

Get the duration value in hours (rounded down).

Get the duration value in milliseconds.

Get the duration value in minutes (rounded down).

Get the duration value in seconds (rounded down).

note

These methods are also available as `getDays()`, `getHours()`, `getMillis()`, `getMinutes()`, and `getSeconds()`.

A float is a floating-point number (i.e. real number) that can be positive or negative.

The following operations are supported for floats:

- `+`, `-`, `*`, `/`: addition, subtraction, multiplication, division
- `**`: exponentiation

note

Floats in Nextflow can be backed by any of the following Java types: `float`, `double`, `Float`, `Double`.

An integer is a whole number that can be positive or negative.

The following operations are supported for integers:

- `+`, `-`, `*`, `/`: addition, subtraction, multiplication, integer division
- `%`: remainder of integer division (i.e. modulus)
- `**`: exponentiation
- `&`, `|`, `^`, `~`: bitwise AND, OR, XOR, NOT
- `<<`, `>>`: left and right bit-shift

note

Integers in Nextflow can be backed by any of the following Java types: `int`, `long`, `Integer`, `Long`.

Implemented by the following types: [stdlib-types-bag](#), [stdlib-types-list](#), [stdlib-types-set](#)

`Iterable` is a trait implemented by collection types that support iteration.

Types that implement `Iterable` can be passed as an `Iterable` parameter of a method, and they can use all of the methods described below.

The following methods are available for iterables:

Returns `true` if any element in the iterable satisfies the given condition.

Returns a new iterable with each element transformed by the given closure.

Transforms each element in the iterable into a collection with the given closure and concatenates the resulting collections into a list.

Returns `true` if the iterable contains the given value.

Invokes the given closure for each element in the iterable.

Returns `true` if every element in the iterable satisfies the given condition.

Returns the elements in the iterable that satisfy the given condition.

Collect the elements of an iterable into groups based on a matching key. The closure should return the key for a given element.

Apply the given accumulator to each element in the iterable and return the final accumulated value. The closure should accept two parameters, corresponding to the current accumulated value and the current iterable element, and return the next accumulated value. The first element from the iterable is used as the initial accumulated value.

Apply the given accumulator to each element in the iterable and return the final accumulated value. The closure should accept two parameters, corresponding to the current accumulated value and the current iterable element, and return the next accumulated value.

Returns `true` if the iterable is empty.

Concatenates the string representation of each element in the iterable, with the given string as the separator between each element.

Returns the maximum element in the iterable.

Returns the maximum element in the iterable according to the given closure. The closure should follow the same semantics as the closure parameter of `toSorted()`.

Returns the maximum element in the iterable according to the given closure. The closure should follow the same semantics as the closure parameter of `toSorted()`.

Returns the minimum element in the iterable.

Returns the minimum element in the iterable according to the given closure. The closure should follow the same semantics as the closure parameter of `toSorted()`.

Returns the minimum element in the iterable according to the given closure. The closure should follow the same semantics as the closure parameter of `toSorted()`.

Returns the number of elements in the iterable.

Returns the sum of the elements in the iterable. The elements should support addition (+).

Transforms each element in the iterable with the given closure and returns the sum. The values returned by the closure should support addition (+).

Converts the iterable to a list.

danger

Converting an unordered collection to a list can lead to non-deterministic behavior. Consider using `toSorted()` instead to ensure a deterministic ordering. See [non-deterministic process inputs](#) for more information.

Converts the iterable to a set. Duplicate elements are excluded.

Returns a sorted list of the iterable's elements.

Returns the iterable as a list sorted according to the given closure. The closure should accept one parameter and transform each element into the value that will be used for comparisons.

Returns the iterable as a list sorted according to the given closure. The closure should accept two parameters and return a negative integer, zero, or a positive integer to denote whether the first argument is less than, equal to, or greater than the second.

Returns a shallow copy of the iterable with duplicate elements excluded.

Returns a shallow copy of the iterable with duplicate elements excluded. The closure should follow the same semantics as the closure parameter of `toSorted()`.

note

Iterables in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

Implements the [stdlib-types-iterable](#) trait.

A list is an ordered collection of values of type `E`. See [Lists](#) for an overview of lists.

The following operations are supported for lists:

Concatenates two lists.

Given a list and an integer `n`, repeats the list `n` times.

Given a list and an index, returns the element at the given index in the list, or `null` if the index is out of range.

Given a value and a list, returns `true` if the list contains the value (or not).

The following methods are available for a list:

Collates the list into a list of sub-lists of length `size`. If `keepRemainder` is `true`, any remaining elements are included as a partial sub-list, otherwise they are excluded.

For example:

```
assert [1, 2, 3, 4, 5, 6, 7].collate(3) == [[1, 2, 3], [4, 5, 6], [7]]
assert [1, 2, 3, 4, 5, 6, 7].collate(3, false) == [[1, 2, 3], [4, 5, 6]]
```

Collates the list into a list of sub-lists of length `size`, stepping through the list `step` elements for each sub-list. If `keepRemainder` is `true`, any remaining elements are included as a partial sub-list, otherwise they are excluded.

For example:

```
assert [1, 2, 3, 4].collate(3, 1) == [[1, 2, 3], [2, 3, 4], [3, 4], [4]]
assert [1, 2, 3, 4].collate(3, 1, false) == [[1, 2, 3], [2, 3, 4]]
```

Returns the first element in the list that satisfies the given condition.

Returns the first element in the list. Raises an error if the list is empty.

Returns the list of integers from 0 to `n - 1`, where `n` is the number of elements in the list.

Equivalent to `first()`.

Returns the index of the first occurrence of the given value in the list, or `-1` if the list does not contain the value.

Returns a shallow copy of the list with the last element excluded.

Returns the last element in the list. Raises an error if the list is empty.

Returns a shallow copy of the list with the elements reversed.

Returns the portion of the list between the given `fromIndex` (inclusive) and `toIndex` (exclusive).

Returns a shallow copy of the list with the first element excluded.

Returns the first `n` elements of the list.

Returns the longest prefix of the list where each element satisfies the given condition.

Returns a list of 2-tuples corresponding to the value and index of each element in the list.

note

Lists in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

A map associates or "maps" keys of type `K` to values of type `V`. Each key can map to at most one value -- a map cannot contain duplicate keys. See [Maps](#) for an overview of maps.

The following operations are supported for maps:

Concatenates two maps. If a key exists in both maps, the mapping from the right-hand side is used.

Given a map and a key, returns the value for the given key in the map, or `null` if the key is not in the map.

Given a key and a map, returns `true` if the map contains the key and the corresponding value is truthy (e.g. not `null`, `0`, or `false`).

The following methods are available for a map:

Returns `true` if any key-value pair in the map satisfies the given condition. The closure should accept two parameters corresponding to the key and value of an entry.

Returns `true` if the map contains a mapping for the given key.

Returns `true` if the map maps one or more keys to the given value.

Invokes the given closure for each key-value pair in the map. The closure should accept two parameters corresponding to the key and value of an entry.

Returns a set of the key-value pairs in the map.

Returns `true` if every key-value pair in the map satisfies the given condition. The closure should accept two parameters corresponding to the key and value of an entry.

Returns `true` if the map is empty.

Returns a set of the keys in the map.

Returns the number of key-value pairs in the map.

Returns a sub-map containing the given keys.

Returns a collection of the values in the map.

note

Maps in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

A `MemoryUnit` represents a quantity of bytes.

A `MemoryUnit` can be created by adding a unit suffix to an integer (e.g. `1.GB`), or by explicitly casting to `MemoryUnit`:

```
// integer with suffix
oneMegabyte = 1.MB

// integer value (bytes)
oneKilobyte = 1024 as MemoryUnit

// string value
oneGigabyte = '1 GB' as MemoryUnit
```

The following suffixes are available:

Unit	Description
B	Bytes
KB	Kilobytes
MB	Megabytes
GB	Gigabytes
TB	Terabytes
PB	Petabytes
EB	Exabytes
ZB	Zettabytes

note

Technically speaking, a kilobyte is equal to 1000 bytes, whereas 1024 bytes is called a "kibibyte" and abbreviated as "KiB", and so on for the other units. In practice, however, kilobyte is commonly understood to mean 1024 bytes, and Nextflow follows this convention in its implementation as well as this documentation.

Memory units can be compared like numbers, and they support basic arithmetic operations:

```
a = 1.GB
b = 2.GB

assert a < b
assert a + a == b
assert b - a == a
assert a * 2 == b
assert b / 2 == a
```

The following methods are available for a `MemoryUnit` object:

Get the memory value in bytes (B).

Get the memory value in gigabytes (rounded down), where 1 GB = 1024 MB.

Get the memory value in kilobytes (rounded down), where 1 KB = 1024 B.

Get the memory value in megabytes (rounded down), where 1 MB = 1024 KB.

Get the memory value in terms of a given unit (rounded down). The unit can be one of: `'B'`, `'KB'`, `'MB'`, `'GB'`, `'TB'`, `'PB'`, `'EB'`, `'ZB'`.

note

These methods are also available as `getBytes()`, `getGiga()`, `getKilo()`, and `getMega()`.

A Path is a handle for hierarchical paths such as local files and directories, HTTP/FTP URLs, and object storage paths (e.g. Amazon S3).

The `file()` function can be used to get a Path for a given filename or URL:

```
def hello = file('hello.txt')
println hello.text
```

The `files()` function can be used to get a collection of Paths from a glob pattern:

```
def inputs = files('*.txt')
inputs.each { input ->
    println "${input.name}: ${input.text}"
}
```

The following sections describe the methods that are available for paths.

note

Paths in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

Operations

The following operations are supported for paths:

Resolves a relative path string against a directory path. Equivalent to `resolve()`.

Appends text to a file without replacing existing content. Equivalent to `append()`.

Filesystem attributes

The following properties are available:

The path name without its extension, e.g. `/some/path/file.tar.gz -> file.tar`.

The path extension, e.g. `/some/path/file.txt -> txt`.

The path name, e.g. `/some/path/file.txt -> file.txt`. For files staged as a task input, the path name is the path relative to the task directory (e.g., `my-dir/file.txt`). Use `fileName.name` for task paths to get only the file name.

The path parent path, e.g. `/some/path/file.txt -> /some/path`.

The path URI scheme, e.g. `s3://some-bucket/hello.txt -> s3`.

The path name without any extension, e.g. `/some/path/file.tar.gz -> file`.

The following methods are available for getting filesystem attributes:

Returns `true` if the path exists.

Returns `true` if the path is a directory.

Gets the path name without any extension, e.g. `/some/path/file.tar.gz -> file`.

Gets the path parent path, e.g. `/some/path/file.txt -> /some/path`.

Gets the path URI scheme, e.g. `s3://some-bucket/hello.txt -> s3`.

Returns `true` if the path is a directory.

Returns `true` if the path is empty or does not exist.

Returns `true` if the path is a file (i.e. not a directory).

Returns `true` if the path is hidden.

Returns `true` if the path is a symbolic link.

Returns the path last modified timestamp in Unix time (i.e. milliseconds since January 1, 1970).

Returns the relative path between this path and the given path.

Resolves the given path string against this path.

Resolves the given path string against this path's parent path.

Gets the file size in bytes.

Gets the file path along with the protocol scheme:

```
def ref = file('s3://some-bucket/hello.txt')

assert ref.toString() == '/some-bucket/hello.txt'
assert "$ref" == '/some-bucket/hello.txt'
assert ref.toUriString() == 's3://some-bucket/hello.txt'
```

Reading

The following methods are available for reading files:

Iterates over the file, applying the specified closure to each line.

Returns the file content as a string.

Reads the file line by line and returns the content as a list of strings.

Invokes the given closure with a [BufferedReader](#), which can be used to read the file one line at a time using the `readLine()` method.

Writing

The following methods are available for writing to files:

Appends text to a file without replacing existing content.

Writes text to a file, replacing any existing content. Equivalent to setting the `text` property.

Writes text to a file, replacing any existing content. Equivalent to `setText()`.

Filesystem operations

The following methods are available for manipulating files and directories in a filesystem:

Copies a source file or directory to a target file or directory.

When copying a file to another file: if the target file already exists, it will be replaced.

```
file('/some/path/my_file.txt').copyTo('/another/path/new_file.txt')
```

When copying a file to a directory: the file will be copied into the directory, replacing any file with the same name.

```
file('/some/path/my_file.txt').copyTo('/another/path')
```

When copying a directory to another directory: if the target directory already exists, the source directory will be copied into the target directory, replacing any sub-directory with the same name. If the target path does not exist, it will be created automatically.

```
file('/any/dir_a').moveTo('/any/dir_b')
```

The result of the above example depends on the existence of the target directory. If the target directory exists, the source is moved into the target directory, resulting in the path `/any/dir_b/dir_a`. If the target directory does not exist, the source is just renamed to the target name, resulting in the path `/any/dir_b`.

note

The `copyTo()` function follows the semantics of the Linux command `cp -r <source> <target>`, with the following caveat: while Linux tools often treat paths ending with a slash (e.g. `/some/path/name/`) as directories, and those not (e.g. `/some/path/name`) as regular files, Nextflow (due to its use of the Java files API) views both of these paths as the same file system object. If the path exists, it is handled according to its actual type (i.e. as a regular file or as a directory). If the path does not exist, it is treated as a regular file, with any missing parent directories created automatically.

Deletes the file or directory at the given path, returning `true` if the operation succeeds, and `false` otherwise:

```
myFile = file('some/file.txt')
result = myFile.delete()
println result ? "OK" : "Cannot delete: $myFile"
```

If a directory is not empty, it will not be deleted and `delete()` will return `false`.

Deletes a directory and all of its contents.

```
file('any/path').deleteDir()
```

Returns a file's permissions using the symbolic notation, e.g. 'rw-rw-r--'.

Returns the first-level elements (files and directories) of a directory as a list of strings.

Returns the first-level elements (files and directories) of a directory as a list of Paths.

Creates a directory at the given path, returning `true` if the directory is created successfully, and `false` otherwise:

```
myDir = file('any/path')
result = myDir.mkdir()
println result ? "OK" : "Cannot create directory: $myDir"
```

If the parent directories do not exist, the directory will not be created and `mkdir()` will return `false`.

Creates a directory at the given path, including any nonexistent parent directories:

```
file('any/path').mkdirs()
```

Creates a filesystem link to a given path:

```
myFile = file('/some/path/file.txt')
myFile.mklink('/user/name/link-to-file.txt')
```

Returns the Path of the link to create.

Available options:

When `true`, creates a hard link, otherwise creates a soft (aka symbolic) link (default: `false`).

When `true`, overwrites any existing file with the same name, otherwise throws a [FileAlreadyExistsException](#) (default: `false`).

Moves a source file or directory to a target file or directory. Follows the same semantics as `copyTo()`.

Rename a file or directory:

```
file('my_file.txt').renameTo('new_file_name.txt')
```

Sets a file's permissions using the [symbolic notation](#):

```
myFile.setPermissions('rwxr-xr-x')
```

Sets a file's permissions using the [numeric notation](#), i.e. as three digits representing the **owner**, **group**, and **other** permissions:

```
myFile.setPermissions(7,5,5)
```

The following methods are available for listing and traversing directories:

Iterates through first-level directories only.

Iterates through directories whose names match the given filter.

Iterates through directories depth-first (regular files are ignored).

Iterates through first-level files and directories.

Iterates through files and directories whose names match the given filter.

Iterates through files and directories depth-first.

Added in version 26.04

Returns the first-level elements (files and directories) in a directory.

Deprecated in version 26.04

Use `listDirectory()` instead.

Returns the first-level elements (files and directories) in a directory.

Splitting files

The following methods are available for splitting and counting the records in files:

Counts the number of records in a [FASTA](#) file. See the [operator-splitfasta](#) operator for available options.

Counts the number of records in a [FASTQ](#) file. See the [operator-splitfastq](#) operator for available options.

Counts the number of records in a JSON file. See the [operator-splitjson](#) operator for available options.

Counts the number of lines in a text file. See the [operator-splittext](#) operator for available options.

Splits a CSV file into a list of records. See the [operator-splitcsv](#) operator for available options.

Splits a [FASTA](#) file into a list of records. See the [operator-splitfasta](#) operator for available options.

Splits a [FASTQ](#) file into a list of records. See the [operator-splitfastq](#) operator for available options.

Splits a JSON file into a list of records. See the [operator-splitjson](#) operator for available options.

Splits a text file into a list of lines. See the [operator-splittext](#) operator for available options.

A record is an immutable map of fields to values (i.e., `Map<String,?>`). Each value can have its own type.

A record can be created using the `record` function:

```
sample = record(id: '1', fastq_1: file('1_1.fastq'), fastq_2: file('1_2.fastq'))
```

Record fields can be accessed as properties:

```
sample.id  
// -> '1'
```

The following operations are supported for records:

Given two records, returns a new record containing the fields and values of both records. When a field is present in both records, the value of the right-hand record takes precedence.

The following methods are available for a record:

Returns a new record containing only the given fields.

Implements the [stdlib-types-iterable](#) trait.

A set is an unordered collection of values of type `E` which cannot contain duplicates.

A set can be created from a list using the `toSet()` method:

```
[1, 2, 2, 3].toSet()  
// -> [1, 2, 3]
```

The following operations are supported for sets:

Given a set and an iterable, returns a new set containing the elements of both collections.

Given a set and an iterable, returns a shallow copy of the set minus the elements of the iterable.

Given a value and a set, returns `true` if the set contains the value (or not).

The following methods are available for a set:

Returns the intersection of the set and the given iterable.

note

Sets in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented here.

A string is an immutable sequence of characters. See [Strings](#) for an overview of strings.

The following operations are supported for strings:

Concatenates two strings.

Given a string and an integer `n`, repeats the string `n` times.

Given a string and an index, returns the character at the given index in the string.

Creates a regular expression from a string. See [Pattern](#) in the Java standard library for more information.

Given a string and a pattern, creates a matcher that is truthy if the pattern occurs anywhere in the string. See [Matcher](#) in the Java standard library for more information.

Given a string and a pattern, returns `true` if the string matches the pattern exactly.

The following methods are available for a string:

Returns `true` if the given substring occurs anywhere in the string.

Returns `true` if the string ends with the given suffix.

Execute the string as a command. Returns a `Process` which provides the exit status and standard input/output/error of the executed command.

Returns the index within the string of the first occurrence of the given substring. Returns -1 if the string does not contain the substring.

Returns the index within the string of the first occurrence of the given substring, starting the search at the given index. Returns -1 if the string does not contain the substring.

Returns `true` if the string is empty or contains only whitespace characters.

Returns `true` if the string is empty (i.e. `length()` is 0).

Returns `true` if the string can be parsed as a 64-bit (double precision) floating-point number.

Returns `true` if the string can be parsed as a 32-bit floating-point number.

Returns `true` if the string can be parsed as a 32-bit integer.

Returns `true` if the string can be parsed as a 64-bit (long) integer.

Returns the index within the string of the last occurrence of the given substring. Returns -1 if the string does not contain the substring.

Returns the index within the string of the last occurrence of the given substring, searching backwards starting at the given index. Returns -1 if the string does not contain the substring.

Returns the length of the string.

Returns the MD5 checksum of the string.

Returns a new string in which each occurrence of the target string is replaced with the given replacement string.

Returns a new string in which each occurrence of the given regular expression is replaced with the given replacement string.

Returns a new string in which the first occurrence of the given regular expression is replaced with the given replacement string.

Returns the SHA-256 checksum of the string.

Returns `true` if the string ends with the given prefix.

Returns a copy of the string with all leading and trailing whitespace removed.

Returns a copy of the string with leading spaces on each line removed. The number of spaces to remove is determined by the line with the least number of leading spaces, excluding lines with only whitespace.

Returns a copy of the string with all leading whitespace removed.

Returns a copy of the string with all trailing whitespace removed.

Returns a substring of this string.

Returns a substring of this string.

Returns `true` if the trimmed string is "true", "y", or "1" (ignoring case).

Parses the string into a 64-bit (double precision) floating-point number.

Parses the string into a 32-bit floating-point number.

Parses the string into a 32-bit integer.

Parses the string into a 64-bit (long) integer.

Returns a copy of this string with all characters converted to lower case.

Returns a copy of this string with all characters converted to upper case.

Splits the string into a list of substrings using the given delimiters. Each character in the delimiter string is treated as a separate delimiter.

note

Strings in Nextflow are backed by the [Java](#) and [Groovy](#) standard libraries, which may expose additional methods. Only methods which are recommended for use in Nextflow are documented [here](#).

A tuple is a fixed sequence of values, where each value can have its own type. Tuples can be created using the `tuple` function.

The following operations are supported for tuples:

Given a tuple and an index, returns the tuple element at the index.

A dataflow value (also known as a value channel) is an asynchronous value of type `V`. It is used to facilitate dataflow logic in a workflow.

See [dataflow-page](#) for an overview of dataflow types.

The following methods are available for a dataflow value:

Transforms the dataflow value into a collection with the given closure and emits the resulting values in a dataflow channel.

Transforms the dataflow value into another dataflow value with the given closure.

Invokes the given closure on the dataflow value.

Prints the dataflow value to standard output.

Transforms the dataflow value using the given closure and print the result to standard output.

Emits the combination of the dataflow value with another dataflow value.

When called on a dataflow value of records with named arguments, the named arguments are appended to each record.

A `VersionNumber` represents a semantic or calendar version number.

The following methods are available for a `VersionNumber`:

Get the major version number, i.e. the first version component.

Get the minor version number, i.e. the second version component.

Get the patch version number, i.e. the third version component.

Check whether the version satisfies a version requirement.

The version requirement string can be prefixed with the usual comparison operators:

- `=` or `==`: equal to
- `<` (`<=`): less than (or equal to)
- `>` (`>=`): greater than (or equal to)
- `!=` or `<>`: not equal

For example:

```
if( !nextflow.version.matches('>=23.10') ) {  
    error "This workflow requires Nextflow version 23.10 or greater -- You are running version $nextflow.version"  
}
```

Multiple constraints can be specified as a comma-separated list, e.g. `>=23.10, <=24.10`.

Alternatively, the version can be suffixed with `+`, which is similar to `==` but also allows the last version part to be greater. For example, `23.10.1+` is satisfied by `23.10.1` and `23.10.2`, but not `23.11.x` or `23.09.x`. Additionally, `23.10.+` is equivalent to `23.10.0+`. This operator is a useful way to enforce a specific version while allowing for newer patch releases.

Groovy and Java classes

Any Groovy or Java class that is available to Nextflow at runtime can be used in a Nextflow script. The following classes are imported by default:

- `groovy.lang.*`
- `groovy.util.*`
- `java.io.*`
- `java.lang.*`
- `java.math.BigDecimal`
- `java.math.BigInteger`
- `java.net.*`
- `java.util.*`

All other classes must be referenced by their fully-qualified name:

```
def vals = [1, 2, 3]
println groovy.json.JsonOutput.toJson(vals)
```

note

The set of classes in Nextflow's runtime classpath can change between different Nextflow versions. As a best practice, any code that uses classes outside the Nextflow standard library should either be refactored to only use the Nextflow standard library or be refactored as a [plugin](#) with explicit dependencies.

Source: <https://docs.seqera.io/nextflow/reference/stdlib-groovy>

Getting started with data lineage

Data lineage in Nextflow provides comprehensive tracking of workflow runs, task executions, and output files. This feature helps you verify the integrity and reproducibility of your pipeline results by maintaining a complete history of computations and intermediate data.

warning

Data lineage is an experimental feature added in Nextflow 25.04. The functionality may change in future releases.

Data lineage tracks the complete history of your Nextflow runs, including:

- Workflow runs and their configurations
- Task executions and their inputs/outputs
- File outputs and their provenance

Each lineage record has a unique identifier called a lineage ID (LID) that you can use to access and query the data.

note

The data model for every lineage record is defined in the Nextflow [source code](#).

To enable data lineage tracking, add the following to your Nextflow configuration:

```
lineage.enabled = true
```

By default, lineage data is stored in the `.lineage` directory in your current working directory. You can customize this location:

```
lineage.store.location = '<PATH_TO_STORAGE>'
```

tip

For global configuration, add these settings to `$HOME/.nextflow/config`.

See the [lineage](#) configuration scope for details.

Run a Nextflow pipeline to generate some lineage metadata. For example:

```
$ nextflow run rnaseq-nf -profile conda
```

Nextflow will automatically record the workflow run, task executions, and output files in the lineage store.

Now that you have generated some lineage metadata, you can explore it from the command line using the `lineage` command.

First, use the `list` subcommand to list the workflow runs in the lineage store:

```
$ nextflow lineage list
TIMESTAMP      RUN NAME      SESSION ID      LINEAGE ID
2025-05-09 13:28:30 CDT peaceful_blackwell 065bdc6b-89b4-42ee-92c1-2a5af37f2c50 lid://
16b31030474f2e96c55f4940bca3ab64
```

The lineage ID (LID) is the unique identifier for the workflow run and the entrypoint for exploring the lineage.

Use the `view` subcommand to view the lineage record for the workflow run:

```
$ nextflow lineage view lid://16b31030474f2e96c55f4940bca3ab64
{
  "version": "lineage/v1beta1",
  "type": "WorkflowRun",
  "workflow": {
    "scriptFiles": [
      ...
    ],
    "repository": "https://github.com/nextflow-io/rnaseq-nf",
    "commitId": "86165b8c81d43a1f57363964431395152e353e56"
  },
  "sessionId": "065bdc6b-89b4-42ee-92c1-2a5af37f2c50",
  "name": "peaceful_blackwell",
  "params": [
    ...
  ],
  "config": {
    ...
  }
}
```

Every workflow run is represented in the lineage store as a `WorkflowRun` record. It includes information such as the pipeline repository, revision, run name, parameters, and resolved config.

The output files of a workflow run can be accessed as `lid://<WORKFLOW_RUN_HASH>/<PATH>`, where `<PATH>` is the file path relative to the workflow output directory.

note

Files must be published to the workflow output directory as defined by the `outputDir` config option (or `-output-dir` command line option) in order to be recorded as workflow outputs in the lineage store.

List the output directory to see the available files:

```
$ find results
results
results/fastqc_ggal_gut_logs
results/fastqc_ggal_gut_logs/ggal_gut_1_fastqc.html
results/fastqc_ggal_gut_logs/ggal_gut_1_fastqc.zip
results/fastqc_ggal_gut_logs/ggal_gut_2_fastqc.html
results/fastqc_ggal_gut_logs/ggal_gut_2_fastqc.zip
results/multiqc_report.html
```

Now, use the workflow LID and relative path to view the lineage record for an output file:

```
$ nextflow lineage view lid://16b31030474f2e96c55f4940bca3ab64/multiqc_report.html
{
  "version": "lineage/v1beta1",
  "type": "FileOutput",
  "path": "/results/multiqc_report.html",
  "checksum": {
    "value": "03fd5ed150c7862e1fad5efd4f574a47",
    "algorithm": "nextflow",
    "mode": "standard"
  },
  "source": "lid://862df53160e07cd823c0c3960545e747/multiqc_report.html",
  "workflowRun": "lid://16b31030474f2e96c55f4940bca3ab64",
  "taskRun": null,
  "size": 5079806,
  "createdAt": "2025-05-09T13:27:34.576590545-05:00",
  "modifiedAt": "2025-05-09T13:27:34.586590551-05:00",
  "labels": null
}
```

Every output file is represented in the lineage store as a `FileOutput` record. It includes basic file information, such as the real path, checksum, file size and created/modified timestamps, as well as lineage information, such as the workflow run and task run that produced it.

As this record is a workflow output, it is not linked directly to a task run. Instead, it is linked to the original task output.

note

The `labels` field is `null` because no labels were assigned to this file. Labels are set using the `label` directive in the `output` block.

Any LID in a lineage record can be viewed, allowing you to traverse the lineage metadata interactively. Use the value of `source` to view the original task output:

```
$ nextflow lineage view lid://862df53160e07cd823c0c3960545e747/multiqc_report.html
{
  "version": "lineage/v1beta1",
  "type": "FileOutput",
  "path": "/work/86/2df53160e07cd823c0c3960545e747/multiqc_report.html",
  "checksum": {
    "value": "b14f5171a48ce5c22ea27d7b8e57b6c4",
    "algorithm": "nextflow",
    "mode": "standard"
  },
  "source": "lid://862df53160e07cd823c0c3960545e747",
  "workflowRun": "lid://16b31030474f2e96c55f4940bca3ab64",
  "taskRun": "lid://862df53160e07cd823c0c3960545e747",
  "size": 5079806,
  "createdAt": "2025-05-09T13:27:34.236590379-05:00",
  "modifiedAt": "2025-05-09T13:27:34.246590383-05:00",
  "labels": null
}
```

This record is the task output for the same file -- it has a value for `taskRun` which is the same as its `source`.

View the lineage record for the task that produced this file:

```
$ nextflow lineage view lid://862df53160e07cd823c0c3960545e747
{
  "version": "lineage/v1beta1",
  "type": "TaskRun",
  "sessionId": "065bdc6b-89b4-42ee-92c1-2a5af37f2c50",
  "name": "MULTIQC",
  "codeChecksum": {
    "value": "edf2e9f84cd3a18ee9259012b660f2dd",
    "algorithm": "nextflow",
    "mode": "standard"
  },
  "script": "\n  cp multiqc/* .\n  echo \"custom_logo: $PWD/nextflow_logo.png\" \u003e multiqc_config.yaml\n  multiqc -n multiqc_report.html .\n  ",
  "input": [
    {
      "type": "path",
      "name": "**",
      "value": [
        "lid://eff8846883b46c5a76f11e7e4480a6c8/ggal_gut",
        "lid://2d8bd92c69f732605bc99941e60d5319/fastqc_ggal_gut_logs"
      ]
    },
    {
      "type": "path",
      "name": "config",
      "value": [
        {
          "path": "https://github.com/nextflow-io/rnaseq-nf/tree/86165b8c81d43a1f57363964431395152e353e56/multiqc",
          "checksum": {
            "value": "2aac500cdfb292e961e678433e7dc3d8",
            "algorithm": "nextflow",
            "mode": "standard"
          }
        }
      ]
    }
  ],
  "container": null,
  "conda": "file:///conda/env-4a436c230263dfdbbf4ddddd0623505d1",
  "spack": null,
  "architecture": null,
  "globalVars": {},
  "binEntries": [],
  "workflowRun": "lid://16b31030474f2e96c55f4940bca3ab64"
}
```

Every task run is represented in the lineage store as a `TaskRun`, which includes information such as the name, script, inputs, and software dependencies. From here, you can continue traversing through the file inputs to view upstream tasks.

Finally, use the `render` subcommand to render the entire lineage of the MULTIQC report as an HTML report:

```
$ nextflow lineage render lid://16b31030474f2e96c55f4940bca3ab64/multiqc_report.html
Rendered lineage graph for lid://16b31030474f2e96c55f4940bca3ab64/multiqc_report.html to lineage.html
```

Open the HTML report in a web browser to view the lineage graph.

To find a lineage record, you normally have to know the LID of the record or a downstream record (such as a workflow run) from which you can traverse to the desired record. However, you can also query the entire lineage store by fields to quickly find relevant records and aggregate records from different runs.

Use the `find` subcommand to find all tasks executed by a workflow run:

```
$ nextflow lineage find type=TaskRun workflowRun=lid://16b31030474f2e96c55f4940bca3ab64
[
  "lid://2d8bd92c69f732605bc99941e60d5319",
  "lid://eff8846883b46c5a76f11e7e4480a6c8",
  "lid://862df53160e07cd823c0c3960545e747",
  "lid://6d3bff36bf2c3c14c2d383384621e8ca"
]
```

You can use any field defined in the [lineage data model](#).

tip

Since the `find` and `view` subcommands always output JSON, you can use JSON processing tools such as `jq` to further query and transform results.

Task run LIDs are based on the standard [task hash](#), which makes it easy to compare two task runs in the lineage metadata. For example, if a task is unexpectedly re-executed during a resumed run, as long as lineage is enabled for both the initial and resumed runs, the two tasks can be compared without any additional runs.

This section builds on the above [rnaseq-nf example](#) to demonstrate how to compare two task runs in the event of a cache invalidation.

First, modify the pipeline in a way that invalidates the cache for the `MULTIQC` process. For example, modify the process script.

Resume the pipeline. It will re-execute the `MULTIQC` process:

```
$ nextflow run rnaseq-nf -profile conda -resume

...

[6d/3bff36] process > RNASEQ:INDEX (ggal_1_48850000_49020000) [100%] 1 of 1, cached: 1 ✓
[2d/8bd92c] process > RNASEQ:FASTQC (FASTQC on ggal_gut) [100%] 1 of 1, cached: 1 ✓
[ef/f88468] process > RNASEQ:QUANT (ggal_gut) [100%] 1 of 1, cached: 1 ✓
[94/33dda7] process > MULTIQC [100%] 1 of 1 ✓
```

Retrieve the hash of the `MULTIQC` run from the log file or work directory. Compare it to the task hash of the initial run:

```
$ nextflow lineage diff lid://862df53160e07cd823c0c3960545e747 lid://9433dda73f2193491f9a26e3e23cd8a1
diff --git 862df53160e07cd823c0c3960545e747 9433dda73f2193491f9a26e3e23cd8a1
--- 862df53160e07cd823c0c3960545e747
+++ 9433dda73f2193491f9a26e3e23cd8a1
@@ -3,11 +3,11 @@
  "sessionId": "065bdc6b-89b4-42ee-92c1-2a5af37f2c50",
  "name": "MULTIQC",
  "codeChecksum": {
-   "value": "edf2e9f84cd3a18ee9259012b660f2dd",
+   "value": "9615a8da3a3f9e935cfc8e4042cdf5e0",
    "algorithm": "nextflow",
    "mode": "standard"
  },
-  "script": "\n    cp multiqc/* .\n    echo \"custom_logo: $PWD/nextflow_logo.png\" \u003e\u003e
multiqc_config.yaml\n    multiqc -n multiqc_report.html .\n    ",
+  "script": "\n    cp multiqc/* . # hello!\n    echo \"custom_logo: $PWD/nextflow_logo.png\" \u003e\u003e
multiqc_config.yaml\n    multiqc -n multiqc_report.html .\n    ",
  "input": [
    {
      "type": "path",
@@ -38,5 +38,5 @@
  "architecture": null,
  "globalVars": {},
  "binEntries": [],
-  "workflowRun": "lid://16b31030474f2e96c55f4940bca3ab64"
+  "workflowRun": "lid://65044872aad36f97e42336b9ba0dee57"
  }
}
```

Note the difference between the task scripts, highlighting the change that caused the task to be re-executed.

Workflow outputs declared in the `output` block are also recorded in the lineage store. The output of a workflow run is accessible as `lid://<WORKFLOW_RUN_HASH>#output`.

For example, run the `rnaseq-nf` pipeline with the `preview-25-04` branch, which uses the `output` block to publish outputs:

```
$ nextflow run rnaseq-nf -r preview-25-04 -profile conda
```

View the workflow output in the lineage metadata:

```
$ nextflow lineage view lid://9410d13abeec617640b5fe9735ba12fc#output
[
  {
    "type": "Collection",
    "name": "samples",
    "value": "lid://9410d13abeec617640b5fe9735ba12fc/samples.json"
  },
  {
    "type": "Path",
    "name": "summary",
    "value": "lid://9410d13abeec617640b5fe9735ba12fc/multiqc_report.html"
  }
]
```

This view can be used to traverse output files directly instead of inferring LIDs from the workflow output directory.

Type	JSON representation	Nextflow type
Collection	array (or index file path)	Bag, List, Set
Map	object	Map, Record
Path	string with <code>lid://</code> prefix	Path

When labels are assigned to a workflow output with the `label` directive, they appear in the `labels` field of each corresponding `FileOutput` record:

```
$ nextflow lineage view lid://9410d13abeec617640b5fe9735ba12fc/multiqc_report.html
{
  "version": "lineage/v1beta1",
  "type": "FileOutput",
  "path": "/results/multiqc_report.html",
  ...
  "labels": ["qc", "summary"]
}
```

Labels can be used to filter files when querying lineage records with the `nextflow lineage find` command.

See [Labels](#) for details on assigning labels to workflow outputs.

Since lineage IDs are valid URIs, output files in the lineage store can be accessed by their LID in a Nextflow script, like any other path. The LID path returns the real path as defined by the `path` field in the `FileOutput` record.

The following script uses the `samples.json` from the previous example as an input samplesheet:

```
channel.fromPath('lid://9410d13abeec617640b5fe9735ba12fc/samples.json')
    .splitJson()
    .view()
```

It should produce the following output:

```
[id:gut, quant:/results/gut/quant, fastqc:/results/gut/fastqc]
```

The `fromLineage` channel factory can also be used to query lineage records in a similar manner as the `find` subcommand. See [fromLineage](#) for details.

Migrating to workflow outputs

The [workflow output definition](#) is a new way to define the top-level outputs of a workflow. It is a replacement for the `publishDir` directive. This tutorial demonstrates how to migrate from `publishDir` to workflow outputs using the `rnaseq-nf` pipeline as an example.

note

Workflow outputs are stable in Nextflow 25.10. However, for large pipelines, it is recommended that you migrate to static typing and records first. See [Migrating to static typing](#) for more information.

In Nextflow DSL1, pipelines were defined in a single script and there was no concept of workflows. Each process used the `publishDir` directive to publish task outputs, which captured output files with glob patterns and copied them from the work directory to an external location.

Nextflow DSL2 introduced workflows and modules, making it easier to develop large and complex pipelines. However, DSL2 retained the same process-based publishing syntax and became unwieldy for several reasons:

- **Mismatch with reusable modules:** Publishing rules often depend on how a process is used in a pipeline. This made it impractical to set `publishDir` in a reusable way for processes that are shared across many pipelines. Publishing rules could be defined in the configuration, but this approach requires extensive use of [process selectors](#), which are difficult to use for large pipelines.
- **Fragmented outputs:** It is difficult to get a concise view of a workflow's outputs when publishing rules are separated across many different modules.
- **Redundant configuration:** Certain settings, such as the base output directory and publish mode, must be repeated for each `publishDir` declaration, leading to duplicated code.
- **Mismatch with channels:** Channels, the primary data structure in Nextflow, contain files and associated metadata that can be accessed by name. However, `publishDir` uses glob patterns to match files, and cannot publish metadata unless it happens to be in a file. This mismatch makes it difficult to translate channels into pipeline outputs.

Workflow outputs were introduced to address these problems by providing a unified, structured, and flexible way to publish outputs:

- **Unified output definition:** Workflow outputs are declared in an `output` block alongside the entry workflow, ensuring that there is a single comprehensive view of what a pipeline produces.
- **Channel-based publishing:** Instead of publishing files from individual processes, workflow outputs are assigned from channels in the entry workflow. The channel itself can be saved as an index file, such as a CSV or JSON file, which provides a structured view of the output directory and can be ingested by downstream pipelines.
- **Flexible file selection:** By default, all files in a published channel are included. However, the published channel can be configured to publish specific files by name, instead of using glob patterns. This approach to publishing files is a natural extension of workflows and channels.

- **Simple configuration:** The base output directory is defined as a global configuration setting, and all files are published into this directory. Publish settings such as the mode are also defined as configuration settings under the `workflow.output` scope, reducing code duplication.

Workflow outputs are stable in Nextflow [25.10](#).

This feature is also available as a preview in Nextflow [24.04](#), [24.10](#), and [25.04](#). If you are upgrading from a preview version of workflow outputs, you must remove the `nextflow.preview.output` feature flag, as it is not supported in Nextflow 25.10.

The `publishDir` directive will continue to be supported, but will be deprecated. It may be removed in a future release.

This section describes how to migrate from `publishDir` to workflow outputs using the `rnaseq-nf` pipeline (v2.4) as an example. To view the completed migration, see the [preview-25-10](#) branch of the `rnaseq-nf` repository.

See [Getting started with rnaseq-nf](#) for an introduction to the `rnaseq-nf` pipeline.

Start by removing each `publishDir` directive and publishing the corresponding process output channel in the entry workflow.

Declare an output for each channel in the `output` block and publish the corresponding channel in the `publish:` section of the entry workflow:

```
workflow {
  main:
    read_pairs_ch = channel.fromFilePairs(params.reads, checkIfExists: true, flat: true)

    (fastqc_ch, quant_ch) = RNASEQ(read_pairs_ch, params.transcriptome)

    multiqc_files_ch = fastqc_ch.mix(quant_ch).collect()

    multiqc_report = MULTIQC(multiqc_files_ch, params.multiqc)

    publish:
      fastqc_logs = fastqc_ch
      multiqc_report = multiqc_report
}

output {
  fastqc_logs {
  }

  multiqc_report {
  }
}
```

note

Each output assigned in the `publish:` section must be declared in the `output` block, and vice versa.

Nextflow copies all files in the published channels into the output directory, which is `results` by default. You can set the output directory using the `outputDir` config setting or the `-output-dir` command-line option.

You can set the publish mode in the config. For example:

```
workflow.output.mode = 'copy'
```

Run the pipeline with the `all-reads` profile to verify the published outputs:

```
$ nextflow run . -profile conda,all-reads
```

The pipeline runs `FASTQC` and `QUANT` for each input sample. However, the workflow publishes only the `FASTQC` results. The workflow passes the `QUANT` results to `MULTIQC` but doesn't publish them directly.

Improve the workflow outputs by also publishing the outputs of `QUANT`:

```
workflow {
  main:
  // ...

  publish:
  fastqc_logs = fastqc_ch
  quant = quant_ch
  multiqc_report = multiqc_report
}

output {
  fastqc_logs {
  }

  quant {
  }

  multiqc_report {
  }
}
```

Running the pipeline with the `all-reads` profile will produce the following output directory:

```
results
├── fastqc_gut_logs
├── fastqc_liver_logs
├── fastqc_lung_logs
├── fastqc_spleen_logs
├── multiqc_report.html
├── quant_gut
├── quant_liver
├── quant_lung
└── quant_spleen
```

This directory will quickly become cluttered as you process more samples. It would be better to group the `FASTQC` and `QUANT` results into separate subdirectories:

```
results
├── fastqc
│   ├── gut
│   ├── liver
│   ├── lung
│   └── spleen
├── multiqc_report.html
└── quant
    ├── gut
    ├── liver
    ├── lung
    └── spleen
```

Achieve this directory structure by customizing the `output` block.

First, update the `FASTQC` and `QUANT` processes to also emit the sample ID alongside the output files:

```

process FASTQC {
  // ...

  output:
  tuple val(id), path("fastqc_${id}")

  // ...
}

process QUANT {
  // ...

  output:
  tuple val(id), path("quant_${id}")

  // ...
}

```

Configure the `fastqc_logs` and `quant` outputs in the `output` block to use dynamic publish paths:

```

output {
  fastqc_logs {
    path { id, fastqc -> "fastqc/${id}" }
  }

  quant {
    path { id, quant -> "quant/${id}" }
  }

  multiqc_report {
  }
}

```

Preserving the sample ID in each output channel allows you to customize the publish path without trying to parse the file name. The dynamic path is applied to each channel value to determine the target name for the given file.

note

The closure parameters for the dynamic publish path must match the structure of the published channel.

An index file is a manifest, or index, of the published files and their metadata for a workflow output. Nextflow can create an index file for each workflow output by saving the channel as a CSV, JSON, or YAML file.

For example, if you enable the index file for `fastqc_logs`:

```

output {
  fastqc_logs {
    path { id, fastqc -> "fastqc/${id}" }
    index {
      path 'fastqc.csv'
      header true
    }
  }
}

```

The workflow produces the following index file:

```
$ cat results/fastqc.csv
"id","fastqc"
"lung","results/fastqc/lung"
"spleen","results/fastqc/spleen"
"gut","results/fastqc/gut"
"liver","results/fastqc/liver"
```

The index file mirrors the structure of the published channel, and it provides a structured view of the output directory. Index files are equivalent to samplesheets, and can be used as inputs to downstream pipelines.

You could define two index files for `fastqc_logs` and `quant`. However, since these outputs essentially provide different slices of data for the same set of samples, you can also combine them into a single output with one index file.

Use the `join` operator to combine the `FASTQC` and `QUANT` results into a single channel:

```
workflow {
  main:
  // ...

  samples_ch = fastqc_ch
    .join(quant_ch)
    .map { id, fastqc, quant ->
      [id: id, fastqc: fastqc, quant: quant]
    }

  multiqc_files_ch = samples_ch
    .flatMap { sample -> [sample.fastqc, sample.quant] }
    .collect()
  multiqc_report = MULTIQC( multiqc_files_ch, params.multiqc )

  publish:
  samples = samples_ch
  multiqc_report = multiqc_report
}
```

This example uses maps instead of tuples so that you can access fields by name, and so that the index file can use the map keys as column names.

Declare the `samples` output with an index file:

```
output {
  samples {
    path { sample ->
      sample.fastqc >> "fastqc/${sample.id}"
      sample.quant >> "quant/${sample.id}"
    }
    index {
      path 'samples.csv'
      header true
    }
  }

  multiqc_report {
  }
}
```

Since each channel value now contains multiple files that go to different subdirectories, you must use publish statements in the `path` directive to route each file to the appropriate location.

Run the pipeline, then verify the index file:

```
$ nextflow run . -profile conda,all-reads -resume
$ cat results/samples.csv
"id","fastqc","quant"
"lung","results/fastqc/lung","results/quant/lung"
"gut","results/fastqc/gut","results/quant/gut"
"liver","results/fastqc/liver","results/quant/liver"
"spleen","results/fastqc/spleen","results/quant/spleen"
```

In the future, if you add a tool with per-sample outputs, you only need to join the tool output into the `samples_ch` channel and update the output `path` directive accordingly. This approach keeps the output definition concise as you add more tools to the pipeline. Additionally, a single unified index file for all per-sample outputs is easier for downstream pipelines to consume, rather than cross-referencing multiple related index files.

Migrating to static typing

Nextflow 26.04 brings full support for static typing in Nextflow code. This tutorial demonstrates how to migrate to static typing using the [rnaseq-nf](#) pipeline as an example.

note

Static typing is optional. All existing code will continue to work.

Static typing allows you to precisely model and validate the structure of your data as it flows through your pipeline. It consists of several new language features:

- **Type annotations** can be added to inputs and outputs at every level of a pipeline, from pipeline parameters to process inputs and outputs, using the [standard Nextflow types](#). These annotations make your code easier to understand and are used by the Nextflow language server to identify type-related errors during development.
- **Records** are a new data structure for modeling composite data. They serve as an alternative to tuples -- whereas tuple elements must be accessed by index, record fields are accessed by name. This allows you to model data with meaningful names instead of keeping track of how tuple elements are ordered.
- **Record types** are custom type definitions that can be used to guarantee a minimum set of requirements for a record in a particular context. Records are duck-typed, which means that a record can be used as an input as long as it meets the minimum requirements of that input (given by a record type).

Static typing works best with the [Nextflow language server](#) and [Nextflow VS Code extension](#).

tip

See [Environment setup](#) for instructions on how to setup VS Code and the Nextflow extension.

When using static typing, the language server can check your code for type-related errors. For example, it can validate that a channel of records has all the required fields when it is passed as input to a process.

The language server performs type checking on every script that enables the `nextflow.enable.types` feature flag.

The Nextflow VS Code extension provides a command for automatically migrating Nextflow pipelines to static types. To migrate a script, open the Command Palette, search for **Convert script to static types**, and select it.

note

Automatic migration is an experimental feature and may not be able to convert an entire pipeline to static types. Always review generated code for correctness.

This section demonstrates how to migrate a pipeline to static typing using [rnaseq-nf](#) (v2.4) as an example. See [Getting started with rnaseq-nf](#) for an introduction to the pipeline.

The approach is as follows:

1. Convert legacy parameters to a `params` block
2. Convert the primary input (`params.reads`) from a glob pattern to a samplesheet
3. Convert each process to static typing
4. Convert each workflow to static typing

The completed migration is available in the [preview-26-04](#) branch.

The pipeline defines the following parameters in the main script using the legacy syntax:

```
params.reads = "$baseDir/data/ggal/ggal_gut_{1,2}.fq"
params.transcriptome = "$baseDir/data/ggal/ggal_1_48850000_49020000.Ggal71.500bpflank.fa"
params.outdir = "results"
params.multiqc = "$baseDir/multiqc"
```

The pipeline also has a `nextflow_schema.json` schema with the following properties:

```
"reads": {
  "type": "string",
  "description": "The input read-pair files",
  "default": "${projectDir}/data/ggal/ggal_gut_{1,2}.fq"
},
"transcriptome": {
  "type": "string",
  "format": "file-path",
  "description": "The input transcriptome file",
  "default": "${projectDir}/data/ggal/ggal_1_48850000_49020000.Ggal71.500bpflank.fa"
},
"outdir": {
  "type": "string",
  "format": "directory-path",
  "description": "The output directory where the results will be saved",
  "default": "results"
},
"multiqc": {
  "type": "string",
  "format": "directory-path",
  "description": "Directory containing the configuration for MultiQC",
  "default": "${projectDir}/multiqc"
}
```

To migrate the pipeline parameters, use the schema and legacy parameters to define the equivalent `params` block:

```
params {
  // The input read-pair files
  reads: String = "${projectDir}/data/ggal/ggal_gut_{1,2}.fq"

  // The input transcriptome file
  transcriptome: Path = "${projectDir}/data/ggal/ggal_1_48850000_49020000.Ggal71.500bpflank.fa"

  // The output directory where the results will be saved
  outdir: Path = 'results'

  // Directory containing the configuration for MultiQC
  multiqc: Path = "${projectDir}/multiqc"
}
```

See [Parameters](#) for more information about the `params` block.

note

Parameters used only in the config file should be declared in the config, not in the script. Since `rnaseq-nf` has no such parameters, all parameters are declared in the script. See [Parameters](#) for more information.

tip

The `rnaseq-nf` pipeline initializes the `reads` and `transcriptome` parameters to a test dataset by default, as it is designed as a toy example. In practice, defaults for test data should be defined in a config profile (e.g., `test`).

The `rnaseq-nf` pipeline takes a glob pattern of FASTQ pairs (e.g., `data/ggal/ggal_gut_{1,2}.fq`) and uses the `channel.fromFilePairs()` factory to load the files as a channel of tuples:

```
read_pairs_ch = channel.fromFilePairs(params.reads, checkIfExists: true, flat: true)
```

Each tuple has three elements -- the sample ID (inferred from the file names) and the two FASTQ files.

This approach will not work with static typing because `fromFilePairs()` does not have a well-defined return type. A more robust way to model a collection of samples is with a samplesheet, such as a CSV file specifying samples as rows and sample fields as columns.

Create the following samplesheet to represent the test data:

data/allreads.csv

```
id,fastq_1,fastq_2
ggal_gut,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_gut_1.fq,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_gut_2.fq
ggal_liver,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_liver_1.fq,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_liver_2.fq
ggal_lung,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_lung_1.fq,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_lung_2.fq
ggal_spleen,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_spleen_1.fq,https://raw.githubusercontent.com/nextflow-io/rnaseq-nf/refs/heads/master/data/ggal/
ggal_spleen_2.fq
```

Refactor `params.reads` to refer to the samplesheet file path instead of a glob pattern:

```
params {
  // The input samplesheet of paired-end reads
  reads: Path = "${projectDir}/data/allreads.csv"

  // ...
}
```

Refactor the `read_pairs_ch` to load the samplesheet as a channel of records:

```
read_pairs_ch = channel.of(params.reads)
  .flatMap { csv -> csv.splitCsv() }
  .map { row ->
    record(id: row[0], fastq_1: file(row[1]), fastq_2: file(row[2]))
  }
```

See [Processes \(typed\)](#) for an overview of typed processes.

note

You must enable the `nextflow.enable.types` feature flag in each script that uses typed processes.

FASTQC

The `FASTQC` process is defined as follows:

```
process FASTQC {
  tag id
  conda 'bioconda::fastqc=0.12.1'

  input:
  tuple val(id), path(fastq_1), path(fastq_2)

  output:
  path "fastqc_${id}_logs"

  script:
  """
  fastqc.sh "${id}" "${fastq_1}" "${fastq_2}"
  """
}
```

To migrate the `FASTQC` process, rewrite the inputs and outputs as follows:

```
nextflow.enable.types = true

process FASTQC {
  tag id
  conda 'bioconda::fastqc=0.12.1'

  input:
  record(
    id: String,
    fastq_1: Path,
    fastq_2: Path
  )

  output:
  record(
    id: id,
    fastqc: file("fastqc_${id}_logs")
  )

  script:
  """
  fastqc.sh "${id}" "${fastq_1}" "${fastq_2}"
  """
}
```

The tuple input is converted to a record input using the `record()` destructor. The field types are specified alongside the field names. The `path` input qualifier is replaced by the `Path` type.

Whereas tuple elements must be specified in a particular order, record fields can be specified in any order. The records supplied by the calling workflow must have the same field names and types as the process definition.

The tuple output is converted to a record using the `record()` function and specifying a name for each record field. The `path` output qualifier is replaced by the `file()` function (or `files()` if multiple files are expected). See [process outputs](#) for the list of special functions that can be used in the `output:` section to retrieve task outputs.

QUANT

The `QUANT` process is defined as follows:

```
process QUANT {
  tag id
  conda 'bioconda::salmon=1.10.3'

  input:
  tuple val(id), path(fastq_1), path(fastq_2)
  path index

  output:
  path "quant_${id}"

  script:
  """
  salmon quant \
    --threads ${task.cpus} \
    --libType=U \
    -i ${index} \
    -1 ${fastq_1} \
    -2 ${fastq_2} \
    -o quant_${id}
  """
}
```

To migrate the `QUANT` process, rewrite the inputs and outputs as follows:

```

nextflow.enable.types = true

process QUANT {
  tag id
  conda 'bioconda::salmon=1.10.3'

  input:
  record(
    id: String,
    fastq_1: Path,
    fastq_2: Path
  )
  index: Path

  output:
  record(
    id: id,
    quant: file("quant_${id}")
  )

  script:
  """
  salmon quant \
    --threads ${task.cpus} \
    --libType=U \
    -i ${index} \
    -1 ${fastq_1} \
    -2 ${fastq_2} \
    -o quant_${id}
  """
}

```

MULTIQC

The MULTIQC process is defined as follows:

```

process MULTIQC {
  conda 'bioconda::multiqc=1.27.1'

  input:
  path '*'
  path config

  output:
  path 'multiqc_report.html'

  script:
  """
  cp ${config}/* .
  echo "custom_logo: \${PWD}/nextflow_logo.png" >> multiqc_config.yaml
  multiqc -n multiqc_report.html .
  """
}

```

To migrate this process, rewrite the inputs and outputs as follows:

```

nextflow.enable.types = true

process MULTIQC {
    // ...

    input:
    logs: Bag<Path>
    config: Path

    // stage:
    // stageAs logs, '*'

    output:
    file('multiqc_report.html')

    // ...
}

```

In a typed process, file patterns for `path` inputs must be declared using a stage directive. In this example, the first input uses the variable name `logs`, and the `stageAs` directive stages the input using the glob pattern `*`.

In this case, you can omit the stage directive because `*` matches Nextflow's default staging behavior. Inputs of type `Path` or a `Path` collection (e.g., `Bag<Path>`) are staged by default using the pattern `'*'`.

note

In a legacy process, you can use the `arity` option to specify whether a `path` qualifier expects a single file or collection of files. When using typed inputs and outputs, the type determines this behavior, i.e., `Path` vs `Bag<Path>`.

INDEX

Apply the same migration principles from the previous processes to migrate `INDEX`.

Once you migrate every process called by a workflow to static typing, you can migrate the workflow itself.

See [Workflows \(typed\)](#) for an overview of typed workflows.

note

You must enable the `nextflow.enable.types` feature flag in each script that uses typed workflows.

RNASEQ

The `RNASEQ` workflow is defined as follows:

```

workflow RNASEQ {
  take:
    read_pairs_ch
    transcriptome

  main:
    index = INDEX(transcriptome)
    fastqc_ch = FASTQC(read_pairs_ch)
    quant_ch = QUANT(index, read_pairs_ch)

  emit:
    fastqc = fastqc_ch
    quant = quant_ch
}

```

You can infer the type of each workflow input by examining how the workflow is called. In this case, `RNASEQ` is called by the entry workflow with the following arguments:

```

workflow {
  read_pairs_ch = channel.of(params.reads)
    .flatMap { csv -> csv.splitCsv() }
    .map { row ->
      record(id: row[0], fastq_1: file(row[1]), fastq_2: file(row[2]))
    }

  RNASEQ(read_pairs_ch, params.transcriptome)

  // ...
}

```

You can determine the type of each input as follows:

- The channel `read_pairs_ch` has type `Channel<Record>`, where each record contains the fields `id`, `fastq_1`, `fastq_2`.
- The parameter `params.transcriptome` has type `Path` as defined in the `params` block.

Specify the workflow input types as follows:

```

nextflow.enable.types = true

workflow RNASEQ {
  take:
    read_pairs_ch: Channel<Sample>
    transcriptome: Path

  // ...
}

record Sample {
  id: String
  fastq_1: Path
  fastq_2: Path
}

```

The `read_pairs_ch` channel also needs to provide all of the record fields required by downstream processes. It is used by `FASTQC` and `QUANT`, which both declare the following record input:

```

input:
  record(
    id: String,
    fastq_1: Path,
    fastq_2: Path
  )

```

The `Sample` record type contains all of the required fields.

note

In this case, the records in `read_pairs_ch` are identical to the record inputs of `FASTQC` and `QUANT`. However, `read_pairs_ch` would still be compatible if it contained additional record fields, as long as it contains the fields required by the two processes.

The `FASTQC` and `QUANT` processes produce the channels `fastqc_ch` and `quant_ch`, both of which have type `Channel<Record>`:

- `fastqc_ch` contains records with the fields `id` and `fastqc`
- `quant_ch` contains records with the fields `id` and `quant`

You can infer this type information from the respective process outputs, as shown in the previous section.

These channels are emitted as the outputs of `RNASEQ`. However, with records it is usually simpler to join related channels into a single channel (e.g., to publish the channel as a [workflow output](#)).

Use the `join` operator to join `fastqc_ch` and `quant_ch` by sample ID:

```
nextflow.enable.types = true

workflow RNASEQ {
  take:
  read_pairs_ch: Channel<Sample>
  transcriptome: Path

  main:
  index = INDEX(transcriptome)
  fastqc_ch = FASTQC(read_pairs_ch)
  quant_ch = QUANT(read_pairs_ch, index)
  samples_ch = fastqc_ch.join(quant_ch, by: 'id')

  // ...
}
```

Finally, the workflow needs to be updated to only emit the `samples_ch` channel. Type annotations are not required for emits, but they are still useful as documentation and as a sanity check -- if the declared output type doesn't match the assigned value's type, the language server will report it.

While `samples_ch` could be emitted as type `Channel<Record>`, the best practice to use an explicit record type so that downstream workflows know which record fields are available.

Define a new record type based on the available fields in `samples_ch`:

```
record AlignedSample {
  id: String
  fastqc: Path
  quant: Path
}
```

Update the workflow to emit `samples_ch` with the new record type:


```

nextflow.enable.types = true

workflow RNASEQ {
  take:
  read_pairs_ch: Channel<Sample>
  transcriptome: Path

  main:
  index = INDEX(transcriptome)
  fastqc_ch = FASTQC(read_pairs_ch)
  quant_ch = QUANT(read_pairs_ch, index)
  samples_ch = fastqc_ch.join(quant_ch, by: 'id')

  emit:
  samples: Channel<AlignedSample> = samples_ch
}

```

Entry workflow

The entry workflow is defined as follows:

```

workflow {
  read_pairs_ch = channel.of(params.reads)
  .flatMap { csv -> csv.splitCsv() }
  .map { row ->
    record(id: row[0], fastq_1: file(row[1]), fastq_2: file(row[2]))
  }

  (fastqc_ch, quant_ch) = RNASEQ(read_pairs_ch, params.transcriptome)

  multiqc_files_ch = fastqc_ch.mix(quant_ch).collect()

  MULTIQC(multiqc_files_ch, params.multiqc)
}

```

Rewrite this workflow based on the updated params, processes, and subworkflows:

```

nextflow.enable.types = true

workflow {
  read_pairs_ch = channel.of(params.reads)
  .flatMap { csv -> csv.splitCsv() }
  .map { row ->
    record(id: row[0], fastq_1: file(row[1]), fastq_2: file(row[2]))
  }

  samples_ch = RNASEQ(read_pairs_ch, params.transcriptome)

  multiqc_files_ch = samples_ch
  .flatMap { id, fastqc, quant -> [fastqc, quant] }
  .collect()

  MULTIQC(multiqc_files_ch, params.multiqc)
}

```

The `reads` param was refactored as a `Path`, so it is loaded into a channel of records using `splitCsv`. It is compatible with the records expected by `RNASEQ`.

The `RNASEQ` workflow now returns a single combined channel, so the `mix` operation is no longer needed. The `flatMap` operator is used to extract the files from each record in `samples_ch`.

While static typing can be adopted progressively with existing code, many coding patterns are not compatible with static typing. Following best practices and avoiding anti-patterns beforehand will make it easier to adopt static typing.

The [strict syntax parser](#) is required to use static typing. It is enabled by default in Nextflow 26.04.

Before you migrate to static typing, ensure your code complies with the strict parser using `nextflow lint` or the language server.

When preparing for the strict parser, try to address [deprecation warnings](#) as much as possible. For example:

```
Channel.from(1, 2, 3).map { it * 2 } // deprecated
channel.of(1, 2, 3).map { it -> it * 2 } // best practice
```

The above example shows how to avoid three deprecated patterns:

1. Using `Channel` to access channel factories (use `channel` instead)
2. Using the deprecated `channel.from` factory (use `channel.of` or `channel.fromList` instead)
3. Using the implicit `it` closure parameter (declare the parameter explicitly instead)

Nextflow provides three ways to assign a channel: a standard assignment, the `set` operator, and the `tap` operator:

```
ch = channel.of(1, 2, 3) // standard assignment
channel.of(10, 20, 30).set { ch } // set
channel.of(10, 20, 30).tap { ch } // tap
```

However, `set` and `tap` are not supported in typed workflows. Use standard assignments instead.

The [special operators](#) `|` and `&` provide shorthands for writing dataflow logic:

```
channel.of('Hello', 'Hola', 'Ciao')
| greet
| map { v -> v.toUpperCase() }
| view
```

However, these special operators are not supported in typed workflows. Use standard assignments and method calls instead:

```
ch_input = channel.of('Hello', 'Hola', 'Ciao')
ch_greet = greet(ch_input)
ch_greet
    .map { v -> v.toUpperCase() }
    .view()
```

The `.out` property can be used to access process and workflow outputs in legacy workflows:

```
MY_WORKFLOW()
MY_WORKFLOW.out.foo.view()
MY_WORKFLOW.out.bar.view()
```

However, this pattern is not supported in typed workflows. Use standard assignments instead:

```
my_out = MY_WORKFLOW()
my_out.foo.view()
my_out.bar.view()
```

The [each](#) input qualifier is not supported in typed processes. Use the [combine](#) operator to create a single tuple channel instead.

For example:

```
process align {
  input:
  path seq
  each mode

  script:
  """
  t_coffee -in $seq -mode $mode > result
  """
}

workflow {
  sequences = channel.fromPath('*.fa')
  methods = ['regular', 'espresso', 'psicoffee']

  align(sequences, methods)
}
```

Rewrite the script to use the `combine` operator. It becomes:

```
process align {
  input:
  tuple path(seq), val(mode)

  script:
  """
  t_coffee -in $seq -mode $mode > result
  """
}

workflow {
  sequences = channel.fromPath('*.fa')
  methods = ['regular', 'espresso', 'psicoffee']

  align(sequences.combine(methods))
}
```

tip

The `each` qualifier is discouraged in modern Nextflow code. While it provides a convenient shorthand for combining multiple inputs, it couples the process definition with external workflow logic. Since the introduction of DSL2, Nextflow aims to treat processes as standalone modules that are decoupled from workflow logic.

Many [operators](#) are not statically typed. While you can still use them in typed workflows, the type checker will not be able to fully validate your code. These operators can usually be replaced by another operator and/or a standard library function.

For example, the `splitCsv` operator is not statically typed. Use `flatMap` and the equivalent [Path](#) method instead:

```
// before
channel.fromPath('samplesheet.csv')
  .splitCsv(sep: ',')
  .view()

// after
channel.fromPath('samplesheet.csv')
  .flatMap { csv -> csv.splitCsv(sep: ',') }
  .view()
```

See [Migrating operators](#) for more information.

See the following links to learn more about static typing:

- [Processes \(typed\)](#)
- [Workflows \(typed\)](#)
- [Types](#)
- [Records](#)
- [Record type syntax](#)

Using operators with static typing

Nextflow 26.04 brings updates to the operator library in order to support static typing and records. This page provides best practices for using operators with static typing.

See [Migrating to static typing](#) for more information about migrating pipelines to static typing.

All operators can be used with or without static typing (i.e. [typed workflows](#)). However, only a core subset of operators are recommended for use with static typing, while the rest are discouraged. They are distinguished here as core operators and legacy operators.

The [core operators](#) are recommended for use with static typing. When static typing is enabled (via `nextflow.enable.types`), some of these operators have stricter semantics which may require minor changes to pipeline code. These cases are described below.

When using `collect` with static typing, it has the same semantics as `toList`. Collected values are not flattened, and when the source channel is empty, an empty list is emitted.

When using `combine` with static typing, the right operand should be a channel, dataflow value, or named arguments corresponding to record fields.

When `combine` is called with named arguments, they are appended to each source record as additional fields:

```
sample = channel.value( record(id: 1, fastq: file('1.fq')) )
index = channel.value( file('index.fa') )
sample.combine(strandedness: 'auto', index: index)

// [id:1, fastq:1.fq, strandedness:auto, index:index.fa]
```

For uses of `combine` with the `by` option, use `join` instead:

```
// before
left = channel.of( [1, 'alpha'], [2, 'beta'] )
right = channel.of( [1, 'x'], [1, 'y'], [2, 'p'] )
left.combine(right, by: 0).view()

// [1, alpha, x]
// [1, alpha, y]
// [2, beta, p]

// after (static typing enabled)
left = channel.of(
  record(id: 1, name: 'alpha'),
  record(id: 2, name: 'beta')
)
right = channel.of(
  record(id: 1, code: 'x'),
  record(id: 1, code: 'y'),
  record(id: 2, code: 'p')
)
left.join(right, by: 'id').view()

// [id:1, name:alpha, code:x]
// [id:1, name:alpha, code:y]
// [id:2, name:beta, code:p]
```

When using `filter` with static typing, the predicate should be a closure.

```
// before
ch.filter( ~/^a.* / )

// after (static typing enabled)
ch.filter { v -> v == ~/^a.* / }
```

When using `flatMap` with static typing, the mapping closure should always return a collection. Maps and tuples are not automatically flattened because they are not collection types.

```
// before
channel.of( 1, 2, 3 )
  .flatMap { n -> [ number: n, square: n*n, cube: n*n*n ] }
  .view { entry -> "${entry.key}: ${entry.value}" }

// after (static typing enabled)
channel.of( 1, 2, 3 )
  .flatMap { n -> [ tuple('number', n), tuple('square', n*n), tuple('cube', n*n*n) ] }
  .view { key, value -> "${key}: ${value}" }
```

The `groupBy` operator is a replacement for `groupTuple` that is statically typed.

While `groupTuple` accepts tuples of arbitrary length, `groupBy` accepts either a 2-tuple of (`<key>`, `<value>`) or a 3-tuple of (`<key>`, `<size>`, `<value>`). Specifying the group size with each input tuple provides the same behavior as using the `size` option (or wrapping each key with `groupKey()`) does with `groupTuple`.

While `groupTuple` can group multiple lists in a group, `groupBy` always emits 2-tuples of the form (`<key>`, `<values>`), where `<values>` is an unordered collection (`Bag`). This approach avoids a pitfall with `groupTuple` where the grouped lists can be ordered inconsistently.

When using `join` with static typing, the `by` option is required. It should be either an integer (for joining tuples by index) or a string (for joining records by field name).

When using `join` with records, the `failOnDuplicate` and `failOnMismatch` options are not supported. Duplicate matches are handled by emitting each matching combination (like a relational join). Unmatched records are either emitted or discarded depending on whether the `remainder` option is set. To fail on mismatches, use the `remainder` option and check for unmatched records in downstream logic.

```
// tuples
left = channel.of( ['X', 1], ['Y', 2], ['Z', 3], ['P', 7] )
right = channel.of( ['Z', 6], ['Y', 5], ['X', 4] )
left.join(right).view()

// [X, 1, 4]
// [Y, 2, 5]
// [Z, 3, 6]

// records
left = channel.of(
  record(id: 'X', a: 1),
  record(id: 'Y', a: 2),
  record(id: 'Z', a: 3),
  record(id: 'P', a: 7)
)
right = channel.of(
  record(id: 'Z', b: 6),
  record(id: 'Y', b: 5),
  record(id: 'X', b: 4)
)
left.join(right, by: 'id').view()

// [id: X, a: 1, b: 4]
// [id: Y, a: 2, b: 5]
// [id: Z, a: 3, b: 6]
```

When using `map` with static typing, `null` values are not automatically discarded. Use `filter` to discard `null` values explicitly.

```
// before
ch.map { r -> r.id }

// after (static typing enabled)
ch.map { r -> r.id }.filter { id -> id != null }
```

When using `mix` with static typing, only one argument should be supplied for each `mix` call.

```
// before
ch1.mix(ch2, ch3)

// after (static typing enabled)
ch1.mix(ch2).mix(ch3)
```

The [legacy operators](#) are discouraged from use with static typing. They can still be used, but the type checker will not be able to validate them.

This section describes how to rewrite each legacy operator with core operators.

Use `filter` and `map` for each branch instead. Using records instead of tuples can eliminate much of the need for `branch`.

Example requiring only `filter`:

```
// before
ch_gvcf_branch = ch_gvcf.branch { meta, gvcf, tbi ->
  no_tbi: !tbi
    return tuple(meta, gvcf)
  tbi: tbi
    return tuple(meta, gvcf, tbi)
}

ch_gvcf_branch.no_tbi.view()
ch_gvcf_branch.tbi.view()

// after
ch_gvcf_no_tbi = ch_gvcf.filter { s -> !s.tbi }
ch_gvcf_tbi = ch_gvcf.filter { s -> s.tbi }
```

Example requiring `filter` and `map`:

```
// before
ch_input_by_type = ch_input.branch { meta, platform, fastq_1, fastq_2 ->
  fastq: meta.single_end || fastq_2
    return tuple(meta + [type: "short"], fastq_2 ? [fastq_1, fastq_2] : [fastq_1])

  nanopore: platform == 'OXFORD_NANOPORE'
    meta.single_end = true
    return tuple(meta + [type: "long"], [fastq_1])

  pacbio: platform == 'PACBIO_SMRT'
    meta.single_end = true
    return tuple(meta + [type: "long"], [fastq_1])
}

ch_input_by_type.fastq.view()
ch_input_by_type.nanopore.view()
ch_input_by_type.pacbio.view()

// after -- no more fastq_1/fastq_2 wrangling
ch_input_fastq = ch_input
  .filter { s -> s.single_end || s.fastq_2 }
  .map { s -> s + record(type: 'short') }

ch_input_nanopore = ch_input
  .filter { s -> s.platform == 'OXFORD_NANOPORE' }
  .map { s -> s + record(single_end: true, type: 'long') }

ch_input_pacbio = ch_input
  .filter { s -> s.platform == 'PACBIO_SMRT' }
  .map { s -> s + record(single_end: true, type: 'long') }

ch_input_fastq.view()
ch_input_nanopore.view()
ch_input_pacbio.view()
```

These operators are [non-deterministic](#). Use `groupBy` or `List::collate()` instead.

```
// before
channel.of(1..9)
  .collate(3)
  .view()

// [1, 2, 3]
// [4, 5, 6]
// [7, 8, 9]

// after
channel.of(1..9)
  .map { i -> tuple((i - 1).intdiv(3) + 1, i) }
  .groupBy()
  .view()

// [1, [1, 2, 3]]
// [2, [4, 5, 6]]
// [3, [7, 8, 9]]
```

The `collectFile` operator is useful for collecting intermediate results into a final output file, or writing a samplesheet. In many cases, `collectFile` can be replaced by a [workflow output](#), which can generate an index file for a published channel.

For other cases, consider the following alternatives:

- Use the `collect` and `groupBy` operators to collect and group items
- Use `Iterable::toSorted` to sort collected items
- Use an `exec` process to write files (see [Working with files](#))

You can compose these functions and operators as needed to achieve the desired functionality.

For example:

```
nextflow.enable.types = true

process COLLECT_FILE {
    input:
    name: String
    items: List<String>

    output:
    file(name)

    exec:
    def path = task.workDir.resolve(name)
    items.each { item ->
        path << item
        path << '\n'
    }
}

workflow {
    val_names = channel.of('alpha', 'beta', 'gamma')
        .collect()
        .map { names -> names.toSorted() }

    COLLECT_FILE('sample.txt', val_names)
        .view { result -> result.text }
}
```

Prints:

```
alpha
beta
gamma
```

Use `mix` instead.

Use `collect` and the corresponding `Iterable` methods instead.

Use `join` with records instead.

```
// before
left = channel.of( [1, 'alpha'], [2, 'beta'] )
right = channel.of( [1, 'x'], [1, 'y'], [2, 'p'] )
left.cross(right).view()

// [[1, alpha], [1, x]]
// [[1, alpha], [1, y]]
// [[2, beta], [2, p]]

// after
left = channel.of(
    record(id: 1, name: 'alpha'),
    record(id: 2, name: 'beta')
)
right = channel.of(
    record(id: 1, code: 'x'),
    record(id: 1, code: 'y'),
    record(id: 2, code: 'p')
)
left.join(right, by: 'id').view()

// [id:1, name:alpha, code:x]
// [id:1, name:alpha, code:y]
// [id:2, name:beta, code:p]
```

This operator is `non-deterministic`. Use `unique` instead.

Use `view` instead. The `view` operator now supports the `tag` option, allowing it to be used like `dump`.

These operators are **non-deterministic**. Use the corresponding [List](#) methods instead.

Use `flatMap` instead.

The `ifEmpty` operator is typically used to either (1) raise an error if a channel is empty or (2) provide a fallback for a null dataflow value.

With static typing, both cases can be implemented without `ifEmpty`:

```
// (1) fail if channel is empty
files_ch = channel.fromPath('*.txt')
files_ch.collect().subscribe { files ->
    if( files.isEmpty() )
        error 'no input files were found'
}
files_ch.view()

// (2) provide a fallback for dataflow value
index_file = FETCH_INDEX().map { index ->
    index ?: file('index_default.txt')
}
index_file.view()
```

The example for (2) assumes that `FETCH_INDEX` is a typed process. Typed processes emit `null` when an optional output is missing, whereas legacy processes emit nothing.

This operator is **non-deterministic**. Use `join` instead.

Use `map` for each branch instead. Using records instead of tuples can eliminate much of the need for `branch`.

For example:

```
// before
ch_input_by_type = ch_input.multiMap { families, meta, cram, crai, gvcf, tbi, roi ->
    def new_meta = meta + [
        family_count: families[meta.family].size(),
        type: gvcf && cram ? "gvcf_cram" : gvcf ? "gvcf" : "cram"
    ]

    gvcf: tuple(new_meta, gvcf, tbi)
    cram: tuple(new_meta, cram, crai)
    roi: tuple(new_meta, roi)
}

ch_input_by_type.gvcf.view()
ch_input_by_type.cram.view()
ch_input_by_type.roi.view()

// after -- just keep everything in a single record
ch_input = ch_input.map { s ->
    s + record(
        family_count: s.families[s.family].size(),
        type: s.gvcf && s.cram ? "gvcf_cram" : s.gvcf ? "gvcf" : "cram"
    )
}
```

This operator is **non-deterministic**. It should not be used.

If needed, it is possible to implement a function that samples a collection (e.g., using `Math.random()` from the Java standard library).

Use standard assignments instead:

```
// before
channel.of(10, 20, 30).set { my_channel }

// after
my_channel = channel.of(10, 20, 30)
```

Use standard assignments instead:

```
// before
channel.of(10, 20, 30)
  .tap { log1 }
  .map { v -> v * 2 }
  .tap { log2 }

// after
log1 = channel.of(10, 20, 30)
log2 = log1.map { v -> v * 2 }
```

Use the equivalent `Path` method with `flatMap` instead:

```
// before
channel.fromPath('samplesheet.csv')
  .splitCsv(sep: ',')
  .view()

// after
channel.fromPath('samplesheet.csv')
  .flatMap { csv -> csv.splitCsv(sep: ',') }
  .view()
```

Use `collect` instead.

Use `collect` and `Iterable::toSorted` instead:

```
// before
channel.of(3, 2, 1, 4)
  .toSortedList()
  .view()

// after
channel.of(3, 2, 1, 4)
  .collect()
  .map { vals -> vals.toSorted() }
  .view()
```

Use `flatMap` instead:

```
// before
channel.of(
    tuple(1, ['A', 'B', 'C']),
    tuple(2, ['C', 'A']),
    tuple(3, ['B', 'D']),
)
.view()

// after
channel.of(
    tuple(1, ['A', 'B', 'C']),
    tuple(2, ['C', 'A']),
    tuple(3, ['B', 'D']),
)
.flatMap { key, values ->
    values.collect { value -> tuple(key, value) }
}
.view()
```

Understanding task resource metrics

This tutorial explains how resource usage metrics are computed from execution reports. See [Execution report](#) for more information about how to enable them.

CPU Usage plots report how CPU resources are used by each process.

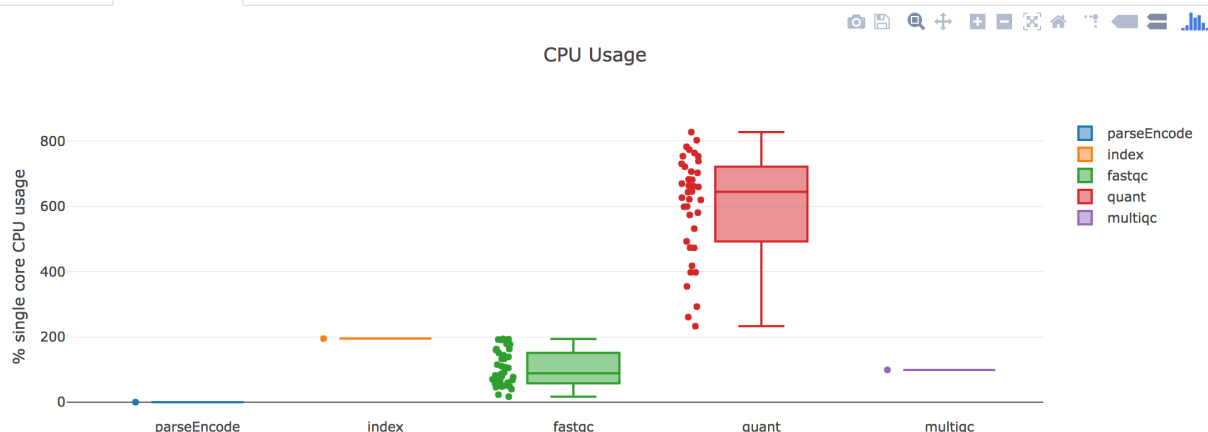
Resource Usage

These plots give an overview of the distribution of resource usage for each process.

CPU Usage

% Allocated

Raw Usage



Raw Usage tabs are expected to show 100% core usage if processes perform one task of pure computation. If tasks are distributed over, 2, 3, or 4 CPUs, the raw usage will be 200%, 300%, or 400%, respectively. **% Allocated** tabs rescale raw usage values relative to the number of CPUs that are set with the `cpus` directive. If the `cpus` directive is not set, CPUs are set to `1` and **% Allocated** tabs will show the same values **Raw Usage** tabs.

For example, using the `stress` program, the following script would report 100% CPU usage in the **Raw Usage** tab and 50% CPU usage in the **% Allocated** tab as the process requested double the number of CPUs that are required:

```
process cpu_usage_ex1 {
  cpus 2

  script:
  """
  stress -c 1 -t 10 # compute square-root of random numbers during 10s using 1 CPU
  """
}

workflow{
  cpu_usage_ex1() // Stress using 1 CPU
}
```

tip

See [Linux stress command with examples](#) for more information about the stress command.

CPU usage decreases if processes spend some time performing pure computation and some time waiting for CPUs. For example, using the `stress` and `sleep` commands, the following script would report 75% CPU usage in the **Raw Usage** tab:

```

process cpu_usage_ex2 {
  cpus 1

  script:
  """
  stress -c 1 -t 10 # compute square-root of random numbers during 10s using 1 CPU
  stress -c 1 -t 5 # compute square-root of random numbers during 5s using 1 CPU
  sleep 5 # use no CPU during 5s
  """
}

workflow{
  cpu_usage_ex2() // Stress using 1 CPU and sleep
}

```

In the above example, CPU usage is a weighted average that accounts for the percentage of the CPU used and duration of each individual program over the job duration:

$$100\% \times 10s + 100\% \times 5s + 0\% \times 5s = 75\% \frac{100\% \times 10s + 100\% \times 5s + 0\% \times 5s}{10s + 5s + 5s} = 75\% \frac{100\% \times 10s + 100\% \times 5s + 0\% \times 5s}{10s + 5s + 5s} = 75\%$$

CPU usage increases if a single step is forked on multiple CPUs:

```

process cpu_usage_ex3 {
  cpus 2

  script:
  """
  stress -c 2 -t 10 # compute square-root of random numbers during 10 s using 2 CPUs
  sleep 10 # use no CPU during 10s
  """
}

workflow{
  cpu_usage_ex3() // Stress using 2 CPUs and sleep
}

```

In the above example, the **Raw Usage** tab would report 100%:

$$200\% \times 10s + 0\% \times 10s = 100\% \frac{200\% \times 10s + 0\% \times 10s}{10s + 10s} = 100\% \frac{200\% \times 10s + 0\% \times 10s}{10s + 10s} = 100\%$$

However, the **% Allocated** tab would report 50%. It would not be relevant to change the `cpus` directive from 2 to 1 as the process uses 2 CPUs at it peak load.

Memory Usage plots report how memory was used by each process. It has three tabs, **Physical (RAM)**, **Virtual (RAM + Disk swap)**, and **% RAM Allocated**, showing the usage of the physical memory (RAM), the virtual memory (vmem), and the percentage of RAM used by the process relative to the memory that the `memory` directive set, respectively.

Peak usage during process executions is reported for both physical and virtual memories. The total amount of memory used by a process is the `virtual memory (vmem)`. The `vmem` contains all memory areas, including in the physical memory (RAM), in the swap space, on the disk, or shared with other processes. The `resident set size (RSS)` is the amount of `physical memory (RAM)` held by a process.

The relationship is:

$$vmem \geq RSS + Swap \quad vmem \geq RSS + Swap$$

The behavior of **Memory Usage** plots can be examined using two programs written in C. The first program allocates a variable of 1 GiB:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/resource.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <time.h>

/* Get vmem and rss usage from /proc/<pid>/statm */
static int mem_used(pid_t pid, unsigned long* vmem, unsigned long* rss) {
    FILE* file;
    char path[40];
    unsigned int page_size;

    snprintf(path, 40, "/proc/%ld/statm", (long) pid);
    file = fopen(path, "r");
    // vmem and rss are the first values in the file
    fscanf(file, "%lu %lu", vmem, rss);
    // values in statm are in pages so to get bytes we need to know page size
    page_size = (unsigned) getpagesize();
    *vmem = *vmem * page_size;
    *rss = *rss * page_size;

    fclose(file);
    return 0;
}

int main(int argc, char **argv) {
    unsigned char *address;
    char input;
    size_t size = 1024*1024*1024; // 1 GiB
    unsigned long i;
    unsigned long vmem = 0;
    unsigned long rss = 0;
    pid_t pid;

    pid = getpid();
    printf("Pid: %ld\n", (long) pid);

    mem_used(pid, &vmem, &rss);
    printf("VMEM: %lu RSS: %lu\n", vmem, rss);

    address = malloc(size);
    printf("Allocated %d Bytes of memory\n", (int) size);

    mem_used(pid, &vmem, &rss);
    printf("VMEM: %lu RSS: %lu\n", vmem, rss);

    // Leave time for nextflow to get information
    sleep(15);

    free(address);
    return 0;
}
```

The second program allocates a variable of 1 GiB and fills it with data:


```

#include <stdio.h>
#include <stdlib.h>
#include <sys/resource.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <time.h>

/* Get vmem and rss usage from /proc/<pid>/statm */
static int mem_used(pid_t pid, unsigned long* vmem, unsigned long* rss) {
    FILE* file;
    char path[40];
    unsigned int page_size;

    snprintf(path, 40, "/proc/%ld/statm", (long) pid);
    file = fopen(path, "r");
    // vmem and rss are the first values in the file
    fscanf(file, "%lu %lu", vmem, rss);
    // values in statm are in pages so to get bytes we need to know page size
    page_size = (unsigned) getpagesize();
    *vmem = *vmem * page_size;
    *rss = *rss * page_size;

    fclose(file);
    return 0;
}

int main(int argc, char **argv) {
    unsigned char *address;
    char input;
    size_t size = 1024*1024*1024; // 1 GiB
    unsigned long i;
    unsigned long vmem = 0;
    unsigned long rss = 0;
    pid_t pid;

    pid = getpid();
    printf("Pid: %ld\n", (long) pid);

    mem_used(pid, &vmem, &rss);
    printf("VMEM: %lu RSS: %lu\n", vmem, rss);

    address = malloc(size);
    printf("Allocated %d Bytes of memory\n", (int) size);

    mem_used(pid, &vmem, &rss);
    printf("VMEM: %lu RSS: %lu\n", vmem, rss);

    printf("Filling memory with data...");
    fflush(stdout);
    for (i = 0; i < size; i++) {
        *(address + i) = 123;
    }

    mem_used(pid, &vmem, &rss);
    printf("\nVMEM: %lu RSS: %lu\n", vmem, rss);

    // Leave time for nextflow to get information
    sleep(15);

    free(address);
    return 0;
}

```

The first and second programs are executed as `malloc` and `malloc_fill`, respectively, in the following script:

```

process malloc {
  memory '1.5 GB'

  script:
  """
  memory_vmem_1GiB_ram_0Gib
  """
}

process malloc_fill {
  memory '1.5 GB'

  script:
  """
  memory_vmem_1GiB_ram_1Gib
  """
}

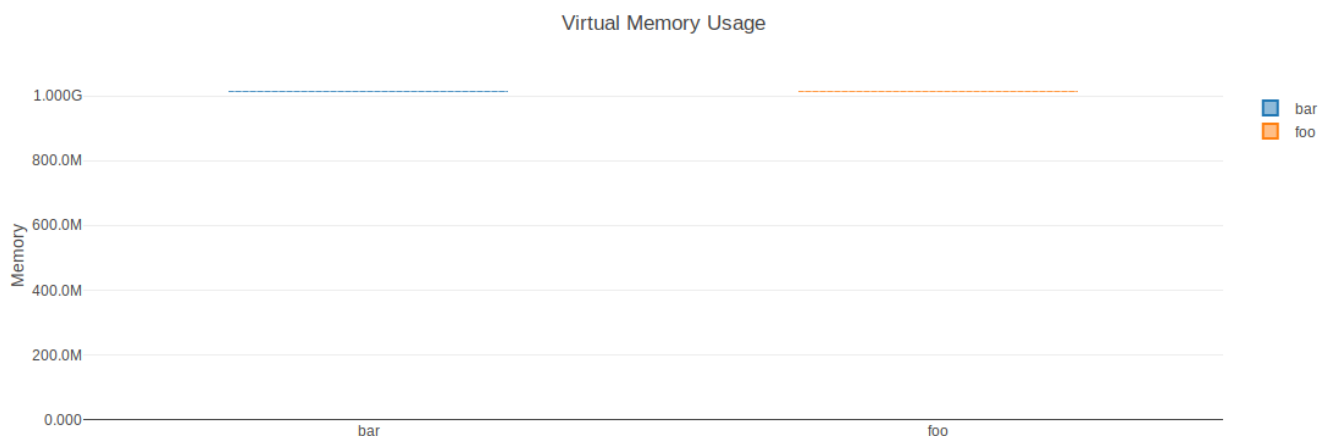
workflow{
  malloc() // Allocates a variable of 1 GiB
  malloc_fill() // Allocates a variable of 1 GiB and fills it with data
}

```

The **Virtual (RAM + Disk swap)** tab shows that both `malloc` and `malloc_fill` use the same amount of virtual memory (~1 GiB):

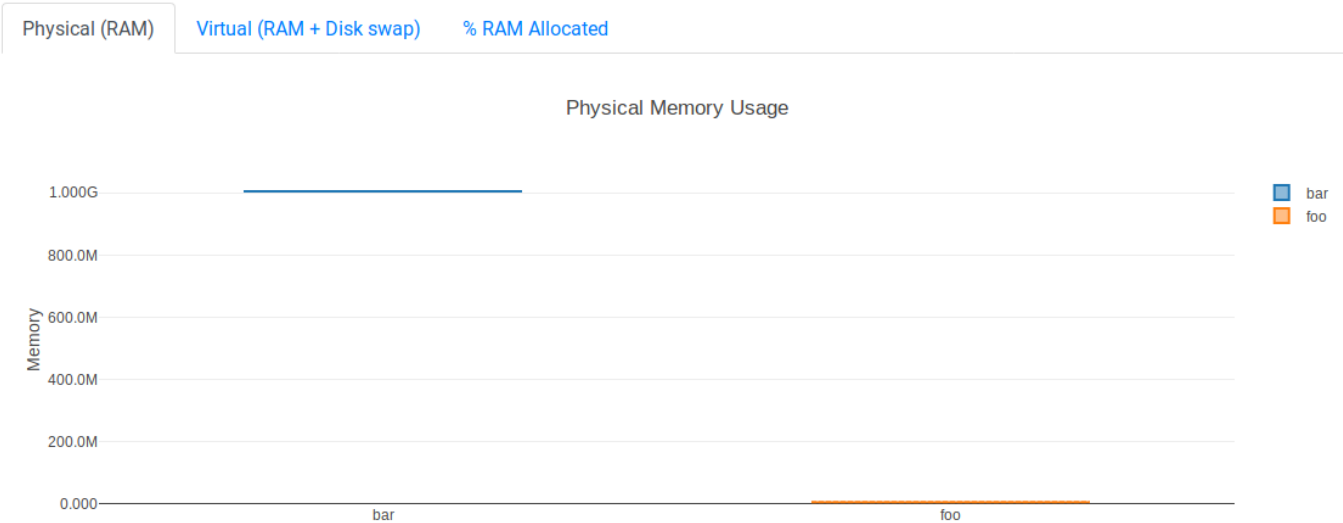
Memory

Physical (RAM) Virtual (RAM + Disk swap) % RAM Allocated



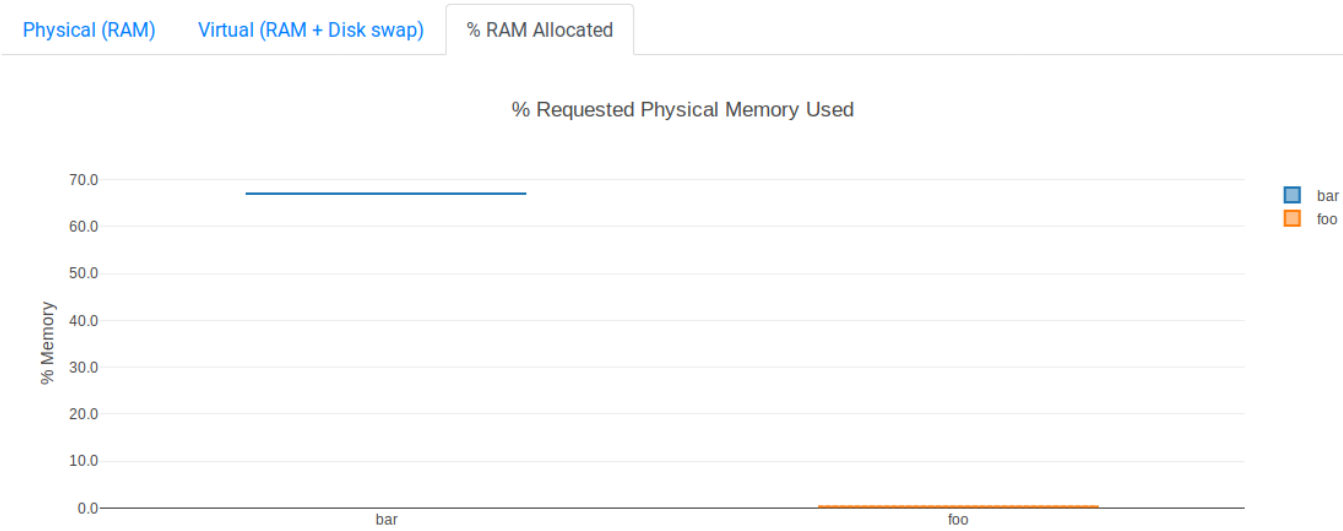
However, the **Physical (RAM)** tab shows that `malloc_fill` uses ~1 GiB of RAM while `malloc` uses ~0 GiB of RAM:

Memory



The **% RAM Allocated** tab shows that `malloc` and `malloc_fill` used 0% and 67% of resources set in the `memory` directive, respectively:

Memory



warning

Memory and storage metrics are reported in bytes. For example, 1 KB = 1024 bytes, 1 MB = 1024² bytes, and 1 GB = 1024³ bytes.

Job Duration plots report how long each process took to run. It has two tabs. The **Raw Usage** tab shows the job duration and the **% Allocated** tab shows the time that was requested relative to what was requested using the `time` directive. Job duration is sometimes known as elapsed real time, real time or wall time.

Job Duration



I/O Usage plots show how much data was read and written by processes. The amount of data that was read by a process (`rchar` in trace files) is the number of bytes the process read using read-like system calls. The amount of data that was written by a process (`wchar` in trace files) is the number of bytes the process wrote using write-like system call. Read and write data is read from the file `/proc/$pid/io`.

Read tabs shows how much data was read and **Write** tabs shows how much data was written by each process. For example, the following script reads and writes different data volumes:

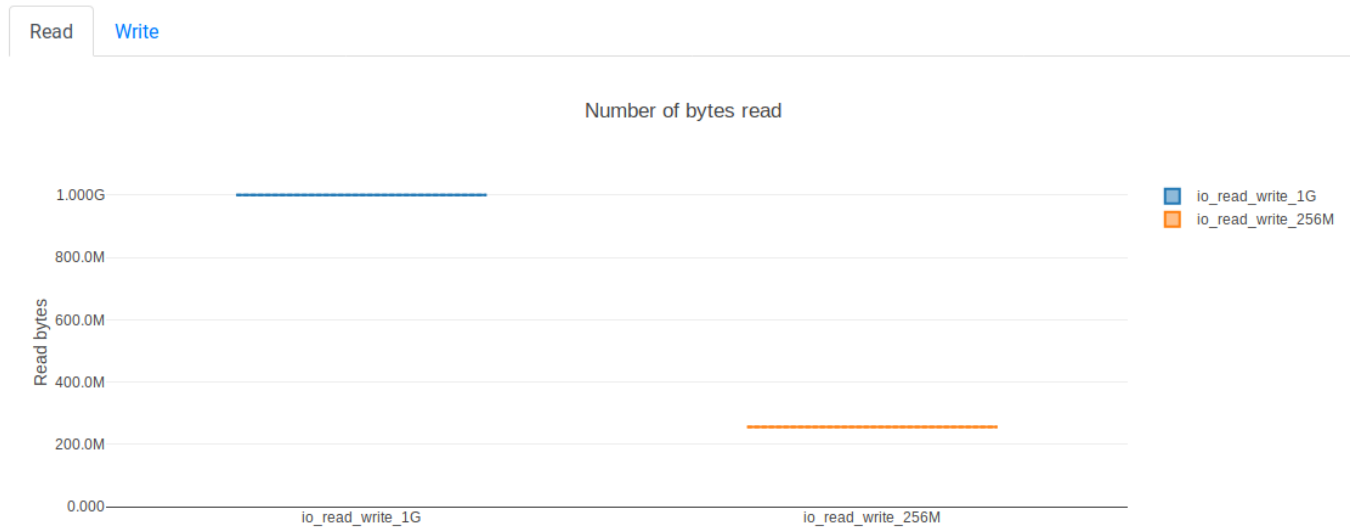
```
process io_read_write_1G {
  script:
  """
  dd if=/dev/zero of=/dev/null bs=1G count=1
  """
}

process io_read_write_256M {
  script:
  """
  dd if=/dev/zero of=/dev/null bs=256M count=1
  """
}

workflow{
  io_read_write_1G() // Read and write 1 GiB
  io_read_write_256M() // Read and write 256 Mb
}
```

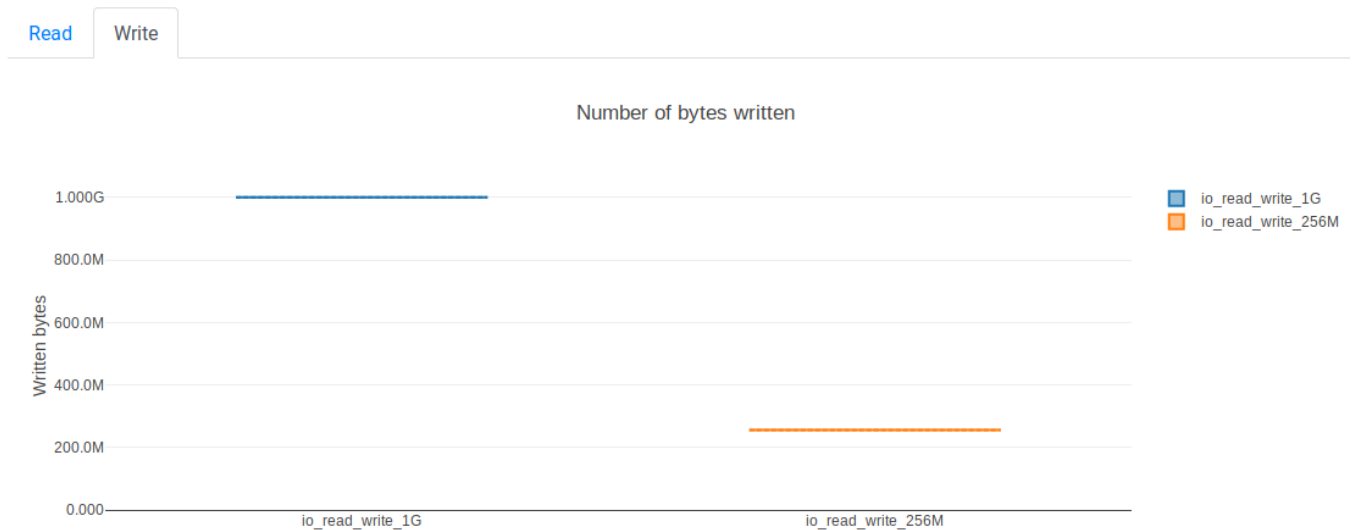
The **Read** tab shows that ~1 Gib and ~256 Mb are read:

I/O



The **Write** tab shows that ~1 Gib and ~256 Mb are written:

I/O



Using Nextflow with Flux

Added in version 23.04

The [Flux Framework](#) is a modern resource manager that can span the space between cloud and HPC. If your center does not provide Flux, you can [build Flux yourself](#) and launch it as a job using your resource manager of choice (e.g. SLURM or a cloud provider).

In the [docker/flux](#) directory we provide a [Dockerfile for interacting with Flux](#) along with a [VSCode Developer Container](#) environment that you can put at the root of the project to be provided with a Flux agent and the dependencies needed to build Nextflow. There are two ways to use this:

- Build a container from scratch and bind your code to it (e.g. for development or testing)
- Use VSCode and DevContainers to create a more seamless environment

Both strategies are described below. For this tutorial, you will generally want to prepare a pipeline to use the `flux` executor, create an environment with Flux, start a Flux instance, and interact with it.

To run your pipeline with Flux, you'll want to specify it in your config. Here is an example `nextflow.config`:

```
manifest {
  mainScript = 'demo.nf'
  homepage = 'https://github.com/nextflow-io/nextflow/tree/master/docker/flux'
  description = 'Demo using Nextflow with Flux'
}

process {
  executor = 'flux'
}
```

For additional Flux settings, see the [Flux executor](#) section.

Here is an example pipeline that we will use:

```
workflow {
  breakfast = channel.of ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '
  haveMeal(breakfast)
}

process haveMeal {
  debug true
  input:
  val food
  script:
  """
  printf '$food for breakfast!'
  """
}
```

You can either build the Docker image from the root of the Nextflow repository:

```
$ docker build -f docker/flux/.devcontainer/Dockerfile --platform linux/amd64 -o type=docker -t nextflow-flux .
```

And then shell into the container for a development environment. You'll need to bind the present working directory to `/code` to see your local changes in the container:

```
$ docker run -it -v $PWD:/code nextflow-flux
```

You can also move the `.devcontainer` directory to the root of your repository, and open it in VSCode:

```
$ cp -R docker/flux/.devcontainer .devcontainer
```

Then open in VSCode, and select **Re-open in container**:

```
$ code .
```

Then you should be able to open a terminal (**Terminal -> New Terminal**) to interact with the command line. Try running `make` again! Whichever of these two approaches you take, you should be in a container environment with the `flux` command available.

Once in your container, you can start an interactive Flux instance (from which you can submit jobs on the command line to test with Nextflow) as follows:

```
$ flux start --test-size=4
```

Here is an example of submitting a job and getting the log for it.

First submit the job:

```
$ flux submit echo "HELLO MOTO"
fEzWqspb
```

Then get the log for it:

```
$ flux job attach fEzWqspb
HELLO MOTO
```

Try submitting a longer job:

```
$ flux submit sleep 60
```

And then seeing it in the jobs listing.

```
$ flux jobs
      JOBID USER   NAME      ST NTASKS NNODES   TIME INFO
      f4tkMUAAT root   sleep      R     1      1  2.546s ab6634a491bb
```

Prepare your `nextflow.config` and `demo.nf` in the same directory.

```
$ ls .
demo.nf  nextflow.config
```

Finally, run the pipeline with Flux:

```
.
$ nextflow -c nextflow.config run demo.nf
.
N E X T F L O W ~ version 22.10.0
Launching `demo.nf` [clever_blackwell] DSL2 - revision: f8cda838cb
executor > flux (5)
[4c/f162db] process > haveMeal (3) [100%] 5 of 5 ✓
    for breakfast!
    for breakfast!
    for breakfast!
    for breakfast!
    for breakfast!
```


Class Diagrams

This directory contains class diagrams of the Nextflow source code, abridged and annotated for relevance and ease of use.

Each node is a class. Fields are selectively documented in order to show only core data structures and the classes that "own" them. Methods are not explicitly documented, but they are mentioned in certain links where appropriate.

Links between classes denote one of the following relationships:

- Inheritance (`A <|-- B`): `B` is a subclass of `A`
- Composition (`A --* B`): `A` contains `B`
- Instantiation (`A --> B : f`): `A` creates instance(s) of `B` at runtime via `A::f()`

Some links are commented out or not included at all, in order to focus on the most important classes and relationships. You can view these "hidden" links by simply uncommenting them, but I have found that their significance is sufficiently clear from the description files.

A separate diagram description is provided for each package. These files are interoperable, which means that you can combine any subset of files into a larger diagram description. The `merge-diagrams.sh` can create a merged file for you automatically, and it includes a sensible default set of packages.

You can use the [Mermaid Live Editor](#) or the [Mermaid CLI](#) to render the diagram in a variety of image formats.

Source: <https://docs.seqera.io/nextflow/developer/diagrams/>